SBTK handbook

Documentation of SBTK (Small Body Toolkit) version 1.0.0: density distributions, gravity coefficients and moments of inertia of small bodies from polyhedral shape models, and estimation of densities from gravity and inertia measurements.

Generated from the SBTK source on 2026-10-20. The website version is at https://ashcav1.github.io/sbtk-docs/.

Contents

1. SBTK: Small Body Toolkit
2. Assumptions and limitations
3. Installation
4. First script
5. Units
6. Coordinate systems and origins
7. Gravity coefficients
8. Frames of a gravity field
9. Moments of inertia
10. Shape models
11. Density distributions
12. Gravity and moment calculations
13. Measurement sets
14. Density estimation
15. Goodness of fit and model comparison
16. Results folders and figures
17. Exchange packages
18. Density models by component type
19. Calculations
20. Tutorial scripts
21. Project template
22. Bennu
23. Hera blind test
24. API reference
25. sbtk
26. sbtk.units
27. sbtk.shape
28. sbtk.body
29. sbtk.interior
30. sbtk.gravity
31. sbtk.data
32. sbtk.invert
33. sbtk.compare
34. sbtk.exchange
35. sbtk.io
36. sbtk.plot
37. Data structures
38. Error identifiers
39. Internal functions
40. Validation and tests

SBTK: Small Body Toolkit

SBTK is a MATLAB toolkit for the interior density distribution of asteroids and other small bodies. It computes the gravity coefficients, center of mass and moments of inertia of a density model inside a polyhedral shape model, and it estimates the densities of such a model from measured gravity coefficients and moments of inertia. This site documents version 1.0.0.

Capabilities

The following are implemented and covered by the test suite (Validation).

Density models
: A background density, uniform components (spheres, triaxial ellipsoids, tori with any axis, layers below the surface, latitude–longitude–radius sectors) and an optional Gaussian random density field, clipped to a closed polyhedral shape model that is star-shaped about the expansion origin (Density distributions).

Gravity and moment calculations
: Fully normalized spherical-harmonic coefficients to any degree, the center of mass and the inertia tensor of such a model, from exact polyhedral integrals over the cones between the expansion origin and each surface facet (Gravity and moment calculations).

Frames
: Coefficients about the center of mass in the shape-model axes, the spin axes or the principal axes of inertia, and bodies, measurements and models carried into another frame (Frames of a gravity field).

Measurement sets
: Gravity coefficients with a full covariance, moment-of-inertia elements, linear combinations of them, the libration ratio $(B - A) / C$ and the center of mass, simulated (exact values, or with uncertainties and Gaussian noise) or read from files (Measurement sets).

Density estimation
: Weighted least squares for the background density and the relative density of each component, with the total mass as an equality constraint, bounds on the densities, optional Tikhonov regularization, optional estimation of the position of spheres, ellipsoids and tori and of the axis of ellipsoids and tori, linearized 1σ uncertainties, and a χ^2 test of the fit (Density estimation).

Comparison with a true model
: For synthetic tests: the fraction of the variance of the true density explained by the estimate (R^2), the largest value the chosen components allow, and a cell-by-cell comparison of density grids (Goodness of fit and model comparison).

Exchange packages
: Plain-text packages of measurements, interior models and solutions with their units, frame and normalization, for comparisons between teams (Exchange packages).

The assumptions behind these calculations and what SBTK does not do are listed in Assumptions and limitations.

Status and requirements

SBTK 1.0.0 has not been publicly released; its source repository is private. It requires MATLAB R2024b or newer with the Optimization Toolbox. The polyhedral gravity kernel is a compiled C function, provided for macOS and buildable on other platforms (Installation).

Organization of this documentation

Section	Contents
Getting started	Installation and a complete first script.
Conventions	Units, coordinate systems, coefficient normalization, moments of inertia.
Models and calculations	Shape models, density distributions, the forward model.
Measurements and inversion	Measurement sets, the estimation problem, χ^2 testing and comparison statistics.
Files	Results folders, figures and the exchange-package format.
Examples	One example per component type and per calculation; the eight tutorial scripts and the six maker models.
Projects	The project template and the Bennu and Hera blind-test projects.
Reference	Every supported function, every data structure and field, every error identifier; internal functions separately.
Validation	What each test checks and its tolerance.

A single-page version and a PDF handbook contain the same pages.

Credits

SBTK was written by A. Cavale (University of Colorado Boulder). The polyhedral gravity kernel `ComputePolyhedralCSBar_mex` was written by Y. Takahashi (University of Colorado Boulder). The Bennu project reproduces the inversion of Cavale et al., "Constraints on the Internal Density Distribution of Bennu Using OSIRIS-REx Mission Data" (Icarus). Shape models: DART stereophotoclinometry models of Didymos (v003) and Dimorphos (v004); OSIRIS-REx shape model of Bennu.

Assumptions and limitations

This page lists the physical and numerical assumptions of SBTK 1.0.0 and the calculations it does not perform. Each item links to the page where the behavior is described.

Physical model

- The body is rigid and its density does not change with time. The gravity field is described by spherical-harmonic coefficients only; SBTK does not evaluate the potential or the gravitational acceleration at field points.
- The density is a background density plus uniform components plus an optional Gaussian random field (Density distributions). Gradients within a component, other than through the random field, are not modeled.
- The shape model is fixed and exact: its uncertainty is not propagated into the coefficients or the estimates.
- The total mass is treated as exact. A measured GM enters only as the mass of the model; its uncertainty is not used (`sbtk.exchange.read_shadr` reads it and states that it is not used).

Geometry

- The shape model must be closed, ordered outward and star-shaped about the expansion origin: every facet must face away from it (Shape models). Bodies with overhangs that violate this cannot be integrated.
- Layers and sectors are defined along rays from the expansion origin (the center of mass), not along the local surface normal (Coordinate systems).
- The z-axis of the shape-model frame is taken as the spin axis and as a principal axis in the definition of the libration ratio (Moments of inertia). For a homogeneous body the axis of the largest moment is 3.6° from the shape-model z-axis for Didymos and 0.7° for Dimorphos.
- Measurements, models and estimates of one body are in one frame, that of its shape model. `sbtk.data.to_frame` turns the axes of a measurement set only when its inertia rows are exact (the values of `sbtk.data.simulate`): rows with uncertainties would mix with correlations that the rows cannot hold. An inversion result cannot be transformed; invert in the frame wanted (Frames of a gravity field).
- The spin axes are those of a body in principal-axis rotation (z along the axis of the largest moment); the rotation state itself is not modeled, and a spin pole known with an error is represented only by the noise of $\bar{C}_{21}$, $\bar{S}_{21}$.
- When a field with uncertainties is moved to its center of mass or rotated into its principal axes, the covariance is propagated through the translation and rotation as linear maps; the uncertainty of the center of mass and of the principal axes themselves is not added.

Numerical integration

- Facet-cone integrals of the whole body are exact up to rounding (Gravity and moment calculations).
- A sphere entirely inside the body is integrated as a point mass, which is exact.
- Other components are integrated by sampling their radial extent along 9 sub-rays per facet and averaging the exact cone integrals over them. The result is exact when the radial interval is the same along every sub-ray of a cone and otherwise converges with the mesh resolution. Sector boundaries in latitude and longitude are resolved at the sub-ray spacing. The validation tests bound the errors for particular cases (Validation).
- The crossings of a ray with a sphere, ellipsoid or torus are exact (roots of a quadratic or quartic). The integrals of these components are continuous in their position and orientation, but smooth only on scales larger than the ray spacing: where a ray grazes a component, its piece inside changes like a square root. The derivatives used to estimate geometry are therefore taken over steps of 0.1 m and 0.01° (Estimated geometry).
- The random density field is integrated with one value per radial shell (24 shells per facet cone).

Estimation

- The measurements are linear in the densities. Positions and axes of components, when estimated, are found by Gauss–Newton iteration with a trust region and optional a priori constraints and limits; the geometry of layers and sectors, and of every component not marked for estimation, is fixed. An estimated geometry can absorb signal from structure the model lacks (Estimated geometry).
- The uncertainties are the linearized covariance with the mass constraint and fixed parameters eliminated. The inequality bounds on the densities are not reflected in the covariance; `fit.at_bound` flags densities at a bound.
- The χ^2 test assumes Gaussian measurement errors with the stated covariance.
- Simulated noise is Gaussian, drawn from the stated covariance of the coefficients and from the σ of each inertia row independently (`sbtk.data.add_noise`).
- The libration ratio obtained from a libration amplitude (`sbtk.data.libration_ratio_from_amplitude`) uses planar forced libration to first order in the eccentricity, about a point-mass primary; free libration, the primary's oblateness and the mutual dynamics of a binary are not included.

Data formats

- Coefficients must be fully normalized without the Condon–Shortley phase (Gravity coefficients). `sbtk.exchange.read_shadr` rejects files whose header does not state full normalization; other conversions are the user's responsibility.
- The exchange format gives the center of mass separately from the gravity coefficients, so a covariance between the center of mass and the other coefficients cannot be written (`sbtk:exchange:correlatedCenterOfMass`).
- The SBTK measurement text format stores only the diagonal of the coefficient covariance; the `.mat` file keeps the full matrix (Results folders and figures).

Software

- MATLAB R2024b or newer with the Optimization Toolbox (`lsqlin`). The compiled kernel is provided for macOS (Apple silicon and Intel) and must be compiled once on other platforms (Installation).
- Version 1.0.0 is not publicly released.

Installation

Requirements

- MATLAB R2024b or newer. SBTK 1.0.0 is tested with R2025a.
- Optimization Toolbox, for `lsqlin` in the inversion.
- A compiled copy of the polyhedral gravity kernel `ComputePolyhedralCSBar_mex` for the platform. Binaries for macOS on Apple silicon (`.mexmaca64`) and Intel (`.mexmaci64`) are included; on other platforms it must be compiled once (below).

No other toolboxes or external libraries are used.

Repository layout

Folder	Contents
`toolbox/+sbtk/`	The toolkit: one function per file, one folder per package (`+units`, `+shape`, …).
`sbtk_setup.m`	Adds `toolbox/` to the MATLAB path for the current session.
`data/shapes/`	Shape models in OBJ format, lengths in km.
`data/gravity/`	The OSIRIS-REx gravity field of Bennu (JPL, 2019).
`examples/`	Eight tutorial scripts, `ex01_quick_start.m` … `ex08_moments_of_inertia.m` (Tutorial scripts).
`projects/`	Runner scripts for the project template, Bennu and the Hera blind test (Projects).
`results/`	Output of every run, created on demand (Results folders).
`tests/`	Unit and integration tests, longer studies and test reports (Validation).
`tools/`	Maintenance scripts (reduced shape models, benchmark, README figures).
`docs/`	Notes in Markdown and earlier documents.

Setting the path

Scripts in `examples/` and `projects/` add `toolbox/` to the path themselves, so they can be run directly, also section by section. For the Command Window or other scripts, run `sbtk_setup.m` once per MATLAB session:

MATLAB

```
run("/path/to/SBTK/sbtk_setup.m")     % replace /path/to/SBTK with the location of the repository
```

It prints the version and the repository folder. `sbtk.version` and `sbtk.root` return them.

Runner scripts locate themselves with `fileparts(which('<script name>'))` rather than `mfilename`, because under **Run Section** MATLAB executes a temporary copy of the script. A copied runner must have the name inside `which` changed to its own file name.

Compiling the gravity kernel

On platforms without a compiled kernel, with a C compiler configured for MATLAB (`mex -setup C`):

MATLAB

```
cd(fullfile(sbtk.root, "toolbox", "+sbtk", "+internal"))
mex -O ComputePolyhedralCSBar_mex.c
```

Then run the tests; the regression test compares the kernel output with stored values.

Files that are not in the repository

File	Needed by	Source
`data/shapes/g_01165mm_spc_obj_didy_0000n00000_v003.obj` (Didymos, 1.165 m sampling, about 160 MB)	`sbtk.shape.bundled("didymos_highres")`, `tools/make_reduced_shapes.m`, `examples/ex01_quick_start.m`	DART shape models on ESA's SPICE server, `spiftp.esac.esa.int/data/SPICE/HERA/kernels/dsk/`
`data/shapes/g_00243mm_spc_obj_dimo_0000n00000_v004.obj` (Dimorphos, 0.243 m, about 160 MB)	`sbtk.shape.bundled("dimorphos_highres")`, `tools/make_reduced_shapes.m`	same
Uncertainty files of the Hera end-to-end test	the simulation runners of `projects/hera_hlp6`	Hera working group; not distributed (Hera blind test)

All other shape models used in the examples and tests are in `data/shapes/`.

Checking the installation

```
MATLAB
```

```
run(fullfile(sbtk.root, "tests", "run_tests.m"))
```

This runs 129 unit and integration tests in about three minutes and writes `tests/reports/test_report.txt`. Every test should pass (Validation).

First script

The script below runs the complete SBTK sequence on the shape model of the asteroid Didymos: it defines a body and a known interior (the true model), computes the exact gravity coefficients and moments of inertia of that interior, turns them into measurements by giving them uncertainties and adding noise, estimates the densities of the interior from those measurements, and compares the estimate with the true model. It takes about 15 s.

Script

Replace `/path/to/SBTK` with the location of the repository and run the script.

MATLAB script: first_script.m

```
run("/path/to/SBTK/sbtk_setup.m")

% 1. Body: shape model, total mass [kg], reference radius of the gravity field [km]
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
sbtk.body.summary(body)

% 2. A known interior: a dense core and a low-density surface layer
components = [sbtk.interior.sphere("core", body.centroid_km, 0.170, +450), ...   % center [km]: the center of figure
              sbtk.interior.layer("regolith", 0, 0.012, -900)];        % from 0 to 12 m below the surface
interior = sbtk.interior.create(components);

% 3. The true model and its exact gravity coefficients (degrees 1-4) and moments of inertia
[exact, truth] = sbtk.data.simulate(body, interior, 4, "first_script");

% 4. Measurements: 1-sigma uncertainties, then one realization of noise drawn from them
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), ...   % degree 1: 2 m in the center of mass
                               1e-4, 2e-4, 4e-4], ...                           % degrees 2, 3, 4
                              2e-3, ...                                         % each inertia element / (M R_ref^2)
                              0.02);                                            % libration ratio
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
sbtk.data.summary(obs)

% 5. Estimate the relative densities of the same components (the values given above are not used)
fit = sbtk.invert.solve(body, components, obs);
sbtk.invert.summary(fit)

% 6. Compare the estimate with the true model
cmp = sbtk.compare.with_truth(body, truth, fit);
sbtk.compare.summary(cmp)
sbtk.plot.truth_vs_estimate(body, truth, fit);
```

Output

Output

```
SBTK 1.0.0 ready (/path/to/SBTK)

Didymos
  shape didymos_g_9309mm_spc_obj_0000n00000_v003: 49152 facets, 24578 vertices
  volume 0.20336 km^3, bulk density 2593 kg/m^3, mass 5.273e+11 kg
  reference radius 430 m; center of figure [15.9 42.3 -34.0] m from the shape origin

Didymos measurements (first_script)
  mass 5.273e+11 kg, reference radius 430 m, gravity to degree 4
  24 gravity coefficients: 3 of degree 1, 5 of degree 2, 7 of degree 3, 9 of degree 4
  7 moment-of-inertia measurements: Ixx Iyy Izz Ixy Ixz Iyz libration
  coefficients are about origin_km = [15.9 42.3 -34.0] m (shape-model coordinates)
  coefficient covariance: diagonal (independent uncertainties)
  example: C20 = -3.4877e-02 +- 1.0e-04
  notes: Simulated by SBTK from interior first_script. Values include simulated measurement noise (seed 1).

Didymos inversion of first_script: 27 measurements
  chi2 = 27.77, dof = 25.0, reduced chi2 = 1.11, p = 0.32 (consistent with the measurements at the 5% level)

  component    density [kg/m^3]      relative density [kg/m^3]
  background     2741 +-    51
  core           2871 +-   115          +131 +-   164   not resolved (|relative density| < 1 sigma)
  regolith       1040 +-   323         -1701 +-   373

  largest normalized residual: S42, 2.0 sigma

Didymos: estimate compared with the true model
  R2 = 0.37: fraction of the variance of the true density about its mean explained by the estimate
  R2_max = 1.00: the largest R2 these components can reach
  RMS density error 267 kg/m^3; standard deviation of the true density 337 kg/m^3
  reduced chi2 of the fit 1.11 (p = 0.32)

  component         estimate [kg/m^3]       true    deviation
  core                 2871 +-   115       3083       1.8 sigma
  regolith             1040 +-   323       1733       2.1 sigma
```

The figure shows the true density, the estimated density and their difference on the planes $z = 0$ and $y = 0$ through the center of mass (Figures).

What each step does

1. **Body.** `sbtk.shape.bundled` reads the DART shape model of Didymos (version 3, 9.3 m sampling, 49,152 facets) in km. `sbtk.body.from_mass` attaches the total mass $M = 5.273{\times}10^{11}$ kg and the reference radius $R_{\mathrm{ref}} = 0.43$ km of the coefficients; the bulk density $M\,/\,V$ follows from the shape volume. The center of figure is 57 m from the origin of the shape-model file, which is not the body's center (Coordinate systems and origins).
2. **Interior.** The core is a sphere of radius 170 m centered at the center of figure, `body.centroid_km`, 450 kg/m³ denser than the background; positions of spheres, ellipsoids and tori are shape-model coordinates, and the origin of this file is 57 m from the center of figure (Where to place components). The regolith is everything within 12 m of the surface, measured along rays from the center of mass, 900 kg/m³ lighter than the background. The background density is not given: it is set so that the total mass equals M (Density distributions).
3. **True model.** `sbtk.data.simulate` computes the fully normalized gravity coefficients of degree 1 to 4, the six inertia elements and the libration ratio of this interior, about its center of mass (Gravity and moment calculations). It returns the true model (`truth`) and its exact values (`exact`), with no uncertainties: none are involved in the forward model. Degree-1 coefficients give the center of mass (Gravity coefficients).
4. **Measurements.** `sbtk.data.set_uncertainties` gives every coefficient the 1σ of its degree and every inertia quantity its 1σ; `sbtk.data.add_noise` adds one Gaussian realization drawn from them with seed 1 (Measurement sets).
5. **Estimation.** `sbtk.invert.solve` fits the background density and the relative density of each component to the 24 coefficients and, with the standard settings of `sbtk.invert.options`, to `Ixx`, `Iyy` and `Izz`, with the total mass as a constraint (Density estimation). The reduced χ^2 of 1.11 with $p = 0.32$ means that the model fits the data within their uncertainties (Goodness of fit).
6. **Comparison.** `sbtk.compare.with_truth` evaluates both densities throughout the body. $R^2 = 0.37$ is the fraction of the variance of the true density that the estimate explains; $R^2_{\mathrm{max}} = 1$ because the estimate uses the true geometry. Each component's estimate is within 2.1σ of its true density. The core is not resolved (its relative density is smaller than its 1σ): a core concentric with the body changes the field much as the background and the regolith layer do, so at these uncertainties the three are hard to tell apart. An off-center core, or more precise inertia measurements, separate them better (Tutorial ex08).

Next steps

- Density models by component type and Calculations: short examples of every model type and calculation.
- Results folders and figures: saving runs with logs, settings and standard figures.
- Exchange packages: measurement packages for blind tests between teams.

Units

SBTK uses one set of units in every function, structure and file. Every stored quantity that has a unit carries it at the end of its name; a name without a unit suffix is dimensionless.

Quantity	Unit	Name suffix	Example
Lengths, positions, radii, depths	km	`_km`	`center_km = [0.02 0 0]` (20 m along x)
Densities and relative densities	kg/m³	`_kgm3`	`drho_kgm3 = -900`
Mass	kg	`_kg`	`mass_kg = 5.273e11`
Volume	km³	`_km3`	`volume_km3 = 0.20336`
Gravitational parameter GM	km³/s²	`_km3_s2`	`GM_km3_s2 = 3.519e-8`
Angles: latitude, longitude, rotations	degrees	`_deg`	`lat_deg = [40 90]`
Gravity coefficients $\bar{C}_{lm}$, $\bar{S}_{lm}$	dimensionless		fully normalized (Gravity coefficients)
Moments of inertia in measurement sets	$I\,/\,(MR_{\mathrm{ref}}^2)$, dimensionless	`_norm`	$I_{zz} \approx 0.38$ for Didymos with $R_{\mathrm{ref}} = 0.43$ km
Moments of inertia in reports and text files	kg m²	`_kgm2`, `_kg_m2`	`inertia_tensor_kg_m2`
Gravitational constant G	km³ kg⁻¹ s⁻²		6.67430×10^{-20} (CODATA 2018), `sbtk.units.G`

`sbtk.units.table` prints this table in MATLAB.

Where units are given at input

Two inputs carry a unit chosen by the caller:

- **Shape files.** `sbtk.shape.load` and `sbtk.shape.from_arrays` require the length unit of the vertices, `"km"`, `"m"` or `"cm"`; there is no default. `sbtk.units.length_scale` gives the conversion factor to km.
- **Moments of inertia in kg m².** `sbtk.units.moi_to_norm` converts them to $I\,/\,(MR_{\mathrm{ref}}^2)$ before `sbtk.data.add_moi`; `sbtk.units.norm_to_moi` converts back.

All other arguments take the units of the table; their names say which (`sbtk.interior.sphere(name, center_km, radius_km, drho_kgm3)`).

Conversions inside the toolkit

Volumes are in km³ and densities in kg/m³, so the mass of a region of density ρ and volume V is $\rho\, V \times 10^9$ kg. Moments of inertia are normalized with R_{ref} in km, so $I\,/\,(MR_{\mathrm{ref}}^2)$ needs no further factor; converting to kg m² multiplies by $M\,(10^3 R_{\mathrm{ref}})^2$.

A center-of-mass offset d (km) along one axis corresponds to the fully normalized degree-1 coefficient $d\,/\,(\sqrt{3}\;R_{\mathrm{ref}})$ (`sbtk.units.offset_to_degree1`); this is how a center-of-mass uncertainty enters `sbtk.data.uncertainties`.

Coordinate systems and origins

Shape-model frame

Every position in SBTK is given in the **shape-model frame**: the origin and axes of the shape model in use, lengths in km. The z-axis is taken as the spin axis. Measurements, models and estimates of one body are in the same frame. `sbtk.body.to_frame`, `sbtk.data.to_frame` and `sbtk.interior.to_frame` carry a body, its measurements and its interior into another frame, for example the spin or principal axes of a true model, which then becomes the shape-model frame of everything that follows; `sbtk.gravity.center_of_mass_frame` expresses a gravity field in the shape-model, spin or principal axes (Frames of a gravity field).

Three points of the shape-model frame are distinct in general:

Point	Definition	Where SBTK stores it
Shape-model origin	The point (0, 0, 0) of the shape file, chosen by its makers. For the DART Didymos model v003 it is 57 m from the center of figure; for the OSIRIS-REx Bennu model it is the center of mass found from the tracking data, 1.4 m from the center of figure; in the packages of the HLP6 makers it is the center of mass of the true model (The shape model of a package).	the origin of every coordinate
Center of figure	Centroid of the volume enclosed by the shape model: the center of mass of a homogeneous body.	`body.centroid_km`, `props.centroid_km`
Center of mass	Center of mass of the density model. It differs from the center of figure when the density is not uniform.	`truth.com_km`, `obs.com_minus_cof_km`, `center_of_mass.txt` of a measurement package

Expansion origin

Gravity coefficients, first moments and the inertia tensor are computed **about** a point, the expansion origin. It is stored as `origin_km` in measurement sets (`obs.origin_km`), true models (`truth.origin_km`), inversion results (`fit.origin_km`) and geometries (`geo.origin_km`).

- `sbtk.data.simulate` expands about the true center of mass of the simulated interior. Because layers and sectors are measured from the expansion origin, the center of mass depends on the origin; the simulation repeats the calculation about the new center of mass until it moves by less than 1 mm.
- Measurement packages read with `sbtk.exchange.read_data` are expanded about the *measured* center of mass, and the inversion moves its predictions to the model's own center of mass (Density estimation).
- `sbtk.gravity.geometry` takes any expansion origin about which the shape is star-shaped.

Positions of components

Components are positioned in two different ways:

- **Spheres, ellipsoids and tori** are placed in shape-model coordinates. `center_km = [0 0 0]` is the shape-model origin, not the center of figure. A torus lies about its axis, the third of `component.axes`, through `component.center_km`, in the plane at `height_km` along that axis; as constructed this is the z-axis through the origin. To place components from the center of figure, as recommended, use `body.centroid_km`: a sphere at `body.centroid_km + offset`, a torus with `center_km = body.centroid_km` (Where to place components).
- **Layers and sectors** are measured along rays from the expansion origin: a layer's depth is the distance below the surface along the ray through the point; a sector's radial fraction is the distance from the origin divided by the distance from the origin to the surface along the same ray, and its latitude and longitude are measured about the origin.

Latitude and longitude

For a vector $\mathbf{x} = (x, y, z)$ from the point of reference, latitude ϕ is measured from the xy-plane toward $+z$ and longitude λ from $+x$ toward $+y$:

$$\mathbf{x} = d\,(\cos\phi\cos\lambda,\ \cos\phi\sin\lambda,\ \sin\phi)\,.$$

`sbtk.interior.latlon` returns $\mathbf{x}$ from ϕ, λ (degrees) and d (km). Sector longitudes are compared in $[-180°, 180°)$ with ranges that may extend beyond ±180°; for example $[150°, 240°]$ includes $-170°$.

Ellipsoid orientation

An ellipsoid's semi-axes (a, b, c) lie along x, y, z before rotation. The rotation is $R = R_z(\psi)\,R_y(\theta)\,R_x(\varphi)$ with ψ the yaw (about z), θ the pitch (about y) and φ the roll (about x), given in degrees as `[yaw pitch roll]`. The columns of R (`component.axes`) are the ellipsoid's axes in the shape-model frame, and a point $\mathbf{x}$ is inside when $\sum_i \left([R^{\mathrm{T}}(\mathbf{x} - \mathbf{c})]_i \,/\, a_i \right)^2 < 1$.

Axes of the inertia tensor and the coefficients

Coefficients and inertia elements are given about the center of mass but in the axes of the shape-model frame, not in principal axes of inertia, so $\bar{C}_{21}$, $\bar{S}_{21}$ and $\bar{S}_{22}$ are in general not zero. For a homogeneous body, the axis of the largest moment of inertia is 3.6° from the shape-model z-axis for Didymos (DART v003) and 0.7° for Dimorphos (v004), and the principal axes in the xy-plane are 33.6° and 7.8° from x; in the shape-model axes the homogeneous Didymos has $\bar{C}_{21} = 3.7{\times}10^{-3}$, $\bar{S}_{21} = 8.8{\times}10^{-4}$ and $\bar{S}_{22} = -7.0{\times}10^{-3}$. In the spin axes of a field (z along its axis of the largest moment) $\bar{C}_{21} = \bar{S}_{21} = 0$, and in its principal axes also $\bar{S}_{22} = 0$; the interior-model package states the principal axes of the true model (Frames of a gravity field). A field and its shape model must be in the same frame: when they are not, carry the shape model into the frame of the field with `sbtk.body.to_frame`. The libration ratio uses the

principal moments in the xy-plane computed from the tensor (Moments of inertia), and takes z as a principal axis, which holds exactly in the spin and principal axes.

Gravity coefficients

Expansion of the potential

Outside the smallest sphere about the expansion origin that encloses the body (the Brillouin sphere), the gravitational potential is

$$U(r,\phi,\lambda) = \frac{GM}{r}\sum_{l=0}^{L}\left(\frac{R_{\text{ref}}}{r}\right)^{l}\sum_{m=0}^{l}\bar{P}_{lm}(\sin\phi)\left[\bar{C}_{lm}\cos(m\lambda)+\bar{S}_{lm}\sin(m\lambda)\right]$$

with r, latitude ϕ and longitude λ measured from the expansion origin in the axes of the shape-model frame (Coordinate systems). $U > 0$ and the acceleration is $+\nabla U$. $\bar{C}_{00} = 1$. R_{ref} is a chosen reference radius; SBTK computes the coefficients and does not evaluate the series.

For a density $\rho(\mathbf{x})$ of total mass M the fully normalized coefficients are

$$\bar{C}_{lm} = \frac{1}{(2l+1)M}\int \rho\left(\frac{r}{R_{\text{ref}}}\right)^{l}\bar{P}_{lm}(\sin\phi)\cos(m\lambda)\,dV,$$

$$\bar{S}_{lm} = \frac{1}{(2l+1)M}\int \rho\left(\frac{r}{R_{\text{ref}}}\right)^{l}\bar{P}_{lm}(\sin\phi)\sin(m\lambda)\,dV.$$

Normalization

The coefficients are **fully normalized** (the 4π normalization of geodesy) and the associated Legendre functions have **no Condon–Shortley phase** $(-1)^m$:

$$\bar{P}_{lm} = N_{lm}P_{lm}, \qquad N_{lm} = \sqrt{\frac{(2-\delta_{m0})(2l+1)(l-m)!}{(l+m)!}}, \qquad P_{lm}(x) = (1-x^2)^{m/2}\frac{d^m}{dx^m}P_l(x),$$

so that the mean of $[\bar{P}_{lm}(\sin\phi)\cos(m\lambda)]^2$ over the sphere is 1. Unnormalized coefficients are $C_{lm} = N_{lm}\bar{C}_{lm}$. Examples: $N_{20} = \sqrt{5}$, $N_{21} = \sqrt{5/3}$, $N_{22} = \sqrt{5/12}$, $N_{30} = \sqrt{7}$. The zonal coefficients are $J_l = -C_{l0} = -\sqrt{2l+1}\ \bar{C}_{l0}$.

Coefficients in another convention must be converted before use: multiply by $(-1)^m$ to remove a Condon–Shortley phase, and by $1/N_{lm}$ to normalize. `sbtk.exchange.read_shadr` checks the normalization state in the file header and rejects unnormalized files; `sbtk.io.read_jpl_gravity` converts J_l to $\bar{C}_{l0}$.

Reference radius

Changing the reference radius from R to R' multiplies a coefficient of degree l by $(R/R')^l$; its variance by $(R/R')^{2l}$. `sbtk.data.use_uncertainties` applies this when it takes uncertainties from a data set with another reference radius. The examples on this site use $R_{\text{ref}} = 0.43$ km for Didymos and 0.10 km for Dimorphos.

Order of coefficients

Vectors of coefficients are ordered by degree, and within a degree as C_{l0}, then C_{lm}, S_{lm} for $m = 1 \ldots l$:

`C00, C10, C11, S11, C20, C21, S21, C22, S22, C30, C31, S31, …`

`sbtk.gravity.labels` returns the names to a given degree and `pk` describes the order. Measurement sets (`obs`) leave out C_{00} and store the names in `obs.coef_labels`. Where coefficients are given as matrices (`sbtk.data.from_coefficients`), `C(l+1, m+1)` holds $\bar{C}_{lm}$ and `S(l+1, m+1)` holds $\bar{S}_{lm}$.

Degree 1 and the center of mass

About an expansion origin O, with (x_c, y_c, z_c) the center of mass minus O,

$$\bar{C}_{10} = \frac{z_c}{\sqrt{3}\ R_{\text{ref}}}, \qquad \bar{C}_{11} = \frac{x_c}{\sqrt{3}\ R_{\text{ref}}}, \qquad \bar{S}_{11} = \frac{y_c}{\sqrt{3}\ R_{\text{ref}}}.$$

About the center of mass these vanish. A measured center of mass enters a measurement set as these three coefficients with their uncertainties (`sbtk.data.add_center_of_mass`, `sbtk.units.offset_to_degree1`).

Frame of the coefficients: `coef_about`

Each measurement set states what its coefficients are about (`obs.coef_about`):

- `'origin'`: every coefficient is about `obs.origin_km`. Simulated measurements are of this kind, with the origin at the true center of mass.
- `'center_of_mass'`: the coefficients of degree 2 and higher are about the body's center of mass, as a radio-science solution is; the degree-1 coefficients give the measured position of the center of mass relative to `obs.origin_km`. Measurement packages and fields given to `sbtk.data.add_center_of_mass` are of this kind; the inversion treats them as described in Density estimation.

`sbtk.data.move_origin` changes the origin of such data; only the degree-1 values change.

Frames of a gravity field

A set of gravity coefficients means nothing without the point it is expanded about and the axes its latitude and longitude are measured in. Changing the point or the axes changes every coefficient but not the potential outside the body. This page covers:

- the two changes used for small bodies: moving the expansion point to the center of mass, and turning the axes into the *spin axes* or the *principal axes* of inertia;
- what each change makes zero;
- the frames used for past measured and modeled fields;
- how the frame affects a comparison with a homogeneous body;
- which frame a team that makes data sets (*maker*) or estimates interiors from them (*finder*) should use, including for a body that may be tumbling, such as Dimorphos.

Frame	Expansion point	Axes	Coefficients that vanish
Shape-model origin	origin of the shape-model file	shape-model axes	none
Center of mass, shape-model axes	center of mass	shape-model axes	$\bar{C}_{10}$, $\bar{C}_{11}$, $\bar{S}_{11}$
Center of mass, spin axes	center of mass	z along the axis of the largest moment, x toward the shape-model x-axis	degree 1, and $\bar{C}_{21}$, $\bar{S}_{21}$
Center of mass, principal axes	center of mass	principal axes of inertia	degree 1, and $\bar{C}_{21}$, $\bar{S}_{21}$, $\bar{S}_{22}$

In SBTK, coefficients are always expanded about the center of mass, in the axes of the shape model in use. The functions that work with frames are:

- `sbtk.gravity.center_of_mass_frame` gives any field in the shape-model, spin or principal axes, with its principal axes and moments.
- `sbtk.plot.difference_from_homogeneous` compares fields with the homogeneous body in any of the three.
- `sbtk.body.to_frame`, `sbtk.data.to_frame` and `sbtk.interior.to_frame` carry a body, its measurements and its interior or true model into another frame. Every later step (simulation, packages, density grids, plots and comparisons) then works in that frame.

Summary

- **The information is the same in every frame.** The terms a frame makes zero become its definition: three coordinates of the center of mass and up to three angles. Both must be known relative to the shape model.
- **Spin axes** reproduce fields measured by radio science from a body that rotates about its axis of the largest moment (OSIRIS-REx at Bennu): $\bar{C}_{21} = \bar{S}_{21} = 0$, but $\bar{S}_{22} \neq 0$ because the x-axis is set by a surface feature, not by the mass distribution.
- **Principal axes** suit rotational dynamics and bodies without a fixed spin axis (tumbling), and also a synchronous satellite.
- **Makers** should send the measurements, the shape-model file, the true model and the density grid in one frame: spin axes for a body like Didymos, principal axes for Dimorphos (Which frame to use).
- **Finders** should work in the frame of the package they receive and report in it.
- **Against a homogeneous body**, the frame-independent results are the angles between the principal axes of the field and of the homogeneous body (`axes_deg`) and the power of each degree.

Moving the expansion point to the center of mass

Degree 1

About an expansion point O, with $d = (x_c, y_c, z_c)$ the center of mass minus O, the degree-1 coefficients are (Gravity coefficients)

$$\bar{C}_{10} = \frac{z_c}{\sqrt{3}\ R_{\mathrm{ref}}}, \qquad \bar{C}_{11} = \frac{x_c}{\sqrt{3}\ R_{\mathrm{ref}}}, \qquad \bar{S}_{11} = \frac{y_c}{\sqrt{3}\ R_{\mathrm{ref}}},$$

so they vanish about the center of mass and only there. A radio-science field is expanded about the center of mass because the spacecraft orbit is integrated about it. The position of the center of mass relative to the shape model is then a separate result, not part of the coefficients.

Higher degrees

Moving the expansion point changes the coefficients of every degree. The coefficients of degree l about the new point are a linear combination of those of degree $\leq l$ about the old point, weighted by powers of the displacement: lower degrees enter higher ones, never the reverse. For degree 2 this is the parallel-axis theorem. With $I_O = I_{\mathrm{cm}} + M\left(|d|^2\, 1 - d\, d^{\mathrm{T}}\right)$ and the relations between the inertia tensor and the degree-2 coefficients (Moments of inertia), the unnormalized coefficients about O are

$$C_{20}^{O} = C_{20} + \frac{2z_c^2 - x_c^2 - y_c^2}{2R^2}, \quad C_{21}^{O} = C_{21} + \frac{x_c z_c}{R^2}, \quad S_{21}^{O} = S_{21} + \frac{y_c z_c}{R^2},$$

$$C_{22}^{O} = C_{22} + \frac{x_c^2 - y_c^2}{4R^2}, \quad S_{22}^{O} = S_{22} + \frac{x_c y_c}{2R^2},$$

where C_{2m}, S_{2m} are about the center of mass. The change is of second order in $|d\,| \,/\, R$. For the Didymos model below, the shape-model origin is 57 m from the center of mass ($|d\,| \,/\, R_{\mathrm{ref}} = 0.13$). As a result, $\bar{C}_{21}$ is $3.67{\times}10^{-3}$ about the center of mass but $1.49{\times}10^{-3}$ about the origin.

SBTK applies the translation to all degrees at once with a matrix built from the solid harmonics of point masses (`sbtk.internal.sh_translation`). The map is linear and finite, so the result is exact to rounding. The covariance of a measurement set is propagated through the same matrix.

Rotating the axes

Principal axes from the degree-2 coefficients

About the center of mass, the degree-2 coefficients fix the inertia tensor up to a multiple of the identity (MacCullagh's formula). With unnormalized coefficients,

$$\frac{I - I_{zz}1}{MR_{\mathrm{ref}}^2} = \begin{pmatrix} C_{20} - 2C_{22} & -2S_{22} & -C_{21} \\ -2S_{22} & C_{20} + 2C_{22} & -S_{21} \\ -C_{21} & -S_{21} & 0 \end{pmatrix}.$$

Its eigenvectors are the principal axes of inertia. Its eigenvalues are the principal moments $A \le B \le C$ minus I_{zz}, divided by MR_{ref}^2. SBTK orders the principal axes so that:

- x' is the axis of the smallest moment, y' of the middle one and z' of the largest;
- each axis points to the positive side of the shape-model axis nearest to it;
- the frame is right-handed (`sbtk.internal.principal_axes`).

An eigenvector has no sign, so a field in principal axes must state these choices. Reversing x', for example, changes the sign of every coefficient that is odd in x, such as $\bar{C}_{31}$. When two moments are equal (a body symmetric about an axis), the axes in their plane are not defined.

What a rotation changes

A rotation mixes the $2l + 1$ coefficients of each degree among themselves and with no others. For fully normalized coefficients, the mixing matrix of each degree is orthogonal, so the power of each degree,

$$\sigma_l^2 = \sum_{m=0}^{l} \left(\bar{C}_{lm}^2 + \bar{S}_{lm}^2 \right),$$

is unchanged. SBTK builds the rotation matrix of each degree from the solid harmonics of points and their rotated images (`sbtk.internal.sh_rotation`). It is exact to rounding and also rotates a covariance.

Rotating the coefficients gives the same result as computing the field of the rotated body. The homogeneous field of the shape model with its vertices rotated into the principal axes equals the rotated homogeneous coefficients to 10^{-12} (`testDifferenceInPrincipalAxes`). Simulating a model whose body, components and random field are all carried into another frame also gives the rotated field, density and inertia (`testChangeOfFrameEqualsSimulationInIt`, Validation).

Principal axes

In the principal axes the products of inertia vanish, so

$$\bar{C}_{21} = \bar{S}_{21} = \bar{S}_{22} = 0, \qquad C_{20} = \frac{A + B - 2C}{2MR^2}, \qquad C_{22} = \frac{B - A}{4MR^2} \ \text{(unnormalized)},$$

and the whole degree-2 power is in $\bar{C}_{20}$ and $\bar{C}_{22}$. Degree 1 stays zero. No rotation makes the coefficients of degree 3 and higher zero: their values change but their power does not.

Spin axes

The spin axes remove only the tilt of the axis of the largest moment:

- z' is the axis of the largest moment (the third principal axis);
- x' is the shape-model x-axis projected onto the plane normal to z';
- $y' = z' \times x'$.

Of all frames with z' along that axis, this one turns the shape-model axes the least.

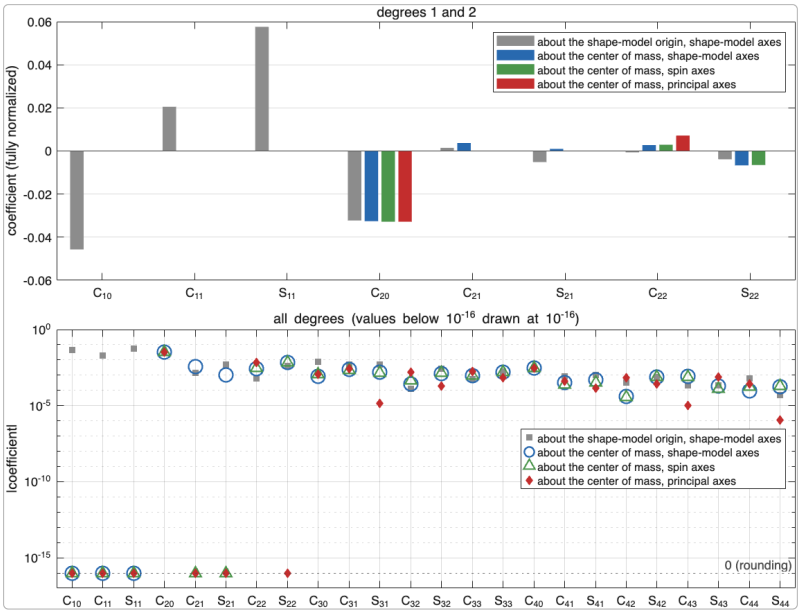
The products of inertia that involve z vanish, so $\bar{C}_{21} = \bar{S}_{21} = 0$. $\bar{S}_{22}$ is in general not zero, and the axis of the smallest moment lies at longitude

$$\lambda_{22} = \frac{1}{2} \operatorname{atan2}\left(\bar{S}_{22}, \bar{C}_{22} \right).$$

These are the conditions of a body-fixed frame of a body in principal-axis rotation, whose spin axis is the axis of its largest moment. The prime meridian here is the direction of the shape-model x-axis.

One field in four frames

Below is the true model of the documentation example (core, torus, regolith, boulder and a random density field in the 9.3 m Didymos model, Results folders and figures) in the four frames, made by `documentation/tools/make_samples.m`:



The degree-1 and degree-2 coefficients (top) and the magnitude of every coefficient (bottom) of one field: about the shape-model origin (gray), and about its center of mass in the shape-model axes (blue), the spin axes (green) and the principal axes (red). Values at the dotted line are zero to rounding.

Output

```
Example true model of Didymos (shape model didymos_g_9309mm_spc_obj_0000n00000_v003), R_ref = 0.43 km

Coefficients of the true model
coef  shape-model origin    center of mass    center of mass    center of mass
       shape-model axes  shape-model axes         spin axes    principal axes
C10       -4.572228e-02      0.000000e+00     -1.289007e-18     -3.710172e-20
C11        2.051808e-02      0.000000e+00      4.758569e-17      1.083458e-17
S11        5.758028e-02      0.000000e+00     -7.278009e-19      1.071706e-18
C20       -3.226924e-02     -3.256747e-02     -3.279488e-02     -3.279488e-02
C21        1.485208e-03      3.665234e-03      4.219222e-18     -1.689108e-17
S21       -5.134086e-03      9.837596e-04      1.553676e-17     -2.088580e-17
C22       -6.163254e-04      2.746776e-03      2.863367e-03      7.152014e-03
S22       -3.893035e-03     -6.638447e-03     -6.553811e-03     -5.354679e-17
```

- **About the shape-model origin**, degree 1 is the largest part of the field: the origin is 57 m from the center of mass.
- **About the center of mass**, degree 1 is zero. The parallel-axis terms above change the degree-2 coefficients; $\bar{S}_{21}$ even changes sign. Degrees 3 and 4 change too.
- **In the spin axes**, $\bar{C}_{21}$ and $\bar{S}_{21}$ are zero to rounding. $\bar{C}_{22}$ and $\bar{S}_{22}$ barely change, because the tilt is only 4.0°.
- **In the principal axes**, $\bar{S}_{22}$ is zero as well. $\bar{C}_{22}$ grows from 2.9×10^{-3} to 7.2×10^{-3}: the axes turn by 33° about z, and the degree-2 power is the same in all three axes.

MATLAB script: A field in four frames

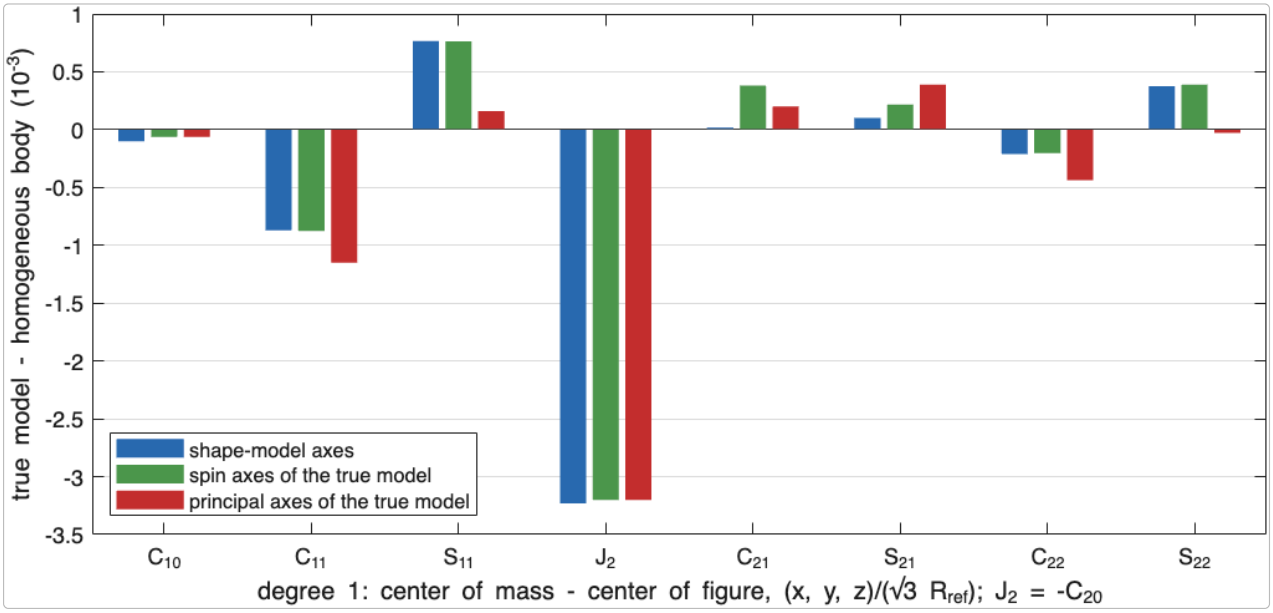
```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
cof = body.centroid_km;                                          % positions from the center of figure
interior = sbtk.interior.create([sbtk.interior.sphere("core", cof, 0.170, +450), ...
                                 sbtk.interior.sphere("boulder", cof + sbtk.interior.latlon(25, 60, 0.250), 0.040, +900)]);
[exact, truth] = sbtk.data.simulate(body, interior, 4, "frames");

pk = sbtk.internal.sh_packing(4);
about_origin = truth.coef*sbtk.internal.sh_translation(-truth.origin_km, pk, body.R_ref_km)';
cs = sbtk.gravity.center_of_mass_frame(truth, body, "shape");        % degree 1 = 0
cz = sbtk.gravity.center_of_mass_frame(truth, body, "spin");         % and C21 = S21 = 0
cp = sbtk.gravity.center_of_mass_frame(truth, body, "principal");    % and C21 = S21 = S22 = 0
disp(table(cs.labels(1:8)', about_origin(2:9)', cs.coef(1:8)', cz.coef(1:8)', cp.coef(1:8)', 'VariableNames', ...
           ["coefficient" "about the origin" "shape axes" "spin axes" "principal axes"]))
power = @(c, l) sum(c(pk.deg(2:end) == l).^2);
fprintf("degree-2 power: %.6e (shape), %.6e (spin), %.6e (principal)\n", power(cs.coef, 2), power(cz.coef, 2), ...
        power(cp.coef, 2))
fprintf("axis of the largest moment %.2f deg from the shape-model z-axis; smallest at longitude %.1f deg\n", ...
        acosd(cp.principal(3, 3)), 0.5*atan2d(cz.coef(cz.labels == "S22"), cz.coef(cz.labels == "C22")))
fprintf("principal moments minus their mean, I/(M R^2): %.5f %.5f %.5f\n", cp.moments)
```

Comparison with the homogeneous body

`sbtk.plot.difference_from_homogeneous` subtracts the field of the homogeneous body (density M / V) from a field. Both are taken about their own center of mass and expressed in the same axes, which belong to the field (`frame` argument). Degree 1 shows the center of mass minus the center of figure in those

axes.



The example true model minus the homogeneous body, degrees 1 and 2, in the shape-model axes, in the spin axes of the true model and in its principal axes.

Output

```
True model minus the homogeneous body (both about their own center of mass)
coef   shape-model axes         spin axes    principal axes
J2        -3.231612e-03     -3.202163e-03     -3.202163e-03
C21        1.721657e-05      3.803159e-04      1.995238e-04
S21        1.005986e-04      2.168021e-04      3.896582e-04
C22       -2.103466e-04     -2.023064e-04     -4.383788e-04
S22        3.748967e-04      3.900043e-04     -2.924388e-05

axis of the largest moment from the shape-model z-axis:   true model 3.99 deg, homogeneous body 3.56 deg
axis of the smallest moment from the x-axis, in longitude: true model -33.3 deg, homogeneous body -33.2 deg
principal axes of the homogeneous body from those of the true model: tilt 0.44 deg, turn 0.11 deg
in the spin axes, axis of the smallest moment at (1/2) atan2(S22, C22) = -33.2 deg
principal moments minus their mean, I/(M R_ref^2): true model -0.03368 -0.01521 0.04889
                                                   homogeneous body -0.03663 -0.01703 0.05367
center of mass - center of figure: -0.6 0.6 -0.1 m
```

What each frame shows:

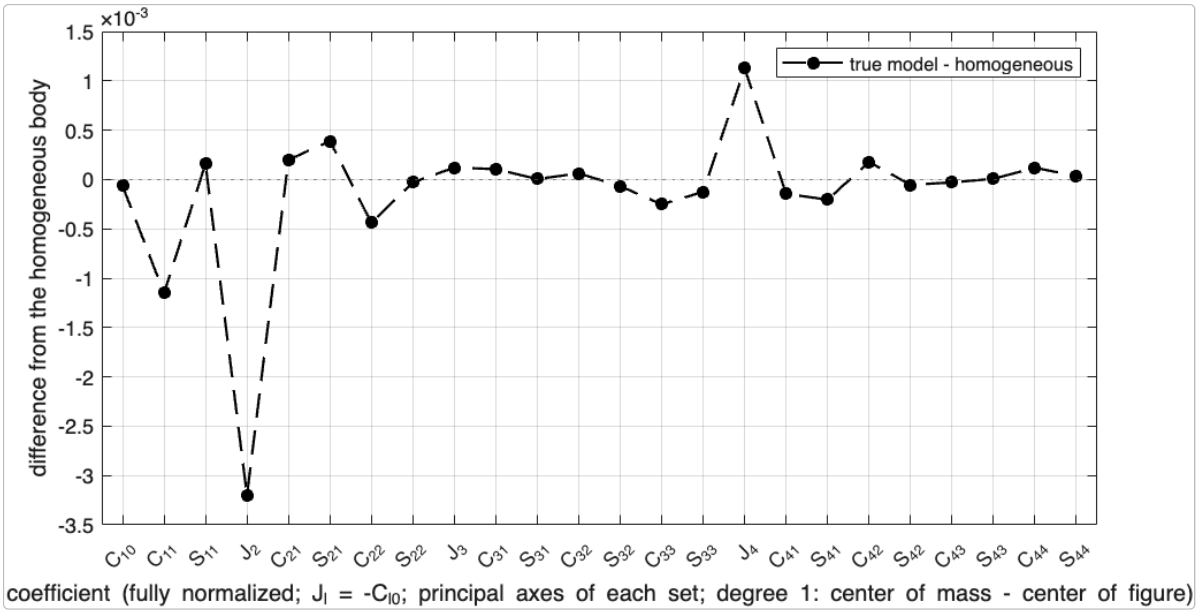
- **Shape-model axes.** Both fields keep their own $\bar{C}_{21}$, $\bar{S}_{21}$, $\bar{S}_{22}$. The differences mix two things: how far the interior turns the principal axes, and how much it changes the moments.
- **Spin axes.** The field has $\bar{C}_{21} = \bar{S}_{21} = 0$. The differences in these two terms are therefore minus the homogeneous body's own terms, and measure the tilt between the two axes of the largest moment (0.44° here). The differences in $\bar{C}_{22}$ and $\bar{S}_{22}$ contain the turn of the equatorial axes and the change in $B - A$.
- **Principal axes.** The field has $\bar{C}_{21} = \bar{S}_{21} = \bar{S}_{22} = 0$, so all of the misalignment is in the homogeneous body's terms: $\bar{C}_{21}$, $\bar{S}_{21}$ for the tilt and $\bar{S}_{22}$ for the turn (0.11° here). The difference in $\bar{C}_{22}$ is then the change in $B - A$.

For small angles, a rotation ε about one principal axis gives the homogeneous body a product of inertia proportional to ε times the difference of the other two moments:

$$|\bar{C}_{21}| \approx \frac{C-A}{\sqrt{5/3}\ MR^2} |\varepsilon_y|, \qquad |\bar{S}_{21}| \approx \frac{C-B}{\sqrt{5/3}\ MR^2} |\varepsilon_x|, \qquad |\bar{S}_{22}| \approx \frac{B-A}{2\sqrt{5/12}\ MR^2} |\varepsilon_z| .$$

A mass excess m at (x, y, z) adds the products of inertia $-mxz$, $-myz$ and $-mxy$. An excess at mid-latitude, away from the z-axis, tilts the axis of the largest moment. An excess in a direction between the x'- and y'-axes turns the axes in the equatorial plane.

In the example, $(C-A)/(MR^2) = 0.083$ and the tilt is 0.44°, which gives about $5{\times}10^{-4}$, shared between $\bar{C}_{21}$ and $\bar{S}_{21}$ according to the direction of the tilt ($2.0{\times}10^{-4}$ and $3.9{\times}10^{-4}$ in the principal axes): several times the 1σ of 10^{-4} assumed for degree 2. The plot returns the tilt and the turn as `axes_deg`. They, and the power of each degree, do not depend on the frame. Which frame to plot in depends on what the plot is for: the spin axes show the tilt directly in $\bar{C}_{21}$, $\bar{S}_{21}$, as a mission field would; the principal axes isolate all orientation differences in the homogeneous body's terms.



The example true model minus the homogeneous body, degrees 1 to 4, in the principal axes of the true model.

Frames used for measured and modeled fields

Body-fixed frames of measured fields

Gravity fields from radio science, and the shape models made with them, are given in a body-fixed (cartographic) frame:

- z is the spin pole;
- x points to the prime meridian, which is defined by a surface feature;
- the origin is the center of mass found from the tracking data.

NEAR at Eros, Dawn at Vesta and Ceres, and OSIRIS-REx at Bennu all used frames of this kind. For a body in principal-axis rotation, the spin pole is the axis of the largest moment, so $\bar{C}_{21} = \bar{S}_{21} = 0$. The prime meridian, however, has nothing to do with the mass distribution, so $\bar{S}_{22}$ is not zero.

Example: Bennu

The OSIRIS-REx field of Bennu (JPL, 2019) and the Bennu shape model of the Bennu project, with the homogeneous Bennu in the same frame (`documentation/tools/make_samples.m`):

Output

```
Bennu: OSIRIS-REx gravity field (JPL 2019), shape model bennu_higher_qual, R_ref = 0.290 km
Body-fixed frame of the shape model and the field: z the spin pole, x the prime meridian, origin the center of mass
center of figure (centroid of the shape model): 1.34 -0.44 0.24 m from the origin (1.43 m)

coef           measured      1-sigma       homogeneous
C20       -1.926101e-02     5.20e-05     -1.875290e-02
C21       -2.178217e-14     7.75e-10      4.653539e-04
S21        3.000970e-15     7.75e-10     -9.420563e-05
C22        3.064995e-03     5.27e-05      3.538331e-03
S22       -1.094504e-03     1.07e-04     -8.917091e-04

axis of the largest moment from z:      measured 0.000 deg, homogeneous 0.754 deg
axis of the smallest moment, longitude: measured -9.83 deg, homogeneous -7.08 deg
between the axes of the largest moment of the field and of the homogeneous body: 0.754 deg
```

- **The origin is the center of mass.** The field has no degree-1 terms. The centroid of the shape model (the center of figure) is 1.4 m from the origin, which itself is a sign of a non-uniform interior.
- $\bar{C}_{21}$ **and** $\bar{S}_{21}$ **are zero.** Their values are ~10^{-14} with a 1σ of 8×10^{-10}, so they were constrained to zero by taking the spin pole as the axis of the largest moment.
- **The frame is not the principal frame of the shape.** The homogeneous Bennu has $\bar{C}_{21} = 4.7\times10^{-4}$: its axis of the largest moment is 0.75° from the spin pole.
- $\bar{S}_{22}$ **is not zero.** The measured field's axis of the smallest moment is at longitude −9.8°; the homogeneous body's is at −7.1°.

An interior model of Bennu must therefore tilt the axis of the largest moment by 0.75° and turn the equatorial axes by 2.7°. In Cavale et al. (Icarus) this is what the boulders that target $\bar{C}_{21}/\bar{S}_{21}$ and $\bar{C}_{22}/\bar{S}_{22}$ do. Rotating the field 9.8° about z would make $\bar{S}_{22}$ zero. It would also separate the field from the frame of the shape model, the maps and the navigation, and gain nothing, since the angle is already given by $\bar{S}_{22}$ / $\bar{C}_{22}$.

Principal-axis frames

Principal axes are used for:

- rotational dynamics: the equations of rotation (Euler's equations) are diagonal in them;
- model fields made for dynamics studies before a body is visited;
- tumbling asteroids, whose rotation state is solved in the principal axes of the body (for example 4179 Toutatis and 99942 Apophis).

Which frame to use

Makers

1. **Use one frame for everything you send.** The measurements, the shape-model file, the interior-model package and the density grid must all be in the same frame, so the receiving teams never rotate anything and their density grids coincide with yours. Define the interior in the frame of the shape-model file. Choose the frame from the true model: its center of mass as the origin and the axes below. Then carry the body, the exact values and the true model into it (code below).
2. **Spin axes** for a body observed to rotate about one axis (Didymos). They reproduce what a mission field would look like: $\bar{C}_{21} = \bar{S}_{21} = 0$ and $\bar{S}_{22} \neq 0$. They also make the true model consistent with its rotation. In the shape-model axes, a true model whose axis of the largest moment is not z would wobble if it spun about z. The homogeneous Didymos v003 is already 3.6° off, and the example true model 4.0°.
3. **Principal axes** for a body that may be tumbling, or for a synchronous satellite (Dimorphos; Tumbling bodies).
4. **Shape-model axes** keep the original shape-model file and need no rotation. $\bar{C}_{21}$, $\bar{S}_{21}$ are then those of the interior.

Points to check:

- **The frame of the true model is exact.** A real spin pole has an error ε, which leaves $|\bar{C}_{21}|$, $|\bar{S}_{21}| \approx (C-A)\,\varepsilon\,/\,(\sqrt{5/3}\;MR^2)$, about 1.1×10^{-4} for Didymos at $\varepsilon = 0.1°$. A package of exact values has $\bar{C}_{21} = \bar{S}_{21} = 0$ exactly; the 1σ a receiving team gives them stands for the pole error. If that 1σ is much smaller, the data define the pole more precisely than a mission would.
- **The rotation carries information about the interior.** It is the same information the field gives in the shape-model axes ($\bar{C}_{21}$, $\bar{S}_{21}$), and the same a mission's pole and field give together, no more. `conventions.txt` records the change of frame from the original shape-model file.
- **The origin is the center of mass.** Mission body-fixed frames put the origin at the estimated center of mass (Bennu: the centroid is 1.4 m from the origin), and so do the packages. The shape model, the gravity field and the moments of inertia then share one origin, and a receiving team has nothing to move. The origin of the original file has no physical meaning (Didymos v003: 57 m from the center of figure). Moving the origin adds no information: a package of exact values gives the exact center of mass in `center_of_mass.txt` anyway, together with the center of figure, and their offset, which the interior sets, is the same in every frame. For a package with noise, a mission's origin is the *measured* center of mass; translate to it after `sbtk.data.add_noise` (a translation applies to any set).
- **Uncertainties belong to a frame.** A table of one σ per coefficient describes the solution it comes from, in that solution's axes. In other axes the same covariance has correlations, and two coefficients of one degree can have very different σ (in a principal-axis solution, for example, $\sigma(\bar{C}_{22}) \ll \sigma(\bar{S}_{22})$). Applying such a table coefficient by coefficient in other axes is an approximation. `sbtk.data.to_frame` rotates a full covariance exactly.
- **Components tied to the rotation.** A component defined in the shape-model frame, such as a torus about its z-axis, is tilted relative to the spin axes by the angle between them. If it should lie about the spin axis, as an equatorial torus does, set its axis with `sbtk.interior.align_with_spin_axis` before simulating (Axes of components). `interior_model.txt` gives the geometry in the frame of the package.

Finders

1. **Work in the frame of the package.** Its shape model and its measurements are in the same frame (in an SBTK package, with the center of mass at the origin). Define the components of the inversion model in that frame.
2. **If a field and its shape model are in different frames** (for example a field in principal axes and a shape model in its own frame, with the rotation given), rotate the shape model with `sbtk.body.to_frame`, not the measurements.
3. **Report the solution and its density grid in the frame of the package**, so that the cells coincide with those of the interior-model package.
4. **To interpret the orientation of the field**, compare it with the homogeneous body in the spin or principal axes, and quote the angles `axes_deg`.

Packages in another frame

MATLAB script: Measurement and interior-model packages in the spin axes

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
cof = body.centroid_km;                                                  % place from the center of figure
torus = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
torus.center_km = cof;
interior = sbtk.interior.create([sbtk.interior.sphere("core", cof, 0.170, +450), torus]);
interior = sbtk.interior.align_with_spin_axis(body, interior, "torus", 0.01, 20);   % torus about the spin axis
[exact, truth] = sbtk.data.simulate(body, interior, 4, "spin example");      % in the frame of the shape-model file

spin  = sbtk.gravity.center_of_mass_frame(truth, body, "spin");             % or "principal"
body  = sbtk.body.to_frame(body, spin.com_km, spin.axes, "didymos_v003_com_spin_axes");   % origin: center of mass
truth = sbtk.interior.to_frame(truth, spin.com_km, spin.axes);
exact = sbtk.data.to_frame(exact, spin.com_km, spin.axes);                   % C21 = S21 = 0

folder = tempname;                                                       % exact values: no uncertainties, no noise
data  = sbtk.exchange.write_data(folder, "DOC-DATA9-Didymos", body, exact, "Example team");
model = sbtk.exchange.write_model(folder, "DOC-MOD9-Didymos", "DOC-DATA9-Didymos", body, truth, exact, 0.040, ...
                                  "Example team");
[received, rbody] = sbtk.exchange.read_data(data);                           % what a receiving team reads
fprintf("%s: C21 %+.1e, S21 %+.1e\n", rbody.shape.name, received.coef(5:6))
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
received = sbtk.data.set_uncertainties(received, unc);                       % the receiving team's uncertainties
```

The package contains `DOC-DATA9-Didymos_didymos_v003_com_spin_axes.obj`, named after the package because no other package has this shape model: the DART shape model with every vertex moved to the center of mass of the true model and rotated into its spin axes, $x' = (x - c)A$ with c the center of mass and the columns of A the spin axes, both in the frame of the original file. The center of mass is at the origin of that file, and the gravity field, the moments of inertia and every position of both packages are in its frame, so a receiving team uses it as it is. Its `conventions.txt` states what the origin and the axes are (read from the values: the center of mass at the origin, $\bar{C}_{21} = \bar{S}_{21} = 0$) and the change of frame from the original file. `interior_model.txt` describes the torus with its axis in the new frame: along z, because it was aligned with the spin axis. The package has exact values without uncertainties (Packages of exact values). The HLP6 runners have a FRAME setting that does this (Hera blind test).

Tumbling bodies and Dimorphos

The gravity field does not depend on the rotation. The coefficients and the principal axes describe the mass distribution in a body-fixed frame and do not change with the rotation state.

What changes for a tumbling body is the meaning of "spin axis". In principal-axis rotation the body spins about its axis of the largest moment. A tumbling body (non-principal-axis rotation) has angular momentum fixed in space, but its angular velocity moves through the body, so no body-fixed axis is the spin axis. The rotation is described in the principal axes, in which Euler's equations are diagonal.

Dimorphos. Before the DART impact, Dimorphos was taken to rotate synchronously with its orbit about Didymos. In that state the body-fixed frame (z the orbit normal, x toward Didymos on average) is close to its principal axes. The impact changed its orbit, and simulations (Agrusa et al., 2021, Icarus) showed that it could have excited non-principal-axis rotation. Its present rotation state is not known; Hera will measure it. The DART shape model v004 is from before the impact, and the impact may have changed the shape.

For making and finding data sets this means:

- **Frame.** Use the principal axes. They suit a tumbling body and, within a few degrees, a synchronous one. The spin axes remain a valid body-fixed frame for a tumbler, but their z-axis is no longer a spin axis.
- **What the rotation measures.**
- A librating synchronous body gives the libration ratio $(B - A) / C$ (Moments of inertia).
- A tumbling body gives two ratios of principal moments, for example A / C and B / C, because Euler's equations depend only on ratios. Together with the degree-2 field, which fixes $B - A$ and $C - (A + B) / 2$ in units of MR^2, two ratios fix all three moments, including the trace that the gravity field cannot give.

A tumbling Dimorphos would therefore constrain the radial density structure better than a librating one. - **Moment-ratio measurements in SBTK.** In principal axes the ratios are the linear rows $I_{xx} - r_A I_{zz} = 0$ and $I_{yy} - r_B I_{zz} = 0$, with σ = $\sigma_r I_{zz}$, added with `sbtk.data.add_moi_row`. In other axes the weights follow from the principal axes of the degree-2 field. The exchange format gives inertia elements and the libration ratio but not moment ratios. A group can give I_{xx}, I_{yy}, I_{zz} in principal axes, which are A, B, C, or agree on an extension of the format. - **The interior model is made as before.** The true interior is defined in the shape-model frame and carried into the principal axes; the rotation state affects only the rotation measurements. Finders invert as usual in the frame of the package.

MATLAB script: Ratios of principal moments of a tumbling Dimorphos

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Dimorphos", sbtk.shape.bundled("dimorphos"), 2.700e9, 0.10);
interior = sbtk.interior.create(sbtk.interior.ellipsoid("core", body.centroid_km + [0.004 0 0], [0.045 0.040 0.028], ...
                                                        [15 0 0], +400));
[exact, truth] = sbtk.data.simulate(body, interior, 3, "tumbling example");
cp = sbtk.gravity.center_of_mass_frame(truth, body, "principal");
exact = sbtk.data.to_frame(exact, [0 0 0], cp.axes);           % Ixx = A, Iyy = B, Izz = C
I = exact.moi_tensor_norm;
rA = I(1, 1)/I(3, 3);  rB = I(2, 2)/I(3, 3);  sigma_r = 0.005;
obs = sbtk.data.select_moi(exact, strings(1, 0));
obs = sbtk.data.add_moi_row(obs, "A_over_C", [1 0 -rA 0 0 0], 0, sigma_r*I(3, 3));
obs = sbtk.data.add_moi_row(obs, "B_over_C", [0 1 -rB 0 0 0], 0, sigma_r*I(3, 3));
fprintf("A/C = %.4f, B/C = %.4f; off-diagonal elements in principal axes: %.1e\n", rA, rB, max(abs(I(~eye(3)))))
```

Moments of inertia

Inertia tensor

The inertia tensor about the expansion origin, in the axes of the shape-model frame, is

$$I_{ij} = \int \rho \left(r^2 \delta_{ij} - x_i x_j \right) dV,$$

so the products of inertia carry the minus sign: $I_{xy} = -\int \rho \, xy \, dV$. SBTK stores the six independent elements in the order $[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}]$, normalized by MR_{ref}^2 in measurement sets and models (`truth.inertia_norm`, `fm.inertia_norm`) and in kg m² in reports (Units).

Relation to the degree-2 coefficients

About the center of mass, in the same axes and with the same R_{ref}, the unnormalized degree-2 coefficients are

$$C_{20} = \frac{I_{xx} + I_{yy} - 2I_{zz}}{2MR^2}, \quad C_{21} = -\frac{I_{xz}}{MR^2}, \quad S_{21} = -\frac{I_{yz}}{MR^2}, \quad C_{22} = \frac{I_{yy} - I_{xx}}{4MR^2}, \quad S_{22} = -\frac{I_{xy}}{2MR^2},$$

and $\bar{C}_{lm} = C_{lm} \,/\, N_{lm}$ (Gravity coefficients). These relations are verified by `testDegreeTwoAndInertia` (Validation).

The degree-2 field fixes five combinations of the six elements. The sixth, the trace (for example through the polar moment I_{zz}), is the information that a moment-of-inertia measurement adds to the gravity field: it measures how strongly the mass is concentrated toward the center, which the gravity coefficients alone cannot distinguish from other radial distributions of the same field. Tutorial ex08 compares inversions with different inertia measurements.

Measurement rows

A moment-of-inertia measurement is one linear combination of the elements:

$$\mathbf{w} \cdot \frac{[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}]}{MR_{\mathrm{ref}}^2} = v \pm \sigma,$$

stored as a `moi` row with `weights` $\mathbf{w}$, `value` v and `sigma` σ. A single element has one weight equal to 1 (`sbtk.data.add_moi`); any other combination is added with `sbtk.data.add_moi_row`. `sbtk.data.select_moi` chooses the rows a data set contains and `opts.moi_labels` (`opts.moi_labels`) the rows an inversion fits.

Libration ratio

The libration ratio is

$$k = \frac{B - A}{C},$$

with $A \le B$ the principal moments of inertia about the axes in the xy-plane and $C = I_{zz}$; the spin axis z is taken as a principal axis, which holds exactly in the spin or principal axes of the field (Frames of a gravity field). For a tumbling body the rotation gives ratios of principal moments instead (Tumbling bodies). With a the angle of those principal axes from the x-axis,

$$B - A = \cos(2a)\,(I_{yy} - I_{xx}) + \sin(2a) \cdot 2I_{xy},$$

which is linear in the tensor once a is fixed. `sbtk.data.add_libration` takes a from the measured $\bar{C}_{22}$ and $\bar{S}_{22}$, since $(\cos(2a), \sin(2a))$ is proportional to $(C_{22}, -S_{22})$ about the center of mass, and adds the linear row

$$\cos(2a)\,(I_{yy} - I_{xx}) + \sin(2a) \cdot 2I_{xy} - k\,I_{zz} = 0 \pm \sigma_k\,I_{zz},$$

labeled `libration`, with weights $[-\cos(2a), \cos(2a), -k, 2\sin(2a), 0, 0]$ (`sbtk.internal.libration_weights`). Together with C_{22} and S_{22}, which give $B - A$, the ratio measures C with a relative precision of about $\sigma_k \,/\, k$. If $C_{22} = S_{22} = 0$ the principal axes in the xy-plane are undefined and the function stops with `sbtk:data:axisymmetric`.

Libration ratio from a libration amplitude

For a synchronously rotating satellite in a Keplerian orbit of eccentricity e about a point-mass primary, the forced libration angle γ of the long axis from the direction to the primary obeys, to first order in e,

$$\ddot{\gamma} + 3n^2 k\,\gamma = 2en^2 \sin M,$$

with mean motion n and mean anomaly M, so $\gamma = G \sin M$ with

$$G = \frac{2e}{3k - 1}, \qquad k = \frac{1}{3}\left(1 + \frac{2e}{G}\right).$$

`sbtk.data.libration_ratio_from_amplitude` returns k and its 1σ from G, e and their uncertainties by first-order error propagation. $G < 0$ for $k < 1 \,/\, 3$ (elongations up to about 1.4 : 1). The sensitivity of G to k grows near $k = 1 \,/\, 3$, where the free libration period equals the orbital period. Free libration, the

primary's oblateness and the mutual dynamics of a binary are not included.

Shape models

A shape model is a closed triangular mesh, stored in SBTK as a `shape` structure with vertices in km.

Reading and creating shape models

Function	Input
`sbtk.shape.load`	Wavefront OBJ file: `v x y z` and `f i j k` records; normals, texture coordinates and faces written as `f i/t/n` are accepted, other records ignored. The length unit of the file (`"km"`, `"m"` or `"cm"`) is a required argument.
`sbtk.shape.bundled`	A model shipped with SBTK, by name (table below).
`sbtk.shape.from_arrays`	Face and vertex arrays with their length unit.
`sbtk.shape.ellipsoid`	A triaxial ellipsoid: an icosahedron subdivided `level` times (20·4^level facets) with vertices projected onto the ellipsoid.
`sbtk.shape.write_obj`	Writes a shape model as OBJ with vertices in km.

Requirements

The forward model integrates over the cones from the expansion origin to every facet (Gravity and moment calculations). The mesh must therefore be

- **closed**: every edge is shared by exactly two facets that traverse it in opposite directions;
- **ordered outward**: the enclosed volume computed from the facets is positive;
- **star-shaped about the expansion origin**: every facet faces away from it, so that each ray from the origin crosses the surface once.

`sbtk.shape.properties` reports all three, the last about the centroid (`props`). `sbtk.gravity.geometry` stops with `sbtk:gravity:notStarShaped` when a facet faces the expansion origin.

MATLAB script: Checking a shape model

```
run("/path/to/SBTK/sbtk_setup.m")
shape = sbtk.shape.bundled("didymos");
p = sbtk.shape.properties(shape)
fprintf("volume %.5f km^3, centroid [%.1f %.1f %.1f] m\n", p.volume_km3, 1e3*p.centroid_km)
```

Shape models in the repository

Files in `data/shapes/`, lengths in km, in the body-fixed frame of each model (z along the spin axis).

`bundled` name	File	Sampling	Facets	Volume (km³)	Description
`"didymos"`	`didymos_g_9309mm_spc_obj_0000n00000_v003.obj`	9.3 m	49,152	0.20336	DART stereophotoclinometry (SPC) model of Didymos, version 3
`"didymos_highres"`	`g_01165mm_spc_obj_didy_0000n00000_v003.obj`	1.165 m	3,145,728	0.20381	The same model at full resolution (not in the repository, Installation)
`"didymos_every8"`	`g_01165mm_spc_obj_didy_0000n00000_v003_every8.obj`	9.3 m	49,152	0.20375	Every 8th grid point of the 1.165 m model
`"dimorphos"`	`g_00243mm_spc_obj_dimo_0000n00000_v004_every8.obj`	1.94 m	49,152	0.0017597	Every 8th grid point of the DART SPC model of Dimorphos, version 4 (before the DART impact)
`"dimorphos_highres"`	`g_00243mm_spc_obj_dimo_0000n00000_v004.obj`	0.243 m	3,145,728	0.0017602	The Dimorphos model at full resolution (not in the repository)
`"bennu"`	`bennu_higher_qual.obj`		196,608		OSIRIS-REx shape model of Bennu
`"example_asteroid"`	`example_asteroid.obj`				A synthetic body of about 1.2 km, used by the project template
`"dimorphos_placeholder"`	`dimorphos_placeholder.obj`		20,480		A synthetic stand-in used by tests; not a mission model

The 9.3 m Didymos file is distributed separately by the DART team and is not a subsample of the 1.165 m model. For a homogeneous body with $R_{\mathrm{ref}} = 0.43$ km, `tools/make_reduced_shapes.m` gives these differences from the full-resolution models:

Model compared with the full resolution	Volume	Center of figure	Largest coefficient difference, degree 2 / 3 / 4
`didymos_every8`	−0.030%	0.01 m	1.2e-5 / 5.0e-6 / 2.8e-6
`didymos` (9.3 m file)	−0.224%	0.35 m	3.1e-4 / 1.7e-4 / 4.2e-5
`dimorphos` (every 8th point, $R_{\mathrm{ref}} = 0.10$ km)	−0.031%	0.00 m	1.5e-5 / 1.3e-6 / 5.1e-6

Differences of this size between the shape used to simulate data and the shape used to invert them act as a systematic error in the coefficients. Measurement packages therefore include the shape model they were made with (Exchange packages).

Reducing high-resolution models

High-resolution shape models have more facets than coefficients to degree 4 or 5 require. Two functions reduce them:

- `sbtk.shape.reduce_spc` keeps every `step`-th grid point of an SPC model stored in its grid order (6 cube faces of $(q+1)^2$ vertices, $12q^2$ facets), triangulated as the original. The grid is radial, so the result stays star-shaped about the body's center; a cell whose triangles would face the center is split along its other diagonal.
- `sbtk.shape.reduce` applies MATLAB's `reducepatch` to any mesh and checks that the result is closed, outward and star-shaped about its centroid; a general reduction can fold facets, in which case it stops with `sbtk:shape:reduce`.

MATLAB

```
big   = sbtk.shape.load("g_00243mm_spc_obj_dimo_0000n00000_v004.obj", "km");   % q = 512, 3,145,728 facets
small = sbtk.shape.reduce_spc(big, 8);                                        % q = 64, 49,152 facets
sbtk.shape.write_obj(small, "g_00243mm_spc_obj_dimo_0000n00000_v004_every8.obj");
```

Bodies

A body (`body`) is a shape model with its total mass and the reference radius of its gravity coefficients. `sbtk.body.from_mass` takes the mass; `sbtk.body.create` takes the bulk density and computes the mass from the volume. The bulk density is M / V; the background density of a model with components differs from it (Density distributions).

MATLAB fragment. Requires the variables defined in Checking a shape model.

```
body = sbtk.body.from_mass("Didymos", shape, 5.273e11, 0.43);    % mass [kg], reference radius [km]
sbtk.body.summary(body)
sbtk.plot.shape(body);
```

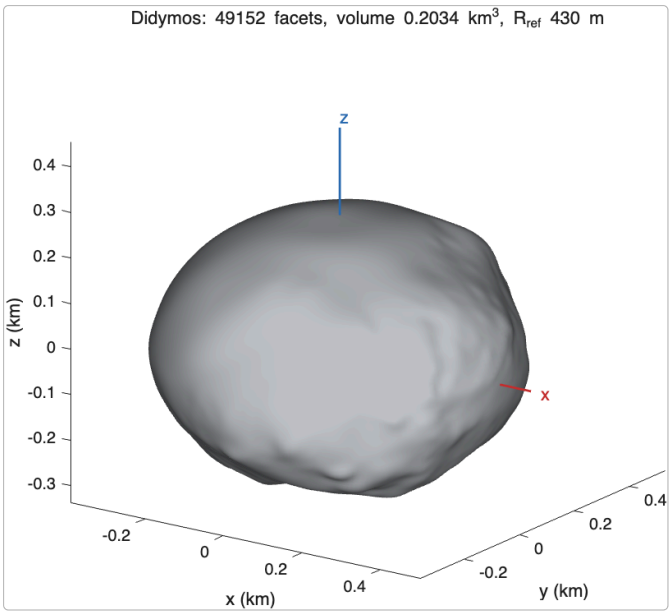


Figure produced by `sbtk.plot.shape` for the 9.3 m Didymos model: the mesh with its center of figure and axes.

Density distributions

Density model

The density at a point $\mathbf{x}$ inside the body is

$$\rho(\mathbf{x}) = \rho_{\mathrm{bg}} + \sum_{c\,:\,\mathbf{x}\,\in\,c} \Delta\rho_c + \delta\rho(\mathbf{x}),$$

where

- ρ_{bg} is the **background density**, the density outside every component;
- $\Delta\rho_c$ is the **relative density** of component c: its density minus the background density (`component.drho_kgm3`). Where components overlap their relative densities add. Everything outside the shape model is excluded;
- $\delta\rho$ is an optional Gaussian **random density field**.

The background density is not an input of a simulation: `sbtk.gravity.forward` sets it so that the total mass equals the mass of the body,

$$\rho_{\mathrm{bg}} = \frac{M - \sum_c \Delta\rho_c V_c - \int \delta\rho \, dV}{V},$$

with V_c the volume of component c inside the body. In an inversion it is estimated or held at the bulk density (Density estimation). `sbtk.data.simulate` stops with `sbtk:data:negativeDensity` if the density is negative anywhere.

An interior model (`interior`) holds the components and the random field; `sbtk.interior.create` builds it. `sbtk.interior.density_at` evaluates the density of any model at given points.

Component types

Each constructor returns one `component`. A point $\mathbf{x}$ (shape-model frame) belongs to the component when the condition in the table holds. $\mathbf{o}$ is the expansion origin, t_s the distance from $\mathbf{o}$ to the surface along the ray through $\mathbf{x}$, $r = |\mathbf{x} - \mathbf{o}|$.

Type	Constructor	Membership condition	Measured from
Sphere	`sbtk.interior.sphere(name, center_km, radius_km, drho_kgm3)`	$\lvert\mathbf{x} - \mathbf{c}\rvert < r_s$	shape-model origin
Ellipsoid	`sbtk.interior.ellipsoid(name, center_km, semi_axes_km, angles_deg, drho_kgm3)`	$\sum_i \left([R^{\mathsf{T}}(\mathbf{x} - \mathbf{c})]_i / a_i \right)^2 < 1$	shape-model origin
Torus	`sbtk.interior.torus(name, ring_radius_km, tube_radius_km, height_km, drho_kgm3)`	$\left(\sqrt{q_1^2 + q_2^2} - R_r \right)^2 + (q_3 - h)^2 < r_t^2$ with $q = A^{\mathsf{T}}(x - c)$	its axis: the third axis of `component.axes` A through `component.center_km` c (the shape-model z-axis unless set)
Layer	`sbtk.interior.layer(name, top_depth_km, bottom_depth_km, drho_kgm3)`	$d_{\mathrm{top}} \le t_s - r < d_{\mathrm{bottom}}$	rays from $\mathbf{o}$
Sector	`sbtk.interior.sector(name, frac, lat_deg, lon_deg, drho_kgm3)`	$f_{\mathrm{in}} \le r / t_s < f_{\mathrm{out}}$, $\phi_1 \le \phi < \phi_2$ ($\le 90°$ if $\phi_2 = 90°$), λ in $[\lambda_1, \lambda_2)$ modulo 360°	rays from $\mathbf{o}$

Notes on the types:

- **Sphere.** A sphere entirely inside the body is integrated exactly as a point mass; one that crosses the surface is clipped. Its center can be estimated in an inversion (`component.estimate_position`).
- **Ellipsoid.** $R = R_z(\mathrm{yaw}) R_y(\mathrm{pitch}) R_x(\mathrm{roll})$ (Coordinate systems).
- **Torus.** Lies about its axis, the shape-model z-axis through the origin as constructed; set `component.center_km` to move it (for example to the center of figure) and turn it with `sbtk.interior.set_axis`. It may be buried, cross the surface or extend outside the body. Only the part inside the body counts.
- **In another frame.** `sbtk.interior.to_frame` moves and turns every component with the frame: centers, the axes of ellipsoids, the axis of a torus and the axes in which a sector's latitude and longitude are measured (`component.axes`), and the random field. The density at every point is unchanged (Frames of a gravity field).
- **Layer.** Depth is measured along the ray from the expansion origin, not along the surface normal; `layer("regolith", 0, 0.012, …)` is the outermost 12 m along each ray.
- **Sector.** Radial fractions follow the shape: `sector("shell", [0.5 0.75], [-90 90], [-180 180], …)` is a shell between half and three quarters of the way to the surface. `sbtk.interior.sector` stops with `sbtk:interior:sectorRange` unless each range increases and the longitude range spans at most 360°.

`sbtk.interior.latlon` converts latitude, longitude and distance to a position, for example to place a sphere. `sbtk.interior.summary` prints a list of components.

Where to place components

Spheres, ellipsoids and tori are placed in the coordinates of the shape-model file, so a center `[0 0 0]` is the **origin of the file**. Layers and sectors are measured along rays from the expansion origin, which in a simulation is the **center of mass**. The **center of figure** is the centroid of the shape model (`body.centroid_km`), the center of mass of a homogeneous body (Coordinate systems).

Place components from the center of figure:

- **It depends only on the shape.** The center of mass of a model depends on its components, so positions measured from it would depend on what they define.
- **"Central" means concentric with the figure.** An offset of a component from the center of figure is what moves the center of mass away from it, which the data measure (degree 1). That offset should be chosen, not inherited from where the file has its origin.
- **The origin of a shape-model file can be far from the body's center.** It is 57 m from the center of figure in the DART Didymos model v003 (13% of its radius), 0.7 m in the DART Dimorphos model v004, and 1.4 m in the OSIRIS-REx Bennu model, whose origin is the center of mass found from the tracking data.

MATLAB fragment. Requires the variables defined in An interior model of Didymos.

```
cof = body.centroid_km;                                % center of figure
core = sbtk.interior.sphere("core", cof + [0.010 0 0], 0.170, +450);   % 10 m from the center of figure along x
t = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
t.center_km = cof;                                     % about the z-axis through the center of figure
```

In the interior of Cavale et al. (Icarus) for Bennu (Bennu) every position is from the origin of the Bennu shape model, which is its measured center of mass: there the two choices differ by 1.4 m. With a pre-mission model such as the DART Didymos v003 they differ by 57 m. A central core placed at the origin is then 57 m off center, and in a Didymos model it moves the center of mass about 2 m from the center of figure.

Axes of components

The axis of a torus, the axes of an ellipsoid and the axes in which a sector's latitude and longitude are measured are `component.axes`; their third axis is "the axis" of the component.

- `sbtk.interior.set_axis(c, direction)` turns a component so that its axis points along a direction (the smallest rotation, about its center).
- `sbtk.interior.align_with_spin_axis(body, interior, names, tol_deg, max_iter)` sets the axis of the named components along the axis of the largest moment of the whole model: the spin axis of a body in principal-axis rotation, about which an equatorial torus forms. That axis depends on the components themselves, so the function iterates (simulate to degree 2, take the axis, turn the components, repeat) until the axis moves less than `tol_deg`, at most `max_iter` times (`sbtk.internal.fixed_point`, `sbtk:iteration:notConverged`). Other components are not changed.

MATLAB fragment. Requires the variables defined in An interior model of Didymos.

```
cof = body.centroid_km;
t = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
t.center_km = cof;
model = sbtk.interior.create([sbtk.interior.sphere("boulder", cof + sbtk.interior.latlon(25, 60, 0.250), 0.040, +900), t], ...
                             sbtk.interior.no_random_field());
[model, spin_axis, n] = sbtk.interior.align_with_spin_axis(body, model, "torus", 0.01, 20);
fprintf("torus axis %.2f deg from the shape-model z-axis after %d simulations\n", acosd(spin_axis(3)), n)
```

For a true model these are modeling choices. In an inversion the geometry of a torus or an ellipsoid (center, height, axis) can be estimated instead, with bounds and priors (Density estimation).

Sector grids

`sbtk.interior.sector_grid(frac_edges, lat_edges_deg, lon_edges_deg)` divides the whole body into sectors, one per radial shell, latitude band and longitude band, named `s<shell><lat><lon>` (`s111` is the innermost shell, the southernmost band, the first longitude band). This is a density model without an assumed structure. Because the sectors fill the body, the background density and a common change of all sector densities are indistinguishable: an inversion must hold the background at the bulk density (`background = "bulk"`) or use regularization (Density estimation).

Random density field

`sbtk.interior.random_field(sigma_kgm3, corr_length_km, seed)` is a Gaussian random field with standard deviation σ and correlation $\exp(-|\Delta\mathbf{x}|^2/(2L^2))$, represented by 300 Fourier modes (`random_field`). It is defined at every point independently of the expansion origin and is the same for the same seed. `sbtk.interior.no_random_field` gives no field. A random field is used in simulations; inversions do not estimate one.

Prescribed components

A component with `component.estimate_density` set to false keeps its relative density in an inversion, for example a surface layer of known density. The constructors set it to true.

Example

MATLAB script: An interior model of Didymos

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
cof = body.centroid_km;                                          % positions from the center of figure
torus = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
torus.center_km = cof;                                           % about the z-axis through it
components = [ ...
    sbtk.interior.sphere("core", cof, 0.170, +450), ...
    torus, ...
    sbtk.interior.layer("regolith", 0, 0.012, -900), ...
    sbtk.interior.sphere("boulder", cof + sbtk.interior.latlon(25, 60, 0.250), 0.040, +900)];
interior = sbtk.interior.create(components, sbtk.interior.random_field(40, 0.06, 7));
sbtk.interior.summary(interior.components)

geo = sbtk.gravity.geometry(body, body.centroid_km, 0);          % degree 0: no coefficients needed
fm  = sbtk.gravity.forward(geo, interior, body.mass_kg);
fprintf("background density %.0f kg/m^3\n", fm.rho_bg_kgm3)
model = interior;
model.rho_bg_kgm3 = fm.rho_bg_kgm3;                               % background + components + field
rho = sbtk.interior.density_at(model, body.centroid_km + [0 0 0; 0.30 0 0], geo)
```

Here the geometry is built about the center of figure, so the layer is measured from it; `sbtk.data.simulate` uses the center of mass instead and returns the model with its background density as `truth`. The last line evaluates the density at the center of figure and 300 m from it along x. Further examples: Density models by component type.

Gravity and moment calculations

The forward model computes, for a density model inside a shape model, the gravity coefficients (Gravity coefficients), the center of mass and the inertia tensor (Moments of inertia) about an expansion origin $\mathbf{o}$.

Facet cones

`sbtk.gravity.geometry(body, origin_km, degree)` divides the body into tetrahedral cones, one per surface facet, with apex at $\mathbf{o}$ and the facet as base. This partition is exact when the shape is star-shaped about $\mathbf{o}$. For every cone f it stores, per unit density (`geo`):

Quantity	Definition	Field
Spherical-harmonic integrals	$T_{f,lm} = \int_f (r / R_{\text{ref}})^l \bar{Y}_{lm}\, dV$, with $\bar{Y}_{lm} = \bar{P}_{lm}(\sin\phi)\{\cos, \sin\}(m\lambda) / (2l+1)$	`geo.T`
Volume	V_f	`geo.V`
First moments	$\int_f \mathbf{x}\, dV$	`geo.m1`
Second moments	$\int_f [x^2, y^2, z^2, xy, xz, yz]\, dV$	`geo.Q`

$T_{f,lm}$ comes from the polyhedral kernel `sbtk.internal.ComputePolyhedralCSBar_mex` applied to the tetrahedron of each cone. With this definition a body of uniform density ρ has $\bar{C}_{lm} = \rho \sum_f T_{f,lm} / M$ (with the unit factor of Units).

Scaling within a cone

Shrinking a cone about its apex by a factor s multiplies $r^l\, dV$ by s^{l+3}. The part of cone f between the scaled facets s_a and s_b $(0 \le s_a < s_b \le 1)$ therefore contributes exactly

$$T_{f,lm}\,(s_b^{l+3} - s_a^{l+3}), \qquad V_f\,(s_b^3 - s_a^3), \qquad \mathbf{m}_{1,f}\,(s_b^4 - s_a^4), \qquad Q_f\,(s_b^5 - s_a^5)\,.$$

The validation test `testConeScaling` checks this law against the kernel to a relative error below 10^{-13}.

Integrals of components

`sbtk.gravity.component_integrals(geo, c)` returns the integrals of the part of a component inside the body per unit density (`integrals`):

- **Sphere entirely inside the body:** exact point-mass integrals.
- **Layer:** along each of 9 sub-rays per facet, the radial interval between the two depths gives s_a and s_b directly.
- **Sector:** the radial fractions give s_a and s_b; whether a sub-ray lies within the latitude and longitude range is tested at the middle of the interval.
- **Sphere crossing the surface, ellipsoid, torus:** a sub-ray crosses the component's surface where a polynomial in the distance along it vanishes: a quadratic for a sphere or an ellipsoid, a quartic for a torus (solved as the eigenvalues of its companion matrix). Its real roots between the expansion origin and the body's surface split the sub-ray, and each piece whose midpoint is inside the component contributes $s_b^p - s_a^p$, so a ray may enter and leave the component several times, and a ray that only grazes it contributes a correspondingly short piece.

For each facet the contributions are averaged over its 9 sub-rays and multiplied by the cone's integrals. The result is exact when the radial interval is the same along every sub-ray of a cone, and converges with mesh resolution otherwise (Assumptions and limitations).

`sbtk.gravity.random_field_integrals` evaluates the random field at the centroid of each of 24 radial shells of every cone (`samples`) and integrates each shell exactly with that value.

Coefficients, center of mass and inertia

`sbtk.gravity.forward(geo, interior, mass_kg)` combines the integrals of the body (U, V, $\mathbf{m}_1$, Q), of the components (U_c, V_c, ...) and of the random field (U_δ, ...):

$$U = \rho_{\text{bg}} U_{\text{body}} + \sum_c \Delta\rho_c U_c + U_\delta, \qquad \bar{C}_{lm}, \bar{S}_{lm} = \frac{U_{lm}}{M},$$

$$\mathbf{x}_{\text{cm}} = \mathbf{o} + \frac{\rho_{\text{bg}} \mathbf{m}_{1,\text{body}} + \sum_c \Delta\rho_c \mathbf{m}_{1,c} + \mathbf{m}_{1,\delta}}{M}, \qquad I_{xx} = Q_{yy} + Q_{zz},\ I_{xy} = -Q_{xy},\ \ldots$$

with ρ_{bg} chosen so that the mass is M (Density distributions). It returns the coefficients to degree L, the center of mass and $[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}] / (MR_{\text{ref}}^2)$ (`fm`).

Simulated measurements

`sbtk.data.simulate(body, interior, degree, name)` runs the forward model about the center of mass:

1. geometry and forward model about the shape-model origin;
2. geometry and forward model about the resulting center of mass, repeated until the center of mass and the expansion origin differ by less than 1 mm (layers and sectors depend on the origin);
3. a check that the density is not negative at the sample points of `sbtk.internal.body_samples` (24 radial shells per cone), otherwise `sbtk:data:negativeDensity`;

4. the exact coefficients of degree 1 to L, the six inertia elements and the libration row, with the ratio $k = \sqrt{(I_{yy} - I_{xx})^2 + 4I_{xy}^2} \,/\, I_{zz}$ of the principal moments, as a measurement set without uncertainties (`obs`), and the true model (`truth`).

No measurement uncertainties enter the forward model. To make measurements of the true model, `sbtk.data.set_uncertainties` (or `sbtk.data.use_uncertainties`, `sbtk.data.set_covariance`, `sbtk.data.set_moi_sigma`) gives the exact values uncertainties and `sbtk.data.add_noise` draws one noisy realization (Measurement sets). Selecting which inertia quantities a data set contains: `sbtk.data.select_moi`.

Computation time

Times measured by `tools/benchmark.m` on Didymos (49,152 facets, degree 4), MATLAB R2025a on an Apple-silicon laptop:

Step	Time
Geometry, degree 4	0.22 s
Simulation: 4 components and a random field, center-of-mass frame	3.6 s
Inversion setup: 3 components	1.3 s
Inversion run	0.25 s
Comparison with the true model	0.53 s

The geometry scales with the number of facets; a 3,145,728-facet model takes about a minute to read and integrate.

Example

MATLAB script: Coefficients, center of mass and inertia of a model

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
interior = sbtk.interior.create(sbtk.interior.sphere("core", [0 0 0], 0.170, +450));

geo = sbtk.gravity.geometry(body, body.centroid_km, 4);       % about the center of figure, to degree 4
fm  = sbtk.gravity.forward(geo, interior, body.mass_kg);
lab = sbtk.gravity.labels(4);
fprintf("C20 = %.5e, C22 = %.5e\n", fm.coef(strcmp(lab, "C20")), fm.coef(strcmp(lab, "C22")))
fprintf("center of mass - center of figure = [%.2f %.2f %.2f] m\n", 1e3*(fm.com_km - body.centroid_km))
fprintf("Izz / (M R_ref^2) = %.4f\n", fm.inertia_norm(3))
```

The core is centered at the shape-model origin, 57 m from the center of figure, so the center of mass moves toward it and the degree-1 coefficients about the center of figure are not zero.

Measurement sets

A measurement set (`obs`) holds fully normalized gravity coefficients with their covariance, moment-of-inertia measurement rows, the total mass, the reference radius and the expansion origin.

Sources

Source	Function	Notes
A simulated interior	`sbtk.data.simulate`	Exact values about the true center of mass, without uncertainties (they are NaN until set, below); `coef_about = 'origin'`.
An SBTK measurement package	`sbtk.exchange.read_data`	Coefficients about the measured center of mass with their full covariance; returns the body as well (Exchange packages).
A gravity file in the PDS SHADR layout	`sbtk.exchange.read_shadr`	Header row (R, GM, σGM, degree, order, normalization state, ...) then rows `l, m, C, S, sigma_C, sigma_S`; diagonal covariance. The caller gives the shape model and the point the coefficients are about.
A JPL SSD gravity file	`sbtk.io.read_jpl_gravity`	MATLAB-assignment form with `NAMES`, `VALS`, `COV`; parsed as text; full covariance; J_l converted to $\bar{C}_{l0}$.
Coefficient matrices	`sbtk.data.from_coefficients`	`C(l+1, m+1)`, `S(l+1, m+1)` and their 1σ; diagonal covariance.
The SBTK text format	`sbtk.io.read_measurements`, `sbtk.io.load_measurements`	One keyword per line; diagonal covariance only (Results folders).

Uncertainties and covariance

- **Exact values have none.** `sbtk.data.simulate` returns the true model's exact values without uncertainties: `coef_cov` and every inertia σ are NaN. Measurements need them: `sbtk.data.add_noise`, `sbtk.invert.setup` and `sbtk.exchange.write_data` stop with `sbtk:data:noUncertainties` otherwise.
- **By degree.** `sbtk.data.set_uncertainties(obs, unc)` gives every coefficient of degree l the 1σ of `sbtk.data.uncertainties` for that degree (a diagonal covariance), every inertia element its 1σ (as a fraction of MR_{ref}^2), and the libration row the 1σ of the ratio times $I_{zz} / (MR_{\mathrm{ref}}^2)$.
- **From another data set.** `sbtk.data.use_uncertainties` gives every coefficient that a reference data set also contains the reference's variances and covariances, converted to the reference radius of the target by $(R / R')^l$. Coefficients the reference lacks keep their variances and become uncorrelated with the others.
- **Full covariance.** `sbtk.data.set_covariance` replaces the coefficient covariance with any symmetric positive definite matrix in the order of `obs.coef_labels`.
- **Inertia rows.** `sbtk.data.set_moi_sigma` sets the σ of named rows; the field `libration` gives the 1σ of the ratio.

Noise

`sbtk.data.add_noise(obs, seed)` returns one realization: the coefficients plus $L\mathbf{z}$, with L the lower Cholesky factor of the coefficient covariance and $\mathbf{z}$ standard normal; each inertia row plus its σ times a standard normal variable. The same seed gives the same noise; `obs.noise_seed` records it.

Inertia measurements and the center of mass

Function	Adds
`sbtk.data.add_moi`	One element `Ixx` ... `Iyz` as $I / (MR_{\mathrm{ref}}^2)$ (`sbtk.units.moi_to_norm` converts from kg m²).
`sbtk.data.add_moi_row`	Any linear combination of the six elements.
`sbtk.data.add_libration`	The libration ratio $(B - A) / C$ as a linear row (Moments of inertia). Requires $\bar{C}_{22}$ and $\bar{S}_{22}$.
`sbtk.data.libration_ratio_from_amplitude`	Computes the ratio and its 1σ from a forced libration amplitude.
`sbtk.data.add_center_of_mass`	The measured center of mass as degree-1 coefficients; sets `coef_about` to `'center_of_mass'`.

Selecting measurements

`sbtk.data.select_degree` keeps the coefficients up to a degree, `sbtk.data.select_coefficients` a list of coefficients, and `sbtk.data.select_moi` a list of inertia rows; the covariance keeps the matching rows and columns. `sbtk.data.move_origin` changes the expansion origin of data about the center of mass. `sbtk.data.summary` prints the contents of a measurement set.

Example

MATLAB script: Measurements from coefficient matrices

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Dimorphos", sbtk.shape.bundled("dimorphos"), 2.700e9, 0.10);

% coefficient matrices, C(l+1, m+1) = C_lm, fully normalized, about the shape-model origin
L = 2;
C = zeros(L + 1);  S = zeros(L + 1);
C(1,1) = 1;  C(3,1) = -0.08;  C(3,3) = 0.03;  S(3,3) = 0.002;     % C00, C20, C22, S22
Cs = 1e-3*ones(L + 1);  Ss = 1e-3*ones(L + 1);                     % 1-sigma of each
obs = sbtk.data.from_coefficients(body, "example", [0 0 0], C, S, Cs, Ss);

% a polar moment given in kg m^2, and a libration ratio
Izz = sbtk.units.moi_to_norm(1.6e14, body.mass_kg, body.R_ref_km);
obs = sbtk.data.add_moi(obs, "Izz", Izz, 0.01*Izz);
obs = sbtk.data.add_libration(obs, 0.20, 0.03, Izz);

% 30% correlation between all coefficients
sig = sqrt(diag(obs.coef_cov));
obs = sbtk.data.set_covariance(obs, (0.3 + 0.7*eye(numel(sig))).*(sig*sig'));
sbtk.data.summary(obs)
```

Calculations has further examples: simulated data with uncertainties from another data set, selection of inertia rows, and reading files.

Density estimation

The inversion estimates the background density and the relative densities of a set of components from a measurement set, and optionally parts of their geometry: the position of spheres, ellipsoids and tori and the axis of ellipsoids and tori (Estimated geometry).

Procedure

`sbtk.invert.setup(body, components, obs, opts)` builds the geometry about `obs.origin_km` and the design matrix once (`prob`); `sbtk.invert.run(prob)` solves. `sbtk.invert.solve` does both. Reusing `prob` is faster when the same problem is solved more than once:

- with other settings that do not select measurements (`prob.opts.regularization`, bounds, positions);
- for a homogeneous body (`sbtk.invert.homogeneous`);
- for other realizations of the same measurements (`sbtk.invert.replace_data`).

Estimation problem

Let $\mathbf{y}$ be the n measurements (the selected coefficients followed by the inertia rows named in `opts.moi_labels`) with covariance $\Sigma = LL^{\mathsf{T}}$. The predictions are linear in the densities:

$$\mathbf{y}_{\mathrm{pred}} = \rho_{\mathrm{bg}}\,\mathbf{g}_{\mathrm{body}} + \sum_{c=1}^{n_c} \Delta\rho_c\,\mathbf{g}_c,$$

where $\mathbf{g}_{\mathrm{body}}$ and $\mathbf{g}_c$ are the measurements of a unit density in the whole body and in component c (`prob.gbody`, `prob.G`). The unknowns $\mathbf{x} = (\rho_{\mathrm{bg}}, \Delta\rho_1, \ldots, \Delta\rho_{n_c})$ minimize

$$\lVert L^{-1}(\mathbf{y}_{\mathrm{pred}} - \mathbf{y}) \rVert^2 + \lambda \sum_c \left(\frac{\Delta\rho_c}{\bar{\rho}}\right)^2$$

subject to

- the **mass constraint** $\rho_{\mathrm{bg}} V + \sum_c \Delta\rho_c V_c = M$, with V_c the volume of component c inside the body and M = `obs.mass_kg`;
- the **bounds** $0 \le \rho_{\mathrm{bg}} \le \rho_{\max}$ and $0 \le \rho_{\mathrm{bg}} + \Delta\rho_c \le \rho_{\max}$ for every component, with $\rho_{\max}$ = `opts.rho_max_kgm3`;
- **fixed parameters**: with `opts.background = "bulk"` the background is held at the bulk density $\bar{\rho} = M / V$; components with `component.estimate_density` false keep their relative density.

λ is the Tikhonov parameter `opts.regularization`. The problem is solved with `lsqlin`, first with the active-set algorithm and, if that does not converge, with the interior-point algorithm; if neither converges the run stops with `sbtk:invert:solver`.

Estimated geometry

Besides the densities, an inversion can estimate the geometry of components. The geometry parameters are listed by `sbtk.internal.geometry_parameters`:

Field of the component	Components	Parameters
`component.estimate_position = true`	sphere, ellipsoid, torus	the three center coordinates, km (`<name>_x`, `_y`, `_z`)
`component.estimate_position = [false false true]`	sphere, ellipsoid, torus	only the coordinates of the mask, for example the height of a torus
`component.estimate_axis = true`	torus	the direction of its axis: two tilts about its own first and second axes, deg (`<name>_tilt1`, `_tilt2`)
`component.estimate_axis = true`	ellipsoid	its orientation: the two tilts of its third axis and the turn about it, deg (`<name>_tilt1`, `_tilt2`, `_turn`)

Setting either field on a component without a center or an axis stops the inversion with `sbtk:invert:geometryParameter`. The geometry is found by Gauss–Newton iteration:

- **Each step** linearizes the measurements in the geometry parameters and solves the bounded problem. The derivatives are central differences of the integrals: 10^{-6} km for a sphere inside the body (a point mass), 10^{-4} km and 10^{-2} deg for components integrated along rays, whose integrals are smooth only on scales larger than the ray spacing (Assumptions and limitations).
- **Trust region.** The steps are limited to `opts.step_km` for a center coordinate and `opts.step_deg` for a tilt or turn. A step that does not lower the objective is undone and the trust region halved; a successful step restores it.
- **A priori constraints.** With `opts.position_sigma_km` or `opts.axis_sigma_deg` finite, each center coordinate, tilt or turn has an a priori constraint of that 1σ about its starting value. These are needed when the measurements do not determine all of the geometry, and they keep a component from absorbing structure the model lacks (below).
- **Hard limits.** `opts.max_shift_km` bounds the distance of an estimated center from its start, `opts.max_tilt_deg` the angle of an estimated axis from its starting direction, and `opts.max_turn_deg` the turn of an ellipsoid about its axis. Spheres are always kept inside the body.
- **Stopping.** Iteration stops after `opts.max_iter` steps, or earlier when any of these holds:
- the objective changes by less than 10^{-10} relative;
- an accepted step changes every parameter by less than `opts.geometry_tol` of its step limit;
- the trust region falls below 10^{-6} km.

Without the step test, a model that lacks structure the data contain keeps lowering χ^2 by ever smaller steps. That runs every repetition about the center of mass to its limit.

For data about the center of mass the fit is repeated about the model's own center of mass (below), and each repetition continues from the geometry of the previous one. The priors and limits always refer to the geometry the components were given.

Absorbed structure. An estimated geometry can absorb structure the model lacks, so freeing it can lower χ^2 while moving the component away from the truth. Example: a Didymos true model has a torus about its spin axis at the center of figure, plus a boulder and a random density field. The data are its exact values weighted by the Hera radio-science precision. An inversion model with a core, the torus and the regolith has a reduced χ^2 of 121 ($R^2 = -5.3$) with fixed geometry. With the torus axis and height estimated (a priori 1σ of 1° and 10 m, limits of 10° and 50 m), χ^2 drops to 39 while R^2 falls to −5.7. The torus tilts 5.6° and moves 41 m up to stand in for the boulder. At this precision such priors are weak. Priors, or limits tight enough to bind, must express how well the geometry is actually known.

Local minima. A fit whose geometry absorbs structure can have more than one local minimum. The same Didymos fit, with the derivatives taken over smaller steps, settled with the torus 47 m *below* the center of figure and a reduced χ^2 of 37; which minimum Gauss–Newton reaches depends on its path. Compare fits started from different geometries before reading a position or an axis as a result.

`sbtk.interior.align_with_spin_axis` is the counterpart for making a true model: it sets the axis of tori, ellipsoids or sectors along the model's own axis of the largest moment by fixed-point iteration (Density distributions).

Uncertainties

The covariance of the parameters is the linearized one at the solution, with the mass constraint and the fixed parameters eliminated:

$$P = Z\,(Z^{\mathsf{T}}J^{\mathsf{T}}JZ)^{-1}Z^{\mathsf{T}},$$

where J is the whitened Jacobian of the measurements with the regularization and a priori rows appended, and the columns of Z span the null space of the mass constraint and the fixed parameters. If $Z^{\mathsf{T}}J^{\mathsf{T}}JZ$ is singular (the measurements do not determine every free parameter), the pseudo-inverse is used and `fit.determined` is false. The bounds are not included in P; `fit.at_bound` marks densities at a bound.

The density of a component and its 1σ are $\rho_{\mathrm{bg}} + \Delta\rho_c$ and $\sqrt{P_{\mathrm{bg,bg}} + P_{cc} + 2P_{\mathrm{bg},c}}$ (`fit.rho_kgm3`, `fit.rho_sigma_kgm3`). `fit.resolved` marks components whose relative density exceeds its 1σ. The validation tests check over 40 noise realizations that the standard deviation of (estimate − truth)/σ is within 0.25 of 1, with diagonal and with strongly correlated covariances (Validation).

Regularization

Models with many components, such as a sector grid, or weak data need Tikhonov regularization to keep noise from producing large alternating densities. `sbtk.invert.choose_regularization` returns the largest λ for which

$$\chi^2 \le \max\left(n,\ \chi^2_{\min} + \sqrt{2n}\ \right),$$

with n the number of measurements and $\chi^2_{\min}$ the value at $\lambda = 10^{-2}$ (the discrepancy principle): χ^2 may rise by about one standard deviation of its distribution. It searches $\log_{10}\lambda \in [-2, 8]$ by bisection with 9 solves; the result is returned, not stored, so set `prob.opts.regularization` before the final run.

Data about the center of mass

When `obs.coef_about` is `'center_of_mass'`, the coefficients of degree 2 and higher are about the body's center of mass, which is known only through its measurement (the degree-1 rows). Expanding the model about the *measured* center of mass would carry the error of that measurement into the predictions: a translation of the expansion point changes the coefficients of degree l by terms proportional to those of lower degrees. `sbtk.invert.run` therefore

1. keeps the model where it is: its geometry is built about the measured center of mass (`fit.origin_km`), from which layers and sectors are measured;
2. moves its predicted coefficients of degree ≥ 2 and inertia to the model's own center of mass, by the exact translation of solid harmonics (`sbtk.internal.sh_translation`) and the parallel-axis theorem;
3. iterates (Gauss–Newton, including the dependence of the center of mass on the densities, `prob.D`) until that center moves by less than 1 mm (`fit.frame_iterations`, `fit.frame_offset_km`);
4. compares the model's center of mass with the measured one through the degree-1 rows.

`fit.com_shift_km` is the model's center of mass minus `fit.origin_km`. The tests `testCenterOfMassFrameMatchesTheTrueFrame` and `testPredictionsAboutTheModelCenterOfMass` check that the densities agree with an inversion about the true center of mass to within 0.2σ and that the predictions equal a forward model computed from scratch about the fitted center of mass (Validation).

Settings

Setting (`opts`)	Standard value	Meaning
`opts.moi_labels`	`{'Ixx','Iyy','Izz'}`	Inertia rows fitted; `{}` for gravity coefficients only.
`opts.regularization`	0	Tikhonov parameter λ.
`opts.background`	`"estimate"`	`"bulk"` holds the background at M / V.
`opts.rho_max_kgm3`	4500	Upper bound on every density, kg/m³.
`opts.max_iter`	20	Gauss–Newton iterations for estimated geometry.

Setting (`opts`)	Standard value	Meaning
`opts.step_km`	0.010	Trust-region radius of a center coordinate, km.
`opts.step_deg`	1	Trust-region radius of an axis tilt or turn, deg.
`opts.position_sigma_km`	`Inf`	A priori 1σ of center coordinates, km.
`opts.axis_sigma_deg`	`Inf`	A priori 1σ of axis tilts and turns, deg.
`opts.max_shift_km`	`Inf`	Largest distance of an estimated center from its start, km.
`opts.max_tilt_deg`	`Inf`	Largest angle of an estimated axis from its start, deg.
`opts.max_turn_deg`	`Inf`	Largest turn of an estimated ellipsoid about its axis from its start, deg.
`opts.geometry_tol`	10^{-3}	Converged when every geometry step is below this fraction of its step limit.

`sbtk.invert.options(Name=Value)` changes any of them; the fields can also be set afterwards.

Example

MATLAB script: Estimation with several settings

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
unc  = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
cof = body.centroid_km;                                          % positions from the center of figure
true_components = [sbtk.interior.sphere("core", cof + [0.03 -0.02 0.01], 0.17, +500), ...
                   sbtk.interior.torus("torus", 0.37, 0.055, 0.015, -500), ...
                   sbtk.interior.layer("regolith", 0, 0.012, -900)];
true_components(2).center_km = cof;                              % the torus about the z-axis through cof
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(true_components), 4, "estimation");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);

components = [sbtk.interior.sphere("core", cof, 0.17, 0), ...        % geometry fixed, densities estimated
              sbtk.interior.torus("torus", 0.37, 0.055, 0, 0), ...
              sbtk.interior.layer("regolith", 0, 0.012, 0)];
components(2).center_km = cof;
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
hom  = sbtk.invert.homogeneous(prob);                            % is structure required?
fit  = sbtk.invert.run(prob);
sbtk.invert.summary(fit)

moving = components;  moving(1).estimate_position = true;        % estimate the core center as well
fit_p  = sbtk.invert.solve(body, moving, obs);
fprintf("core center [%.0f %.0f %.0f] m from the center of figure +- [%.0f %.0f %.0f] m\n", ...
        1e3*(fit_p.components(1).center_km - cof), 1e3*fit_p.center_sigma_km(1, :))

raised = components;  raised(2).estimate_position = [false false true];  % estimate the height of the torus
fit_z  = sbtk.invert.solve(body, raised, obs, sbtk.invert.options(max_shift_km=0.030));
fprintf("torus height %.1f m +- %.1f m (true 15 m)\n", 1e3*(fit_z.components(2).center_km(3) - cof(3)), 1e3*fit_z.center_sigma_km(2, 3))
```

The true core is at (30, −20, 10) m from the center of figure and the true torus 15 m above it; the fixed-geometry fit places both at the center of figure. More examples: Calculations.

Goodness of fit and model comparison

Chi-square test

For a fit with residuals $\mathbf{r} = \mathbf{y}_{\text{pred}} - \mathbf{y}$ and measurement covariance Σ:

Quantity	Definition	Field
χ^2	$\mathbf{r}^{\mathsf{T}}\Sigma^{-1}\mathbf{r}$, with the full covariance	`fit.chi2`
Effective number of parameters	$p_{\text{eff}} = \operatorname{tr}(J_d P J_d^{\mathsf{T}})$, the trace of the data resolution matrix (J_d whitened measurement Jacobian, P parameter covariance); it accounts for fixed parameters and regularization	
Degrees of freedom	$\nu = n - p_{\text{eff}}$	`fit.dof`
Reduced χ^2	χ^2 / ν; close to 1 when the model and the stated uncertainties are correct	`fit.chi2_reduced`
p-value	$P(\chi^2_\nu \geq \chi^2)$, the probability that a correct model gives a χ^2 at least as large, computed as the upper regularized incomplete gamma function $Q(\nu/2, \chi^2/2)$	`fit.p_value`
Consistent	$p \geq 0.05$: the model is consistent with the measurements at the 5% significance level	`fit.consistent`

`sbtk.invert.goodness_of_fit` computes the last three from χ^2 and ν. When $\nu \leq 10^{-6}$ (at least as many parameters as measurements) the test is undefined: the reduced χ^2 and the p-value are NaN and `consistent` is false.

Homogeneous body

`sbtk.invert.homogeneous` fits a body of uniform density M / V to the same measurements; it has no free parameters ($\nu = n$). If this fit is already consistent with the measurements, the data do not require internal structure at their precision.

Normalized residuals

$(y_{\text{pred}} - y) / \sigma$ for every measurement (`fit.normalized_residual`, `sbtk.plot.residuals`). For a correct model about 68% lie within ±1. Large residuals concentrated in one group indicate structure that the model lacks: in the degree-1 rows, a model that cannot place the center of mass; at one degree, structure at the corresponding scale.

When a fit is rejected ($p < 0.05$), its densities and uncertainties describe the best model of the chosen form, not the body. At high measurement precision, small structures that a component model omits, such as a random density field of a few percent of the bulk density or a boulder tens of meters across, can be enough to reject an otherwise correct model (Hera blind test).

Comparison with a true model

In synthetic tests the true model is known. `sbtk.compare.with_truth(body, truth, fit)` evaluates both densities at the centroids of 24 radial shells of every facet cone (1,179,648 points for the 49,152-facet Didymos model), each weighted by the shell volume w_i, with each model's layers and sectors measured from its own origin. It returns (`cmp`):

$$R^2 = 1 - \frac{\sum_i w_i(\rho_{\text{est},i} - \rho_{\text{true},i})^2}{\sum_i w_i(\rho_{\text{true},i} - \bar{\rho}_{\text{true}})^2},$$

the fraction of the volume-weighted variance of the true density about its mean that the estimate explains: 1 for an exact estimate, 0 for one no better than a uniform density, negative for a worse one.

- $R^2_{\max}$: the largest R^2 attainable with the same components, from a weighted least-squares fit of the true density by the background and the components. It measures the limit set by the choice of components; R^2 close to $R^2_{\max}$ means the measurements determined the densities as well as that geometry allows.
- The volume-weighted RMS error and the standard deviation of the true density.
- For each component present in both models: estimate, 1σ, true density and $|\text{estimate} - \text{true}| / \sigma$.

For many fits of the same components against one true model, `sbtk.compare.reference` samples the true model once and `sbtk.compare.evaluate` compares each fit in milliseconds. `sbtk.compare.summary` prints a comparison and `sbtk.plot.comparison` plots it.

Comparison of density grids

Any two models can be compared on a common cubic grid, independently of how they are parametrized. `sbtk.exchange.density_grid(body, model, spacing_km)` evaluates a model at cell centers that lie at integer multiples of the spacing along the axes of the shape-model frame; cells whose center is inside the shape model (tested along rays from the center of figure) are kept, so all models of a body have the same cells (`grid`). `sbtk.compare.grids(truth, estimate)` compares the cells present in both grids (`gridcmp`): R^2 with equal weights, RMS and mean error, and the fraction of cells whose error is at most twice the 1σ of the estimate. Both grids must have the same spacing (`sbtk:compare:gridSpacing`). This is how solutions in exchange packages are compared with interior-model packages.

Calibration of the uncertainties

The uncertainties of a fit can be checked with repeated noise realizations: if they are correct, the standard deviation of $(\hat{\rho} - \rho_{\text{true}}) / \sigma$ over many realizations is close to 1.

MATLAB script: Calibration over noise realizations

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
unc  = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
components = [sbtk.interior.sphere("core", body.centroid_km, 0.17, +450), sbtk.interior.layer("regolith", 0, 0.012, -900)];
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(components), 4, "calibration");
exact = sbtk.data.set_uncertainties(exact, unc);

prob = sbtk.invert.setup(body, components, sbtk.data.add_noise(exact, 1), sbtk.invert.options());
z = zeros(30, 3);
for k = 1:30
    f = sbtk.invert.run(sbtk.invert.replace_data(prob, sbtk.data.add_noise(exact, 100 + k)));
    z(k, :) = ([f.rho_bg_kgm3, [f.components.drho_kgm3]] - [truth.rho_bg_kgm3, [components.drho_kgm3]]) ...
              ./[f.rho_bg_sigma_kgm3, f.drho_sigma_kgm3];
end
fprintf("standard deviation of (estimate - truth)/sigma: %.2f\n", std(z(:)))
```

Results folders and figures

Run folders

`sbtk.io.new_run(project, name)` creates the folder `results/<project>/<yyyy-mm-dd_HHMMSS>_<name>/` in the repository and starts recording everything printed in the Command Window to `log.txt` in it, until `sbtk.io.finish_run` closes the log. It returns an `out` structure, which every save and report function takes. A new run never overwrites an earlier one.

- `sbtk.io.latest_run(project, name)` returns the newest folder whose name after the time stamp equals `name` exactly: `"didymos_T1"` does not match `didymos_T1_inversion`. Inversion scripts use it to find the newest simulation.
- `sbtk.io.subfolder(out, name)` makes the save and report functions write into a subfolder while the log stays in the run folder. The Hera simulation runners use it to keep the true model apart from the data package (Hera blind test).

MATLAB

```
out = sbtk.io.new_run("my_project", "T1_inversion");     % results/my_project/<date_time>_T1_inversion/
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
sbtk.io.report_inversion(out, body, sbtk.invert.run(prob), sbtk.invert.homogeneous(prob));
sbtk.io.finish_run(out);
```

This fragment requires `body`, `components` and `obs` as defined in the first script.

Files

File	Written by	Contents	Read with
`log.txt`	`sbtk.io.new_run`	Everything printed during the run.	a text editor
`settings.json`	`sbtk.io.save_settings`, called by the report functions	Body, degree, uncertainties, components, random field, noise seed and inversion settings of the run.	`jsondecode(fileread(file))`
`measurements.txt`, `.mat`	`sbtk.io.save_measurements`	The measurement set (`obs`); the text has only the diagonal of the coefficient covariance.	`sbtk.io.load_measurements`
`truth.txt`, `.mat`	`sbtk.io.save_truth`	The true model of a simulation (`truth`).	`sbtk.io.load_truth`
`fit.txt`, `.mat`	`sbtk.io.save_fit`	Densities with 1σ, χ^2 test, every residual (`fit`).	`sbtk.io.load_fit`
`homogeneous.txt`	`sbtk.io.report_inversion`	The same data fitted by a homogeneous body.	
`comparison.txt`, `.mat`	`sbtk.io.save_comparison`	R^2, $R^2_{\max}$, RMS error, per-component comparison (`cmp`).	`load`
`figures/*.png`	`sbtk.io.save_figure` (200 dpi)	The figures below.	

The report functions write a standard set in one call:

Function	Writes
`sbtk.io.report_simulation(out, body, obs, truth)`	`measurements.*`, `truth.*`, `settings.json`, `figures/truth_3d.png`, `truth_slices.png`, `measurements.png`
`sbtk.io.report_inversion(out, body, fit, homogeneous)`	`fit.*`, `homogeneous.txt`, `settings.json`, `figures/residuals.png`, `densities.png`, `estimate_3d.png`, `estimate_slices.png`
`sbtk.io.report_comparison(out, body, truth, fit, cmp)`	`comparison.*`, `settings.json`, `figures/comparison.png`, `truth_vs_estimate.png`

Text formats

Measurements (`sbtk.io.write_measurements`, `sbtk.io.read_measurements`): one keyword per line, values separated by spaces, `#` lines are comments. `coef` lines hold l, m, $\dot{C}_{lm}$, $\dot{S}_{lm}$ and their 1σ; `inertia` lines hold the label, value, 1σ and the six weights. The first lines of a file:

Output

```
# Didymos measurements: didymos_example (SBTK 1.0.0)
# Simulated by SBTK from interior didymos_example. Values include simulated measurement noise (seed 1).
# Lines: keyword then values. coef = gravity coefficient: degree l, order m, C, S and their
# 1-sigma uncertainties (fully normalized, Kaula, no Condon-Shortley phase; dimensionless).
# inertia = one moment-of-inertia measurement on I/(M R_ref^2): label, value, 1-sigma, then
# its weights on Ixx Iyy Izz Ixy Ixz Iyz. Lengths in km, mass in kg.
body Didymos
name didymos_example
mass_kg 5.272999999999999e+11
GM_km3_s2 3.519358390000000e-08
R_ref_km 4.300000000000000e-01
degree 4
origin_km 1.528148834993066e-02 4.288469572137803e-02 -3.405288560049095e-02
coef_about origin   (origin: coefficients about origin_km; center_of_mass: degree >= 2 about the center of mass)
com_minus_cof_km -6.480457727157593e-04 5.708425733001801e-04 -7.435287904191540e-05
noise_seed 1   (NaN = exact values or real data)
libration_sigma 2.000000000000000e-02   (1-sigma of the libration ratio; NaN = none)
# gravity field about origin_km: l m C S sigma_C sigma_S
coef 1 0 -1.743147996446580e-03  0.000000000000000e+00 2.685350089254074e-03 0.000000000000000e+00
coef 1 1  3.171855274043025e-03 -2.036642465133304e-03 2.685350089254074e-03 2.685350089254074e-03
coef 2 0 -3.267843300100998e-02  0.000000000000000e+00 1.000000000000000e-04 0.000000000000000e+00
coef 2 1  3.580678708085953e-03  9.264930696445466e-04 1.000000000000000e-04 1.000000000000000e-04
coef 2 2  2.690908094312223e-03 -6.620608536237520e-03 1.000000000000000e-04 1.000000000000000e-04
coef 3 0  8.207508643558385e-04  0.000000000000000e+00 2.000000000000000e-04 0.000000000000000e+00
coef 3 1 -2.161190799036665e-03  1.345792464405454e-03 2.000000000000000e-04 2.000000000000000e-04
coef 3 2 -1.185030577784691e-04  1.023223669886617e-03 2.000000000000000e-04 2.000000000000000e-04
coef 3 3 -7.109585763745573e-04 -1.687530510730563e-03 2.000000000000000e-04 2.000000000000000e-04
coef 4 0  3.025957726300028e-03  0.000000000000000e+00 4.000000000000000e-04 0.000000000000000e+00
coef 4 1 -1.108795363729265e-03 -9.890871671108035e-04 4.000000000000000e-04 4.000000000000000e-04
coef 4 2  5.087506921525737e-04  1.560247138787098e-03 4.000000000000000e-04 4.000000000000000e-04
```

Inversion results (`sbtk.io.write_fit`): lines beginning with `#` describe the lines below them.

Output

```
# Didymos inversion of didymos_example (SBTK 1.0.0)
# chi-square, degrees of freedom, reduced chi-square, p-value (consistent at 5%: true)
chi2 25.9429   dof 24.000   chi2_reduced 1.0810   p_value 0.3561
n_measurements 27
regularization 0
moi_used Ixx Iyy Izz
origin_km 1.528148834993066e-02 4.288469572137803e-02 -3.405288560049095e-02
coef_about origin   com_shift_km 0.000000000 0.000000000 0.000000000   (degree >= 2 and inertia predicted about origin_km + com_shift_km)
# density <component> <relative density> <its 1-sigma> <density> <its 1-sigma>   [kg/m^3]
#   (+ center x y z and their 1-sigma in km, if the center was estimated; + axis x y z and the
#    1-sigma of the rotations about the first, second and third axes in deg, if estimated)
density background 0 0 2666.936 129.322
density core 642.329 533.014 3309.265 405.126
density torus -414.298 108.921 2252.638 52.039
density regolith -1082.035 716.363 1584.901 588.859
# component <name> <type> <geometry>
component core sphere center [15.9 42.3 -34.0] m, radius 170.0 m
component torus torus ring 370.0 m, tube 55.0 m, ring center [15.9 42.3 -34.0] m, axis [0.0000 0.0000 1.0000]
component regolith layer 0.0 to 12.0 m below the surface
# residual <label> <measured> <predicted> <sigma> <(predicted - measured)/sigma>
residual C10 -1.743147996447e-03  8.859490892471e-05 2.685350e-03  0.682
residual C11  3.171855274043e-03  7.515844247841e-04 2.685350e-03 -0.901
```

True models (`sbtk.io.write_truth`) and **comparisons** (`sbtk.io.write_comparison`) use the same layout.

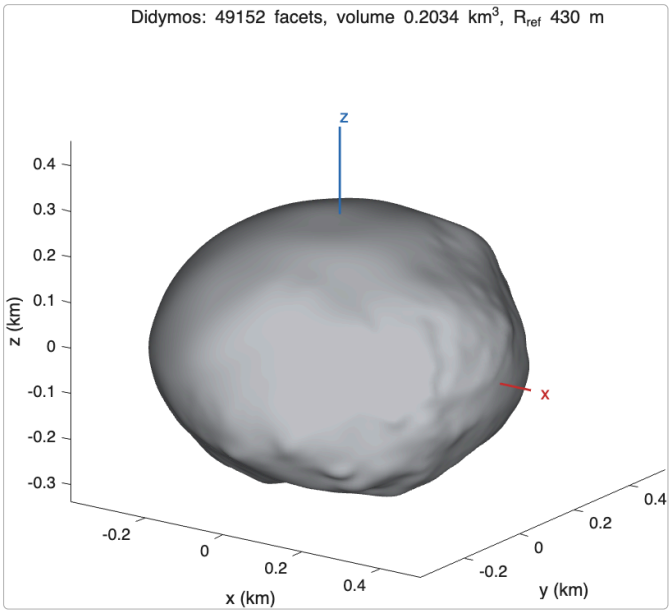
Figures

Every plot function returns a figure handle. `sbtk.io.save_figure(out, fig, name)` saves it as `figures/<name>.png` at 200 dpi in a run folder; `exportgraphics` saves it anywhere. Densities are drawn with a sequential color map (dark blue to yellow); signed differences with blue for lower and red for higher density.

The figures below come from one synthetic Didymos run, made by `documentation/tools/make_samples.m`: a core (+450 kg/m³) at the center of figure, an equatorial torus (−500) about the z-axis through it, a 12 m regolith layer (−900), a boulder of radius 40 m (+900) and a random field of 40 kg/m³ with a 60 m correlation length; measurements to degree 4 with the uncertainties of the first script; an inversion for core, torus and regolith (the boulder and the field are not in the inversion model). The fit has $\chi^2 / \nu = 1.08$ ($p = 0.36$), the homogeneous body $\chi^2 / \nu = 45.0$; $R^2 = 0.93$ with $R^2_{\max} = 0.98$.

Shape model

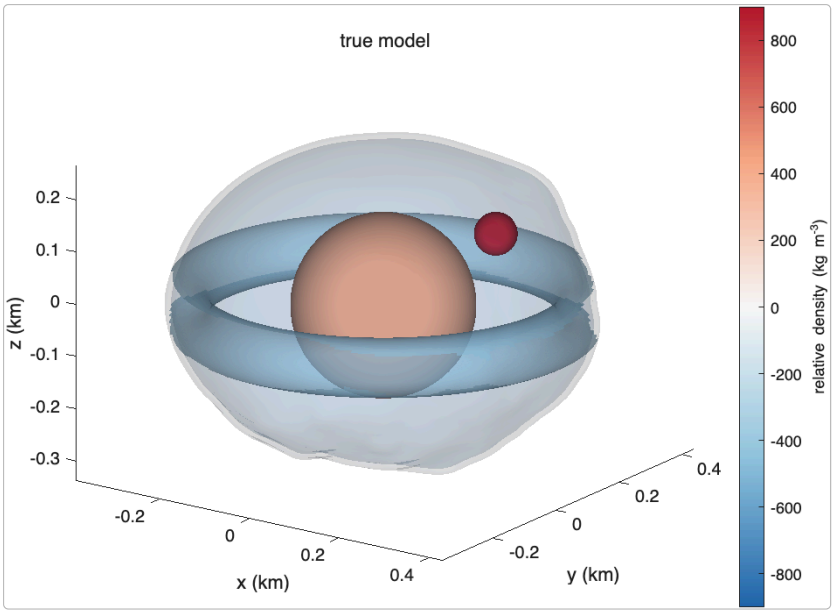
`sbtk.plot.shape(body)`: the mesh with its center of figure and axes.



Didymos shape model (DART v003, 9.3 m sampling).

Interior in three dimensions

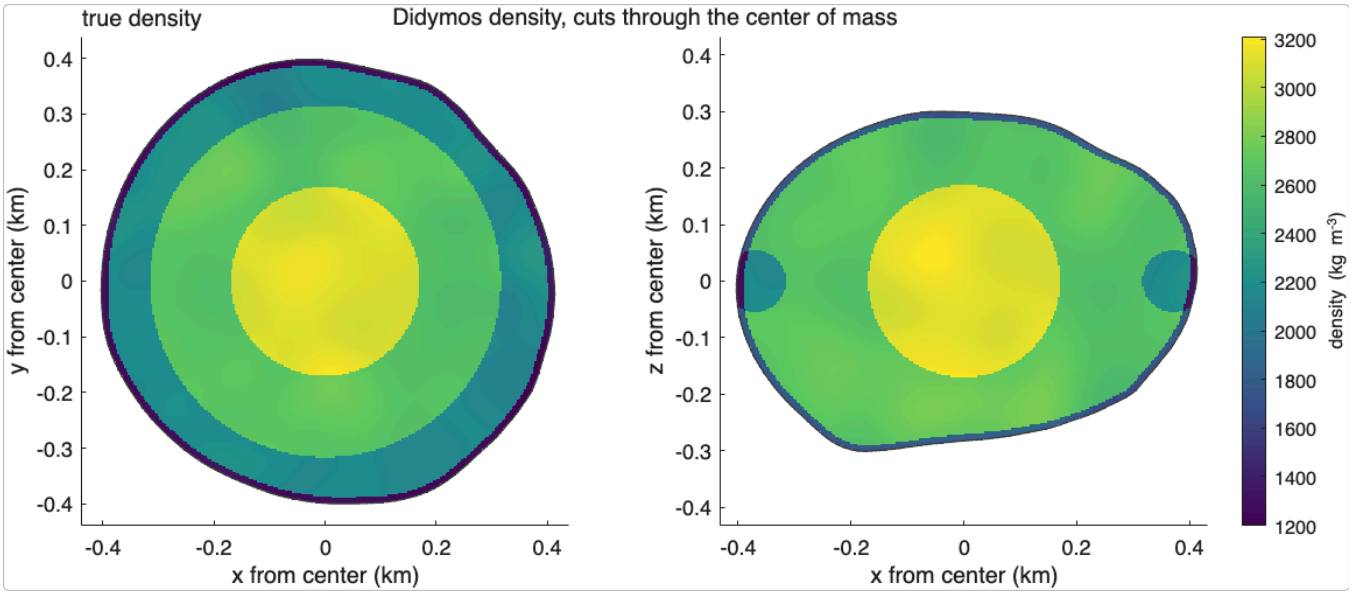
`sbtk.plot.interior3d(body, model, title)`: the surface drawn translucent; spheres, ellipsoids and tori drawn where they lie inside the body, layers by their lower boundary; color is the relative density. Sectors are not drawn (use slices). Saved as `truth_3d.png` and `estimate_3d.png` by the report functions.



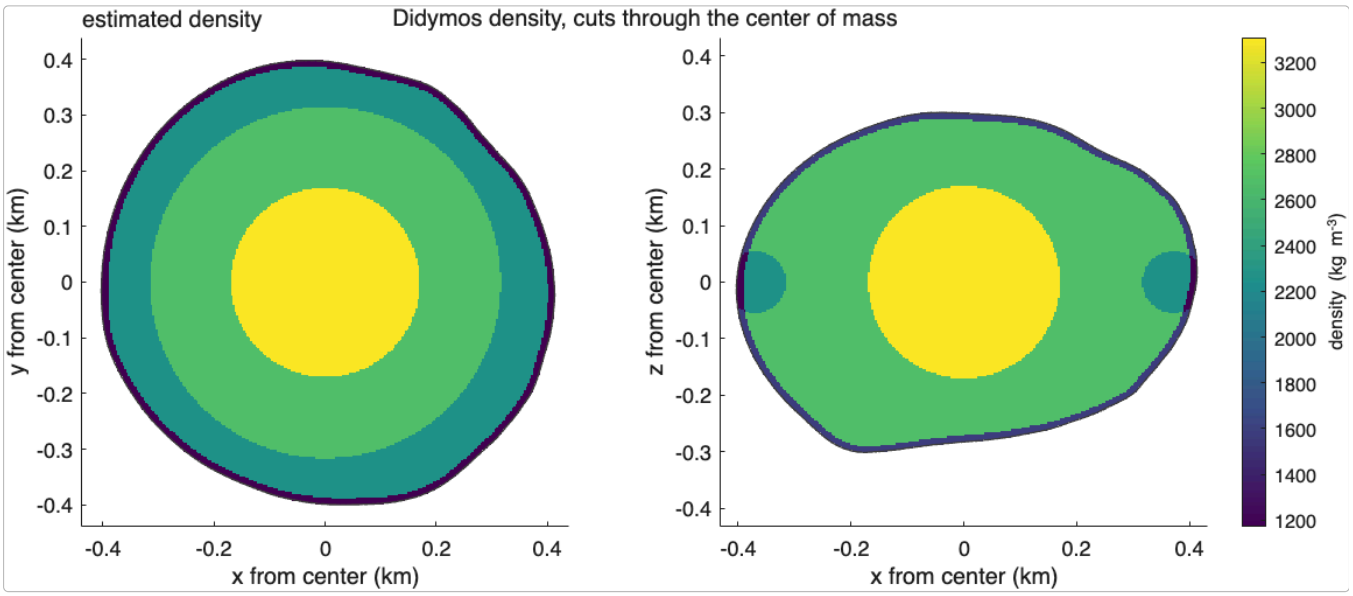
True model: core, torus, lower boundary of the regolith layer, boulder.

Density slices

`sbtk.plot.slices(body, {models}, {titles})`: the density on the plane $z = 0$ (left) and the plane $y = 0$ (right), both through the first model's origin, one row per model with a common color scale. Saved as `truth_slices.png` and `estimate_slices.png`.



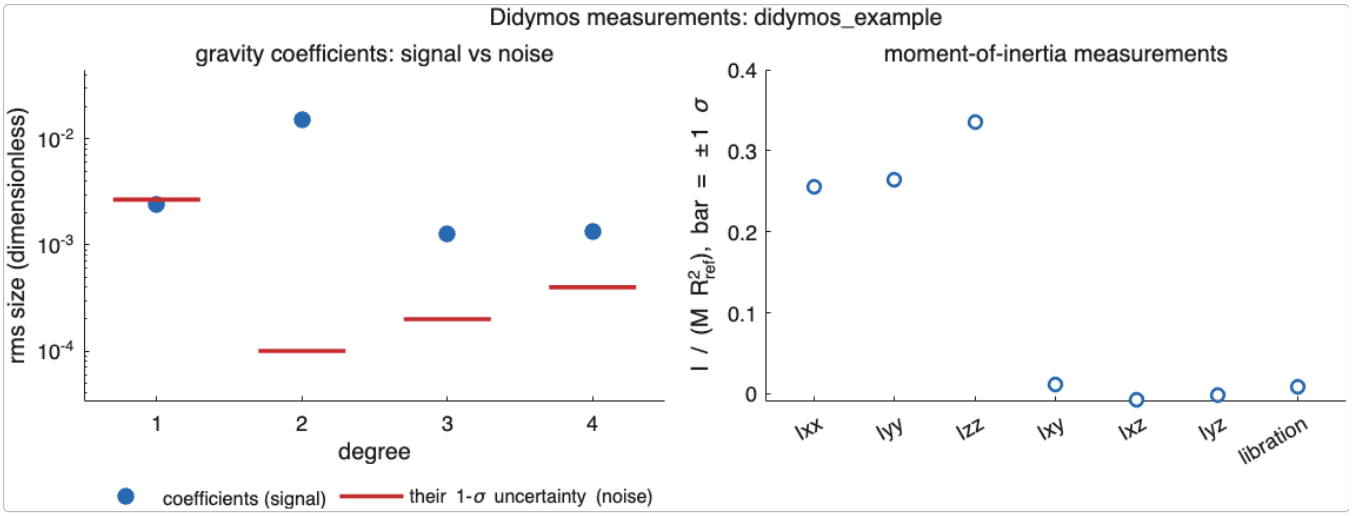
True density; the random field appears as small-scale variation.



Estimated density of the three-component model on the same planes.

Measurements

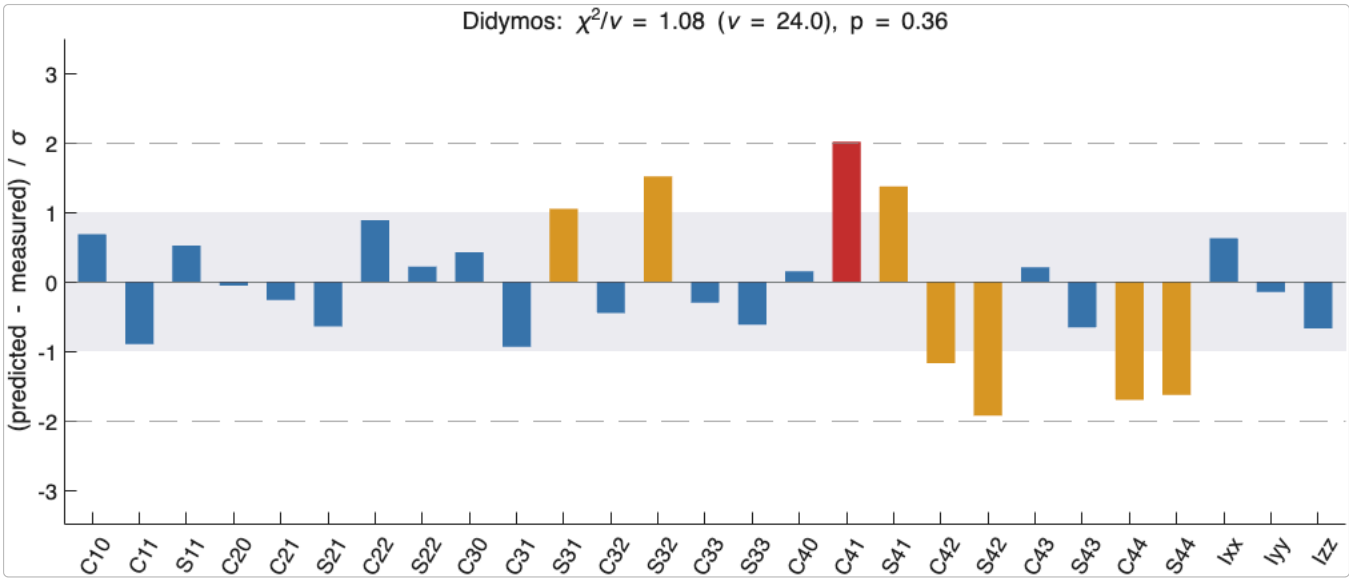
`sbtk.plot.measurements(obs)`: left, the root-mean-square size of the coefficients of each degree (points) and of their 1σ uncertainties (line); degrees where the points lie well above the line carry information about the interior. Degree 1 is close to zero about the center of mass. Right, each inertia measurement with its 1σ. Saved as `measurements.png`.



Coefficients of degrees 1 to 4 and the seven inertia rows of the example.

Normalized residuals

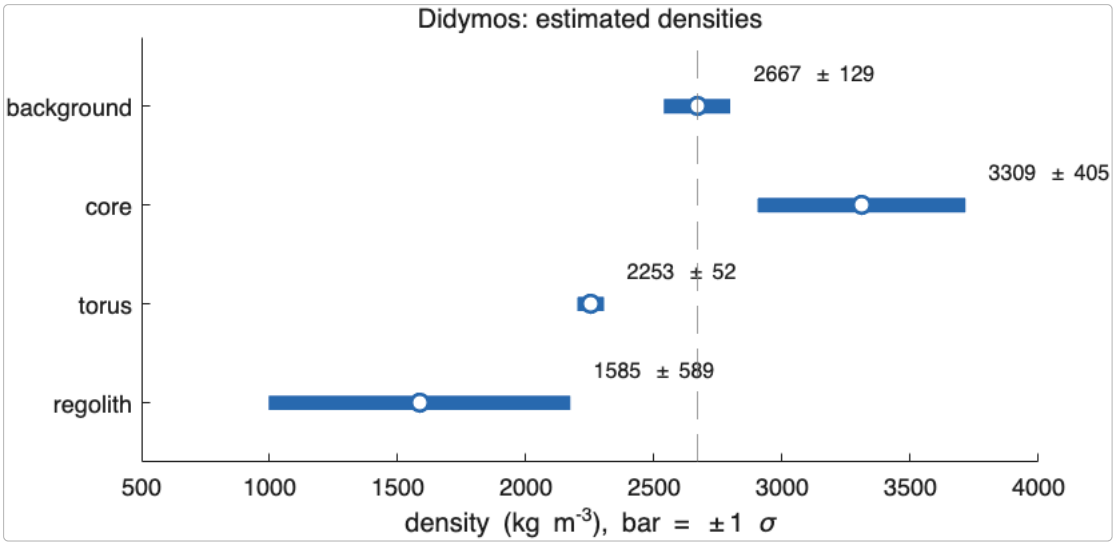
`sbtk.plot.residuals(fit)`: (predicted – measured)/σ for every measurement; gray band ±1σ, dashed lines ±2σ. Saved as `residuals.png`.



Residuals of the three-component fit.

Estimated densities

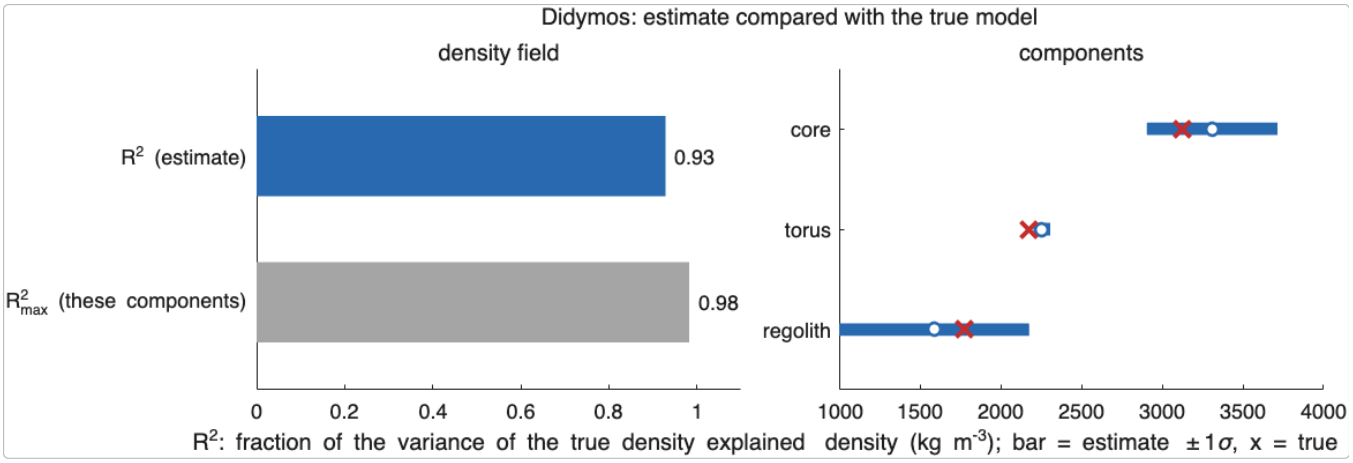
`sbtk.plot.densities(fit)`: the density of the background and each component with its ±1σ interval; the dashed line is the background. Components whose relative density is smaller than its 1σ are drawn in gray. Saved as `densities.png`.



Estimated densities of the example.

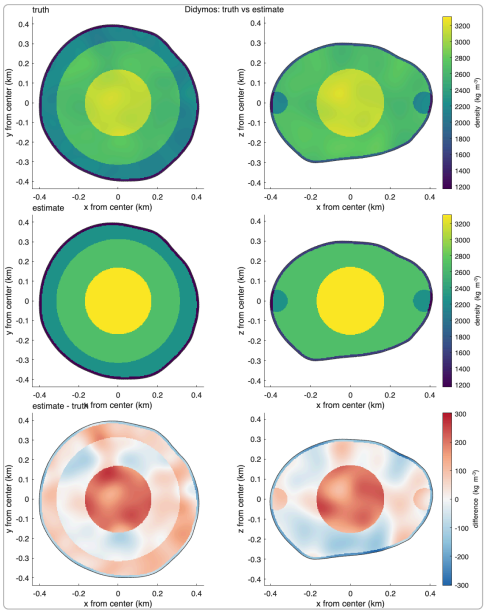
Comparison with the true model

`sbtk.plot.comparison(cmp)`: left, R^2 of the estimate and $R^2_{\max}$; right, each component's estimate with ±1σ and its true density (cross). Saved as `comparison.png`.



Comparison of the example fit with the true model.

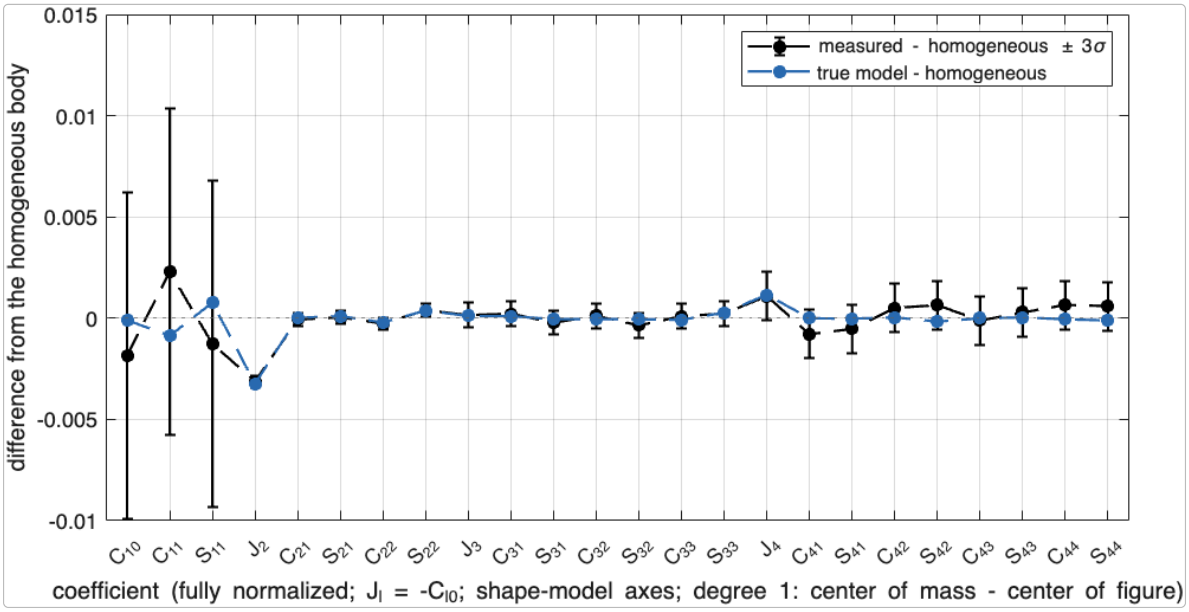
`sbtk.plot.truth_vs_estimate(body, truth, fit)`: rows for the true density, the estimate (same color scale) and their difference, on the two planes through the true center of mass. Saved as `truth_vs_estimate.png`.



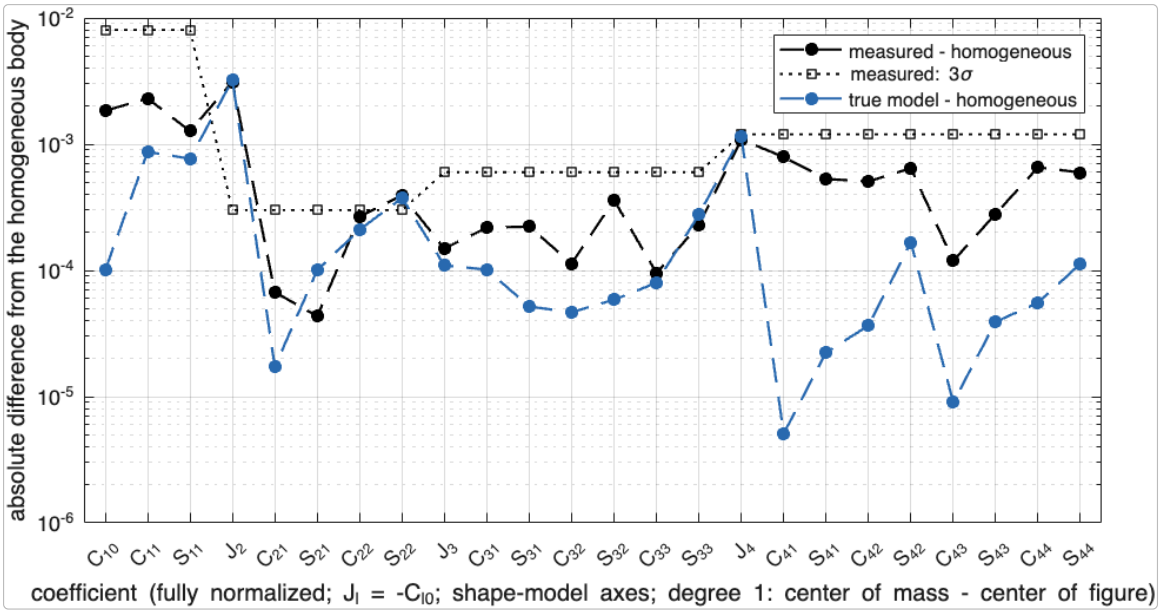
True density, estimate and difference.

Difference from the homogeneous body

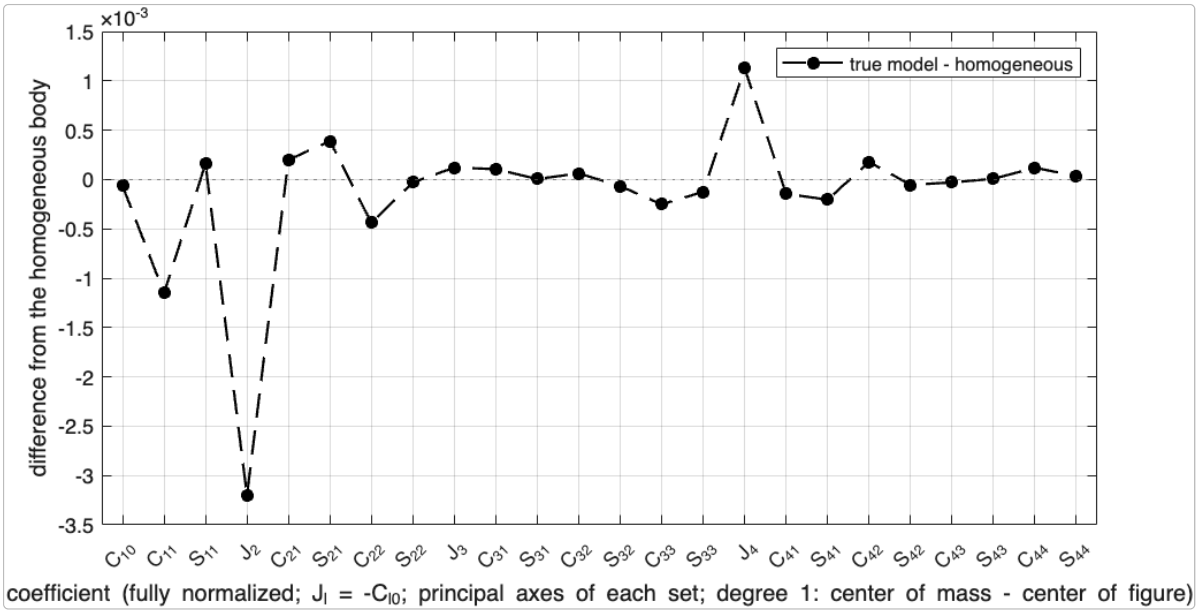
`sbtk.plot.difference_from_homogeneous(body, sets, names, scale, frame)`: for each set of gravity coefficients (a measurement set, a true model or an inversion result), the coefficients minus those of the homogeneous body (density M / V), both about their own center of mass, as radio-science fields are. Degree 1 shows the set's center of mass minus the center of figure. A measurement set about another point (for example a field about the shape-model origin) is first moved to its center of mass by the exact translation of solid harmonics. Zonal coefficients are shown as $J_l = -\bar{C}_{l0}$. With `scale = "linear"` the signed difference has ±3σ bars for sets with uncertainties; with `"log"` the absolute difference is drawn with 3σ as a separate line, which shows which departures from a uniform density the measurements resolve. With `frame = "shape"` both fields are in the shape-model axes; with `"spin"` in the spin axes of the set (z along its axis of the largest moment), where the set's $\bar{C}_{21}$, $\bar{S}_{21}$ vanish; with `"principal"` in its principal axes, where also $\bar{S}_{22}$ vanishes. The differences in the vanishing terms are the homogeneous body's own terms in those axes: they measure how far the principal axes of the interior are tilted and turned from those of the homogeneous body (Frames of a gravity field). Not saved by the report functions; save it with `sbtk.io.save_figure`.



Measured coefficients (±3σ) and those of the true model, minus the homogeneous body, in the shape-model axes.



The same on a logarithmic scale, with the 3σ level of the measurements.



The true model minus the homogeneous body in the principal axes of the true model.

Exchange packages

Exchange packages carry measurements, interior models and solutions between teams, for example in a blind test where one team simulates measurements of an interior it keeps hidden and other teams invert them. They are plain text with the units, frame and normalization written in them, and can be read without SBTK.

Package	Name	Written by	Contents	Released
Measurements	`<team>-DATA<n>-<comment>`	`sbtk.exchange.write_data`	Gravity field, covariance, center of mass, moments of inertia, shape model	with the data set
Interior model	`<team>-MOD<n>-<comment>`	`sbtk.exchange.write_model`	True model, density grid, noise-free measurements, figures	after the solutions
Solution	`<team>-SOL_<data team><n>`	`sbtk.exchange.write_solution`	Estimated densities, covariance, fitted measurements, density grid, figures	with the solution

`sbtk.exchange.solution_name` forms solution names; `sbtk.exchange.locate` finds the package of a given kind in a run folder and its `redacted/` and `unredacted/` subfolders.

File layout

Each file starts with metadata lines `# key: value`, followed by one line of column names and comma-separated rows. Every column and key with a unit ends with it; NaN marks a value that is not provided.

Reader	Statement
MATLAB	`T = readtable('gravity_field.txt', 'FileType', 'text', 'CommentStyle', '#');`
Python with pandas	`T = pandas.read_csv('gravity_field.txt', comment='#')`
Python, standard library	`rows = list(csv.DictReader(l for l in open('gravity_field.txt') if not l.startswith('#')))`
SBTK	`sbtk.exchange.read_data(folder)`, `sbtk.exchange.read_grid(file)`

Measurement package

File	Contents
`README.txt`	Summary (GM and mass, bulk density, degree and reference radius of the field; the uncertainties and the noise when the values have them) and the list of files.
`gravity_field.txt`	`l,m,C,S` for degrees 2 and higher, fully normalized, about the center of mass, with `sigma_C,sigma_S` when the values have uncertainties.
`gravity_covariance.txt`	Full covariance of those coefficients; rows and columns labeled. Only when the values have uncertainties.
`center_of_mass.txt`	Center of mass in the shape-model frame (with its 3 × 3 covariance when it has one) and the center of figure.
`moments_of_inertia.txt`	`quantity,value,unit` (and `sigma`): inertia elements as $I / (MR_{\mathrm{ref}}^2)$ about the center of mass and/or the libration ratio. The sign convention of the products of inertia and the definition of the libration ratio are given when the file has them.
`conventions.txt`	Units; the frame (what its origin and axes are, and the change of frame from the original shape-model file); the normalization, with MATLAB and Python code; degree 1 and the center of mass.
`<shape>.obj`	The shape model the data were made with, in km, in the frame of every position of the package (The shape model of a package).

The gravity field is expanded about the center of mass, as a radio-science solution is, so it has no degree-1 terms; the position of the center of mass is a separate measurement in `center_of_mass.txt`. The package contains nothing about the interior: not the expansion origin of the simulation (the true center of mass), the noise seed, the model name or any component; `testNothingAboutTheInterior` checks this (Validation).

`sbtk.exchange.read_data` returns the measurement set with `coef_about = 'center_of_mass'`: the coefficients of degree ≥ 2 with their covariance, degree-1 coefficients 0 ± σ that carry the uncertainty of the center of mass, the inertia rows, and the body built from the package's shape model and GM. A libration ratio becomes the row $(B - A) - k\,I_{zz} = 0$ with σ = $\sigma_k I_{zz}$, using the measured I_{zz} if the package has one and that of a homogeneous body otherwise.

The files state only what applies to the package. Those of a measurement package with uncertainties and noise, made by `documentation/tools/make_samples.m`:

Output

```
DOC-DATA1-Didymos: measurements of Didymos
Prepared by Example team

SUMMARY
  GM                3.51935839e-08 km^3/s^2, mass 5.273000e+11 kg
  bulk density      2592.98 kg/m^3 (mass / shape model volume 0.203356 km^3)
  gravity field     degrees 2 to 4, reference radius 0.43 km, about the center of mass
  center of mass    see center_of_mass.txt file
  inertia           see moments_of_inertia.txt file
  frame             see conventions.txt file
  values            with 1-sigma uncertainties (covariance of the coefficients in gravity_covariance.txt) and Gaussian noise drawn from them

FILES
  gravity_field.txt
  gravity_covariance.txt
  center_of_mass.txt
  moments_of_inertia.txt
  conventions.txt
  didymos_g_9309mm_spc_obj_0000n00000_v003.obj shape model
```

Output

```
# product: gravity field, spherical-harmonic coefficients
# data_set: DOC-DATA1-Didymos
# body: Didymos
# shape_model: didymos_g_9309mm_spc_obj_0000n00000_v003.obj (24578 vertices, 49152 facets, lengths in km)
# expansion_point: the center of mass (center_of_mass.txt), so the degree-1 coefficients are zero; they are not listed
# axes: those of the shape model file (conventions.txt)
# normalization: fully normalized (conventions.txt)
# reference_radius_km: 0.43
# GM_km3_s2: 3.51935839e-08
# mass_kg: 5.273000e+11
# max_degree: 4
# C00: 1 (not listed)
# uncertainties: 1-sigma; full covariance in gravity_covariance.txt
# noise: Gaussian noise drawn from the 1-sigma uncertainties was added to the values
l,m,C,S,sigma_C,sigma_S
2,0,-3.267843300100998e-02,0.000000000000000e+00,1.000000000000000e-04,0.000000000000000e+00
2,1,3.580678708085953e-03,9.264930696445466e-04,1.000000000000000e-04,1.000000000000000e-04
2,2,2.690908094312223e-03,-6.620608536237520e-03,1.000000000000000e-04,1.000000000000000e-04
3,0,8.207508643558385e-04,0.000000000000000e+00,2.000000000000000e-04,0.000000000000000e+00
3,1,-2.161190799036665e-03,1.345792464405454e-03,2.000000000000000e-04,2.000000000000000e-04
3,2,-1.185030577784691e-04,1.023223669886617e-03,2.000000000000000e-04,2.000000000000000e-04
3,3,-7.109585763745573e-04,-1.687530510730563e-03,2.000000000000000e-04,2.000000000000000e-04
4,0,3.025957726300028e-03,0.000000000000000e+00,4.000000000000000e-04,0.000000000000000e+00
4,1,-1.108795363729265e-03,-9.890871671108035e-04,4.000000000000000e-04,4.000000000000000e-04
```

Output

```
# product: center of mass and center of figure
# data_set: DOC-DATA1-Didymos
# body: Didymos
# shape_model: didymos_g_9309mm_spc_obj_0000n00000_v003.obj (24578 vertices, 49152 facets, lengths in km)
# frame: that of the shape model file (conventions.txt)
# center_of_mass: the point the gravity field and the moments of inertia are about
# center_of_figure: centroid of the volume enclosed by the shape model (the center of mass if the density were uniform)
# center_of_mass_minus_center_of_figure_km: 0.001714294458 -0.000946011764 -0.001372617864
# uncertainties: 1-sigma of the center of mass; full 3 x 3 covariance in the cov columns
# noise: Gaussian noise drawn from the 1-sigma uncertainties was added to the values
axis,center_of_mass_km,sigma_km,center_of_figure_km,cov_x_km2,cov_y_km2,cov_z_km2
x,0.017643828580,0.002000000000,0.015929534123,4.000000000000001e-06,0.000000000000000e+00,0.000000000000000e+00
y,0.041367841384,0.002000000000,0.042313853148,0.000000000000000e+00,4.000000000000001e-06,0.000000000000000e+00
z,-0.035351150585,0.002000000000,-0.033978532721,0.000000000000000e+00,0.000000000000000e+00,4.000000000000001e-06
```

Output

```
# product: moments of inertia about the center of mass
# data_set: DOC-DATA1-Didymos
# body: Didymos
# shape_model: didymos_g_9309mm_spc_obj_0000n00000_v003.obj (24578 vertices, 49152 facets, lengths in km)
# axes: those of the shape model file (conventions.txt)
# normalization: values are I / (M R_ref^2), with M = mass_kg and R_ref = reference_radius_km
# reference_radius_km: 0.43
# mass_kg: 5.273000e+11
# products_of_inertia: I_ij = integral rho (r^2 delta_ij - x_i x_j) dV, so Ixy = -integral rho x y dV
# libration_ratio: (B - A) / C, with A <= B the principal moments about axes in the xy-plane and C = Izz
# uncertainties: 1-sigma, uncorrelated
# noise: Gaussian noise drawn from the 1-sigma uncertainties was added to the values
quantity,value,sigma,unit
Ixx,2.553972158114155e-01,2.000000000000000e-03,M R_ref^2
Iyy,2.641173924187360e-01,2.000000000000000e-03,M R_ref^2
Izz,3.346618502230035e-01,2.000000000000000e-03,M R_ref^2
Ixy,1.137452222834483e-02,2.000000000000000e-03,M R_ref^2
Ixz,-7.467290512494866e-03,2.000000000000000e-03,M R_ref^2
Iyz,-1.855098124365382e-03,2.000000000000000e-03,M R_ref^2
libration_ratio,8.070726840715539e-02,2.000000000000000e-02,1
```

The shape model of a package

A package's shape model is the one the data were made with, in the frame of every position of the package. A maker who changes the frame, as the HLP6 makers do (Frames of a gravity field), sends the original shape model moved and rotated into the new frame with `sbtk.body.to_frame`: every vertex becomes $x' = (x - c)A$, with c the center of mass of the true model and the columns of A the new axes, both in the frame of the original file. The shape itself does not change, only its coordinates.

Body	Shape model of the package	Origin	Axes
Didymos	`UCB-DATA1-Didymos_didymos_v003_com_spin_axes.obj`	center of mass	spin axes: z the axis of the largest moment, x the DART x-axis projected onto the plane normal to z
Dimorphos	`UCB-DATA4-Dimorphos_dimorphos_v004_com_principal_axes.obj`	center of mass	principal axes: x the axis of the smallest and z of the largest moment

Each true model has its own center of mass and axes, so each package has its own shape model, and its file name starts with the name of the package: `<package>_<shape name>.obj` (`sbtk.internal.shape_file_name`). The interior-model package of the data set and a solution of it use the same name. A shape model taken unchanged from a file keeps the file's name.

The center of mass is then at the origin (to rounding), and the center of figure at $-(r_{\mathrm{CoM}} - r_{\mathrm{CoF}})A$. The gravity field, the moments of inertia and `center_of_mass.txt`, and in the interior-model package every component and the density grid, are in this frame, so a receiving team uses the shape model as it is, with nothing to move or rotate. The FRAME section of `conventions.txt` states the origin and the axes, and the change of frame from the original file ($o = c$ and A), so the coordinates of the original file are $x = x'A^T + c$. The shape model records this change as `shape.frame_origin_km` and `shape.frame_axes`, combined over successive changes of frame.

Packages of exact values

A set without any uncertainties, such as the exact values of a true model (`sbtk.data.simulate`), is written as a package of exact values:

- `gravity_field.txt` has the columns `l,m,C,S`, `center_of_mass.txt` the columns `axis,center_of_mass_km,center_of_figure_km`, and `moments_of_inertia.txt` the columns `quantity,value,unit`;
- there is no `gravity_covariance.txt`;
- no file has an `uncertainties` or `noise` line.

A maker's package of exact values in the frame with the center of mass as origin and the spin axes, made by `documentation/tools/make_samples.m`:

Output

```
DOC-DATA2-Didymos: measurements of Didymos
Prepared by Example team

SUMMARY
  GM               3.51935839e-08 km^3/s^2, mass 5.273000e+11 kg
  bulk density     2592.98 kg/m^3 (mass / shape model volume 0.203356 km^3)
  gravity field    degrees 2 to 4, reference radius 0.43 km, about the center of mass
  center of mass   see center_of_mass.txt file
  inertia          see moments_of_inertia.txt file
  frame            see conventions.txt file

FILES
  gravity_field.txt
  center_of_mass.txt
  moments_of_inertia.txt
  conventions.txt
  DOC-DATA2-Didymos_didymos_v003_com_spin_axes.obj shape model
```

Output

```
# product: center of mass and center of figure
# data_set: DOC-DATA2-Didymos
# body: Didymos
# shape_model: DOC-DATA2-Didymos_didymos_v003_com_spin_axes.obj (24578 vertices, 49152 facets, lengths in km)
# frame: that of the shape model file, whose origin is the center of mass (conventions.txt)
# center_of_mass: the point the gravity field and the moments of inertia are about
# center_of_figure: centroid of the volume enclosed by the shape model (the center of mass if the density were uniform)
# center_of_mass_minus_center_of_figure_km: -0.000652459894 0.000568745609 -0.000047559533
axis,center_of_mass_km,center_of_figure_km
x,0.000000000000,0.000652459894
y,0.000000000000,-0.000568745609
z,0.000000000000,0.000047559533
```

In a blind test the makers send such packages, and each receiving team applies uncertainties of its own choosing. `sbtk.exchange.read_data` reads them with NaN uncertainties. The inversion then stops with `sbtk:data:noUncertainties` until uncertainties are given, for example with `sbtk.data.use_uncertainties` from a precision file and `sbtk.data.set_moi_sigma`, after truncating with `sbtk.data.select_degree` to the degrees the precision covers. A set with some uncertainties but not all is not written (`sbtk:data:noUncertainties`).

Interior-model package

File	Contents
`README.txt`	Summary (mass, bulk and background density, number of components, random field, density grid) and the list of files.
`interior_model.txt`	`name,type,relative_density_kgm3,density_kgm3,geometry` for the background and every component; metadata: mass, bulk and background density, the point layers and sectors are measured from, the random field, the principal axes of inertia (unit vectors in the shape-model axes) and the principal moments $A \le B \le C$ (Frames of a gravity field).
`density_grid.txt`	`x_km,y_km,z_km,density_kgm3,density_sigma_kgm3` : the true density at the cell centers (σ = 0), random field included.
`noise_free_data/`	The measurement files without noise.
`figures/`	`true_density_3d.png`, `true_density_slices.png`.

Output

```
# product: true interior model
# model: DOC-MOD1-Didymos
# data_set: DOC-DATA1-Didymos
# body: Didymos
# shape_model: didymos_g_9309mm_spc_obj_0000n00000_v003.obj (24578 vertices, 49152 facets, lengths in km)
# frame: that of the shape model file (noise_free_data/conventions.txt)
# mass_kg: 5.273000e+11
# bulk_density_kgm3: 2592.984
# background_density_kgm3: 2672.002
# center_of_mass_km: 0.015281 0.042885 -0.034053
# layers_and_sectors_from_km: 0.015281 0.042885 -0.034053 (the center of mass: layer depths and sector boundaries are measured along rays from this point)
# overlaps: relative densities of overlapping components add; components are clipped to the shape model
# principal_axes: x' = [0.835027009 -0.548783743 0.039576476], y' = [0.546420442 0.835555749 0.057195193], z' = [-0.064456144 -0.026134136 0.997578274] (unit
vectors in the axes of the shape model file; x' of the smallest, z' of the largest moment)
# principal_moments: A B C = 0.253270820 0.271737242 0.335835620 (I / (M R_ref^2), R_ref = 0.43 km)
# random_field: Gaussian, standard deviation 40.0 kg/m^3, correlation exp(-d^2/(2 L^2)) with L = 0.0600 km, seed 7 (SBTK sbtk.interior.random_field); included in
density_grid.txt
# relative_density: density minus background density
name,type,relative_density_kgm3,density_kgm3,geometry
background,background,0.000,2672.002,"everything not in a component"
core,sphere,450.000,3122.002,"center [15.9 42.3 -34.0] m; radius 170.0 m"
torus,torus,-500.000,2172.002,"ring 370.0 m; tube 55.0 m; ring center [15.9 42.3 -34.0] m; axis [0.0000 0.0000 1.0000]"
regolith,layer,-900.000,1772.002,"0.0 to 12.0 m below the surface"
```

Solution package

File	Contents
`README.txt`	Method, the χ^2 test of the fit and of a homogeneous body, the measurements used.
`solution.txt`	Background and each component: density and relative density with 1σ, estimated or prescribed, geometry.
`solution_covariance.txt`	Covariance of the estimated parameters.
`fitted_measurements.txt`	`label,measured,predicted,sigma,normalized_residual` ; degree-1 rows are the center of mass.
`density_grid.txt`	Estimated density and its 1σ at the cell centers, propagated from the parameter covariance.
`figures/`	Residuals, densities, 3-D view, slices.

Output

```
# product: estimated interior
# solution: DOC-SOL_DOC1
# data_set: DOC-DATA1-Didymos
# body: Didymos
# shape_model: didymos_g_9309mm_spc_obj_0000n00000_v003.obj (24578 vertices, 49152 facets, lengths in km)
# fit: chi2 24.69, degrees of freedom 24.0, reduced chi2 1.03, p-value 0.423 (consistent with the measurements at the 5% level)
# homogeneous_body: reduced chi2 44.86, p-value 3.61e-238
# measurements_used: C10 C11 S11 C20 C21 S21 C22 S22 C30 C31 S31 C32 S32 C33 S33 C40 C41 S41 C42 S42 C43 S43 C44 S44 Ixx Iyy Izz
# regularization: 0 (Tikhonov parameter)
# layers_and_sectors_from_km: 0.017644 0.041368 -0.035351 (the measured center of mass: layer depths and sector boundaries are measured along rays from this
point)
# relative_density: density minus background density
# uncertainties: 1-sigma; covariance in solution_covariance.txt
name,type,density_kgm3,density_sigma_kgm3,relative_density_kgm3,relative_density_sigma_kgm3,density,geometry
background,background,2665.863,129.767,0.000,0.000,estimated (total mass as a constraint),"everything not in a component"
core,sphere,3310.739,406.372,644.877,534.712,estimated,"center [15.9 42.3 -34.0] m; radius 170.0 m"
torus,torus,2252.418,52.225,-413.445,109.017,estimated,"ring 370.0 m; tube 55.0 m; ring center [15.9 42.3 -34.0] m; axis [0.0000 0.0000 1.0000]"
regolith,layer,1591.601,591.154,-1074.262,719.114,estimated,"0.0 to 12.0 m below the surface"
```

Density grid

Cell centers lie at integer multiples of `spacing_km` along each axis of the shape-model frame, so grids of the same spacing coincide regardless of who made them. A cell is listed when its center is inside the shape model, tested along rays from the center of figure, so all models of a body have the same cells. Each cell stands for the volume `spacing_km`³ and carries the density at its center. `sbtk.compare.grids` compares two grids (Comparison of density grids); the comparison of the example solution with its model:

Output

```
density grids: 3179 common cells (3179 true, 3179 estimated)
  R2 = 0.922 (fraction of the variance of the true density explained)
  RMS error 98.3 kg/m^3, mean error +1.2 kg/m^3, standard deviation of the true density 351.3 kg/m^3
  97.9% of the common cells have abs(estimate - true) <= 2 sigma of the estimate
```

conventions.txt

Every measurement package includes this file, written by `sbtk.internal.exchange_conventions`; the tests check the normalization code it prints. That of the maker's package above, whose shape model was moved and rotated:

Output

```
UNITS
  Lengths and positions in km, mass in kg, time in s, GM in km^3/s^2, densities in kg/m^3,
  angles in degrees.

FRAME
  All positions, the gravity field and the moments of inertia of this package are given
  in the frame of its shape model file, DOC-DATA2-Didymos_didymos_v003_com_spin_axes.obj:
    origin  the center of mass of the body
    z       the axis of the largest moment of inertia, so C21 = S21 = 0 (to rounding
            error); for a body in principal-axis rotation it is the spin axis
    x       the x-axis of the original file projected onto the plane normal to z
    y       completes the right-handed frame (y = z cross x)
  The shape model file was made by moving and rotating the original file,
    didymos_g_9309mm_spc_obj_0000n00000_v003.obj
  into this frame:
    x' = (x - o) A
    o = [0.015281496 0.042884748 -0.034053123] km  (the center of mass, in the original file)
    A = [  0.997920540676  0.000000000000 -0.064456143974
          -0.001688015795  0.999657019950 -0.026134136041
           0.064434036803  0.026188594157  0.997578273840 ]
  where x and x' are the coordinates of a point in the original file and in this one
  (row vectors, km), and o and the columns of A are the origin and the axes of this frame
  in the original file. Use this package's shape model file with its data, not the
  original file.

GRAVITY FIELD COEFFICIENT NORMALIZATION
  The files give fully normalized coefficients:
    Cbar_lm = C_lm / N_lm,   Sbar_lm = S_lm / N_lm,   Pbar_lm = N_lm P_lm,
    N_lm = sqrt( (2 - delta_m0) (2l + 1) (l - m)! / (l + m)! )
  where P_lm are the associated Legendre functions without the factor (-1)^m. For example,
    N_20 = sqrt(5), N_21 = sqrt(5/3), N_22 = sqrt(5/12), N_30 = sqrt(7).
  Zonal coefficients: J_l = -C_l0 = -sqrt(2l + 1) Cbar_l0.

  MATLAB
    N = sqrt((2 - (m == 0)) .* (2*l + 1) .* factorial(l - m) ./ factorial(l + m));
  Python
    N = math.sqrt((2 - (m == 0)) * (2*l + 1) * math.factorial(l - m) / math.factorial(l + m))

DEGREE 1 AND THE CENTER OF MASS
  The gravity field is expanded about the center of mass, so its degree-1 coefficients are
  zero and are not listed. The position of the center of mass in the frame of the shape
  model file is a separate measurement (center_of_mass.txt).
```

Other formats

Format	Function	Notes
PDS SHADR layout	`sbtk.exchange.read_shadr`	Header row then `l, m, C, S, sigma_C, sigma_S`; commas or spaces; `#` comments. The caller gives the shape model and the point the coefficients are about.
JPL SSD gravity file	`sbtk.io.read_jpl_gravity`	Full covariance; J_l converted.
Coefficient matrices	`sbtk.data.from_coefficients`	
Uncertainties of another data set	`sbtk.data.use_uncertainties`	Converted to the target reference radius by $(R / R')^l$.

Density models by component type

One example per component type and model feature. Each example after the first is a fragment: it uses the body and uncertainties defined in the setup script below, which must be run first. Building a true model and its exact values (`sbtk.data.simulate`) needs no uncertainties; only the examples that make measurements and estimate densities use `unc`. Geometry definitions: Density distributions.

Setup

MATLAB script: Body and uncertainties used by the examples on this page

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);    % mass [kg], R_ref [km]
unc  = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);   % for measurements
```

Sphere

A sphere is given by its center (shape-model frame, km), radius and relative density. A sphere entirely inside the body is integrated as a point mass. `sbtk.interior.latlon` gives a position by latitude, longitude and distance; added to the center of figure, as here, it is measured from there (Where to place components).

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
core    = sbtk.interior.sphere("core", body.centroid_km, 0.170, +450);
boulder = sbtk.interior.sphere("boulder", body.centroid_km + sbtk.interior.latlon(25, 60, 0.250), 0.040, +900);
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create([core, boulder]), 4, "spheres");
fprintf("center of mass - center of figure: [%.1f %.1f %.1f] m\n", 1e3*exact.com_minus_cof_km)
```

Ellipsoid

Semi-axes along x, y, z before the rotation by yaw (about z), pitch (about y) and roll (about x), in degrees.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
lens = sbtk.interior.ellipsoid("lens", body.centroid_km + [0.05 0.10 0.12], [0.20 0.08 0.05], [40 -30 15], +300);
disp(lens.axes)                                  % columns: the ellipsoid axes in the shape-model frame
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(lens), 4, "ellipsoid");
sbtk.plot.slices(body, {truth}, {'ellipsoid'});
```

Torus

A torus about the z-axis: ring radius, tube radius, height of its plane. Only the part inside the body counts, so a torus that crosses the surface at the equator represents a belt of different density along the equator. Set its `center_km` to place it about the center of figure, and align it with the spin axis with `sbtk.interior.align_with_spin_axis` (Axes of components).

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
torus = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
torus.center_km = body.centroid_km;                          % about the z-axis through the center of figure
geo = sbtk.gravity.geometry(body, body.centroid_km, 0);
I = sbtk.gravity.component_integrals(geo, torus);
fprintf("torus volume inside the body: %.5f km^3 (%.1f%% of the body)\n", I.V, 100*I.V/body.volume_km3)
```

Layer

Top and bottom depth below the surface, along rays from the expansion origin.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
regolith = sbtk.interior.layer("regolith", 0, 0.012, -900);          % the outermost 12 m
mantle   = sbtk.interior.layer("mantle", 0.012, 0.080, -200);        % from 12 m to 80 m deep
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create([regolith, mantle]), 4, "layers");
fprintf("background density %.0f kg/m^3\n", truth.rho_bg_kgm3)
```

Sector

Radial fractions of the distance from the expansion origin to the surface, a latitude range and a longitude range (degrees). Longitude ranges may extend beyond ±180°.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
block = sbtk.interior.sector("block", [0.3 0.9], [40 90], [150 240], +200);    % northern block across 180 deg
shell = sbtk.interior.sector("shell", [0.5 0.75], [-90 90], [-180 180], -300); % a shell that follows the shape
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create([block, shell]), 4, "sectors");
sbtk.plot.slices(body, {truth}, {'sectors'});
```

Sector grid

A grid of sectors that fills the body is a model without assumed structure. As an estimation model it needs the background held at the bulk density and, with many sectors, regularization. Here the true interior is a dense core and the estimate uses 36 sectors.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(sbtk.interior.sphere("core", body.centroid_km, 0.17, +450)), ...
                                    4, "core");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);       % measurements of it
sectors = sbtk.interior.sector_grid([0 0.5 0.85 1], [-90 -30 30 90], [-180 -90 0 90 180]);   % 3 x 3 x 4
prob = sbtk.invert.setup(body, sectors, obs, sbtk.invert.options(background="bulk"));
prob.opts.regularization = sbtk.invert.choose_regularization(prob);
fit = sbtk.invert.run(prob);
fprintf("lambda %.3g, reduced chi2 %.2f, dof %.1f\n", fit.regularization, fit.chi2_reduced, fit.dof)
sbtk.plot.slices(body, {truth, fit}, {'true density', 'sector estimate'});
```

Random density field

Standard deviation (kg/m³), correlation length (km) and seed. The field is added to the components; it is used in simulations and is not estimated by the inversion.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
field = sbtk.interior.random_field(40, 0.06, 7);                    % sigma 40 kg/m^3, correlation length 60 m
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(sbtk.interior.no_components(), field), 4, "field");
grid = sbtk.exchange.density_grid(body, truth, 0.020);              % density at 20 m cell centers inside the body
fprintf("%d cells: mean %.0f, standard deviation %.1f kg/m^3\n", numel(grid.density_kgm3), ...
        mean(grid.density_kgm3), std(grid.density_kgm3))
```

Homogeneous body

`sbtk.interior.no_components` without a random field gives a uniform density M / V.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(sbtk.interior.no_components()), 4, "uniform");
fprintf("background %.1f kg/m^3, bulk density %.1f kg/m^3\n", truth.rho_bg_kgm3, body.bulk_density_kgm3)
```

Prescribed component

A component with `estimate_density = false` keeps its relative density in the inversion; here a regolith layer of known density while the core is estimated.

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
truth_components = [sbtk.interior.sphere("core", body.centroid_km, 0.17, +450), sbtk.interior.layer("regolith", 0, 0.012, -900)];
exact = sbtk.data.simulate(body, sbtk.interior.create(truth_components), 4, "prescribed");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
model = truth_components;
model(1).drho_kgm3 = 0;                  % estimated: the value is not used
model(2).estimate_density = false;       % prescribed: -900 kg/m^3 is kept
fit = sbtk.invert.solve(body, model, obs);
sbtk.invert.summary(fit)
```

Estimated sphere position

The center of a sphere with `estimate_position = true` is estimated with its density; `position_sigma_km` adds an a priori constraint about the starting position. The center, height and orientation of tori and ellipsoids can be estimated the same way (Estimated geometry).

MATLAB fragment. Requires the variables defined in Body and uncertainties used by the examples on this page.

```
truth_components = sbtk.interior.sphere("core", body.centroid_km + [0.03 -0.02 0.01], 0.17, +500);
exact = sbtk.data.simulate(body, sbtk.interior.create(truth_components), 4, "position");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
model = sbtk.interior.sphere("core", body.centroid_km, 0.17, 0);       % started at the center of figure
model.estimate_position = true;
fit = sbtk.invert.solve(body, model, obs, sbtk.invert.options(position_sigma_km=0.050));
fprintf("center [%.1f %.1f %.1f] m from the center of figure +- [%.1f %.1f %.1f] m after %d iterations\n", ...
        1e3*(fit.components(1).center_km - body.centroid_km), 1e3*fit.center_sigma_km(1, :), fit.iterations)
```

Calculations

Short examples of individual calculations. Fragments on this page use the variables of the setup script below.

Setup

MATLAB script: Body, interior and measurements used by the examples on this page

```
run("/path/to/SBTK/sbtk_setup.m")
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
components = [sbtk.interior.sphere("core", body.centroid_km, 0.170, +450), sbtk.interior.layer("regolith", 0, 0.012, -900)];
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(components), 4, "calculations");   % the true model
unc  = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
exact = sbtk.data.set_uncertainties(exact, unc);                    % exact values with uncertainties
obs = sbtk.data.add_noise(exact, 1);                                % one noisy realization
```

Unit conversions

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
Izz = sbtk.units.moi_to_norm(1.6e16, body.mass_kg, body.R_ref_km)    % kg m^2 -> I / (M R_ref^2)
I_kgm2 = sbtk.units.norm_to_moi(Izz, body.mass_kg, body.R_ref_km)    % and back
c = sbtk.units.offset_to_degree1(0.002, body.R_ref_km)               % 2 m center-of-mass offset -> degree 1
f = sbtk.units.length_scale("m")                                     % m -> km: 1e-3
```

Volume and moments of one component

`sbtk.gravity.component_integrals` returns the integrals of the part of a component inside the body, per unit density, about the origin of the geometry.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
geo = sbtk.gravity.geometry(body, truth.origin_km, 4);
I = sbtk.gravity.component_integrals(geo, components(2));            % the regolith layer
fprintf("regolith volume %.5f km^3 = %.1f%% of the body\n", I.V, 100*I.V/geo.V_body)
mass_deficit_kg = components(2).drho_kgm3*I.V*1e9                   % its mass relative to the background
```

Coefficients and inertia about another origin

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
geo0 = sbtk.gravity.geometry(body, [0 0 0], 4);                      % about the shape-model origin
fm = sbtk.gravity.forward(geo0, sbtk.interior.create(components), body.mass_kg);
lab = sbtk.gravity.labels(4);
fprintf("C11 about the shape origin %.4e; about the center of mass %.1e\n", ...
        fm.coef(strcmp(lab, "C11")), truth.coef(strcmp(lab, "C11")))
```

Difference from the homogeneous body

The gravity coefficients of the true model and of the measurements, minus those of a body of uniform density, with the 3σ of the measurements (Figures). The second output holds the plotted values; `axes_deg` gives the angles between the principal axes of each set and of the homogeneous body.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
[fig, d] = sbtk.plot.difference_from_homogeneous(body, {obs, truth}, ["measured" "true model"], "linear", "shape");
fprintf("%s: %+.2e\n", [d(2).labels(1:4); string(d(2).difference(1:4))])   % C10, C11, S11, J2 of the true model
sbtk.plot.difference_from_homogeneous(body, {obs, truth}, ["measured" "true model"], "log", "shape");
[~, dp] = sbtk.plot.difference_from_homogeneous(body, {truth}, "true model", "linear", "principal");
fprintf("principal axes of the true model vs the homogeneous body: tilt %.2f deg, turn %.2f deg\n", dp.axes_deg)
```

Field about the center of mass, in spin and principal axes

`sbtk.gravity.center_of_mass_frame` moves a field to its center of mass (degree 1 becomes 0) and, with `"spin"`, turns its axis of the largest moment to z ($\bar{C}_{21} = \bar{S}_{21} = 0$) or, with `"principal"`, rotates it into its principal axes of inertia ($\bar{C}_{21} = \bar{S}_{21} = \bar{S}_{22} = 0$) (Frames of a gravity field).

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
cs = sbtk.gravity.center_of_mass_frame(truth, body, "shape");
cz = sbtk.gravity.center_of_mass_frame(truth, body, "spin");
cp = sbtk.gravity.center_of_mass_frame(truth, body, "principal");
disp(table(cs.labels(1:8)', cs.coef(1:8)', cz.coef(1:8)', cp.coef(1:8)', 'VariableNames', ...
          ["coefficient" "shape axes" "spin axes" "principal axes"]))
fprintf("axis of the largest moment %.2f deg from the shape-model z-axis\n", acosd(cp.principal(3, 3)))
mo = sbtk.gravity.center_of_mass_frame(obs, body, "principal");      % a measurement set: with its covariance
fprintf("C22 in principal axes %.4e +- %.1e\n", mo.coef(mo.labels == "C22"), mo.sigma(mo.labels == "C22"))
```

A body and its true model in another frame

`sbtk.body.to_frame` and `sbtk.interior.to_frame` carry the shape model and the true model into the frame with the center of mass as origin and the spin axes; the density grid is then on cells of that frame (Frames of a gravity field).

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
spin = sbtk.gravity.center_of_mass_frame(truth, body, "spin");
body_s  = sbtk.body.to_frame(body, spin.com_km, spin.axes, "didymos_v003_com_spin_axes");
truth_s = sbtk.interior.to_frame(truth, spin.com_km, spin.axes);
fprintf("center of mass %s km in the shape-model file, %s km in the new frame\n", mat2str(truth.com_km, 4), ...
        mat2str(truth_s.com_km, 4))
fprintf("center of figure %s km in the new frame\n", mat2str(body_s.centroid_km, 4))
grid_s = sbtk.exchange.density_grid(body_s, truth_s, 0.040);
fprintf("%d cells of 40 m in the spin axes\n", numel(grid_s.density_kgm3))
```

Density at points and on a grid

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
geo = sbtk.gravity.geometry(body, truth.origin_km, 0);              % geometry in the model's frame
x = linspace(-0.3, 0.3, 61)';
rho = sbtk.interior.density_at(truth, truth.origin_km + [x, 0*x, 0*x], geo);
plot(1e3*x, rho); xlabel("x from the center of mass [m]"); ylabel("density [kg/m^3]");

grid = sbtk.exchange.density_grid(body, truth, 0.040);              % 40 m cells
fprintf("%d cells, mean density %.0f kg/m^3\n", numel(grid.density_kgm3), mean(grid.density_kgm3))
```

Uncertainties from another data set

`sbtk.data.use_uncertainties` gives simulated coefficients the covariance of a reference data set, converted to the target reference radius. Here the reference has $R_{\mathrm{ref}} = 0.40$ km.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
L = 4;
[C, S] = deal(zeros(L + 1));  C(1, 1) = 1;
sig = 5e-5*ones(L + 1);
reference = sbtk.data.from_coefficients(sbtk.body.from_mass("Didymos", body.shape, body.mass_kg, 0.40), ...
                                        "precision", [0 0 0], C, S, sig, sig);
noisy = sbtk.data.add_noise(sbtk.data.use_uncertainties(exact, reference), 2);
fprintf("1-sigma of C20: %.3e (reference %.1e at 0.40 km)\n", sqrt(noisy.coef_cov(4, 4)), 5e-5)
```

Inertia measurements

`sbtk.data.select_moi` chooses the inertia rows of a data set; `opts.moi_labels` the rows a fit uses. `sbtk.data.libration_ratio_from_amplitude` converts a libration amplitude into a libration ratio.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
gravity_only = sbtk.data.select_moi(obs, {});
with_izz = sbtk.data.add_moi(gravity_only, "Izz", exact.moi(3).value, 2e-3);
with_lib = sbtk.data.add_libration(gravity_only, exact.libration_ratio, 0.001, exact.moi(3).value);
[k, sk] = sbtk.data.libration_ratio_from_amplitude(-12, 0.2, 0.027, 0.001)   % G [deg], its sigma, e, its sigma

f0 = sbtk.invert.solve(body, components, gravity_only, sbtk.invert.options(moi_labels={}));
f1 = sbtk.invert.solve(body, components, with_izz, sbtk.invert.options(moi_labels={'Izz'}));
f2 = sbtk.invert.solve(body, components, with_lib, sbtk.invert.options(moi_labels={'libration'}));
fprintf("regolith density 1-sigma: gravity only %.0f, with Izz %.0f, with the libration ratio %.0f kg/m^3\n", ...
        f0.rho_sigma_kgm3(2), f1.rho_sigma_kgm3(2), f2.rho_sigma_kgm3(2))
```

Measured center of mass

A gravity field about the center of mass has no degree-1 terms; `sbtk.data.add_center_of_mass` adds the measured position, after which the inversion expands about the model's own center of mass (Density estimation).

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
field = sbtk.data.select_coefficients(obs, setdiff(obs.coef_labels, {'C10', 'C11', 'S11'}, "stable"));
field = sbtk.data.add_center_of_mass(field, obs.origin_km + [0.0005 0 0], 0.002);   % measured 0.5 m off
fit = sbtk.invert.solve(body, components, field);
fprintf("frame iterations %d, model center of mass − origin: [%.2f %.2f %.2f] m\n", fit.frame_iterations, 1e3*fit.com_shift_km)
```

Reading a SHADR-layout gravity file

The example writes a small file in the PDS SHADR layout and reads it back. The coefficients are about the shape-model origin, so the file includes degree-1 terms and `origin_km = [0 0 0]`.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
geo0 = sbtk.gravity.geometry(body, [0 0 0], 2);
fm = sbtk.gravity.forward(geo0, sbtk.interior.create(components), body.mass_kg);
lab = sbtk.gravity.labels(2);
coef = @(name) fm.coef(strcmp(lab, name));
file = [tempname '.tab'];
rows = sprintf("%.4f, %.6e, 0, 2, 2, 1, 0, 0", body.R_ref_km, body.GM_km3_s2);   % R, GM, sigma GM, degree, order, normalized
rows(2) = sprintf("1, 0, %.10e, 0, 1e-5, 1e-5", coef("C10"));
rows(3) = sprintf("1, 1, %.10e, %.10e, 1e-5, 1e-5", coef("C11"), coef("S11"));
rows(4) = sprintf("2, 0, %.10e, 0, 1e-5, 1e-5", coef("C20"));
rows(5) = sprintf("2, 1, %.10e, %.10e, 1e-5, 1e-5", coef("C21"), coef("S21"));
rows(6) = sprintf("2, 2, %.10e, %.10e, 1e-5, 1e-5", coef("C22"), coef("S22"));
writelines(rows, file);
[shadr, shadr_body] = sbtk.exchange.read_shadr(file, "Didymos", body.shape, [0 0 0], 2);
sbtk.data.summary(shadr)
```

Packages for another team

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
folder = tempname;
data  = sbtk.exchange.write_data(folder, "TEAM-DATA1-Didymos", body, obs, "Team name");
model = sbtk.exchange.write_model(folder, "TEAM-MOD1-Didymos", "TEAM-DATA1-Didymos", body, truth, exact, 0.040, "Team name");
[obs2, body2] = sbtk.exchange.read_data(data);                      % what the other team reads
prob = sbtk.invert.setup(body2, components, obs2, sbtk.invert.options());
solution = sbtk.exchange.write_solution(folder, sbtk.exchange.solution_name("OTHER", obs2.name), body2, ...
                                        sbtk.invert.run(prob), sbtk.invert.homogeneous(prob), 0.040, "Other team");
cmp = sbtk.compare.grids(sbtk.exchange.read_grid(fullfile(model, "density_grid.txt")), ...
                         sbtk.exchange.read_grid(fullfile(solution, "density_grid.txt")));
sbtk.compare.grid_summary(cmp);
```

Many fits against one true model

`sbtk.compare.reference` samples the true model once; `sbtk.compare.evaluate` then compares each fit in milliseconds. `sbtk.invert.replace_data` reuses the setup for each noise realization.

MATLAB fragment. Requires the variables defined in Body, interior and measurements used by the examples on this page.

```
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
ref = sbtk.compare.reference(body, truth, components, prob.geo.origin_km);
R2 = zeros(1, 20);
for k = 1:20
    fit = sbtk.invert.run(sbtk.invert.replace_data(prob, sbtk.data.add_noise(exact, 10 + k)));
    R2(k) = sbtk.compare.evaluate(ref, fit).R2;
end
fprintf("R2 over 20 noise realizations: %.2f +− %.2f\n", mean(R2), std(R2))
```

Tutorial scripts

The repository folder `examples/` contains eight tutorial scripts with sections. Each adds `toolbox/` to the path itself and can be run whole or section by section. All eight run in the test suite (`tests/integration/test_examples.m`, also section by section for ex01). The code below is included from the repository files.

Script	Topic	Run time
`ex01_quick_start.m`	Body, interior, simulation, inversion, comparison on the 1.165 m Didymos model	about 90 s
`ex02_shapes_and_bodies.m`	Shape models from files, arrays and ellipsoids; checks; bodies	about 5 s
`ex03_forward_model.m`	Every component type, a random field, the simulated measurements, plots	about 20 s
`ex04_inversion.m`	Homogeneous test, inertia data, sphere positions, regularization, calibration	about 30 s
`ex05_import_data.m`	Measurements from matrices, kg m², a full covariance, the text format	about 5 s
`ex06_results_folders.m`	Run folders, reports, logs, settings	about 30 s
`ex07_exchange_packages.m`	Measurement, interior-model and solution packages	about 40 s
`ex08_moments_of_inertia.m`	What inertia data and the libration ratio add to the gravity field	about 20 s

The same folder has six maker models, `make_didymos_1.m` to `make_dimorphos_3.m`: true models of Didymos and Dimorphos written as exchange packages for the Hera blind test (Maker models).

ex01: quick start

The sequence of the first script on the full-resolution Didymos model (3,145,728 facets), which is not in the repository (Installation). With `"didymos"` instead of `"didymos_highres"` it runs in about 15 s.

MATLAB script: examples/ex01_quick_start.m

```
%% EX01_QUICK_START  Simulate measurements of an interior, invert them, compare with the true model. (~90 s with the 1.165 m shape model)
%
% The SBTK workflow on the Didymos shape model:
%   1. a body: shape model, bulk density, reference radius
%   2. forward model: components -> gravity coefficients and moments of inertia
%   3. inversion: measurements + component geometry -> densities with 1-sigma uncertainties
%   4. comparison with the true model

addpath(fullfile(fileparts(fileparts(which('ex01_quick_start'))), 'toolbox'));

%% 1. Body
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos_highres"), 5.273e11, 0.43);   % mass [kg], R_ref [km] (Hera end-to-end test)
sbtk.body.summary(body)

%% 2. Forward model: a dense core and a low-density surface layer
components = [sbtk.interior.sphere("core", body.centroid_km, 0.170, +450), ...  % center [km] (the center of figure), radius [km], relative density [kg/m^3]
              sbtk.interior.layer("regolith", 0, 0.012, -900)];           % from 0 to 12 m below the surface
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(components), 4, "quick_start");   % exact values

% measurements: 1-sigma uncertainties, then one noise realization
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), ...  % degree 1: 2 m center of mass
                               1e-4, 2e-4, 4e-4], ...                          % degrees 2, 3, 4
                              2e-3, ...                                        % each MOI / (M R_ref^2)
                              0.02);                                           % libration ratio
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
sbtk.data.summary(obs)

%% 3. Inversion: the same components, relative densities unknown
% (the relative densities given with the components are not used: they are estimated)
fit = sbtk.invert.solve(body, components, obs);
sbtk.invert.summary(fit)

%% 4. Comparison with the true model
cmp = sbtk.compare.with_truth(body, truth, fit);
sbtk.compare.summary(cmp)
sbtk.plot.densities(fit);
sbtk.plot.truth_vs_estimate(body, truth, fit);
```

ex02: shapes and bodies

Shape models from an OBJ file with a stated unit, from arrays (a tetrahedron in meters) and as an ellipsoid; the checks of `sbtk.shape.properties`; writing OBJ; a body; the unit table.

MATLAB script: examples/ex02_shapes_and_bodies.m

```
%% EX02_SHAPES_AND_BODIES  Shape models from files or arrays, units, checks. (~5 s)
%
% Any closed polyhedral shape works if it is star-shaped about its center of
% mass (every facet visible from it). Lengths are converted to km on input:
% the unit of the file must be given.

addpath(fullfile(fileparts(fileparts(which('ex02_shapes_and_bodies'))), 'toolbox'));

%% From an OBJ file (unit stated explicitly)
didymos = sbtk.shape.load(fullfile(sbtk.root, "data", "shapes", "didymos_g_9309mm_spc_obj_0000n00000_v003.obj"), "km");
% the shape models that ship with SBTK also load by name:
%   sbtk.shape.bundled("didymos"), ("bennu"), ("dimorphos_placeholder"), ("example_asteroid")
props = sbtk.shape.properties(didymos);
fprintf('%s: %d facets, volume %.5f km^3, centroid [%.1f %.1f %.1f] m, star-shaped: %d\n', didymos.name, ...
        props.n_facets, props.volume_km3, 1e3*props.centroid_km, props.star_shaped_about_centroid);

%% From arrays (here a tetrahedron given in meters)
V = [0 0 0; 100 0 0; 0 100 0; 0 0 100] - 25;          % vertices [m]
F = [1 3 2; 1 2 4; 1 4 3; 2 3 4];                      % facets, outward
tetra = sbtk.shape.from_arrays("tetra", F, V, "m");
disp(sbtk.shape.properties(tetra))

%% A triaxial ellipsoid (exact test shape), saved to OBJ
ell = sbtk.shape.ellipsoid("test_ellipsoid", [0.40 0.35 0.30], 4);     % 5,120 facets
file = fullfile(tempdir, 'test_ellipsoid.obj');
sbtk.shape.write_obj(ell, file);
ell2 = sbtk.shape.load(file, "km");

%% Bodies: shape + bulk density + reference radius of the gravity field
body = sbtk.body.create("Ellipsoid", ell2, 2200, 0.40);
sbtk.body.summary(body)
sbtk.plot.shape(body);
sbtk.units.table()
```

---

ex03: forward model

Every component type and a random field, the simulated measurements as a table, the density at given points, and the three-dimensional, slice and measurement figures.

MATLAB script: examples/ex03_forward_model.m

```
%% EX03_FORWARD_MODEL  Every component type, a random density field, and the simulated measurements. (~20 s)
%
% Components are regions of uniform relative density (density minus the
% background density). Overlapping relative densities add and components are
% clipped to the body. The background density follows from the total mass.

addpath(fullfile(fileparts(fileparts(which('ex03_forward_model'))), 'toolbox'));
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);

%% Components
% Spheres, ellipsoids and tori are placed in shape-model coordinates, here from the center of
% figure (the origin of this shape-model file is 57 m from it); layers and sectors are
% measured from the center of mass.
cof = body.centroid_km;
components = [ ...
  sbtk.interior.sphere("core", cof, 0.170, +250), ...                                    % center, radius
  sbtk.interior.ellipsoid("lens", cof + [0.05 0.10 0.12], [0.20 0.08 0.05], [40 -30 15], +300), ... % center, semi-axes, yaw/pitch/roll
  sbtk.interior.torus("torus", 0.370, 0.055, 0.000, -500), ...                           % ring radius, tube radius, height
  sbtk.interior.layer("regolith", 0.000, 0.012, -900), ...                               % top, bottom depth
  sbtk.interior.sector("block", [0.3 0.9], [40 90], [150 240], +200), ...                % radial fraction, lat, lon
  sbtk.interior.sphere("boulder", cof + sbtk.interior.latlon(25, 60, 0.250), 0.040, +900)];   % position from lat/lon/distance
components(3).center_km = cof;                                                           % the torus about the z-axis through cof
sbtk.interior.summary(components)

%% Gaussian random density field (standard deviation 40 kg/m^3, correlation length 60 m, seed 7)
interior = sbtk.interior.create(components, sbtk.interior.random_field(40, 0.06, 7));

%% Forward model: the true model and its exact gravity coefficients and moments of inertia
[exact, truth, geo] = sbtk.data.simulate(body, interior, 4, "example_interior");

%% Measurements of it: uncertainties, then noise drawn from them
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
fprintf('background density %.0f kg/m^3, center of mass %.1f m from the center of figure\n', ...
        truth.rho_bg_kgm3, 1e3*norm(obs.com_minus_cof_km));

% What the measurements look like
disp(table(obs.coef_labels(1:8), obs.coef(1:8), sqrt(diag(obs.coef_cov(1:8, 1:8))), ...
           'VariableNames', {'label', 'value', 'sigma'}))
disp(struct2table(obs.moi))

%% Density anywhere inside (truth frame = center of mass)
rho = sbtk.interior.density_at(truth, [0 0 0; 0.3 0 0] + truth.origin_km, geo);
fprintf('density at the center %.0f, 300 m out along x %.0f kg/m^3\n', rho);

%% Plots
sbtk.plot.interior3d(body, truth, "example interior");
sbtk.plot.slices(body, {truth}, {'truth'});
sbtk.plot.measurements(obs);
```

ex04: inversion options

The homogeneous test; fits with and without inertia data; estimation of a sphere center; a 36-sector grid with Tikhonov regularization chosen by the discrepancy principle; and the standard deviation of (estimate − truth)/σ over 50 noise realizations using `sbtk.invert.replace_data`.

MATLAB script: examples/ex04_inversion.m

```
%% EX04_INVERSION  Inversion options: homogeneous model, MOI, sphere positions, regularization. (~30 s)
%
% setup does the slow part once (geometry and each component's effect on every
% measurement); run solves in milliseconds, so trying options is cheap.

addpath(fullfile(fileparts(fileparts(which('ex04_inversion'))), 'toolbox'));
body  = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
unc   = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
cof   = body.centroid_km;                  % components are placed from the center of figure
truth_components = [sbtk.interior.sphere("core", cof + [0.03 -0.02 0.01], 0.17, +500), ...
              sbtk.interior.torus("torus", 0.37, 0.055, 0, -500), sbtk.interior.layer("regolith", 0, 0.012, -900)];
truth_components(2).center_km = cof;       % the torus about the z-axis through the center of figure
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(truth_components), 4, "ex04");
exact = sbtk.data.set_uncertainties(exact, unc);
obs = sbtk.data.add_noise(exact, 1);

%% Set up once
% Component geometry is fixed (the core is placed at the center of figure, 37 m from the true
% center); relative densities are estimated, so their values here are not used.
components = [sbtk.interior.sphere("core", cof, 0.17, 0), sbtk.interior.torus("torus", 0.37, 0.055, 0, 0), ...
             sbtk.interior.layer("regolith", 0, 0.012, 0)];
components(2).center_km = cof;
opts = sbtk.invert.options();             % every setting and its standard value
disp(opts)
prob = sbtk.invert.setup(body, components, obs, opts);

%% Is a homogeneous body consistent with the measurements?
u = sbtk.invert.homogeneous(prob);
fprintf('homogeneous body: reduced chi2 %.2f, p = %.2g\n', u.chi2_reduced, u.p_value);

%% Fit, and the same with gravity only (no MOI): MOI shrink the uncertainties
fit = sbtk.invert.run(prob);
sbtk.invert.summary(fit)
fit_g = sbtk.invert.solve(body, components, obs, sbtk.invert.options(moi_labels={}));
fprintf('core 1-sigma: %.0f kg/m^3 with MOI, %.0f without\n', fit.rho_sigma_kgm3(1), fit_g.rho_sigma_kgm3(1));

%% Estimate the core position as well
moving = components;  moving(1).estimate_position = true;
fit_p = sbtk.invert.solve(body, moving, obs);
fprintf('core center [%.0f %.0f %.0f] m +- [%.0f %.0f %.0f] m (truth [30 -20 10] m from the shape origin)\n', ...
        1e3*fit_p.components(1).center_km, 1e3*fit_p.center_sigma_km(1, :));

%% Tikhonov regularization for many components (36-sector grid), parameter by the discrepancy principle
% The sectors fill the body, so the background is held at the bulk density.
sectors = sbtk.interior.sector_grid([0 0.5 0.85 1], [-90 -30 30 90], [-180 -90 0 90 180]);
probc = sbtk.invert.setup(body, sectors, obs, sbtk.invert.options(background="bulk"));
probc.opts.regularization = sbtk.invert.choose_regularization(probc);
fitc = sbtk.invert.run(probc);
fprintf('sector grid: lambda %.3g, reduced chi2 %.2f (dof %.1f)\n', fitc.regularization, fitc.chi2_reduced, fitc.dof);

%% Calibration of the uncertainties: 50 noise realizations
z = zeros(50, 4);
for k = 1:50
    f = sbtk.invert.run(sbtk.invert.replace_data(prob, sbtk.data.add_noise(exact, 100 + k)));
    z(k, :) = ([f.rho_bg_kgm3, [f.components.drho_kgm3]] - [truth.rho_bg_kgm3, [truth_components.drho_kgm3]]) ...
              ./[f.rho_bg_sigma_kgm3, f.drho_sigma_kgm3];
end
fprintf('standard deviation of (estimate - truth)/sigma: %.2f (1 for calibrated uncertainties)\n', std(z(:)));
sbtk.plot.residuals(fit);
```

ex05: importing data

A measurement set from coefficient matrices; inertia given in kg m²; a libration ratio; a full covariance; the text format written and read back; selection of degrees.

MATLAB script: examples/ex05_import_data.m

```
%% EX05_IMPORT_DATA  Measurements from arrays, kg m^2, a full covariance, and text files. (~5 s)
%
% Real data arrive as coefficient matrices with uncertainties (and maybe a full
% covariance) and as moments of inertia. This builds an SBTK measurement set
% from them, in the units SBTK uses.

addpath(fullfile(fileparts(fileparts(which('ex05_import_data'))), 'toolbox'));
body = sbtk.body.from_mass("Dimorphos", sbtk.shape.bundled("dimorphos"), 2.700e9, 0.10);

%% Coefficient matrices: C(l+1, m+1) = C_lm, fully normalized about origin_km
L = 2;
C = zeros(L + 1);  S = zeros(L + 1);
C(1,1) = 1;  C(3,1) = -0.08;  C(3,3) = 0.03;  S(3,3) = 0.002;     % C00, C20, C22, S22
Cs = 1e-3*ones(L + 1);  Ss = 1e-3*ones(L + 1);                     % 1-sigma of each
obs = sbtk.data.from_coefficients(body, "example_import", [0 0 0], C, S, Cs, Ss);

%% Moments of inertia, given in kg m^2 -> I/(M R_ref^2)
Izz_kgm2 = 1.6e14;
Izz = sbtk.units.moi_to_norm(Izz_kgm2, body.mass_kg, body.R_ref_km);
obs = sbtk.data.add_moi(obs, "Izz", Izz, 0.01*Izz);
obs = sbtk.data.add_libration(obs, 0.20, 0.03, Izz);               % (B-A)/C and its 1-sigma

%% A full covariance (here: 30% correlation between all coefficients)
sig = sqrt(diag(obs.coef_cov));
corr = 0.3*ones(numel(sig)) + 0.7*eye(numel(sig));
obs = sbtk.data.set_covariance(obs, corr.*(sig*sig'));
sbtk.data.summary(obs)

%% The SBTK text format: write it, read it back
% (for data to send to other teams see ex07_exchange_packages.m)
file = fullfile(tempdir, 'example_measurements.txt');
sbtk.io.write_measurements(obs, file);
type(file)
back = sbtk.io.load_measurements(file);     % (the text keeps only the diagonal of the covariance)

%% Use fewer degrees
obs1 = sbtk.data.select_degree(obs, 1);
fprintf('%d coefficients up to degree 1\n', numel(obs1.coef));
```

ex06: results folders

A simulation, an inversion of the newest simulation found with `sbtk.io.latest_run`, and a comparison, each in its own run folder with log, settings and figures (Results folders).

MATLAB script: examples/ex06_results_folders.m

```
%% EX06_RESULTS_FOLDERS  Standard outputs: run folders, reports, logs, settings. (~30 s)
%
% Every run gets results/<project>/<date_time>_<name>/ with standard file names,
% settings.json (every number used) and log.txt (everything printed). The
% report_* functions save, print and plot the standard set in one call.

addpath(fullfile(fileparts(fileparts(which('ex06_results_folders'))), 'toolbox'));
shape = sbtk.shape.ellipsoid("ellipsoid", [0.40 0.35 0.30], 4);
body  = sbtk.body.create("Ellipsoid", shape, 2200, 0.40);
unc   = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.40), 1e-4, 2e-4], 2e-3, 0.02);

%% Simulation
components = [sbtk.interior.sphere("core", [0.03 0 0], 0.15, +600), sbtk.interior.layer("regolith", 0, 0.02, -700)];
out = sbtk.io.new_run("examples", "ellipsoid_T1");
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(components), 3, "ellipsoid_T1");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
sbtk.io.report_simulation(out, body, obs, truth);
sbtk.io.finish_run(out);

%% Inversion of the newest simulation
obs = sbtk.io.load_measurements(sbtk.io.latest_run("examples", "ellipsoid_T1"));
out = sbtk.io.new_run("examples", "ellipsoid_T1_inversion");
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
fit = sbtk.invert.run(prob);
sbtk.io.report_inversion(out, body, fit, sbtk.invert.homogeneous(prob));
sbtk.io.finish_run(out);

%% Comparison with the true model
truth = sbtk.io.load_truth(sbtk.io.latest_run("examples", "ellipsoid_T1"));
out = sbtk.io.new_run("examples", "ellipsoid_T1_comparison");
sbtk.io.report_comparison(out, body, truth, fit, sbtk.compare.with_truth(body, truth, fit));
sbtk.io.finish_run(out);
dir(out.folder)
```

ex07: exchange packages

A blind test in miniature: one team writes a measurement package and an interior-model package; another reads the measurement package, inverts it and writes a solution package; the solution is compared with the hidden model on the density grid (Exchange packages).

MATLAB script: examples/ex07_exchange_packages.m

```
%% EX07_EXCHANGE_PACKAGES  Data for other teams: measurement, interior-model and solution packages. (~40 s)
%
% A blind test: one team simulates measurements of an interior it keeps hidden and
% sends only the measurement package, of exact values; another team gives them its own
% uncertainties, inverts them and sends a
% solution package; at the end both are compared with the hidden model on a common
% density grid. Every package is plain text with its units, frame and normalization
% written in it (docs/exchange_format.md). projects/hera_hlp6 does the same with
% results folders.

addpath(fullfile(fileparts(fileparts(which('ex07_exchange_packages'))), 'toolbox'));
body = sbtk.body.create("Example asteroid", sbtk.shape.bundled("example_asteroid"), 2000, 0.60);
unc  = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.005, 0.60), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
folder = tempname;                        % where the packages go (normally a results folder)

%% The team that makes the data
hidden = [sbtk.interior.sphere("core", body.centroid_km + [0.03 0 0], 0.25, +500), ...   % 30 m from the center of figure
          sbtk.interior.layer("regolith", 0, 0.02, -800)];
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(hidden), 4, "hidden_model");
data = sbtk.exchange.write_data(folder, "TEAM-DATA1-example", body, exact, "Team, contact");       % send now: exact values
model = sbtk.exchange.write_model(folder, "TEAM-MOD1-example", "TEAM-DATA1-example", body, truth, ...
                                  exact, 0.04, "Team, contact");                                  % send at the end
type(fullfile(data, 'README.txt'))

%% The team that inverts it: everything comes from the package
[obs2, body2] = sbtk.exchange.read_data(data);          % shape model, GM and reference radius included
obs2 = sbtk.data.add_noise(sbtk.data.set_uncertainties(obs2, unc), 1);   % its own uncertainties (and noise, optionally)
components = [sbtk.interior.sphere("core", body2.centroid_km, 0.25, 0), sbtk.interior.layer("regolith", 0, 0.02, 0)];
prob = sbtk.invert.setup(body2, components, obs2, sbtk.invert.options());
fit  = sbtk.invert.run(prob);
sbtk.invert.summary(fit)
solution = sbtk.exchange.write_solution(folder, sbtk.exchange.solution_name("OTHER", obs2.name), body2, fit, ...
                                        sbtk.invert.homogeneous(prob), 0.04, "Other team, contact");

%% At the end: the solution against the hidden model, cell by cell
cmp = sbtk.compare.grids(sbtk.exchange.read_grid(fullfile(model, 'density_grid.txt')), ...
                         sbtk.exchange.read_grid(fullfile(solution, 'density_grid.txt')));
sbtk.compare.grid_summary(cmp);
```

ex08: moments of inertia and the libration ratio

The same inversion with no inertia data, I_{zz}, the libration ratio at two precisions, and I_{xx}, I_{yy}, I_{zz}; the libration ratio from an observed libration amplitude; adding measured inertia values (Moments of inertia).

MATLAB script: examples/ex08_moments_of_inertia.m

```
%% EX08_MOMENTS_OF_INERTIA  What moments of inertia and the libration ratio add to the gravity field. (~20 s)
%
% The degree-2 gravity coefficients fix five combinations of the inertia tensor
% (C20 ~ (Ixx + Iyy - 2 Izz), C22 ~ (Iyy - Ixx), C21, S21, S22 ~ products of inertia),
% but not its trace. A dense core and a light surface layer can therefore give the
% same gravity field, and the inversion cannot separate them. A moment of inertia
% such as the polar moment Izz (from the spin state) adds the missing combination:
% it measures how concentrated the mass is toward the center.
%
% The libration ratio (B - A)/C (principal moments: A <= B about the axes in the
% xy-plane, C = Izz) comes from the libration amplitude of a synchronously rotating body
% (e.g. Dimorphos). The degree-2 field gives B - A, so the ratio gives C: it carries the
% same information as a polar moment, with a relative precision of sigma/ratio.

addpath(fullfile(fileparts(fileparts(which('ex08_moments_of_inertia'))), 'toolbox'));
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);

%% Simulated measurements: every inertia quantity is available
truth_components = [sbtk.interior.sphere("core", body.centroid_km, 0.170, +450), ...   % at the center of figure
                    sbtk.interior.layer("regolith", 0, 0.012, -900)];
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(truth_components), 4, "ex08");
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], ...
                              2e-3, ...        % 1-sigma of each inertia element, as a fraction of M R_ref^2
                              0.02);           % 1-sigma of the libration ratio
exact = sbtk.data.set_uncertainties(exact, unc);
obs = sbtk.data.add_noise(exact, 1);
disp(struct2table(rmfield(obs.moi, 'weights')))    % Ixx Iyy Izz Ixy Ixz Iyz libration, as I/(M R_ref^2)

%% The same inversion with different inertia measurements
% opts.moi_labels chooses which of the data's inertia rows are fitted. The libration
% ratio of this body is about 0.056, so the simulated 1-sigma of 0.02 (36%) says little;
% a ratio measured to 0.001 (1.8%) is added as a separate data set (sbtk.data.add_libration).
gravity  = sbtk.data.select_moi(obs, {});                              % gravity coefficients only
precise  = sbtk.data.add_libration(gravity, exact.libration_ratio, 0.001, obs.moi(3).value);
cases = {'none (gravity only)',    obs,     {}
         'Izz',                    obs,     {'Izz'}
         'libration +- 0.02',      obs,     {'libration'}
         'libration +- 0.001',     precise, {'libration'}
         'Ixx Iyy Izz',            obs,     {'Ixx', 'Iyy', 'Izz'}};
fprintf('\n%-22s %18s %18s %12s %8s\n', 'inertia used', 'core [kg/m^3]', 'regolith [kg/m^3]', 'correlation', 'chi2/nu');
for k = 1:size(cases, 1)
    fit = sbtk.invert.solve(body, truth_components, cases{k, 2}, sbtk.invert.options(moi_labels=cases{k, 3}));
    fprintf('%-22s %10.0f +- %4.0f %10.0f +- %4.0f %12.2f %8.2f\n', cases{k, 1}, fit.rho_kgm3(1), ...
            fit.rho_sigma_kgm3(1), fit.rho_kgm3(2), fit.rho_sigma_kgm3(2), fit.corr(2, 3), fit.chi2_reduced);
end
fprintf('%-22s %10.0f %20.0f\n', 'true', truth.rho_bg_kgm3 + truth_components(1).drho_kgm3, ...
        truth.rho_bg_kgm3 + truth_components(2).drho_kgm3);
% correlation: between the core and regolith relative densities (near 1 = the data
% cannot tell them apart).
% Izz halves the regolith uncertainty. The libration ratio gives C = (B - A)/ratio, so it
% constrains C as well as the ratio is known relative to its value: +-0.001 is 1.8% of
% C and helps, Izz +-0.002 is 0.5% and helps more.

%% A libration ratio from the observed libration amplitude
% A synchronous satellite in an orbit of eccentricity e librates with amplitude
% 2e/(3 ratio - 1) (forced libration, first order in e). Near ratio = 1/3 (elongation
% about 1.4 : 1) the amplitude is large and fixes the ratio precisely; a nearly
% axisymmetric body librates little and the ratio stays uncertain.
for amp = [-4 -12]
    [k, sk] = sbtk.data.libration_ratio_from_amplitude(amp, 0.2, 0.027, 0.001);   % deg, deg, e, sigma e
    fprintf('amplitude %5.1f +- 0.2 deg, e = 0.027 +- 0.001: (B - A)/C = %.3f +- %.4f (%.0f%%)\n', ...
            amp, k, sk, 100*sk/k);
end

%% Your own inertia measurements
% Start from the gravity coefficients (a package, a SHADR file, coefficient matrices,
% ...) and add each measured quantity. Values given in kg m^2 convert with
% sbtk.units.moi_to_norm; inertia is about the center of mass, shape-model axes.
Izz_kgm2 = sbtk.units.norm_to_moi(obs.moi(3).value, body.mass_kg, body.R_ref_km);   % e.g. a measured polar moment
mine = sbtk.data.add_moi(gravity, "Izz", sbtk.units.moi_to_norm(Izz_kgm2, body.mass_kg, body.R_ref_km), 2e-3);
mine = sbtk.data.add_libration(mine, exact.libration_ratio, 0.001, mine.moi(1).value);  % (B - A)/C, 1-sigma, Izz
fit = sbtk.invert.solve(body, truth_components, mine, sbtk.invert.options(moi_labels={'Izz', 'libration'}));
sbtk.invert.summary(fit)
```

Project template

A project is a folder of runner scripts for one body that write to `results/<project>/`. The folder `projects/template/` is the starting point for a new body. Body-specific numbers are kept in one settings function, `my_body.m`; nothing body-specific is in the toolbox.

Files

File	Purpose
`my_body.m`	Returns the body (shape model and its length unit, bulk density, reference radius), the gravity degree and the density-grid spacing.
`simulate_measurements.m`	Simulates exact measurements of a prescribed interior and writes a measurement package and an interior-model package.
`invert_measurements.m`	Gives the measurements of a package uncertainties, estimates component densities and writes a solution package.
`compare_with_truth.m`	Compares an inversion with the true model of a simulation, by component and on the density grid.

As delivered, the template runs on `data/shapes/example_asteroid.obj`, a synthetic body of about 1.2 km. `tests/integration/test_template_project.m` runs the three scripts.

Use

1. Copy `projects/template/` to `projects/<body>/`.
2. In `my_body.m`, replace the shape file, its length unit, the bulk density (or use `sbtk.body.from_mass`), the reference radius, the degree and the grid spacing. The shape must be closed and star-shaped about its center of mass (Shape models).
3. Set `project` at the top of each script.
4. In `simulate_measurements.m`, edit the sections PROJECT, NAME AND DELIVERY and INTERIOR. The simulation writes exact values, without uncertainties or noise.
5. In `invert_measurements.m`, edit UNCERTAINTIES (the 1σ the receiving team gives the exact values), MODEL and OPTIONS.
6. Run the scripts in order. Each run writes its own folder (Results folders); the inversion finds the newest simulation of the given name with `sbtk.io.latest_run`.

A simulation run writes

Simulation run folder

```
results/<project>/<date_time>_<name>/
  log.txt                              everything printed (includes the true model)
  redacted/<team>-DATA<n>-<name>/      measurement package: nothing about the interior
  unredacted/<team>-MOD<n>-<name>/     interior-model package: true model, density grid, the same exact values
  unredacted/                          measurements.*, truth.*, settings.json, figures/
```

Code

MATLAB: projects/template/my_body.m

```
function [body, degree, grid_km] = my_body()
%MY_BODY  Everything specific to your asteroid, used by every runner in this project.
%
%   [body, degree, grid_km] = my_body()
%
%   Replace the shape file (and its length unit), the bulk density, the reference
%   radius, the degree and the grid spacing with your own. The measurement
%   uncertainties are set by the receiving team in each inversion run (invert_measurements.m).

shape = sbtk.shape.load(fullfile(sbtk.root, 'data', 'shapes', 'example_asteroid.obj'), ...   % your shape model
                        "km");                                                               % its length unit: "km", "m" or "cm"
body  = sbtk.body.create("Example asteroid", shape, ...
            2000, ...       % bulk density [kg/m^3]
            0.60);          % reference radius of the gravity coefficients [km]
degree = 4;                 % highest degree of the gravity field
grid_km = 0.040;            % cell edge of the density grids in model and solution packages [km]
end
```

MATLAB script: projects/template/simulate_measurements.m

```
%% SIMULATE_MEASUREMENTS  Synthetic measurements of a prescribed interior, for any body.
%
% Edit NAME and INTERIOR; the body is defined in my_body.m. The measurements are the exact
% values of the interior, without uncertainties or noise: the receiving team applies its own
% (invert_measurements.m). Writes results/<project>/<date_time>_<name>/:
%   redacted/<team>-DATA<n>-<name>/    measurement package to send (nothing about the interior)
%   unredacted/<team>-MOD<n>-<name>/   interior-model package (true model, density grid,
%                                      the same exact values, figures)
%   unredacted/                        measurements.*, truth.*, settings.json, figures/
%   log.txt
% Package format: docs/exchange_format.md.

clear; close all;
here = fileparts(which('simulate_measurements'));
addpath(here, fullfile(fileparts(fileparts(here)), 'toolbox'));
[body, degree, grid_km] = my_body();

%% PROJECT, NAME AND DELIVERY
project     = "my_asteroid";
name        = "T1";
team        = "TEAM";                % team identifier in package names
number      = 1;                     % package numbers: <team>-DATA<number>, <team>-MOD<number>
prepared_by = "Team name, contact";

%% INTERIOR
% Components with their relative density (density minus background, kg/m^3);
% lengths in km. See sbtk.interior: sphere, ellipsoid, torus, layer, sector. Spheres,
% ellipsoids and tori are placed from the center of figure (cof), layers and sectors are
% measured from the center of mass.
cof = body.centroid_km;
components = [ ...
    sbtk.interior.sphere("core",     cof,                                    0.250, +500), ...
    sbtk.interior.layer("regolith",  0.000, 0.020,                                 -800), ...
    sbtk.interior.sphere("boulder",  cof + sbtk.interior.latlon(10, -40, 0.35), 0.060, +900)];
field = sbtk.interior.no_random_field();     % or sbtk.interior.random_field(sigma kg/m^3, correlation length km, seed)

%% RUN
out = sbtk.io.new_run(project, name);
[exact, truth] = sbtk.data.simulate(body, sbtk.interior.create(components, field), degree, name);   % the true model
sbtk.io.report_simulation(sbtk.io.subfolder(out, "unredacted"), body, exact, truth);
data_set = sprintf("%s-DATA%d-%s", team, number, name);
sbtk.exchange.write_data(fullfile(out.folder, "redacted"), data_set, body, exact, prepared_by);
sbtk.exchange.write_model(fullfile(out.folder, "unredacted"), sprintf("%s-MOD%d-%s", team, number, name), ...
                          data_set, body, truth, exact, grid_km, prepared_by);
sbtk.io.finish_run(out);
```

MATLAB script: projects/template/invert_measurements.m

```
%% INVERT_MEASUREMENTS  Component densities from gravity coefficients and moments of inertia, for any body.
%
% Edit DATA, UNCERTAINTIES, MODEL and OPTIONS. Writes results/<project>/<date_time>_<name>_inversion/:
% fit.txt / .mat, homogeneous.txt, settings.json, log.txt, figures/.

clear; close all;
here = fileparts(which('invert_measurements'));
addpath(here, fullfile(fileparts(fileparts(here)), 'toolbox'));
[~, ~, grid_km] = my_body();
team = "TEAM";  prepared_by = "Team name, contact";

%% DATA
project = "my_asteroid";
name    = "T1";
data_folder = sbtk.exchange.locate(sbtk.io.latest_run(project, name), "DATA");   % or any package folder
[obs, body] = sbtk.exchange.read_data(data_folder);

%% UNCERTAINTIES (1-sigma): ours, since the package has exact values
unc = sbtk.data.uncertainties( ...
    [sbtk.units.offset_to_degree1(0.005, body.R_ref_km), ... % degree 1: 5 m center of mass
     1e-4, 2e-4, 4e-4], ...                                  % degrees 2..degree (fully normalized)
    2e-3, ...                                                % each moment of inertia, as a fraction of M R_ref^2
    0.02);                                                   % libration ratio (B-A)/C
obs = sbtk.data.set_uncertainties(obs, unc);
% obs = sbtk.data.add_noise(obs, 1);                         % noise drawn from them, if wanted

%% MODEL (geometry fixed; relative densities estimated)
cof = body.centroid_km;                                      % center of figure of the package's shape model
components = [ ...
    sbtk.interior.sphere("core",     cof,          0.250, 0), ...
    sbtk.interior.layer("regolith",  0.000, 0.020,        0)];
% Without assumed structure:
% components = sbtk.interior.sector_grid([0 0.45 0.8 1], [-90 0 90], [-180 -90 0 90 180]);

%% OPTIONS
opts = sbtk.invert.options();
opts.moi_labels = {'Ixx', 'Iyy', 'Izz'};   % moment-of-inertia measurements used ({} = gravity only)

%% RUN
out  = sbtk.io.new_run(project, name + "_inversion");
prob = sbtk.invert.setup(body, components, obs, opts);
if numel(components) > 6
    prob.opts.regularization = sbtk.invert.choose_regularization(prob);    % many components
end
fit = sbtk.invert.run(prob);
hom = sbtk.invert.homogeneous(prob);
sbtk.io.report_inversion(out, body, fit, hom);
sbtk.exchange.write_solution(out.folder, sbtk.exchange.solution_name(team, obs.name), body, fit, hom, grid_km, prepared_by);
sbtk.io.finish_run(out);
```

MATLAB script: projects/template/compare_with_truth.m

```
%% COMPARE_WITH_TRUTH  Compare an inversion with the true model of a simulation, for any body.
%
% Reports R2 (fraction of the variance of the true density explained by the
% estimate), R2_max for the same components, the RMS density error and each
% component's estimated and true density; then compares the density grids of the
% solution package and the interior-model package cell by cell. Writes
% results/<project>/<date_time>_<inversion>_comparison/: comparison.txt / .mat,
% grid_comparison.txt, figures/comparison.png, figures/truth_vs_estimate.png.

clear; close all;
here = fileparts(which('compare_with_truth'));
addpath(here, fullfile(fileparts(fileparts(here)), 'toolbox'));
body = my_body();

project       = "my_asteroid";
truth_run     = "T1";              % simulation
inversion_run = "T1_inversion";    % inversion

simulation = sbtk.io.latest_run(project, truth_run);
inversion  = sbtk.io.latest_run(project, inversion_run);
truth = sbtk.io.load_truth(simulation);
fit   = sbtk.io.load_fit(inversion);
out   = sbtk.io.new_run(project, inversion_run + "_comparison");
sbtk.io.report_comparison(out, body, truth, fit, sbtk.compare.with_truth(body, truth, fit));

% the solution package against the interior-model package, cell by cell
cmp = sbtk.compare.grids(sbtk.exchange.read_grid(fullfile(sbtk.exchange.locate(simulation, "MOD"), "density_grid.txt")), ...
                         sbtk.exchange.read_grid(fullfile(sbtk.exchange.locate(inversion, "SOL"), "density_grid.txt")));
writelines(sbtk.compare.grid_summary(cmp), fullfile(out.folder, "grid_comparison.txt"));
sbtk.io.finish_run(out);
```

Bennu

`projects/bennu/` estimates the interior of the asteroid Bennu from the OSIRIS-REx gravity field (JPL, 2019; degree 10, R_{ref} = 0.290 km) following Cavale et al., "Constraints on the Internal Density Distribution of Bennu Using OSIRIS-REx Mission Data" (Icarus), and reproduces that paper's inversion.

Files

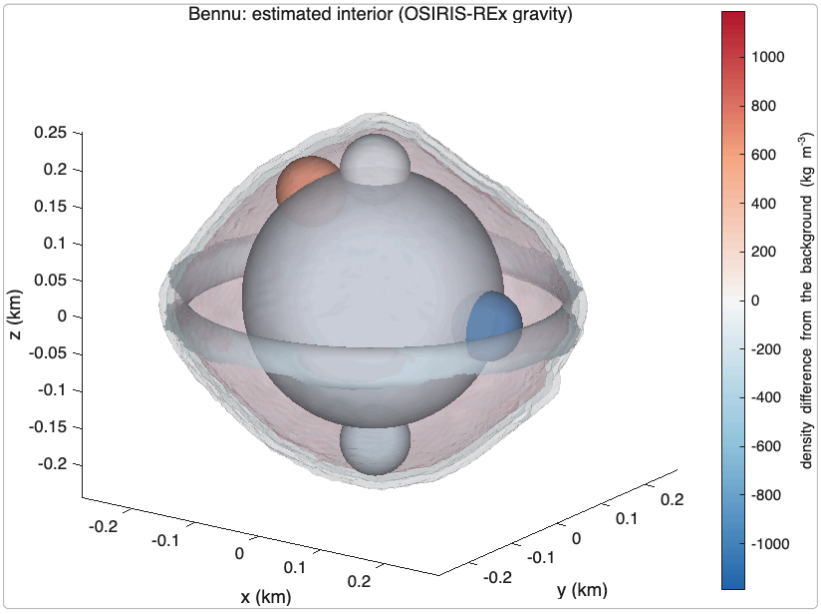
File	Purpose
`bennu_body.m`	Shape model `bennu_higher_qual.obj` (196,608 facets, body-fixed frame with origin at the center of mass), bulk density 1190 kg/m³, R_{ref} = 0.290 km.
`bennu_measurements.m`	The field as a measurement set, with the paper's weighting (C10 = C11 = S11 = 0, J_2, C21, S21, C22, S22, J_3, J_4 with equal weights) or with all coefficients of degrees 2–4 and the full JPL covariance. Reads the file with `sbtk.io.read_jpl_gravity`.
`bennu_paper_components.m`	The paper's nominal model: prescribed surface (0–10 m, −297.5 kg/m³) and subsurface (10–20 m, +1190 kg/m³) layers, a core of radius 170 m with estimated center, an equatorial torus and four boulders. Positions are from the origin of the shape model, as in the paper; that origin is Bennu's center of mass from the OSIRIS-REx tracking data, 1.4 m from the center of figure (Where to place components).
`invert_bennu_paper.m`	Reproduces the paper's inversion with the background held at the bulk density and compares the result with the paper's values (`tests/reference/bennu_paper_nominal.json`).
`invert_bennu_boulder_positions.m`	The same with the centers of the two boulders that target C21/S21 and C22/S22 also estimated, with an a priori 1σ of 10 m per coordinate.
`check_bennu_full_field.m`	Compares the reproduced interior's coefficients of degrees 2–4 with the full field and covariance.
`bennu_compare_with_paper.m`	Prints a fit next to the paper's values.

Reproduction of the paper

Relative densities from `tools/make_readme_examples.m`, which runs the paper configuration:

Relative density (kg/m³)	Paper	SBTK
core	−241.9	−242.1
torus	−239.4	−234.6
boulder C21	+664.5	+660.8
boulder C22	−1049.3	−1037.3
boulder north	−148.0	−146.4
boulder south	−251.5	−250.9
core center (m)	(1.86, −7.89, 12.59)	(2.06, −7.91, 12.56)

The paper's code counted a surface facet as part of the torus only when all three of its vertices were inside the tube; SBTK integrates the exact intersection, whose volume is 2.4% larger, which accounts for most of the torus difference. `tests/integration/test_bennu_paper.m` checks the reproduction: the core relative density within 1 kg/m³, the boulders within 2%, the torus mass contrast within 1%, the core center within 0.5 m, and the coefficients the paper did not fit (C31 … S44) within 0.15 of the JPL 1σ (Validation).



Estimated interior of Bennu in the paper configuration (`sbtk.plot.interior3d`).

Hera blind test

`projects/hera_hlp6/` contains the runner scripts SBTK uses in a blind test of interior-modeling methods for the binary asteroid Didymos–Dimorphos, the target of ESA's Hera mission. In such a test one team simulates measurements of an interior it keeps hidden and shares only the measurements; other teams estimate the interior; the hidden models are released afterwards and compared with the estimates on a common density grid. The makers deliver exact values, without uncertainties or noise; each receiving team weights them with uncertainties of its own choosing. SBTK's inversion runners use those of the Hera end-to-end simulation, read from files of the Hera working group that are not distributed with SBTK; the runners need them in `data/hera_rse/`.

Files

File	Purpose
`hlp6_didymos.m`, `hlp6_dimorphos.m`	Settings shared by every run of one body: shape model, mass, reference radius, degree, density-grid spacing.
`hlp6_team.m`	Team identifier and contact written into package names and files.
`hlp6_uncertainties.m`	The receiving team's uncertainties: the Hera radio-science precision of the gravity coefficients (to degree 5 for Didymos, 3 for Dimorphos, to which the coefficients are truncated), the inertia elements and the libration ratio.
`simulate_didymos.m`, `simulate_dimorphos.m`	Simulate measurements of a hidden interior; write the measurement package and the interior-model package.
`invert_didymos.m`, `invert_dimorphos.m`	Invert a measurement package (by default the redacted package of the newest simulation); write a solution package.
`invert_didymos_sectors.m`	The same with a 36-sector grid, the background held at the bulk density and regularization by the discrepancy principle.
`invert_received_data.m`	Invert another team's package (SBTK format) or a gravity file in the PDS SHADR layout.
`compare_didymos.m`, `compare_dimorphos.m`	Compare an inversion with the hidden model, by component and on the density grid.

Settings

Setting	Didymos	Dimorphos
Shape model	DART v003, 9.3 m sampling, 49,152 facets	DART v004 (before the impact), every 8th point of the 0.243 m model, 49,152 facets
Mass	5.273×10^{11} kg	2.700×10^{9} kg
Reference radius	0.43 km	0.10 km
Degree	5	3
Density-grid spacing	0.020 km	0.005 km
Bulk density	2593 kg/m³	1534 kg/m³
Frame of the packages	center of mass and spin axes of the true model	center of mass and principal axes of the true model

Simulation runs

A simulation runner has four sections:

1. **NAME AND DELIVERY**: the run name, the package number n, and the inertia quantities the package contains (any of `Ixx` … `Iyz`, `libration`, or none; `sbtk.data.select_moi`).
2. **INTERIOR**: components and random field (Density distributions), in the axes of the DART shape-model file with positions measured from the center of figure (`cof = body.centroid_km`), not from the origin of the file, which is 57 m from it for Didymos (Where to place components). The torus lies about the z-axis through `cof`. `align_with_spin_axis` names the components whose axis is set along the axis of the largest moment of the whole true model (Didymos: `"torus"`, which forms about the spin axis; Dimorphos: none), with `sbtk.interior.align_with_spin_axis`.
3. **FRAME OF THE PACKAGES**: the origin is the center of mass of the true model, as in mission body-fixed frames, and the axes are `"spin"`, `"principal"` or `"shape"`. Didymos uses the spin axes of the true model (z along its axis of the largest moment, x toward the shape-model x-axis: $\bar{C}_{21} = \bar{S}_{21} = 0$, as in a mission field of a body in principal-axis rotation); Dimorphos, which may be tumbling, uses its principal axes (Frames of a gravity field).
4. **RUN**: `sbtk.interior.create` and `sbtk.interior.align_with_spin_axis`; `sbtk.data.simulate` (the true model and its exact values, without uncertainties); with `sbtk.gravity.center_of_mass_frame` the center of mass and the axes of the chosen frame, and `sbtk.body.to_frame`, `sbtk.interior.to_frame`, `sbtk.data.to_frame` to carry the body (its shape model is moved and rotated and written into the package as `<package>_<shape>_com_<frame>_axes.obj`, a name no other package has, The shape model of a package), the true model and the exact values into it; `sbtk.data.select_moi`; then `sbtk.io.report_simulation`, `sbtk.exchange.write_data` (a package of exact values: no 1σ columns, no covariance file, Exchange packages) and `sbtk.exchange.write_model`.

The run folder separates what may be shared from what may not:

Simulation run folder of projects/hera_hlp6

```
results/hera_hlp6/<date_time>_<name>/
  log.txt                          everything printed, including the true model
  redacted/<team>-DATA<n>-<body>/  the measurement package to share
  unredacted/<team>-MOD<n>-<body>/ the interior-model package, shared after the estimates
  unredacted/                      measurements.*, truth.*, settings.json, figures/
```

Only the package folder in `redacted/` is shared: `log.txt` and everything in `unredacted/` contain the true model.

Maker models

`examples/make_didymos_1.m` to `make_dimorphos_3.m` are six true models made the same way, one script each, into `results/didymos_models/` and `results/dimorphos_models/`, as packages 1 to 6. Didymos and Dimorphos share one numbering because a solution name carries only the number; the T1 demo sets of the simulate scripts are 7 and 8. They send exact values of every degree to 6 and I_{xx}, I_{yy}, I_{zz}.

Script	Interior (relative densities)
`make_didymos_1`	Underdense core, underdense equatorial torus aligned with the spin axis, underdense surface layer
`make_didymos_2`	Model 1 and two dense boulders of 40 m, on the equator and at latitude 45°
`make_didymos_3`	Model 1 and six dense boulders of 35 m, placed to excite $\bar{C}_{21}$, $\bar{S}_{21}$, $\bar{C}_{22}$, $\bar{S}_{22}$, J_3 and $\bar{C}_{33}$
`make_dimorphos_1`	Underdense surface layer and a denser sector on the side of the DART impact
`make_dimorphos_2`	Model 1, a dense body below the impact site and two voids
`make_dimorphos_3`	Model 2, an underdense fractured ellipsoid below the impact site and six voids

The impact side is the side DART imaged on its approach, where the shape model has its finest detail; it is centered near latitude −11°, longitude 260° in the axes of the DART file. `tests/integration/test_maker_scripts.m` runs `make_didymos_3` and `make_dimorphos_3` and checks their packages.

Inversion runs

`invert_didymos.m` reads the redacted package of the newest simulation of the given name with `sbtk.exchange.locate` and `sbtk.exchange.read_data`, exactly as another team receives it, so the inversion expands about the measured center of mass (Density estimation). `hlp6_uncertainties.m` then gives the exact values the receiving team's uncertainties (the Hera radio-science precision, truncating the coefficients to the degree it covers). Its model places the core and the torus at the center of figure of the package's shape model, as a receiving team would, and estimates the torus axis and height (Estimated geometry): `estimate_axis = true`, `estimate_position = [false false true]`, with a priori 1σ of 1° and 10 m and limits of 10° and 50 m. It writes `fit.*`, `homogeneous.txt`, figures and the solution package `<team>-SOL_<data team><n>/`. `invert_received_data.m` does the same for a package from another team, or, with the commented lines, for a SHADR-layout gravity file, for which the shape model and the point the coefficients are about must be given.

Comparison runs

`compare_didymos.m` loads the newest simulation and inversion of the given names and the body from the simulation's measurement package (so that all three are in the frame of the packages), compares the fit with the true model (`sbtk.compare.with_truth`) and compares the solution package's density grid with the interior-model package's (`sbtk.compare.grids`), which is the comparison every team's solution receives.

Example runs

The delivered interiors (named T1) include a boulder and a random density field that the delivered inversion models do not contain. Weighted by the uncertainties of the end-to-end simulation, the χ^2 test rejects the inversion models of both bodies, and the estimated densities are biased accordingly. At this precision an inversion model has to include structure down to the scale of such features to pass the test (Goodness of fit). For Didymos T1, whose torus is aligned with the spin axis and so lies about the z-axis of the package, the model with fixed geometry has a reduced χ^2 of 121 and $R^2 = -5.3$. Estimating the torus axis and height lowers χ^2 to 39 but R^2 to -5.7: the torus tilts 5.6° and moves 41 m to take up the signal of the boulder, which the model lacks (Estimated geometry). The run takes about a minute. For Dimorphos, in its principal axes, the ellipsoidal core of the model is turned 12° about z against the true core, as a receiving team's would be (reduced χ^2 313).

API reference

The supported interface of SBTK: every function in the namespace packages below, generated from the help text and `arguments` block of its source file. Functions are called by their full name, `sbtk.<package>.<function>`. Arguments have no default values except where the Default column states one (three functions). Structures passed between functions are described in Data structures; error identifiers in Error identifiers. Helper functions in `sbtk.internal` are not part of the supported interface and are listed separately in Internal functions.

Package	Contents	Functions
`sbtk`	Version and repository folder	2
`sbtk.units`	Constants and unit conversions	6
`sbtk.shape`	Shape models	8
`sbtk.body`	Bodies	4
`sbtk.interior`	Density distributions	16
`sbtk.gravity`	Gravity and moment calculations	6
`sbtk.data`	Measurement sets	19
`sbtk.invert`	Density estimation	9
`sbtk.compare`	Comparison with a true model	6
`sbtk.exchange`	Exchange packages	9
`sbtk.io`	Results folders and text files	22
`sbtk.plot`	Figures	9

All supported functions

Function	Purpose
`sbtk.body.create`	A small body: shape model, bulk density and reference radius.
`sbtk.body.from_mass`	A small body defined by its mass instead of its bulk density.
`sbtk.body.summary`	Print a body's main numbers.
`sbtk.body.to_frame`	A body with its shape model in another frame: x' = (x - `origin_km`) * axes.
`sbtk.compare.evaluate`	Compare a fit with the true model, using a prepared reference (see sbtk.compare.reference).
`sbtk.compare.grid_summary`	Print a cell-by-cell comparison of two density grids.
`sbtk.compare.grids`	Compare two density grids cell by cell (e.g. a solution package with an interior-model package).
`sbtk.compare.reference`	The true model sampled once for comparisons with fits that use a given set of components.
`sbtk.compare.summary`	Print a comparison with the true model.
`sbtk.compare.with_truth`	Compare an estimated density with the true density (synthetic tests).
`sbtk.data.add_center_of_mass`	Add degree-1 coefficients that place the center of mass at a given point.
`sbtk.data.add_libration`	Add a measured libration ratio (B - A)/C.
`sbtk.data.add_moi`	Add one measured inertia element to a measurement set.
`sbtk.data.add_moi_row`	Add a measurement of any weighted sum of the inertia elements.
`sbtk.data.add_noise`	One noisy realization of a measurement set.
`sbtk.data.from_coefficients`	Measurement set from gravity-coefficient matrices (e.g. a mission solution).
`sbtk.data.libration_ratio_from_amplitude`	(B - A)/C from the forced libration of a synchronous satellite.
`sbtk.data.move_origin`	Express measurements about the center of mass relative to a new expansion origin.
`sbtk.data.select_coefficients`	Keep only the listed gravity coefficients.
`sbtk.data.select_degree`	Keep only the coefficients up to a degree.
`sbtk.data.select_moi`	Keep only the listed moment-of-inertia measurements.
`sbtk.data.set_covariance`	Replace the coefficient covariance with a full matrix.
`sbtk.data.set_moi_sigma`	Set the 1-sigma of moment-of-inertia measurements by label.
`sbtk.data.set_uncertainties`	Give measurements a 1-sigma uncertainty per degree and per inertia quantity.
`sbtk.data.simulate`	True model of an interior and its exact gravity coefficients and moments of inertia.
`sbtk.data.summary`	Print what is in a measurement set.
`sbtk.data.to_frame`	A measurement set in another frame: x' = (x - `origin_km`) * axes.
`sbtk.data.uncertainties`	1-sigma measurement uncertainties by degree and by inertia quantity, for sbtk.data.set_uncertainties.
`sbtk.data.use_uncertainties`	Give measurements the coefficient uncertainties of another data set (e.g. a precision file).
`sbtk.exchange.density_grid`	Density of a model at the centers of a cubic grid inside the body.

Function	Purpose
sbtk.exchange.locate	The exchange package of a given kind in a run folder.
sbtk.exchange.read_data	Measurements and body from a measurement package (sbtk.exchange.write_data format).
sbtk.exchange.read_grid	Density grid from an interior-model or solution package (`density_grid.txt`).
sbtk.exchange.read_shadr	Measurements and body from a gravity file in the PDS SHADR layout (e.g. other teams' .tab files).
sbtk.exchange.solution_name	Name of a solution package: <team>-SOL_<data team><number>.
sbtk.exchange.write_data	Write a measurement package to send to other teams (nothing about the interior).
sbtk.exchange.write_model	Write the interior-model package behind a measurement package (released after the solutions).
sbtk.exchange.write_solution	Write an inversion result as a solution package for the other teams.
sbtk.gravity.center_of_mass_frame	A gravity field about its center of mass, in the shape-model, spin or principal axes.
sbtk.gravity.component_integrals	Integrals over one component per unit density, clipped to the body.
sbtk.gravity.forward	Forward model: gravity coefficients, center of mass and inertia of an interior.
sbtk.gravity.geometry	Facet-cone geometry of a body about an expansion origin (the integration workspace).
sbtk.gravity.labels	Names of the gravity coefficients up to a degree, in SBTK order.
sbtk.gravity.random_field_integrals	Density-weighted integrals of the random density field.
sbtk.interior.align_with_spin_axis	Set the axis of components along the axis of the largest moment of the whole model.
sbtk.interior.create	An interior model: components plus an optional random density field.
sbtk.interior.density_at	Density [kg/m^3] of a model at points inside the body.
sbtk.interior.ellipsoid	Uniform triaxial ellipsoid component, rotated, clipped to the body.
sbtk.interior.latlon	Position [x y z] (km) from latitude, longitude (deg) and distance (km).
sbtk.interior.layer	Uniform layer component between two depths below the surface.
sbtk.interior.no_components	An empty list of components (a homogeneous body).
sbtk.interior.no_random_field	No random density field.
sbtk.interior.random_field	Gaussian random density field added to the interior.
sbtk.interior.sector	Uniform sector component: radial fractions x latitude/longitude range.
sbtk.interior.sector_grid	Sectors that tile the whole body (a model without assumed structure).
sbtk.interior.set_axis	Turn a component so that its axis points along a given direction.
sbtk.interior.sphere	Uniform sphere component (e.g. a core, a boulder or a void).
sbtk.interior.summary	Print a list of components.
sbtk.interior.to_frame	An interior or a true model in another frame: x' = (x - `origin_km`) * axes.
sbtk.interior.torus	Uniform torus component about the z-axis, clipped to the body.
sbtk.invert.choose_regularization	Tikhonov regularization parameter by the discrepancy principle.
sbtk.invert.goodness_of_fit	Reduced chi-square and chi-square test of a fit.
sbtk.invert.homogeneous	Fit a homogeneous body (no components) to the same measurements.
sbtk.invert.options	Inversion settings with their standard values.
sbtk.invert.replace_data	Use another realization of the same measurements without repeating the setup.
sbtk.invert.run	Estimate the densities for a prepared inversion (bounded, mass-constrained least squares).
sbtk.invert.setup	Prepare an inversion: geometry and the design-matrix column of every component, computed once.
sbtk.invert.solve	Setup and run in one call.
sbtk.invert.summary	Print an inversion result.
sbtk.io.finish_run	Stop the run's log and say where the results are.
sbtk.io.latest_run	Folder of the most recent run with a given name.
sbtk.io.load_fit	Inversion result from a run folder (`fit.mat`) or a .mat file holding the variable fit.
sbtk.io.load_measurements	Measurements from a run folder, a .mat file or a text file.
sbtk.io.load_truth	True model from a run folder or a .mat file.
sbtk.io.new_run	Create a results folder for one run and start its log.
sbtk.io.read_jpl_gravity	Measurements from a JPL SSD gravity-field file (normalized coefficients + covariance).
sbtk.io.read_measurements	Measurement set from the SBTK text format.
sbtk.io.report_comparison	Save, print and plot a comparison with the true model.
sbtk.io.report_inversion	Save, print and plot an inversion result.
sbtk.io.report_simulation	Save, print and plot synthetic measurements and their true model.
sbtk.io.save_comparison	comparison.mat and comparison.txt in the run folder.
sbtk.io.save_figure	Save a figure as figures/<name>.png (200 dpi) in the run folder.

Function	Purpose
sbtk.io.save_fit	`fit.mat` and `fit.txt` in the run folder.
sbtk.io.save_measurements	measurements.mat and measurements.txt in the run folder.
sbtk.io.save_settings	Write the settings of a run to `settings.json` (readable, reproducible).
sbtk.io.save_truth	`truth.mat` and `truth.txt` in the run folder.
sbtk.io.subfolder	A subfolder of a run folder that the save and report functions write to.
sbtk.io.write_comparison	Plain-text copy of a comparison with the true model.
sbtk.io.write_fit	Plain-text summary of an inversion result.
sbtk.io.write_measurements	Plain-text copy of a measurement set (read back by sbtk.io.read_measurements).
sbtk.io.write_truth	Plain-text description of a true model (keep private in blind tests).
sbtk.plot.comparison	Estimate compared with the true model: R2 and each component's density.
sbtk.plot.densities	Estimated density of the background and each component with its 1-sigma uncertainty.
sbtk.plot.difference_from_homogeneous	Gravity coefficients minus those of the homogeneous body, coefficient by coefficient.
sbtk.plot.interior3d	See-through 3D view of the body and the components of a model.
sbtk.plot.measurements	What the data contain: gravity signal and noise per degree, and the inertia rows.
sbtk.plot.residuals	Normalized residuals of a fit, (predicted - measured)/sigma, per measurement.
sbtk.plot.shape	The body's shape model with its center of figure and axes.
sbtk.plot.slices	Absolute density on the equatorial plane and a meridional plane.
sbtk.plot.truth_vs_estimate	Truth, estimate and their difference on two planes.
sbtk.root	Folder of the SBTK repository (the one containing toolbox/, examples/, data/).
sbtk.shape.bundled	A shape model that ships with SBTK, by short name.
sbtk.shape.ellipsoid	Triaxial ellipsoid shape model (an icosphere stretched to the semi-axes).
sbtk.shape.from_arrays	Shape model from face and vertex arrays.
sbtk.shape.load	Read a polyhedral shape model (Wavefront OBJ) and convert it to km.
sbtk.shape.properties	Volume, centroid and closure checks of a shape model.
sbtk.shape.reduce	A shape model with fewer facets, checked to stay closed, outward and star-shaped.
sbtk.shape.reduce_spc	Every step-th grid point of an SPC shape model: the same surface, sampled step times coarser.
sbtk.shape.write_obj	Write a shape model to a Wavefront OBJ file (vertices in km).
sbtk.units.G	Gravitational constant in SBTK units [km^3 kg^-1 s^-2].
sbtk.units.length_scale	Factor that converts a length in UNIT to km.
sbtk.units.moi_to_norm	Moment of inertia in kg m^2 -> I / (M `R_ref`^2) (dimensionless).
sbtk.units.norm_to_moi	I / (M `R_ref`^2) (dimensionless) -> moment of inertia in kg m^2.
sbtk.units.offset_to_degree1	Fully normalized degree-1 coefficient of a center-of-mass offset.
sbtk.units.table	Print the SBTK unit convention.
sbtk.version	SBTK version string (semantic versioning).

Reference

`sbtk`: Version and repository folder

Function	Purpose
`root`	Folder of the SBTK repository (the one containing toolbox/, examples/, data/).
`version`	SBTK version string (semantic versioning).

`sbtk.root`

Folder of the SBTK repository (the one containing toolbox/, examples/, data/).

Syntax

```
MATLAB
r = sbtk.root()
```

Outputs

Name	Description
`r`	Absolute path of the SBTK repository folder (the folder that contains toolbox/).

Related functions

Called by

`sbtk.io.latest_run`, `sbtk.io.new_run`, `sbtk.shape.bundled`

Implementation: `toolbox/+sbtk/root.m`, line 1.

`sbtk.version`

SBTK version string (semantic versioning).

Syntax

```
MATLAB
v = sbtk.version()      % e.g. '1.0.0'
```

Outputs

Name	Description
`v`	Version text, e.g. '1.0.0' (semantic versioning).

Related functions

Called by

`sbtk.io.save_settings`, `sbtk.io.write_truth`

Implementation: `toolbox/+sbtk/version.m`, line 1.

Reference

`sbtk.units`: Constants and unit conversions

Function	Purpose
`G`	Gravitational constant in SBTK units [km^3 kg^-1 s^-2].
`length_scale`	Factor that converts a length in UNIT to km.
`moi_to_norm`	Moment of inertia in kg m^2 -> I / (M `R_ref`^2) (dimensionless).
`norm_to_moi`	I / (M `R_ref`^2) (dimensionless) -> moment of inertia in kg m^2.
`offset_to_degree1`	Fully normalized degree-1 coefficient of a center-of-mass offset.
`table`	Print the SBTK unit convention.

`sbtk.units.G`

Gravitational constant in SBTK units [km^3 kg^-1 s^-2].

Syntax

MATLAB

```
g = sbtk.units.G()      % 6.67430e-20 km^3/(kg s^2)  (CODATA 2018)
```

Outputs

Name	Description
`g`	6.67430e-20 km³ kg⁻¹ s⁻² (CODATA 2018).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`.

```
GM_km3_s2 = sbtk.units.G()*body.mass_kg;
```

Related functions

Called by

`sbtk.body.create`, `sbtk.data.from_coefficients`, `sbtk.data.simulate`, `sbtk.exchange.read_data`, `sbtk.exchange.read_shadr`, `sbtk.io.read_jpl_gravity`

Implementation: `toolbox/+sbtk/+units/G.m`, line 1.

`sbtk.units.length_scale`

Factor that converts a length in UNIT to km.

Syntax

MATLAB

```
f = sbtk.units.length_scale(unit)      % unit: "km", "m" or "cm"
```

Arguments

Name	Size	Class	Unit	Default	Description
`unit`	1 × 1	string		required	Length unit: "km", "m" or "cm". Validation: `mustBeMember(unit, ["km", "m", "cm"])`.

Outputs

Name	Description
`f`	Factor that converts a length in that unit to km (1, 1e-3 or 1e-5).

Example

sbtk.units.length_scale("m") returns 1e-3.

Related functions

Called by

`sbtk.shape.from_arrays`, `sbtk.shape.load`

Implementation: `toolbox/+sbtk/+units/length_scale.m`, line 1.

`sbtk.units.moi_to_norm`

Moment of inertia in kg m^2 -> I / (M `R_ref`^2) (dimensionless).

Syntax

MATLAB

```
I_norm = sbtk.units.moi_to_norm(I_kgm2, mass_kg, R_ref_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`I_kgm2`		double	kg m²	required	Moment of inertia, or an array of them. Validation: `mustBeFinite`.
`mass_kg`	scalar	double	kg	required	Total mass M. Validation: `mustBePositive`.
`R_ref_km`	scalar	double	km	required	Reference radius R_{ref}. Validation: `mustBePositive`.

Outputs

Name	Description
`I_norm`	$I / (MR_{\mathrm{ref}}^2)$, same size as the input.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`.

```
Izz = sbtk.units.moi_to_norm(1.6e14, body.mass_kg, body.R_ref_km);    % kg m^2 -> I/(M R_ref^2)
```

Implementation: `toolbox/+sbtk/+units/moi_to_norm.m`, line 1.

`sbtk.units.norm_to_moi`

I / (M `R_ref`^2) (dimensionless) -> moment of inertia in kg m^2.

Syntax

MATLAB

```
I_kgm2 = sbtk.units.norm_to_moi(I_norm, mass_kg, R_ref_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`I_norm`		double	M R_ref²	required	$I / (MR_{\mathrm{ref}}^2)$, or an array of such values. Validation: `mustBeFinite`.
`mass_kg`	scalar	double	kg	required	Total mass M. Validation: `mustBePositive`.
`R_ref_km`	scalar	double	km	required	Reference radius R_{ref}. Validation: `mustBePositive`.

Outputs

Name	Description
`I_kgm2`	Moment of inertia in kg m², same size as the input.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
I_kgm2 = sbtk.units.norm_to_moi(obs.moi_tensor_norm, obs.mass_kg, obs.R_ref_km);
```

Related functions

Called by

`sbtk.io.write_measurements`

Implementation: `toolbox/+sbtk/+units/norm_to_moi.m`, line 1.

`sbtk.units.offset_to_degree1`

Fully normalized degree-1 coefficient of a center-of-mass offset.

Syntax

MATLAB

```
c = sbtk.units.offset_to_degree1(offset_km, R_ref_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`offset_km`		double	km	required	offset of the center of mass from the expansion origin along one axis, or its 1-sigma uncertainty [km] Validation: `mustBeFinite`.
`R_ref_km`	scalar	double	km	required	reference radius of the gravity coefficients [km] Validation: `mustBePositive`.

Outputs

Name	Description
`c`	Fully normalized degree-1 coefficient, d / ($\sqrt{3}$ R_{ref}). Fields: `component`.

Description

c = `offset_km` / (sqrt(3) `R_ref_km`): C10 for z, C11 for x, S11 for y. Use it to give the degree-1 entry of sbtk.data.uncertainties as a center-of-mass uncertainty.

Example

MATLAB, from the help text

```
% 2 m center of mass, 1e-4, 2e-4, 4e-4 for degrees 2-4
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
```

Related functions

Mentioned in the help text

`sbtk.data.uncertainties`

Implementation: `toolbox/+sbtk/+units/offset_to_degree1.m`, line 1.

`sbtk.units.table`

Print the SBTK unit convention.

Syntax

MATLAB

```
sbtk.units.table()
```

Example

MATLAB, from the help text

```
sbtk.units.table()                      % prints the unit conventions
```

Implementation: `toolbox/+sbtk/+units/table.m`, line 1.

Reference

`sbtk.shape`: Shape models

Function	Purpose
`bundled`	A shape model that ships with SBTK, by short name.
`ellipsoid`	Triaxial ellipsoid shape model (an icosphere stretched to the semi-axes).
`from_arrays`	Shape model from face and vertex arrays.
`load`	Read a polyhedral shape model (Wavefront OBJ) and convert it to km.
`properties`	Volume, centroid and closure checks of a shape model.
`reduce`	A shape model with fewer facets, checked to stay closed, outward and star-shaped.
`reduce_spc`	Every step-th grid point of an SPC shape model: the same surface, sampled step times coarser.
`write_obj`	Write a shape model to a Wavefront OBJ file (vertices in km).

`sbtk.shape.bundled`

A shape model that ships with SBTK, by short name.

Syntax

MATLAB

```
shape = sbtk.shape.bundled(name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	file in data/shapes/ (all in km) Validation: `mustBeMember(name, ["didymos", "dimorphos", "didymos_highres", "dimorphos_highres", "didymos_every8", "dimorphos_placeholder", "bennu", "example_asteroid"])`.

Outputs

Name	Description
`shape`	The shape model, lengths in km. Fields: `shape`.

Description

`"didymos"`	`didymos_g_9309mm_spc_obj_0000n00000_v003.obj`: DART SPC model v003 at 9.3 m sampling (49,152 facets), the version the Hera radio-science team and the HLP6-G test use
`"dimorphos"`	`g_00243mm_spc_obj_dimo_0000n00000_v004_every8.obj`: DART SPC model v004 (pre-impact) at 1.94 m sampling (49,152 facets), every 8th point of the high-resolution grid (sbtk.shape.reduce_spc)
`"didymos_highres"`	`g_01165mm_spc_obj_didy_0000n00000_v003.obj`: 1.165 m, 3,145,728 facets
`"dimorphos_highres"`	`g_00243mm_spc_obj_dimo_0000n00000_v004.obj`: 0.243 m, 3,145,728 facets
`"didymos_every8"`	`g_01165mm_spc_obj_didy_0000n00000_v003_every8.obj`: every 8th point of "`didymos_highres`" (49,152 facets)
`"dimorphos_placeholder"`	a placeholder shape (not a mission model; kept for the tests)
`"bennu"`	OSIRIS-REx shape model (`bennu_higher_qual`)
`"example_asteroid"`	a synthetic 1.2 km body (projects/template)

The two high-resolution files (about 160 MB each) are not in git: copy them from the Hera server (End-to-End Test/Shape/) or ESA's SPICE server (spiftp.esac.esa.int/data/SPICE/HERA/kernels/dsk/) into data/shapes/. See data/shapes/README.md for every file. For any other shape model use sbtk.shape.load(file, unit).

Example

MATLAB, from the help text

```
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
big  = sbtk.shape.bundled("dimorphos_highres");              % 3 s to read
```

Related functions

Mentioned in the help text

`sbtk.body.from_mass`, `sbtk.shape.load`, `sbtk.shape.reduce_spc`

Calls

`sbtk.root`, `sbtk.shape.load`

Implementation: `toolbox/+sbtk/+shape/bundled.m`, line 1.

`sbtk.shape.ellipsoid`

Triaxial ellipsoid shape model (an icosphere stretched to the semi-axes).

Syntax

MATLAB

```
shape = sbtk.shape.ellipsoid(name, semi_axes_km, level)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the shape.
`semi_axes_km`	1 × 3	double	km	required	[a b c] along x, y, z [km] Validation: `mustBePositive`.
`level`	scalar	double		required	icosphere refinement: 20*4^level facets (5 -> 20,480) Validation: `mustBeInteger, mustBeNonnegative`.

Outputs

Name	Description
`shape`	Triaxial ellipsoid with 20·4^level facets, vertices on the ellipsoid. Fields: shape.

Description

Vertices lie exactly on the ellipsoid. Useful for tests and stand-ins.

Example

MATLAB, from the help text

```
shape = sbtk.shape.ellipsoid("test", [0.40 0.35 0.30], 4);          % 5,120 facets
```

Implementation: `toolbox/+sbtk/+shape/ellipsoid.m`, line 1.

`sbtk.shape.from_arrays`

Shape model from face and vertex arrays.

Syntax

MATLAB

```
shape = sbtk.shape.from_arrays(name, faces, vertices, unit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the shape.
`faces`	any × 3	double		required	nf x 3 vertex indices (1-based), triangles Validation: `mustBeInteger, mustBePositive`.
`vertices`	any × 3	double		required	nv x 3 positions in UNIT ("km", "m" or "cm") Validation: `mustBeFinite`.
`unit`	1 × 1	string		required	Length unit of the vertices: "km", "m" or "cm". Validation: `mustBeMember(unit, ["km", "m", "cm"])`.

Outputs

Name	Description
`shape`	The shape model, vertices converted to km. Fields: shape.

Description

Returns the same struct as sbtk.shape.load.

Example

MATLAB, from the help text

```
V = [0 0 0; 100 0 0; 0 100 0; 0 0 100] - 25;                        % vertices in meters
shape = sbtk.shape.from_arrays("tetra", [1 3 2; 1 2 4; 1 4 3; 2 3 4], V, "m");
```

Related functions

Mentioned in the help text

`sbtk.shape.load`

Calls

`sbtk.units.length_scale`

Called by

`sbtk.shape.reduce`, `sbtk.shape.reduce_spc`

Implementation: `toolbox/+sbtk/+shape/from_arrays.m`, line 1.

`sbtk.shape.load`

Read a polyhedral shape model (Wavefront OBJ) and convert it to km.

Syntax

MATLAB

```
shape = sbtk.shape.load(file, unit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`file`	1 × 1	string		required	path to an .obj file with 'v x y z' and 'f i j k' records (normals, textures and 'f i/t/n' forms are accepted) Validation: `mustBeFile`.
`unit`	1 × 1	string		required	length unit of the file: "km", "m" or "cm" (required: there is no safe way to guess it) Validation: `mustBeMember(unit, ["km", "m", "cm"])`.

Outputs

Name	Description
`shape`	struct Fields: `shape`.

Description

`.name`	file name without extension
`.faces`	nf x 3 vertex indices
`.vertices_km`	nv x 3 vertex positions [km]
`.source`	the file it came from
`.frame_origin_km, .frame_axes`	[0 0 0] and eye(3): the frame of the file (changed by
`sbtk.body.to_frame; vertices = (file vertices - frame_origin_km)*frame_axes)`	

Example

MATLAB, from the help text

```
shape = sbtk.shape.load(fullfile(sbtk.root, 'data', 'shapes', ...
           'didymos_g_9309mm_spc_obj_0000n00000_v003.obj'), "km");
```

Related functions

Mentioned in the help text

`sbtk.body.to_frame`, `sbtk.root`

Calls

`sbtk.units.length_scale`

Called by

`sbtk.exchange.read_data`, `sbtk.shape.bundled`

Implementation: `toolbox/+sbtk/+shape/load.m`, line 1.

`sbtk.shape.properties`

Volume, centroid and closure checks of a shape model.

Syntax

MATLAB

```
props = sbtk.shape.properties(shape)
```

Arguments

Name	Size	Class	Unit	Default	Description
`shape`	1 × 1	struct		required	The shape model to check. Fields: `shape`.

Outputs

Name	Description
`props`	Volume, centroid and checks. Fields: `props`.

Description

`props.volume_km3`	enclosed volume [km^3]
`props.centroid_km`	center of figure [km]

`props.n_facets`, .n_vertices

`props.extent_km`	[min; max] of the vertices per axis [km]
`props.closed`	true if every edge is shared by exactly two facets, in opposite directions (a closed, consistently ordered surface)
`props.outward`	true if the facets are ordered outward
`props.star_shaped_about_centroid`	true if every facet faces away from the
`centroid (required to integrate about it)`	

Example

MATLAB, from the help text

```
p = sbtk.shape.properties(sbtk.shape.bundled("didymos"));
fprintf('volume %.4f km^3, star-shaped: %d\n', p.volume_km3, p.star_shaped_about_centroid);
```

Related functions

Mentioned in the help text

`sbtk.shape.bundled`

Called by

`sbtk.body.create`, `sbtk.body.from_mass`, `sbtk.exchange.write_data`, `sbtk.exchange.write_model`, `sbtk.shape.reduce`, `sbtk.shape.reduce_spc`

Implementation: `toolbox/+sbtk/+shape/properties.m`, line 1.

`sbtk.shape.reduce`

A shape model with fewer facets, checked to stay closed, outward and star-shaped.

Syntax

MATLAB

```
small = sbtk.shape.reduce(shape, n_facets)
```

Arguments

Name	Size	Class	Unit	Default	Description
`shape`	1 × 1	struct		required	from sbtk.shape.load (e.g. a high-resolution SPC model with millions of facets) Fields: `shape`.
`n_facets`	scalar	double		required	target number of facets Validation: `mustBeInteger, mustBePositive`.

Outputs

Name	Description
`small`	The reduced shape model. Fields: `shape`.

Description

Uses MATLAB's reducepatch. Gravity coefficients to degree 4 or so need far fewer facets than a high-resolution shape model has; check what the reduction changes by comparing sbtk.shape.properties (volume, centroid) and the homogeneous-body coefficients of both (tools/make_reduced_shapes.m does both). Errors if the reduced surface is not closed, not ordered outward, or not star-shaped about its centroid. Save it with sbtk.shape.write_obj.

Example

MATLAB, from the help text

```
big   = sbtk.shape.load("g_00243mm_spc_obj_dimo_0000n00000_v004.obj", "km");
small = sbtk.shape.reduce(big, 50000);
sbtk.shape.write_obj(small, "dimorphos_v004_50k.obj");
```

Related functions

Mentioned in the help text

sbtk.shape.load, sbtk.shape.properties, sbtk.shape.write_obj

Calls

sbtk.shape.from_arrays, sbtk.shape.properties

Implementation: `toolbox/+sbtk/+shape/reduce.m`, line 1.

---

sbtk.shape.reduce_spc

Every step-th grid point of an SPC shape model: the same surface, sampled step times coarser.

Syntax

MATLAB

```
small = sbtk.shape.reduce_spc(shape, step)
```

Arguments

Name	Size	Class	Unit	Default	Description
`shape`	1 × 1	struct		required	an SPC (stereophotoclinometry) shape model in its grid order, as in the DART g_<sampling>_spc_obj_<body>_... OBJ files: 6 cube faces of (q+1) x (q+1) vertices each, the first index fastest, 12 q^2 triangles Fields: shape.
`step`	scalar	double		required	a divisor of q, e.g. 8 Validation: `mustBeInteger, mustBePositive`.

Outputs

Name	Description
`small`	12 (q/step)^2 facets and 6 (q/step)^2 + 2 vertices: the grid points (i, j) with i and j multiples of step, triangulated like the original (diagonal from (i, j) to (i+1, j+1)), the copies along the cu hape.

Description

The grid is radial (one surface point per direction from the body's center), so the result stays star-shaped about the center, unlike a general mesh reduction (sbtk.shape.reduce). Each cube face's triangles are ordered outward.

Example

MATLAB, from the help text

```
big   = sbtk.shape.load("g_00243mm_spc_obj_dimo_0000n00000_v004.obj", "km");   % q = 512
small = sbtk.shape.reduce_spc(big, 8);                                        % q = 64, 49,152 facets
sbtk.shape.write_obj(small, "g_00243mm_spc_obj_dimo_0000n00000_v004_every8.obj");
```

Related functions

Mentioned in the help text

sbtk.shape.load, sbtk.shape.reduce, sbtk.shape.write_obj

Calls

sbtk.shape.from_arrays, sbtk.shape.properties

Implementation: `toolbox/+sbtk/+shape/reduce_spc.m`, line 1.

---

sbtk.shape.write_obj

Write a shape model to a Wavefront OBJ file (vertices in km).

Syntax

MATLAB

```
sbtk.shape.write_obj(shape, file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`shape`	1 × 1	struct		required	The shape model. Fields: `shape`.
`file`	1 × 1	string		required	Path of the OBJ file to write; vertices are written in km.

Example

MATLAB, from the help text

```
sbtk.shape.write_obj(sbtk.shape.ellipsoid("test", [0.40 0.35 0.30], 4), "test.obj");
```

Related functions

Mentioned in the help text

`sbtk.shape.ellipsoid`

Implementation: `toolbox/+sbtk/+shape/write_obj.m`, line 1.

sbtk.body: Bodies

Function	Purpose
create	A small body: shape model, bulk density and reference radius.
from_mass	A small body defined by its mass instead of its bulk density.
summary	Print a body's main numbers.
to_frame	A body with its shape model in another frame: x' = (x - origin_km) * axes.

sbtk.body.create

A small body: shape model, bulk density and reference radius.

Syntax

MATLAB

```
body = sbtk.body.create(name, shape, bulk_density_kgm3, R_ref_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
name	1 × 1	string		required	
shape	1 × 1	struct		required	from sbtk.shape.load / from_arrays / ellipsoid Fields: shape.
bulk_density_kgm3	scalar	double	kg/m³	required	mean density [kg/m^3]; sets the total mass Validation: mustBePositive.
R_ref_km	scalar	double	km	required	reference radius of the gravity coefficients [km] Validation: mustBePositive.

Outputs

Name	Description
body	struct Fields: body.

Description

.volume_km3	enclosed volume [km^3]
.mass_kg	bulk_density_kgm3 * volume [kg]
.GM_km3_s2	G * mass [km^3/s^2]
.centroid_km	center of figure [km]

Example

MATLAB, from the help text

```
shape = sbtk.shape.ellipsoid("test", [0.4 0.35 0.3], 4);
body  = sbtk.body.create("Test", shape, 2600, 0.4);
```

Related functions

Mentioned in the help text

sbtk.shape.ellipsoid, sbtk.shape.load

Calls

sbtk.shape.properties, sbtk.units.G

Called by

sbtk.body.from_mass

Implementation: toolbox/+sbtk/+body/create.m, line 1.

sbtk.body.from_mass

A small body defined by its mass instead of its bulk density.

Syntax

```
MATLAB
body = sbtk.body.from_mass(name, shape, mass_kg, R_ref_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the body.
`shape`	1 × 1	struct		required	The shape model. Fields: `shape`.
`mass_kg`	scalar	double	kg	required	Total mass M. Validation: `mustBePositive`.
`R_ref_km`	scalar	double	km	required	Reference radius of the gravity coefficients. Validation: `mustBePositive`.

Outputs

Name	Description
`body`	The body, with bulk density = mass / shape volume. Fields: `body`.

Description

Same as sbtk.body.create with bulk density = mass / shape volume. Use it when the mass (or GM / G) is what was measured.

Example

```
MATLAB fragment from the help text. Uses variables defined by earlier calls: shape.
body = sbtk.body.from_mass("Didymos", shape, 5.273e11, 0.43);
```

Related functions

Mentioned in the help text

`sbtk.body.create`

Calls

`sbtk.body.create`, `sbtk.shape.properties`

Called by

`sbtk.body.to_frame`, `sbtk.exchange.read_data`, `sbtk.exchange.read_shadr`

Implementation: `toolbox/+sbtk/+body/from_mass.m`, line 1.

`sbtk.body.summary`

Print a body's main numbers.

Syntax

```
MATLAB
sbtk.body.summary(body)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body to describe. Fields: `body`.

Example

```
MATLAB, from the help text
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
sbtk.body.summary(body)
```

Related functions

Mentioned in the help text

`sbtk.body.from_mass`, `sbtk.shape.bundled`

Implementation: `toolbox/+sbtk/+body/summary.m`, line 1.

`sbtk.body.to_frame`

A body with its shape model in another frame: x' = (x - `origin_km`) * axes.

Syntax

MATLAB

```
body = sbtk.body.to_frame(body, origin_km, axes, shape_name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create or sbtk.body.from_mass Fields: `body`.
`origin_km`	1 × 3	double	km	required	origin of the new frame in the current frame [km]; [0 0 0] keeps the origin Validation: `mustBeFinite`.
`axes`	3 × 3	double		required	3 x 3 rotation, columns: the new axes as unit vectors in the current frame, e.g. cf.axes of sbtk.gravity.center_of_mass_frame; eye(3) keeps the axes Validation: `mustBeFinite`.
`shape_name`	1 × 1	string		required	name of the transformed shape model; an exchange package writes it as <data set>_<shape_name>.obj

Outputs

Name	Description
`body`	from sbtk.body.create or sbtk.body.from_mass Fields: `body`.

Description

The vertices become (v - `origin_km`)*axes; the mass, reference radius and name of the body are kept, and the center of figure moves with the shape. `shape.source` keeps the name of the original file (not its path, which a package must not carry), and `shape.frame_origin_km` and `shape.frame_axes` give the frame relative to that file, combined over successive changes of frame: vertices = (file vertices - `frame_origin_km`)*`frame_axes`. An exchange package states them in conventions.txt. Use the same `origin_km` and axes for the measurements (sbtk.data.to_frame) and the interior or true model (sbtk.interior.to_frame).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `truth`, `body`.

```
spin   = sbtk.gravity.center_of_mass_frame(truth, body, "spin");
body_s = sbtk.body.to_frame(body, spin.com_km, spin.axes, "didymos_v003_com_spin_axes");
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.body.from_mass`, `sbtk.data.to_frame`, `sbtk.gravity.center_of_mass_frame`, `sbtk.interior.to_frame`

Calls

`sbtk.body.from_mass`

Called by

`sbtk.interior.to_frame`

Implementation: `toolbox/+sbtk/+body/to_frame.m`, line 1.

`sbtk.interior`: Density distributions

Function	Purpose
`align_with_spin_axis`	Set the axis of components along the axis of the largest moment of the whole model.
`create`	An interior model: components plus an optional random density field.
`density_at`	Density [kg/m^3] of a model at points inside the body.
`ellipsoid`	Uniform triaxial ellipsoid component, rotated, clipped to the body.
`latlon`	Position [x y z] (km) from latitude, longitude (deg) and distance (km).
`layer`	Uniform layer component between two depths below the surface.
`no_components`	An empty list of components (a homogeneous body).
`no_random_field`	No random density field.
`random_field`	Gaussian random density field added to the interior.
`sector`	Uniform sector component: radial fractions x latitude/longitude range.
`sector_grid`	Sectors that tile the whole body (a model without assumed structure).
`set_axis`	Turn a component so that its axis points along a given direction.
`sphere`	Uniform sphere component (e.g. a core, a boulder or a void).
`summary`	Print a list of components.
`to_frame`	An interior or a true model in another frame: x' = (x - `origin_km`) * axes.
`torus`	Uniform torus component about the z-axis, clipped to the body.

`sbtk.interior.align_with_spin_axis`

Set the axis of components along the axis of the largest moment of the whole model.

Syntax

MATLAB

```
[interior, axis_dir, iterations] = sbtk.interior.align_with_spin_axis(body, interior, names, tol_deg, max_iter)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	the body Fields: `body`.
`interior`	1 × 1	struct		required	from sbtk.interior.create Fields: `interior`.
`names`	1 × any	string		required	the components to align (string array; tori, ellipsoids or sectors), e.g. "torus"; strings(1, 0) for none
`tol_deg`	scalar	double	deg	required	convergence: the axis moves by at most this angle in the last iteration [deg] Validation: `mustBePositive`.
`max_iter`	scalar	double		required	largest number of iterations (sbtk:iteration:notConverged beyond it) Validation: `mustBeInteger, mustBePositive`.

Outputs

Name	Description
`interior`	from sbtk.interior.create Fields: `interior`.
`axis_dir`	1 x 3 axis of the largest moment of the aligned interior, in the frame of the shape model (pointing to +z's side)
`iterations`	number of simulations made

Description

A body in principal-axis rotation spins about its axis of the largest moment, so a component that formed about the spin axis, such as an equatorial torus, lies about that axis. The axis depends on every component, the aligned ones included, so the alignment is a fixed-point iteration (sbtk.internal.fixed_point): simulate to degree 2, take the axis of the largest moment (sbtk.gravity.center_of_mass_frame), turn the named components to it (sbtk.interior.set_axis), repeat. Each iteration is one simulation; the axis usually settles to 0.01 deg in 2 to 4.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `core`.

```
t = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
t.center_km = body.centroid_km;
interior = sbtk.interior.create([core, t], sbtk.interior.no_random_field());
[interior, axis_dir] = sbtk.interior.align_with_spin_axis(body, interior, "torus", 0.01, 20);
fprintf('axis %.2f deg from the shape-model z-axis\n', acosd(axis_dir(3)));
```

Related functions

Mentioned in the help text

sbtk.gravity.center_of_mass_frame, sbtk.interior.create, sbtk.interior.no_random_field, sbtk.interior.set_axis, sbtk.interior.torus, sbtk.internal.fixed_point

Calls

sbtk.data.simulate, sbtk.gravity.center_of_mass_frame, sbtk.interior.set_axis

Implementation: `toolbox/+sbtk/+interior/align_with_spin_axis.m`, line 1.

---

sbtk.interior.create

An interior model: components plus an optional random density field.

Syntax

MATLAB

```
interior = sbtk.interior.create(components, field)
```

Arguments

Name	Size	Class	Unit	Default	Description
`components`	1 × any	struct		required	1 x n struct array from the component constructors (sphere, ellipsoid, torus, layer, sector); sbtk.interior.no_components() for none. Overlapping relative densities add; everything outside the body is clipped. Fields: `component`.
`field`	1 × 1	struct		`sbtk.interior.no_random_field()`	sbtk.interior.random_field(...); omit it (or pass sbtk.interior.no_random_field()) for none Fields: `random_field`.

Outputs

Name	Description
`interior`	Components and random field. Fields: `interior`.

Description

The background density is not set here: sbtk.data.simulate sets it so that the total mass matches the body.

Example

MATLAB, from the help text

```
components = [sbtk.interior.sphere("core", [0 0 0], 0.170, +450), ...
              sbtk.interior.layer("regolith", 0, 0.012, -900)];
interior = sbtk.interior.create(components);                                % no random field
interior = sbtk.interior.create(components, sbtk.interior.random_field(40, 0.06, 7));
```

Related functions

Mentioned in the help text

sbtk.data.simulate, sbtk.interior.layer, sbtk.interior.no_components, sbtk.interior.no_random_field, sbtk.interior.random_field, sbtk.interior.sphere

Calls

sbtk.interior.no_random_field

Called by

sbtk.plot.difference_from_homogeneous

Implementation: `toolbox/+sbtk/+interior/create.m`, line 1.

---

sbtk.interior.density_at

Density [kg/m^3] of a model at points inside the body.

Syntax

MATLAB

```
rho_kgm3 = sbtk.interior.density_at(model, X_km, geo)
```

Arguments

Name	Size	Class	Unit	Default	Description
`model`	1 × 1	struct		required	anything with `rho_bg_kgm3`, components and `random_field`: a true model from sbtk.data.simulate or a fit from sbtk.invert.run
`X_km`	any × 3	double	km	required	n x 3 points in shape-model coordinates [km] Validation: `mustBeFinite`.
`geo`	1 × 1	struct		required	geometry about the model's frame (sbtk.gravity.geometry(body, `model.origin_km`, 0)) Fields: `geo`.

Outputs

Name	Description
`rho_kgm3`	Density at each point, n × 1. The points are not tested for being inside the body: a point outside gets the background density plus any component that contains it.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`.

```
geo = sbtk.gravity.geometry(body, truth.origin_km, 0);
rho = sbtk.interior.density_at(truth, [0 0 0; 0.3 0 0] + truth.origin_km, geo);   % center, 300 m along x
```

Related functions

Mentioned in the help text

`sbtk.data.simulate`, `sbtk.gravity.geometry`, `sbtk.invert.run`

Implementation: `toolbox/+sbtk/+interior/density_at.m`, line 1.

`sbtk.interior.ellipsoid`

Uniform triaxial ellipsoid component, rotated, clipped to the body.

Syntax

MATLAB

```
c = sbtk.interior.ellipsoid(name, center_km, semi_axes_km, angles_deg, drho_kgm3)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the component; unique within a model.
`center_km`	1 × 3	double	km	required	[x y z] center [km] Validation: `mustBeFinite`.
`semi_axes_km`	1 × 3	double	km	required	[a b c] semi-axes [km]; before rotation along x, y, z Validation: `mustBePositive`.
`angles_deg`	1 × 3	double	deg	required	[yaw pitch roll] [deg]: rotation about z, then y, then x Validation: `mustBeFinite`.
`drho_kgm3`	scalar	double	kg/m³	required	relative density [kg/m^3] Validation: `mustBeFinite`.

Outputs

Name	Description
`p`	The component. Fields: `component`.

Example

MATLAB, from the help text

```
core = sbtk.interior.ellipsoid("core", [0 0 0], [0.20 0.15 0.12], [30 0 0], +400);
```

Implementation: `toolbox/+sbtk/+interior/ellipsoid.m`, line 1.

`sbtk.interior.latlon`

Position [x y z] (km) from latitude, longitude (deg) and distance (km).

Syntax

MATLAB

```
xyz_km = sbtk.interior.latlon(lat_deg, lon_deg, distance_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`lat_deg`	scalar	double	deg	required	Latitude, from the xy-plane toward +z. Validation: `mustBeInRange(lat_deg, -90, 90)`.
`lon_deg`	scalar	double	deg	required	Longitude, from +x toward +y. Validation: `mustBeFinite`.
`distance_km`	scalar	double	km	required	Distance from the origin of the coordinates. Validation: `mustBeNonnegative`.

Outputs

Name	Description
`xyz_km`	Position $d\,[\cos\phi\cos\lambda,\ \cos\phi\sin\lambda,\ \sin\phi]$, a 1 × 3 vector. Add a point to measure from another origin.

Example

a boulder 250 m from the center at 25 N, 60 E

MATLAB, from the help text

```
b = sbtk.interior.sphere("boulder", sbtk.interior.latlon(25, 60, 0.250), 0.040, +900);
```

Related functions

Mentioned in the help text

`sbtk.interior.sphere`

Implementation: `toolbox/+sbtk/+interior/latlon.m`, line 1.

`sbtk.interior.layer`

Uniform layer component between two depths below the surface.

Syntax

MATLAB

```
c = sbtk.interior.layer(name, top_depth_km, bottom_depth_km, drho_kgm3)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the component; unique within a model.
`top_depth_km`	scalar	double	km	required	Depth of the top of the layer below the surface, along rays from the expansion origin (0 = at the surface). Validation: `mustBeNonnegative`.
`bottom_depth_km`	scalar	double	km	required	Depth of the bottom of the layer; larger than the top depth. Validation: `mustBePositive, mustBeGreaterThan(bottom_depth_km, top_depth_km)`.
`drho_kgm3`	scalar	double	kg/m³	required	Relative density: density minus the background density. Validation: `mustBeFinite`.

Outputs

Name	Description
`p`	The component. Fields: `component`.

Description

Depth is measured along the ray from the expansion origin (the center of mass), so sbtk.interior.layer("regolith", 0, 0.012, -900) is the outermost 12 m of the body.

Example

MATLAB, from the help text

```
regolith = sbtk.interior.layer("regolith", 0, 0.012, -900);
```

Implementation: `toolbox/+sbtk/+interior/layer.m`, line 1.

`sbtk.interior.no_components`

An empty list of components (a homogeneous body).

Syntax

MATLAB

```
components = sbtk.interior.no_components()
```

Outputs

Name	Description
`p`	An empty 1 × 0 component array. Fields: `component`.

Example

MATLAB, from the help text

```
interior = sbtk.interior.create(sbtk.interior.no_components());      % a homogeneous body
```

Related functions

Mentioned in the help text

`sbtk.interior.create`

Called by

`sbtk.interior.sector_grid`, `sbtk.plot.difference_from_homogeneous`

Implementation: `toolbox/+sbtk/+interior/no_components.m`, line 1.

`sbtk.interior.no_random_field`

No random density field.

Syntax

MATLAB

```
field = sbtk.interior.no_random_field()
```

Outputs

Name	Description
`field`	A random field with standard deviation 0. Fields: `random_field`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `components`.

```
interior = sbtk.interior.create(components, sbtk.interior.no_random_field());   % same as create(components)
```

Related functions

Mentioned in the help text

`sbtk.interior.create`

Called by

`sbtk.interior.create`, `sbtk.invert.run`

Implementation: `toolbox/+sbtk/+interior/no_random_field.m`, line 1.

`sbtk.interior.random_field`

Gaussian random density field added to the interior.

Syntax

MATLAB

```
field = sbtk.interior.random_field(sigma_kgm3, corr_length_km, seed)
```

Arguments

Name	Size	Class	Unit	Default	Description
`sigma_kgm3`	scalar	double	kg/m³	required	standard deviation of the density perturbation [kg/m^3] Validation: `mustBeNonnegative`.
`corr_length_km`	scalar	double	km	required	correlation length [km]: correlation exp(-|dx|^2 / (2 L^2)) Validation: `mustBePositive`.
`seed`	scalar	double		required	random seed (the same seed gives the same field) Validation: `mustBeInteger, mustBeNonnegative`.

Outputs

Name	Description
`field`	The random field. Fields: `random_field`.

Description

The field is a sum of 300 random Fourier modes, defined at every point and independent of the expansion origin. sbtk.interior.no_random_field() for none.

Example

MATLAB, from the help text

```
field = sbtk.interior.random_field(40, 0.06, 7);   % sigma 40 kg/m^3, correlation length 60 m
```

Related functions

Mentioned in the help text

`sbtk.interior.no_random_field`

Implementation: `toolbox/+sbtk/+interior/random_field.m`, line 1.

`sbtk.interior.sector`

Uniform sector component: radial fractions x latitude/longitude range.

Syntax

MATLAB

```
c = sbtk.interior.sector(name, frac, lat_deg, lon_deg, drho_kgm3)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the component; unique within a model.
`frac`	1 × 2	double		required	[`f_in` `f_out`]: fraction of the local distance from the expansion origin to the surface (0 = center, 1 = surface) Validation: `mustBeInRange(frac, 0, 1)`.
`lat_deg`	1 × 2	double	deg	required	[`lat_min` `lat_max`] in -90..90 [deg] Validation: `mustBeInRange(lat_deg, -90, 90)`.
`lon_deg`	1 × 2	double	deg	required	[`lon_min` `lon_max`] in -360..360 [deg], at most 360 apart; ranges past +-180 wrap around. Every range must increase. Validation: `mustBeInRange(lon_deg, -360, 360)`.
`drho_kgm3`	scalar	double	kg/m³	required	Relative density: density minus the background density. Validation: `mustBeFinite`.

Outputs

Name	Description
`p`	The component. Fields: `component`.

Description

`drho_kgm3` relative density [kg/m^3]

Latitude and longitude are measured about the expansion origin, in the axes of the shape-model frame (`c.axes` = eye(3); sbtk.interior.to_frame turns them with the frame). sbtk.interior.sector("shell", [0.5 0.75], [-90 90], [-180 180], 0) is a shell that follows the shape. See also sbtk.interior.sector_grid.

Example

MATLAB, from the help text

```
block = sbtk.interior.sector("block", [0.3 0.9], [40 90], [150 240], +200);   % radial fraction, latitude, longitude
shell = sbtk.interior.sector("shell", [0.5 0.75], [-90 90], [-180 180], -300);
```

Related functions

Mentioned in the help text

`sbtk.interior.sector_grid`, `sbtk.interior.to_frame`

Called by

`sbtk.interior.sector_grid`

Implementation: `toolbox/+sbtk/+interior/sector.m`, line 1.

`sbtk.interior.sector_grid`

Sectors that tile the whole body (a model without assumed structure).

Syntax

MATLAB

```
components = sbtk.interior.sector_grid(frac_edges, lat_edges_deg, lon_edges_deg)
```

Arguments

Name	Size	Class	Unit	Default	Description
`frac_edges`	1 × any	double		required	radial fraction edges from 0 to 1, e.g. [0 0.45 0.8 1] Validation: `mustBeInRange(frac_edges, 0, 1)`.
`lat_edges_deg`	1 × any	double	deg	required	latitude edges from -90 to 90, e.g. [-90 0 90] Validation: `mustBeInRange(lat_edges_deg, -90, 90)`.
`lon_edges_deg`	1 × any	double	deg	required	longitude edges spanning 360 deg, e.g. [-180 -90 0 90 180] Validation: `mustBeInRange(lon_edges_deg, -360, 360)`.

Outputs

Name	Description
`components`	One sector per radial shell, latitude band and longitude band, named s<shell><lat><lon>, with relative density 0. Fields: `component`.

Description

One sector per (radial shell, latitude band, longitude band), named 's<shell><lat><lon>' (s111 = innermost shell, southernmost band, first longitude band), `drho_kgm3` = 0. The example edges give 3 x 2 x 4 = 24 sectors. Many components usually need regularization (sbtk.invert.choose_regularization). Because the sectors fill the body, the background density and a common shift of all sector densities are interchangeable: use regularization or hold the background at the bulk density (`opts.background` = "bulk").

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `obs`.

```
sectors = sbtk.interior.sector_grid([0 0.5 0.85 1], [-90 -30 30 90], [-180 -90 0 90 180]);   % 36
prob = sbtk.invert.setup(body, sectors, obs, sbtk.invert.options(background="bulk"));
prob.opts.regularization = sbtk.invert.choose_regularization(prob);
fit = sbtk.invert.run(prob);
```

Related functions

Mentioned in the help text

`sbtk.invert.choose_regularization`, `sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`

Calls

`sbtk.interior.no_components`, `sbtk.interior.sector`

Implementation: `toolbox/+sbtk/+interior/sector_grid.m`, line 1.

`sbtk.interior.set_axis`

Turn a component so that its axis points along a given direction.

Syntax

MATLAB

```
c = sbtk.interior.set_axis(c, direction)
```

Arguments

Name	Size	Class	Unit	Default	Description
`c`	1 × 1	struct		required	a torus, an ellipsoid or a sector (a component with axes, `c.axes`) Fields: `component`.
`direction`	1 × 3	double		required	1 x 3 direction in the frame of the shape model (normalized here) Validation: `mustBeFinite`.

Outputs

Name	Description
`c`	a torus, an ellipsoid or a sector (a component with axes, `c.axes`) Fields: `component`.

Description

The axis of a component is its third axis, `c.axes`(:, 3): the axis a torus lies about, the third semi-axis of an ellipsoid, the pole of a sector's latitudes. The component is turned by the smallest rotation that takes this axis to the direction, about its own center (`c.center_km`; a sector about the expansion origin), so its first two axes stay as close as possible to where they were.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `See`, `also`.

```
  t = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
  t.center_km = body.centroid_km;                          % about the center of figure
  t = sbtk.interior.set_axis(t, [sind(5) 0 cosd(5)]);      % axis tilted 5 deg toward +x
See also sbtk.interior.align_with_spin_axis.
```

Related functions

Mentioned in the help text

`sbtk.interior.align_with_spin_axis`, `sbtk.interior.torus`

Called by

`sbtk.interior.align_with_spin_axis`, `sbtk.invert.run`

Implementation: `toolbox/+sbtk/+interior/set_axis.m`, line 1.

`sbtk.interior.sphere`

Uniform sphere component (e.g. a core, a boulder or a void).

Syntax

MATLAB

```
c = sbtk.interior.sphere(name, center_km, radius_km, drho_kgm3)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the component; unique within a model.
`center_km`	1 × 3	double	km	required	[x y z] center in shape-model coordinates [km] (sbtk.interior.latlon converts latitude, longitude and distance) Validation: `mustBeFinite`.
`radius_km`	scalar	double	km	required	radius [km] Validation: `mustBePositive`.
`drho_kgm3`	scalar	double	kg/m³	required	relative density: density minus the background density [kg/m^3] Validation: `mustBeFinite`.

Outputs

Name	Description
`p`	The component. Fields: `component`.

Description

A sphere entirely inside the body is integrated exactly as a point mass; one that intersects the surface is clipped to the body. For the inversion, set `c.estimate_position` = true to estimate its center as well (or a mask such as [false false true] for z only; sbtk.invert.run).

Example

MATLAB, from the help text

```
core = sbtk.interior.sphere("core", [0 0 0], 0.170, +450);
```

Related functions

Mentioned in the help text

`sbtk.interior.latlon`, `sbtk.invert.run`

Implementation: `toolbox/+sbtk/+interior/sphere.m`, line 1.

`sbtk.interior.summary`

Print a list of components.

Syntax

MATLAB

```
sbtk.interior.summary(components)       % components: struct array, or a model's .components
```

Arguments

Name	Size	Class	Unit	Default	Description
`components`	1 × any	struct		required	Component array, or the `components` field of a model. Fields: `component`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `truth`.

```
sbtk.interior.summary(truth.components)
```

Related functions

Called by

`sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+interior/summary.m`, line 1.

`sbtk.interior.to_frame`

An interior or a true model in another frame: x' = (x - `origin_km`) * axes.

Syntax

MATLAB

```
model = sbtk.interior.to_frame(model, origin_km, axes)
```

Arguments

Name	Size	Class	Unit	Default	Description
`model`	1 × 1	struct		required	an interior (sbtk.interior.create) or a true model (truth from sbtk.data.simulate)
`origin_km`	1 × 3	double	km	required	origin of the new frame in the current frame [km]; [0 0 0] keeps the origin Validation: `mustBeFinite`.
`axes`	3 × 3	double		required	3 x 3 rotation, columns: the new axes as unit vectors in the current frame, e.g. cf.axes of sbtk.gravity.center_of_mass_frame; eye(3) keeps the axes Validation: `mustBeFinite`.

Outputs

Name	Description
`model`	an interior (sbtk.interior.create) or a true model (truth from sbtk.data.simulate)

Description

Every position is transformed as x' = (x - `origin_km`)*axes and every direction as d' = d*axes: the centers of spheres, ellipsoids and tori, the axes of ellipsoids, tori and sectors (`c.axes`), the random field, and for a true model `origin_km` and `com_km`. Its coefficients are rotated (exact, sbtk.internal.sh_rotation) and its inertia elements become axes'*I*axes; both stay about the same point. The density at every point of the body is unchanged, so the model, the body from sbtk.body.to_frame and the measurements from sbtk.data.to_frame (same `origin_km` and axes) describe the same body in the new frame: simulating, gridding, plotting and comparing there gives the transformed results. An inversion result cannot be transformed: transform the measurements and the body and invert in the new frame.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`.

```
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");
spin  = sbtk.gravity.center_of_mass_frame(truth, body, "spin");   % origin: center of mass; z: largest moment
truth_s = sbtk.interior.to_frame(truth, spin.com_km, spin.axes);
body_s  = sbtk.body.to_frame(body, spin.com_km, spin.axes, "didymos_com_spin_axes");
grid = sbtk.exchange.density_grid(body_s, truth_s, 0.020);        % the grid in the new frame
```

Related functions

Mentioned in the help text

sbtk.body.to_frame, sbtk.data.simulate, sbtk.data.to_frame, sbtk.exchange.density_grid, sbtk.gravity.center_of_mass_frame, sbtk.interior.create, sbtk.internal.sh_rotation

Calls

sbtk.body.to_frame, sbtk.data.to_frame

Implementation: `toolbox/+sbtk/+interior/to_frame.m`, line 1.

---

sbtk.interior.torus

Uniform torus component about the z-axis, clipped to the body.

Syntax

MATLAB

```
c = sbtk.interior.torus(name, ring_radius_km, tube_radius_km, height_km, drho_kgm3)
```

Arguments

Name	Size	Class	Unit	Default	Description
`name`	1 × 1	string		required	Name of the component; unique within a model.
`ring_radius_km`	scalar	double	km	required	radius of the tube's center circle [km] Validation: `mustBeNonnegative`.
`tube_radius_km`	scalar	double	km	required	radius of the tube [km] Validation: `mustBePositive`.
`height_km`	scalar	double	km	required	z of the center circle's plane [km] Validation: `mustBeFinite`.
`drho_kgm3`	scalar	double	kg/m³	required	relative density [kg/m^3] Validation: `mustBeFinite`.

Outputs

Name	Description
`p`	The component. Fields: `component`.

Description

The torus lies about the z-axis of the shape-model frame (`c.center_km` = [0 0 0], `c.axes` = eye(3); sbtk.interior.to_frame moves and turns both with the frame). It may be buried, intersect the surface or extend outside the body; only its intersection with the body counts. sbtk.interior.set_axis turns it; sbtk.interior.align_with_spin_axis puts it about the model's own axis of the largest moment. An inversion can estimate its center (`c.estimate_position` = true, or a mask such as [false false true] for z only) and the direction of its axis (`c.estimate_axis` = true) besides its density (sbtk.invert.run).

Example

MATLAB, from the help text

```
torus = sbtk.interior.torus("torus", 0.370, 0.055, 0, -500);
```

Related functions

Mentioned in the help text

sbtk.interior.align_with_spin_axis, sbtk.interior.set_axis, sbtk.interior.to_frame, sbtk.invert.run

Implementation: `toolbox/+sbtk/+interior/torus.m`, line 1.

Reference

`sbtk.gravity`: Gravity and moment calculations

Function	Purpose
`center_of_mass_frame`	A gravity field about its center of mass, in the shape-model, spin or principal axes.
`component_integrals`	Integrals over one component per unit density, clipped to the body.
`forward`	Forward model: gravity coefficients, center of mass and inertia of an interior.
`geometry`	Facet-cone geometry of a body about an expansion origin (the integration workspace).
`labels`	Names of the gravity coefficients up to a degree, in SBTK order.
`random_field_integrals`	Density-weighted integrals of the random density field.

`sbtk.gravity.center_of_mass_frame`

A gravity field about its center of mass, in the shape-model, spin or principal axes.

Syntax

MATLAB

```
cf = sbtk.gravity.center_of_mass_frame(set, body, axes)
```

Arguments

Name	Size	Class	Unit	Default	Description
`set`	1 × 1	struct		required	the field: a measurement set (sbtk.data.simulate, sbtk.exchange.read_data, sbtk.exchange.read_shadr, ...), a true model (truth from sbtk.data.simulate) or an inversion result (fit from sbtk.invert.run: the field of the fitted model)
`body`	1 × 1	struct		required	the body: the shape model for a fitted model, the reference radius for a true model or a fitted model Fields: `body`.
`axes`	1 × 1	string		required	"shape": the axes of the shape-model frame; "spin": z along the axis of the largest moment of the field (the spin axis of a body in principal-axis rotation), x the shape-model x-axis projected onto the plane normal to it, y = z x x: the frame of body-fixed radio-science fields such as the OSIRIS-REx field of Bennu, with C21 = S21 = 0 and S22 not 0; "principal": the principal axes of inertia of the field (x the axis of the smallest moment, z of the largest, each pointing to the positive side of the shape-model axis nearest to it, right-handed), with C21 = S21 = S22 = 0 Validation: `mustBeMember(axes, ["shape", "spin", "principal"])`.

Outputs

Name	Description
`cf`	The field about its center of mass in the chosen axes, with its principal axes and moments. Fields: `cf`.

Description

`cf.labels`	1 x n names of the coefficients, 'C10' ... (degree 1 to cf.degree)
`cf.coef`	1 x n fully normalized coefficients about the center of mass: degree 1 is 0; in spin axes C21 = S21 = 0, in principal axes C21 = S21 = S22 = 0
`cf.sigma`	1 x n 1-sigma (NaN without uncertainties); degree 1: of the center of mass
`cf.cov`	n x n covariance (NaN without uncertainties)
`cf.com_km`	the center of mass in the shape-model frame [km]
`cf.axes`	3 x 3, columns: the axes of cf.coef as unit vectors in the shape-model axes (eye(3) for "shape"); sbtk.body.to_frame, sbtk.data.to_frame and sbtk.interior.to_frame take cf.com_km and cf.axes to express everything in the frame with the center of mass as origin and these axes
`cf.principal`	3 x 3, columns: the principal axes (smallest, middle, largest moment) in the shape-model axes
`cf.moments`	1 x 3 principal moments minus their mean, A <= B <= C, as I / (M `R_ref`^2) (the degree-2 field fixes the moments up to their trace)

cf.R_ref_km, cf.degree

Moving the expansion point to the center of mass makes degree 1 vanish: about a point O, C10, C11, S11 = (z, x, y of the center of mass - O)/(sqrt(3) `R_ref`). The translation of solid harmonics mixes lower degrees into higher ones (degree 2 into 3, ...), so all coefficients change; it is exact (sbtk.internal.sh_translation). Rotating the axes mixes the 2l+1 coefficients within each degree and leaves every degree's power unchanged; in the principal axes the degree-2 inertia tensor is diagonal, so C21 = S21 = S22 = 0 and C20 = (A + B - 2C)/(2 M R^2), C22 = (B - A)/(4 M R^2) (unnormalized). The rotation is exact too (sbtk.internal.sh_rotation). In the spin axes only the tilt of the axis of the largest moment is removed: C21 = S21 = 0, and the angle of the axis of the smallest moment from x is (1/2) atan2(S22, C22).

The center of mass: a true model's `com_km`; a fitted model's own center of mass (computed from its densities); for a measurement set, `origin_km` plus the offset its degree-1 coefficients give. A measurement set with `coef_about` 'origin' whose degree-1 coefficients put the center of mass more than 3 sigma from `origin_km` (more than 1 mm for exact values), e.g. a SHADR field about the shape-model origin, is moved to that center of mass; otherwise it is taken

as about its center of mass already (its degree-1 values are then noise about zero). A set without degree-1 coefficients (e.g. sbtk.io.read_jpl_gravity) is taken as about its center of mass, `origin_km`; its degree-1 sigma is NaN. Moving a set and the principal axes need every coefficient of degree >= 2 up to the set's degree; a set that lacks some can be used only with axes "shape" and only if it is about its center of mass (missing coefficients are NaN).

The covariance is propagated through the translation and the rotation as linear maps; the uncertainty of the center of mass and of the principal axes themselves, which the data also determine, is not added.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`.

```
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");
cs = sbtk.gravity.center_of_mass_frame(truth, body, "shape");       % degree 1 = 0
cz = sbtk.gravity.center_of_mass_frame(truth, body, "spin");        % and C21 = S21 = 0
cp = sbtk.gravity.center_of_mass_frame(truth, body, "principal");   % and C21 = S21 = S22 = 0
disp([cs.labels(1:8); string(cs.coef(1:8)); string(cp.coef(1:8))])
tilt_deg = acosd(cp.principal(3, 3))         % largest-moment axis from the shape z-axis
```

Related functions

Mentioned in the help text

sbtk.body.to_frame, sbtk.data.simulate, sbtk.data.to_frame, sbtk.exchange.read_data, sbtk.exchange.read_shadr, sbtk.interior.to_frame, sbtk.internal.sh_rotation, sbtk.internal.sh_translation, sbtk.invert.run, sbtk.io.read_jpl_gravity

Calls

sbtk.gravity.component_integrals, sbtk.gravity.geometry

Called by

sbtk.interior.align_with_spin_axis, sbtk.plot.difference_from_homogeneous

Implementation: `toolbox/+sbtk/+gravity/center_of_mass_frame.m`, line 1.

---

sbtk.gravity.component_integrals

Integrals over one component per unit density, clipped to the body.

Syntax

MATLAB

```
I = sbtk.gravity.component_integrals(geo, c)
```

Arguments

Name	Size	Class	Unit	Default	Description
`geo`	1 × 1	struct		required	Geometry from `sbtk.gravity.geometry`: the expansion origin and degree of the integrals. Fields: `geo`.
`c`	1 × 1	struct		required	One component. Fields: `component`.

Outputs

Name	Description
`I`	Integrals of the part of the component inside the body, per unit density. Fields: `integrals`.

Description

`I.U`	1 x (L+1)^2 int (r/R)^l `Ybar_lm` dV (packed as `geo.pk`) [km^3]
`I.V`	volume [km^3]
`I.m1`	first moment [km^4]
`I.Q`	second moments [xx yy zz xy xz yz] [km^5]

all about `geo.origin_km` and limited to the body.

A sphere entirely inside the body is an exact point mass. Every other component is integrated ray by ray: each interval of a sub-ray inside the component, between scaled facets `s_a` and `s_b`, adds its exact cone-scaling integrals (see sbtk.gravity.geometry). Layers and sectors give the interval directly; for spheres, ellipsoids and tori the crossings of each ray with their surface are the real roots of a quadratic (sphere, ellipsoid) or quartic (torus) in the distance along the ray, so a ray may cross a component any number of times and the integrals change smoothly with the component's geometry.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`.

```
geo = sbtk.gravity.geometry(body, [0 0 0], 4);
I = sbtk.gravity.component_integrals(geo, sbtk.interior.sphere("core", [0 0 0], 0.170, 0));
fprintf('volume inside the body %.4f km^3\n', I.V);
```

Related functions

Mentioned in the help text

`sbtk.gravity.geometry`, `sbtk.interior.sphere`

Called by

`sbtk.gravity.center_of_mass_frame`, `sbtk.gravity.forward`, `sbtk.invert.run`, `sbtk.invert.setup`

Implementation: `toolbox/+sbtk/+gravity/component_integrals.m`, line 1.

`sbtk.gravity.forward`

Forward model: gravity coefficients, center of mass and inertia of an interior.

Syntax

MATLAB

```
fm = sbtk.gravity.forward(geo, interior, mass_kg)
```

Arguments

Name	Size	Class	Unit	Default	Description
`geo`	1 × 1	struct		required	Geometry from `sbtk.gravity.geometry`; the coefficients and inertia are about `geo.origin_km`. Fields: `geo`.
`interior`	1 × 1	struct		required	Components and random field. Fields: `interior`.
`mass_kg`	scalar	double	kg	required	Total mass; sets the background density. Validation: `mustBePositive`.

Outputs

Name	Description
`fm`	Background density, coefficients, center of mass and inertia. Fields: `fm`.

Description

density = background + sum of component relative densities + random field, with the background density set so that the total mass equals `mass_kg`.

`fm.rho_bg_kgm3`	background density [kg/m^3]
`fm.coef`	1 x (L+1)^2 fully normalized coefficients about `geo.origin_km`, packed as `geo.pk` (C00, C10, C11, S11, C20, ...)
`fm.com_km`	center of mass in shape-model coordinates [km]
`fm.inertia_norm`	[Ixx Iyy Izz Ixy Ixz Iyz] about `geo.origin_km` / (M `R_ref`^2)
`fm.mass_kg`	`mass_kg`

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`.

```
geo = sbtk.gravity.geometry(body, [0 0 0], 4);
fm  = sbtk.gravity.forward(geo, interior, body.mass_kg);
```

Related functions

Mentioned in the help text

`sbtk.gravity.geometry`

Calls

`sbtk.gravity.component_integrals`, `sbtk.gravity.random_field_integrals`

Called by

`sbtk.data.simulate`, `sbtk.plot.difference_from_homogeneous`

Implementation: `toolbox/+sbtk/+gravity/forward.m`, line 1.

`sbtk.gravity.geometry`

Facet-cone geometry of a body about an expansion origin (the integration workspace).

Syntax

MATLAB

```
geo = sbtk.gravity.geometry(body, origin_km, degree)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create Fields: `body`.
`origin_km`	1 × 3	double	km	required	expansion origin in shape-model coordinates [km] (1x3). Gravity coefficients, first moments and inertia are all about this point; layers and sectors are measured along rays from it. Validation: `mustBeFinite`.
`degree`	scalar	double		required	highest spherical-harmonic degree (0 when only volumes, inertia or densities are needed: much faster) Validation: `mustBeInteger, mustBeNonnegative`.

Outputs

Name	Description
`geo`	The facet-cone geometry and integrals. Fields: `geo`.

Description

The body is the union of tetrahedral cones with apex at the origin and a surface facet as base. For every cone geo stores, per unit density,

`T`	spherical-harmonic integrals int (r/R)^l `Ybar_lm` dV [km^3] (nf x (L+1)^2)
`V`	volume [km^3], m1 first moment [km^4],
`Q`	second moments [xx yy zz xy xz yz] [km^5]

(T from the polyhedral kernel sbtk.internal.ComputePolyhedralCSBar_mex). Shrinking a cone about its apex by s multiplies r^l dV by s^(l+3), so the portion of a cone between the scaled facets `s_a` and `s_b` contributes exactly T (`s_b`^(l+3) - `s_a`^(l+3)), V (`s_b`^3 - `s_a`^3), m1 (`s_b`^4 - `s_a`^4), Q (`s_b`^5 - `s_a`^5). Interior components are described by such radial intervals along 9 sub-rays of every cone (sbtk.gravity.component_integrals).

geo fields (km units throughout)

```
body, origin_km, degree, R_ref_km, pk (coefficient packing, sbtk.internal.sh_packing)
nf, K                number of facets, sub-rays per facet (9)
faces, verts         mesh with vertices relative to origin_km
D, E, F, det6, EF, FD, DE    facet corners, 6 x cone volumes, edge cross products
T, V, m1, Q          per-cone integrals (above)
U_body, V_body, m1_body, Q_body   whole-body sums
ray_t, ray_dir       sub-ray surface distances [km] and unit directions (nf*K rows)
```

Didymos (49,152 facets): about 1 s at degree 4.

Errors when some facet faces the origin (shape not star-shaped about it).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`.

```
geo = sbtk.gravity.geometry(body, body.centroid_km, 4);
fprintf('%d facets, volume %.4f km^3\n', geo.nf, geo.V_body);
```

Related functions

Mentioned in the help text

sbtk.body.create, sbtk.gravity.component_integrals, sbtk.internal.ComputePolyhedralCSBar_mex, sbtk.internal.sh_packing

Called by

sbtk.compare.reference, sbtk.data.simulate, sbtk.exchange.density_grid, sbtk.exchange.read_data, sbtk.gravity.center_of_mass_frame, sbtk.invert.setup, sbtk.plot.difference_from_homogeneous, sbtk.plot.interior3d, sbtk.plot.slices, sbtk.plot.truth_vs_estimate

Implementation: `toolbox/+sbtk/+gravity/geometry.m`, line 1.

`sbtk.gravity.labels`

Names of the gravity coefficients up to a degree, in SBTK order.

Syntax

MATLAB

```
lab = sbtk.gravity.labels(degree)     % e.g. degree 2 -> {'C00','C10','C11','S11','C20','C21','S21','C22','S22'}
```

Arguments

Name	Size	Class	Unit	Default	Description
`degree`	scalar	double		required	Highest degree L. Validation: `mustBeInteger, mustBeNonnegative`.

Outputs

Name	Description
`lab`	1 × (L+1)² cell array of names in SBTK order, 'C00' … .

Example

MATLAB, from the help text

```
lab = sbtk.gravity.labels(4);              % 25 labels, C00 ... S44
```

Implementation: `toolbox/+sbtk/+gravity/labels.m`, line 1.

`sbtk.gravity.random_field_integrals`

Density-weighted integrals of the random density field.

Syntax

MATLAB

```
I = sbtk.gravity.random_field_integrals(geo, field)
```

Arguments

Name	Size	Class	Unit	Default	Description
`geo`	1 × 1	struct		required	Geometry from `sbtk.gravity.geometry`. Fields: `geo`.
`field`	1 × 1	struct		required	The random field. Fields: `random_field`.

Outputs

Name	Description
`I`	Density-weighted integrals of the random field. Fields: `integrals`.

Description

`I.U` = int drho (r/R)^l `Ybar_lm` dV, `I.V` = int drho dV, `I.m1`, `I.Q` likewise (units: kg/m^3 times km^n). The field is evaluated at the centroid of every radial shell of every facet cone (sbtk.internal.body_samples) and each shell contributes its exact cone-scaling integrals with that value. No field (`sigma_kgm3` = 0) gives zeros without sampling.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`.

```
geo = sbtk.gravity.geometry(body, [0 0 0], 4);
I = sbtk.gravity.random_field_integrals(geo, sbtk.interior.random_field(40, 0.06, 7));
```

Related functions

Mentioned in the help text

`sbtk.gravity.geometry`, `sbtk.interior.random_field`, `sbtk.internal.body_samples`

Called by

`sbtk.gravity.forward`

Implementation: `toolbox/+sbtk/+gravity/random_field_integrals.m`, line 1.

`sbtk.data`: Measurement sets

Function	Purpose
`add_center_of_mass`	Add degree-1 coefficients that place the center of mass at a given point.
`add_libration`	Add a measured libration ratio (B - A)/C.
`add_moi`	Add one measured inertia element to a measurement set.
`add_moi_row`	Add a measurement of any weighted sum of the inertia elements.
`add_noise`	One noisy realization of a measurement set.
`from_coefficients`	Measurement set from gravity-coefficient matrices (e.g. a mission solution).
`libration_ratio_from_amplitude`	(B - A)/C from the forced libration of a synchronous satellite.
`move_origin`	Express measurements about the center of mass relative to a new expansion origin.
`select_coefficients`	Keep only the listed gravity coefficients.
`select_degree`	Keep only the coefficients up to a degree.
`select_moi`	Keep only the listed moment-of-inertia measurements.
`set_covariance`	Replace the coefficient covariance with a full matrix.
`set_moi_sigma`	Set the 1-sigma of moment-of-inertia measurements by label.
`set_uncertainties`	Give measurements a 1-sigma uncertainty per degree and per inertia quantity.
`simulate`	True model of an interior and its exact gravity coefficients and moments of inertia.
`summary`	Print what is in a measurement set.
`to_frame`	A measurement set in another frame: x' = (x - `origin_km`) * axes.
`uncertainties`	1-sigma measurement uncertainties by degree and by inertia quantity, for sbtk.data.set_uncertainties.
`use_uncertainties`	Give measurements the coefficient uncertainties of another data set (e.g. a precision file).

`sbtk.data.add_center_of_mass`

Add degree-1 coefficients that place the center of mass at a given point.

Syntax

MATLAB

```
obs = sbtk.data.add_center_of_mass(obs, com_km, sigma_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set, without degree-1 coefficients. Fields: `obs`.
`com_km`	1 × 3	double	km	required	center of mass in shape-model coordinates [km]; `obs.origin_km` when the coefficients are expanded about the center of mass (degree 1 = 0) Validation: `mustBeFinite`.
`sigma_km`	scalar	double	km	required	1-sigma uncertainty of each component of the center of mass [km] Validation: `mustBePositive`.

Outputs

Name	Description
`obs`	The measurement set with C10, C11 and S11 added and `coef_about` set to '`center_of_mass`'. Fields: `obs`.

Description

With fully normalized coefficients, C10 = dz/(sqrt(3) R), C11 = dx/(sqrt(3) R), S11 = dy/(sqrt(3) R), with [dx dy dz] = `com_km` - `obs.origin_km`. In an inversion these terms constrain the first moment of the density, which sphere positions (`estimate_position`) can satisfy. Errors if the data already have degree-1 terms.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `file`, `body`.

```
% a field about the center of mass, which is at the shape-model origin within 1 m
obs = sbtk.io.read_jpl_gravity(file, body, "bennu_jpl", [0 0 0], 4);
obs = sbtk.data.add_center_of_mass(obs, [0 0 0], 0.001);
```

Related functions

Mentioned in the help text

`sbtk.io.read_jpl_gravity`

Called by

`sbtk.invert.setup`

Implementation: `toolbox/+sbtk/+data/add_center_of_mass.m`, line 1.

`sbtk.data.add_libration`

Add a measured libration ratio (B - A)/C.

Syntax

MATLAB

```
obs = sbtk.data.add_libration(obs, ratio, ratio_sigma, Izz_norm)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set; it must contain C22 and S22. Fields: `obs`.
`ratio`	scalar	double		required	measured (B - A)/C: A <= B are the principal moments of inertia about the axes in the xy-plane (the long and intermediate axes of a body spinning about z) and C = Izz. This is what the libration of a synchronously rotating body depends on (amplitude 2e/(3 ratio - 1) for forced libration in an orbit of eccentricity e). Validation: `mustBeFinite`.
`ratio_sigma`	scalar	double		required	its 1-sigma Validation: `mustBePositive`.
`Izz_norm`	scalar	double	M R_ref²	required	Izz / (M `R_ref`^2) used to scale the uncertainty (a homogeneous-body value is accurate enough) Validation: `mustBePositive`.

Outputs

Name	Description
`obs`	The measurement set with the `libration` row added. Fields: `obs`.

Description

The principal axes need not be the shape-model axes: their direction in the xy-plane comes from the measured C22 and S22 of obs (which must have them), and the measurement is stored as the linear row

```
    cos(2a) (Iyy − Ixx) + sin(2a) 2 Ixy − ratio Izz = 0 +− ratio_sigma Izz_norm,
```

(B - A) written with the principal-axis angle a (sbtk.internal.libration_weights). Together with C22 and S22, which give B - A, the ratio measures C to a relative precision of about ratio_sigma/ratio.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
% (B − A)/C = 0.20 +− 0.03, with Izz/(M R_ref^2) about 0.39
obs = sbtk.data.add_libration(obs, 0.20, 0.03, 0.39);
```

Related functions

Mentioned in the help text

`sbtk.internal.libration_weights`

Calls

`sbtk.data.add_moi_row`

Called by

`sbtk.exchange.read_data`

Implementation: `toolbox/+sbtk/+data/add_libration.m`, line 1.

`sbtk.data.add_moi`

Add one measured inertia element to a measurement set.

Syntax

```
MATLAB
obs = sbtk.data.add_moi(obs, label, value_norm, sigma_norm)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`label`	1 × 1	string		required	'Ixx', 'Iyy', 'Izz', 'Ixy', 'Ixz' or 'Iyz' (about `obs.origin_km`, shape-model axes; products of inertia with the minus sign) Validation: `mustBeMember(label, ["Ixx", "Iyy", "Izz", "Ixy", "Ixz", "Iyz"])`.
`value_norm`	scalar	double	M R_ref²	required	measured I / (M `R_ref`^2) (sbtk.units.moi_to_norm converts kg m^2) Validation: `mustBeFinite`.
`sigma_norm`	scalar	double	M R_ref²	required	its 1-sigma, same unit Validation: `mustBePositive`.

Outputs

Name	Description
`obs`	The measurement set with the row added. Fields: `obs`.

Description

For any other combination of elements use sbtk.data.add_moi_row.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
obs = sbtk.data.add_moi(obs, "Izz", 0.0752, 2e-3);
```

Related functions

Mentioned in the help text

`sbtk.data.add_moi_row`, `sbtk.units.moi_to_norm`

Calls

`sbtk.data.add_moi_row`

Called by

`sbtk.exchange.read_data`

Implementation: `toolbox/+sbtk/+data/add_moi.m`, line 1.

`sbtk.data.add_moi_row`

Add a measurement of any weighted sum of the inertia elements.

Syntax

```
MATLAB
obs = sbtk.data.add_moi_row(obs, label, weights, value, sigma)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`label`	1 × 1	string		required	Name of the new row; must differ from the existing labels.
`weights`	1 × 6	double		required	Weights on $[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}]$ / (MR_{ref}^2). Validation: `mustBeFinite`.
`value`	scalar	double		required	Measured value of the weighted sum. Validation: `mustBeFinite`.
`sigma`	scalar	double		required	Its 1σ. Validation: `mustBePositive`.

Outputs

Name	Description
`obs`	The measurement set with the row added. Fields: `obs`.

Description

The measurement is weights * [Ixx Iyy Izz Ixy Ixz Iyz]' / (M `R_ref`^2) = value +- sigma. label must not already be in `obs.moi`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
% Izz - (Ixx + Iyy)/2 = 0.09 +- 0.002 (as a fraction of M R_ref^2)
obs = sbtk.data.add_moi_row(obs, "polar_minus_mean", [-0.5 -0.5 1 0 0 0], 0.09, 0.002);
```

Related functions

Called by

`sbtk.data.add_libration`, `sbtk.data.add_moi`

Implementation: `toolbox/+sbtk/+data/add_moi_row.m`, line 1.

`sbtk.data.add_noise`

One noisy realization of a measurement set.

Syntax

MATLAB

```
obs = sbtk.data.add_noise(obs, seed)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	Measurement set with exact values and stated uncertainties. Fields: `obs`.
`seed`	scalar	double		required	Seed of the random number stream (non-negative integer); the same seed gives the same noise. Validation: `mustBeInteger, mustBeNonnegative`.

Outputs

Name	Description
`obs`	One noisy realization: coefficients plus noise drawn from `coef_cov` (full covariance), each inertia row plus noise from its σ; `noise_seed` set. Fields: `obs`.

Description

Adds Gaussian noise drawn from the stated uncertainties: the full coefficient covariance and each moment-of-inertia row's sigma. The same seed gives the same noise. The informational tensor `obs.moi_tensor_norm` is rebuilt from the noisy rows and the seed stored in `obs.noise_seed`. Errors if the measurements have no uncertainties (the exact values of sbtk.data.simulate).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`, `unc`.

```
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");
exact = sbtk.data.set_uncertainties(exact, unc);
obs = sbtk.data.add_noise(exact, 1);        % seed 1: the same noise every time
```

Related functions

Mentioned in the help text

`sbtk.data.set_uncertainties`, `sbtk.data.simulate`

Calls

`sbtk.data.set_covariance`, `sbtk.data.set_moi_sigma`, `sbtk.data.set_uncertainties`, `sbtk.data.use_uncertainties`

Called by

`sbtk.data.to_frame`

Implementation: `toolbox/+sbtk/+data/add_noise.m`, line 1.

`sbtk.data.from_coefficients`

Measurement set from gravity-coefficient matrices (e.g. a mission solution).

Syntax

MATLAB

```
obs = sbtk.data.from_coefficients(body, name, origin_km, C, S, C_sigma, S_sigma)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create: gives the mass and `R_ref`. `R_ref` must be the reference radius the coefficients are normalized with. Fields: `body`.
`name`	1 × 1	string		required	name of this data set
`origin_km`	1 × 3	double	km	required	point the coefficients are expanded about, in shape-model coordinates [km] (usually the center of mass) Validation: `mustBeFinite`.
`C`	any × any	double		required	(L+1) x (L+1) fully normalized coefficients, C(l+1, m+1) = `C_lm` (Kaula normalization, no Condon-Shortley phase); entries with m > l, the column S(:,1) and C00 are ignored Validation: `mustBeFinite`.
`S`	any × any	double		required	(L+1) x (L+1) fully normalized coefficients, C(l+1, m+1) = `C_lm` (Kaula normalization, no Condon-Shortley phase); entries with m > l, the column S(:,1) and C00 are ignored Validation: `mustBeFinite`.
`C_sigma`	any × any	double		required	1-sigma uncertainties, same layout (must be > 0 where used) Validation: `mustBeNonnegative`.
`S_sigma`	any × any	double		required	1-sigma uncertainties, same layout (must be > 0 where used) Validation: `mustBeNonnegative`.

Outputs

Name	Description
`obs`	The measurement set: coefficients of degree 1 … L, diagonal covariance from the uncertainties, no inertia rows. Fields: `obs`.

Description

Degrees 1..L are used. Add moments of inertia with sbtk.data.add_moi / `add_libration` and a full covariance with sbtk.data.set_covariance.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `C`, `S`, `Cs`, `Ss`.

```
obs = sbtk.data.from_coefficients(body, "hera_v1", [0 0 0], C, S, Cs, Ss);
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.data.add_moi`, `sbtk.data.set_covariance`

Calls

`sbtk.units.G`

Called by

`sbtk.exchange.read_shadr`

Implementation: `toolbox/+sbtk/+data/from_coefficients.m`, line 1.

`sbtk.data.libration_ratio_from_amplitude`

(B - A)/C from the forced libration of a synchronous satellite.

Syntax

MATLAB

```
[ratio, ratio_sigma] = sbtk.data.libration_ratio_from_amplitude(amplitude_deg, amplitude_sigma_deg, e, e_sigma)
```

Arguments

Name	Size	Class	Unit	Default	Description
`amplitude_deg`	scalar	double	deg	required	signed amplitude G of the forced libration gamma = G sin(M) [deg]: gamma is the angle of the long axis from the direction to the primary, M the mean anomaly. G < 0 (the long axis lags behind the primary direction while the satellite moves fastest) for (B - A)/C < 1/3, which covers elongations up to about 1.4 : 1; G > 0 above that. Validation: `mustBeNonzero`.
`amplitude_sigma_deg`	scalar	double	deg	required	1-sigma of G [deg] Validation: `mustBeNonnegative`.
`e`	scalar	double		required	orbital eccentricity and its 1-sigma Validation: `mustBeInRange(e, 0, 1, 'exclude-upper')`.

Name	Size	Class	Unit	Default	Description
`e_sigma`	scalar	double		required	orbital eccentricity and its 1-sigma Validation: `mustBeNonnegative`.

Outputs

Name	Description
`ratio`	(B - A)/C (principal moments, see sbtk.data.add_libration) and its 1-sigma, from first-order error propagation
`ratio_sigma`	(B - A)/C (principal moments, see sbtk.data.add_libration) and its 1-sigma, from first-order error propagation

Description

Planar forced libration in a Keplerian orbit about a point-mass primary, to first order in e: gamma'' + 3 n^2 ratio gamma = 2 e n^2 sin(M), so G = 2e/(3 ratio - 1) and ratio = (1 + 2e/G)/3. Free libration, the primary's oblateness and the mutual dynamics of a binary are not included. The sensitivity grows near ratio = 1/3, where the free libration period equals the orbital period: an elongated body gives a large amplitude and a precise ratio, a nearly axisymmetric one a small amplitude that depends little on the ratio.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
% amplitude −12 +− 0.2 deg, e = 0.027 +− 0.001: (B − A)/C = 0.247 +− 0.0035 (1.4%)
[k, sk] = sbtk.data.libration_ratio_from_amplitude(−12, 0.2, 0.027, 0.001);
obs = sbtk.data.add_libration(obs, k, sk, 0.39);
% a nearly axisymmetric body: −4.0 +− 0.2 deg gives 0.075 +− 0.016 (21%)
```

Related functions

Mentioned in the help text

sbtk.data.add_libration

Implementation: `toolbox/+sbtk/+data/libration_ratio_from_amplitude.m`, line 1.

---

sbtk.data.move_origin

Express measurements about the center of mass relative to a new expansion origin.

Syntax

MATLAB

```
obs = sbtk.data.move_origin(obs, origin_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	Measurement set with `coef_about` = '`center_of_mass`'. Fields: `obs`.
`origin_km`	1 × 3	double	km	required	New expansion origin in the shape-model frame. Validation: `mustBeFinite`.

Outputs

Name	Description
`obs`	The same measurements with the degree-1 values relative to the new origin. Fields: `obs`.

Description

For `obs.coef_about` = '`center_of_mass`' only: the coefficients of degree >= 2 are about the body's center of mass wherever the origin is, and the degree-1 coefficients give the measured center of mass relative to the origin (C10 = dz/(sqrt(3) R), C11 = dx/(sqrt(3) R), S11 = dy/(sqrt(3) R)). Moving the origin changes only the degree-1 values; uncertainties are unchanged.

Example

MATLAB, from the help text

```
obs = sbtk.exchange.read_data("UCB-DATA1-Didymos");
moved = sbtk.data.move_origin(obs, obs.origin_km + [0.001 0 0]);     % 1 m along x
```

Related functions

Mentioned in the help text

sbtk.exchange.read_data

Implementation: `toolbox/+sbtk/+data/move_origin.m`, line 1.

`sbtk.data.select_coefficients`

Keep only the listed gravity coefficients.

Syntax

```
MATLAB
obs = sbtk.data.select_coefficients(obs, labels)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`labels`	1 × any	string		required	cell array or string array of names, e.g. {'C20', 'C22', 'S22'}

Outputs

Name	Description
`obs`	The measurement set with only the listed coefficients and the matching rows and columns of the covariance. Fields: `obs`.

Description

Use it to fit a subset of the coefficients (e.g. only those a mission determined well). The covariance keeps the matching rows and columns. See also sbtk.data.select_degree.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
obs = sbtk.data.select_coefficients(obs, ["C10" "C11" "S11" "C20" "C22" "S22"]);
```

Related functions

Mentioned in the help text

`sbtk.data.select_degree`

Called by

`sbtk.exchange.read_shadr`

Implementation: `toolbox/+sbtk/+data/select_coefficients.m`, line 1.

`sbtk.data.select_degree`

Keep only the coefficients up to a degree.

Syntax

```
MATLAB
obs = sbtk.data.select_degree(obs, degree)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`degree`	scalar	double		required	Highest degree to keep. Validation: `mustBeInteger, mustBePositive`.

Outputs

Name	Description
`obs`	The measurement set with the coefficients up to that degree. Fields: `obs`.

Description

Useful to test what a lower-degree solution can tell about the interior.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
obs2 = sbtk.data.select_degree(obs, 2);      % degrees 1 and 2 only
```

Implementation: `toolbox/+sbtk/+data/select_degree.m`, line 1.

`sbtk.data.select_moi`

Keep only the listed moment-of-inertia measurements.

Syntax

```
MATLAB
obs = sbtk.data.select_moi(obs, labels)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`labels`	1 × any	string		required	e.g. {'Izz', 'libration'}; {} keeps none

Outputs

Name	Description
`obs`	The measurement set with only the listed inertia rows. Fields: `obs`.

Description

Use it to decide which inertia measurements a data set contains (for example before sbtk.exchange.write_data). See also sbtk.data.select_coefficients.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
obs = sbtk.data.select_moi(obs, ["Izz" "libration"]);
obs = sbtk.data.select_moi(obs, {});                % gravity coefficients only
```

Related functions

Mentioned in the help text

`sbtk.data.select_coefficients`, `sbtk.exchange.write_data`

Called by

`sbtk.data.set_moi_sigma`, `sbtk.data.set_uncertainties`, `sbtk.data.to_frame`

Implementation: `toolbox/+sbtk/+data/select_moi.m`, line 1.

`sbtk.data.set_covariance`

Replace the coefficient covariance with a full matrix.

Syntax

```
MATLAB
obs = sbtk.data.set_covariance(obs, coef_cov)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`coef_cov`	any × any	double		required	n x n covariance of `obs.coef`, in the order of `obs.coef_labels` (symmetric positive definite) Validation: `mustBeFinite`.

Outputs

Name	Description
`obs`	The measurement set with `coef_cov` replaced. Fields: `obs`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
sig = sqrt(diag(obs.coef_cov));
obs = sbtk.data.set_covariance(obs, (0.3 + 0.7*eye(numel(sig))).*(sig*sig'));   % 30% correlation
```

Related functions

Called by

sbtk.data.add_noise, sbtk.exchange.write_data, sbtk.invert.replace_data, sbtk.invert.setup

Implementation: `toolbox/+sbtk/+data/set_covariance.m`, line 1.

---

sbtk.data.set_moi_sigma

Set the 1-sigma of moment-of-inertia measurements by label.

Syntax

MATLAB

```
obs = sbtk.data.set_moi_sigma(obs, sigma)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`sigma`	1 × 1	struct		required	struct whose field names are measurement labels and whose values are 1-sigma: Ixx, Iyy, Izz, Ixy, Ixz, Iyz (or a label of sbtk.data.add_moi_row) as a fraction of M `R_ref`^2 libration of the ratio (B - A)/C; the row's 1-sigma becomes this times the Izz / (M `R_ref`^2) of the Izz row Rows not named keep their sigma. Errors for a label the data do not have.

Outputs

Name	Description
`obs`	The measurement set with the σ of the named rows replaced. Fields: `obs`.

Example

MATLAB fragment. Uses variables defined by earlier calls: `obs`, `body`.

```
% 1-sigma of the diagonal elements given in kg km^2: divide by M R_ref^2 in kg km^2
s = struct('Ixx', 2.0e9, 'Iyy', 2.0e9, 'Izz', 2.0e9);
obs = sbtk.data.set_moi_sigma(obs, structfun(@(v) v/(body.mass_kg*body.R_ref_km^2), s, 'UniformOutput', false));
```

Related functions

Mentioned in the help text

sbtk.data.add_moi_row

Calls

sbtk.data.select_moi

Called by

sbtk.data.add_noise, sbtk.exchange.write_data, sbtk.invert.replace_data, sbtk.invert.setup

Implementation: `toolbox/+sbtk/+data/set_moi_sigma.m`, line 1.

---

sbtk.data.set_uncertainties

Give measurements a 1-sigma uncertainty per degree and per inertia quantity.

Syntax

MATLAB

```
obs = sbtk.data.set_uncertainties(obs, unc)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	measurements, e.g. the exact values from sbtk.data.simulate, which have none Fields: `obs`.
`unc`	1 × 1	struct		required	from sbtk.data.uncertainties Fields: `unc`.

Outputs

Name	Description
`obs`	measurements, e.g. the exact values from sbtk.data.simulate, which have none Fields: `obs`.

Description

Every gravity coefficient of degree l gets the 1-sigma `unc.coef_sigma_by_degree`(l), and the coefficient covariance becomes diagonal. Every inertia element (Ixx, Iyy, Izz, Ixy, Ixz, Iyz) gets `unc.moi_sigma`. The libration row gets `unc.libration_sigma` times the Izz / (M `R_ref`^2) of the Izz row (set the uncertainties before sbtk.data.select_moi removes that row). Rows with other labels (sbtk.data.add_moi_row) keep their sigma. The values themselves do not change; sbtk.data.add_noise then draws noise from these uncertainties.

For other uncertainties: sbtk.data.use_uncertainties (those of another data set), sbtk.data.set_covariance (a full coefficient covariance), sbtk.data.set_moi_sigma (inertia rows by label).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`.

```
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
```

Related functions

Mentioned in the help text

sbtk.data.add_moi_row, sbtk.data.add_noise, sbtk.data.select_moi, sbtk.data.set_covariance, sbtk.data.set_moi_sigma, sbtk.data.simulate, sbtk.data.uncertainties, sbtk.data.use_uncertainties, sbtk.units.offset_to_degree1

Calls

sbtk.data.select_moi

Called by

sbtk.data.add_noise, sbtk.data.summary, sbtk.data.to_frame, sbtk.exchange.write_data, sbtk.invert.replace_data, sbtk.invert.setup

Implementation: `toolbox/+sbtk/+data/set_uncertainties.m`, line 1.

`sbtk.data.simulate`

True model of an interior and its exact gravity coefficients and moments of inertia.

Syntax

MATLAB

```
[exact, truth, geo] = sbtk.data.simulate(body, interior, degree, name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create (shape, total mass, reference radius) Fields: `body`.
`interior`	1 × 1	struct		required	from sbtk.interior.create (components + random field) Fields: `interior`.
`degree`	scalar	double		required	maximum degree of the gravity coefficients Validation: `mustBeInteger, mustBePositive`.
`name`	1 × 1	string		required	name of the data set, e.g. "`didymos_T1`"

Outputs

Name	Description
`exact`	the exact values as a measurement set (docs/data_formats.md), without uncertainties (`coef_cov` and every inertia sigma are NaN); for Didymos to degree 4: `coef_labels` 24 x 1 {'C10';'C11';'S11';'C20';...}, coef 24 x 1, moi 7 rows (Ixx Iyy Izz Ixy Ixz Iyz, I/(M `R_ref`^2); libration: (B - A)/C with principal moments, see sbtk.data.add_libration), `origin_km` the center of mass the coefficients are about [km] Fields: `obs`.
`truth`	the true model (`rho_bg_kgm3`, components, `random_field`, `origin_km`, exact values) Fields: `truth`.
`geo`	geometry about the center of mass (for plots and comparisons) Fields: `geo`.

Description

The gravity field and inertia are expanded about the center of mass. Layers and sectors are defined along rays from the expansion origin, so the interior, and with it the center of mass, depend on the origin: the geometry is rebuilt about the center of mass until it moves by less than 1 mm (sbtk.internal.fixed_point, at most 50 times; sbtk:iteration:notConverged otherwise). No measurement uncertainties are involved: the values are exact.

To make measurements of the true model, give the exact values uncertainties (sbtk.data.set_uncertainties: one 1-sigma per degree and per inertia quantity; sbtk.data.use_uncertainties: those of another data set; sbtk.data.set_covariance, sbtk.data.set_moi_sigma) and draw noise from them (sbtk.data.add_noise).

Errors when the density is negative anywhere.

Example

MATLAB, from the help text

```
body = sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43);
interior = sbtk.interior.create(sbtk.interior.sphere("core", [0 0 0], 0.170, +450));
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");          % the true model
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);  % measurements of it
```

Related functions

Mentioned in the help text

sbtk.body.create, sbtk.body.from_mass, sbtk.data.add_libration, sbtk.data.add_noise, sbtk.data.set_covariance, sbtk.data.set_moi_sigma, sbtk.data.set_uncertainties, sbtk.data.uncertainties, sbtk.data.use_uncertainties, sbtk.interior.create, sbtk.interior.sphere, sbtk.internal.fixed_point, sbtk.shape.bundled, sbtk.units.offset_to_degree1

Calls

sbtk.gravity.forward, sbtk.gravity.geometry, sbtk.units.G

Called by

sbtk.data.to_frame, sbtk.interior.align_with_spin_axis

Implementation: `toolbox/+sbtk/+data/simulate.m`, line 1.

---

sbtk.data.summary

Print what is in a measurement set.

Syntax

MATLAB

```
sbtk.data.summary(obs)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set to describe. Fields: `obs`.

Example output

Output

```
Didymos measurements (didymos_T1)
  mass 5.273e+11 kg, reference radius 430 m, gravity to degree 4
  24 gravity coefficients: 3 of degree 1, 5 of degree 2, 7 of degree 3, 9 of degree 4
  7 moment-of-inertia measurements: Ixx Iyy Izz Ixy Ixz Iyz libration
```

Related functions

Calls

sbtk.data.set_uncertainties

Called by

sbtk.io.report_simulation

Implementation: `toolbox/+sbtk/+data/summary.m`, line 1.

---

sbtk.data.to_frame

A measurement set in another frame: x' = (x - `origin_km`) * axes.

Syntax

MATLAB

```
obs = sbtk.data.to_frame(obs, origin_km, axes)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	a measurement set; to change the axes it must have all six inertia elements without uncertainties, as the exact values of sbtk.data.simulate have (or no inertia rows): a measurement is made in the frame it is reported in, so transform the exact values, then give uncertainties, select and add noise Fields: `obs`.

Name	Size	Class	Unit	Default	Description
`origin_km`	1 × 3	double	km	required	origin of the new frame in the current frame [km]; [0 0 0] keeps the origin Validation: `mustBeFinite`.
`axes`	3 × 3	double		required	3 x 3 rotation, columns: the new axes as unit vectors in the current frame, e.g. cf.axes of sbtk.gravity.center_of_mass_frame; eye(3) keeps the axes Validation: `mustBeFinite`.

Outputs

Name	Description
`obs`	a measurement set; to change the axes it must have all six inertia elements without uncertainties, as the exact values of sbtk.data.simulate have (or no inertia rows): a measurement is m bs.

Description

`origin_km` becomes (`origin_km` - o)*axes and `com_minus_cof_km` becomes `com_minus_cof_km`*axes. The coefficients stay about the same point and are rotated degree by degree, with their covariance (exact, sbtk.internal.sh_rotation); every degree present must be complete. The inertia elements Ixx ... Iyz are recomputed as the elements in the new axes (axes'*I*axes); the libration row and `libration_ratio` use the new z-axis (C = Izz) and the principal axes in the new xy-plane; any other row keeps its meaning (its weights are transformed). A translation alone (axes = eye(3)) applies to any set. Use the same `origin_km` and axes for the body (sbtk.body.to_frame) and the interior or true model (sbtk.interior.to_frame).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `truth`, `body`, `exact`, `unc`.

```
spin    = sbtk.gravity.center_of_mass_frame(truth, body, "spin");
exact_s = sbtk.data.to_frame(exact, spin.com_km, spin.axes);      % C21 = S21 = 0
obs_s   = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact_s, unc), 1);
```

Related functions

Mentioned in the help text

sbtk.body.to_frame, sbtk.data.add_noise, sbtk.data.set_uncertainties, sbtk.data.simulate, sbtk.gravity.center_of_mass_frame, sbtk.interior.to_frame, sbtk.internal.sh_rotation

Calls

sbtk.data.add_noise, sbtk.data.select_moi, sbtk.data.set_uncertainties, sbtk.data.simulate

Called by

sbtk.interior.to_frame

Implementation: `toolbox/+sbtk/+data/to_frame.m`, line 1.

---

sbtk.data.uncertainties

1-sigma measurement uncertainties by degree and by inertia quantity, for sbtk.data.set_uncertainties.

Syntax

MATLAB

```
unc = sbtk.data.uncertainties(coef_sigma_by_degree, moi_sigma, libration_sigma)
```

Arguments

Name	Size	Class	Unit	Default	Description
`coef_sigma_by_degree`	1 × any	double		required	1 x L: 1-sigma of every fully normalized coefficient of degree 1, 2, ..., L (dimensionless). A center-of-mass uncertainty of d km corresponds to d/(sqrt(3) `R_ref`) at degree 1. Validation: `mustBePositive`.
`moi_sigma`	scalar	double		required	1-sigma of each element Ixx..Iyz / (M `R_ref`^2) Validation: `mustBePositive`.
`libration_sigma`	scalar	double		required	1-sigma of the libration ratio (B - A)/C (principal moments) Validation: `mustBePositive`.

Outputs

Name	Description
`unc`	The uncertainties. Fields: `unc`.

Description

Only measurements use it: the true model and its exact values (sbtk.data.simulate) do not. sbtk.data.set_uncertainties gives them to a measurement set.

Example

Didymos placeholders

MATLAB fragment from the help text. Uses variables defined by earlier calls: `exact`.

```
unc = sbtk.data.uncertainties([sbtk.units.offset_to_degree1(0.002, 0.43), 1e-4, 2e-4, 4e-4], 2e-3, 0.02);
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
```

Related functions

Mentioned in the help text

`sbtk.data.add_noise`, `sbtk.data.set_uncertainties`, `sbtk.data.simulate`, `sbtk.units.offset_to_degree1`

Implementation: `toolbox/+sbtk/+data/uncertainties.m`, line 1.

`sbtk.data.use_uncertainties`

Give measurements the coefficient uncertainties of another data set (e.g. a precision file).

Syntax

MATLAB

```
obs = sbtk.data.use_uncertainties(obs, ref)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	measurements whose uncertainties are replaced (usually exact values from sbtk.data.simulate, before sbtk.data.add_noise) Fields: `obs`.
`ref`	1 × 1	struct		required	measurements holding the uncertainties to use, e.g. a precision file read with sbtk.exchange.read_shadr or sbtk.exchange.read_data Fields: `ref`.

Outputs

Name	Description
`obs`	measurements whose uncertainties are replaced (usually exact values from sbtk.data.simulate, before sbtk.data.add_noise) Fields: `obs`.

Description

Every coefficient of obs that ref also has takes ref's variances and covariances; the others (e.g. degree 1 if ref has none) keep theirs and become uncorrelated with them (exact values from sbtk.data.simulate have none: give them with sbtk.data.set_uncertainties first). If the reference radii differ, ref's covariance is converted to `obs.R_ref_km` first: a coefficient of degree l scales by (`R_ref` / `R_obs`)^l. Moment-of-inertia uncertainties are not changed (sbtk.data.set_moi_sigma sets them).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`.

```
shape = sbtk.shape.load("didymos_g_9309mm_spc_obj_0000n00000_v003.obj", "km");
prec  = sbtk.exchange.read_shadr("Didymos_gravity_fromRS.tab", "Didymos", shape, [0 0 0], 4);
[exact, truth] = sbtk.data.simulate(body, interior, 4, "didymos_T1");
exact = sbtk.data.use_uncertainties(exact, prec);      % every coefficient, degree 1 included
exact = sbtk.data.select_moi(exact, {});               % gravity only in this example
obs = sbtk.data.add_noise(exact, 1);
```

Related functions

Mentioned in the help text

`sbtk.data.add_noise`, `sbtk.data.select_moi`, `sbtk.data.set_moi_sigma`, `sbtk.data.set_uncertainties`, `sbtk.data.simulate`, `sbtk.exchange.read_data`, `sbtk.exchange.read_shadr`, `sbtk.shape.load`

Called by

`sbtk.data.add_noise`, `sbtk.exchange.write_data`, `sbtk.invert.replace_data`, `sbtk.invert.setup`

Implementation: `toolbox/+sbtk/+data/use_uncertainties.m`, line 1.

Reference

`sbtk.invert`: Density estimation

Function	Purpose
`choose_regularization`	Tikhonov regularization parameter by the discrepancy principle.
`goodness_of_fit`	Reduced chi-square and chi-square test of a fit.
`homogeneous`	Fit a homogeneous body (no components) to the same measurements.
`options`	Inversion settings with their standard values.
`replace_data`	Use another realization of the same measurements without repeating the setup.
`run`	Estimate the densities for a prepared inversion (bounded, mass-constrained least squares).
`setup`	Prepare an inversion: geometry and the design-matrix column of every component, computed once.
`solve`	Setup and run in one call.
`summary`	Print an inversion result.

`sbtk.invert.choose_regularization`

Tikhonov regularization parameter by the discrepancy principle.

Syntax

MATLAB

```
lambda = sbtk.invert.choose_regularization(prob)
prob.opts.regularization = lambda;  fit = sbtk.invert.run(prob);
```

Arguments

Name	Size	Class	Unit	Default	Description
`prob`	1 × 1	struct		required	Prepared inversion; `prob.opts.regularization` is not used. Fields: `prob`.

Outputs

Name	Description
`lambda`	Tikhonov parameter chosen by the discrepancy principle.

Description

The regularization term lambda * sum((`drho_c` / `rho_mean`)^2) damps the relative densities. Models with many components (a sector grid) or weak data need it to keep noise from producing large alternating densities. This returns the largest lambda for which chi^2 <= max(n, chi^2_min + sqrt(2n)), n measurements, chi^2_min the unregularized value: chi^2 may rise by about one standard deviation of its distribution. Bisection on log10(lambda) over [-2, 8] (9 solves, about 0.2 s with prob reused).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `obs`.

```
sectors = sbtk.interior.sector_grid([0 0.5 1], [-90 0 90], [-180 0 180]);
prob = sbtk.invert.setup(body, sectors, obs, sbtk.invert.options(background="bulk"));
prob.opts.regularization = sbtk.invert.choose_regularization(prob);
fit = sbtk.invert.run(prob);
```

Related functions

Mentioned in the help text

`sbtk.interior.sector_grid`, `sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`

Calls

`sbtk.invert.run`

Implementation: `toolbox/+sbtk/+invert/choose_regularization.m`, line 1.

`sbtk.invert.goodness_of_fit`

Reduced chi-square and chi-square test of a fit.

Syntax

MATLAB

```
[chi2_reduced, p_value, consistent] = sbtk.invert.goodness_of_fit(chi2, dof)
```

Arguments

Name	Size	Class	Unit	Default	Description
`chi2`	scalar	double		required	sum of squared normalized residuals (with the full covariance) Validation: `mustBeNonnegative`.
`dof`	scalar	double		required	degrees of freedom: number of measurements minus the effective number of estimated parameters Validation: `mustBeFinite`.

Outputs

Name	Description
`chi2_reduced`	chi2/dof; about 1 when the model and the stated uncertainties are right
`p_value`	probability that a correct model gives a chi2 at least this large
`consistent`	`p_value` >= 0.05: the model is consistent with the measurements at the 5% significance level

Description

With dof <= 0 (at least as many parameters as measurements) the test is undefined: `chi2_reduced` and `p_value` are NaN and consistent is false.

Example

MATLAB, from the help text

```
[chi2_reduced, p, consistent] = sbtk.invert.goodness_of_fit(30.2, 24);   % 1.26, 0.18, true
```

Related functions

Called by

`sbtk.invert.run`

Implementation: `toolbox/+sbtk/+invert/goodness_of_fit.m`, line 1.

`sbtk.invert.homogeneous`

Fit a homogeneous body (no components) to the same measurements.

Syntax

MATLAB

```
fit = sbtk.invert.homogeneous(prob)
```

Arguments

Name	Size	Class	Unit	Default	Description
`prob`	1 × 1	struct		required	Prepared inversion; its components are ignored. Fields: `prob`.

Outputs

Name	Description
`fit`	Fit of a homogeneous body: the bulk density everywhere, no free parameters. Fields: `fit`.

Description

The density is the bulk density everywhere (fixed by the mass), so there are no free parameters. If this fit is already consistent with the measurements (`fit.consistent`), the data do not require internal structure.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`.

```
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
h = sbtk.invert.homogeneous(prob);
fprintf('homogeneous body: reduced chi2 %.2f, p = %.2g\n', h.chi2_reduced, h.p_value);
```

Related functions

Mentioned in the help text

`sbtk.invert.options`, `sbtk.invert.setup`

Calls

`sbtk.invert.run`

Implementation: `toolbox/+sbtk/+invert/homogeneous.m`, line 1.

`sbtk.invert.options`

Inversion settings with their standard values.

Syntax

MATLAB

```
opts = sbtk.invert.options()                    % the standard values
opts = sbtk.invert.options(Name=Value, ...)     % with some of them changed
```

Arguments

Name	Size	Class	Unit	Default	Description
`moi_labels`				`{'Ixx', 'Iyy', 'Izz'}`	Name-value argument; moment-of-inertia measurements to fit, e.g. {'Ixx','Iyy','Izz'}, {'libration'}, or {} (gravity only) Validation: `mustBeText`.
`regularization`	scalar	double		`0`	Name-value argument; Tikhonov parameter lambda (0 = none). The penalty is lambda * sum((`drho_c` / `rho_mean`)^2) in units of chi^2. Use sbtk.invert.choose_regularization for many components. Validation: `mustBeNonnegative`.
`background`	1 × 1	string		`"estimate"`	Name-value argument; "estimate": the background density is estimated (with the mass constraint); "bulk": it is held at the mean density M/V, so the relative densities of the components conserve the mass by themselves Validation: `mustBeMember(settings.background, ["estimate", "bulk"])`.
`rho_max_kgm3`	scalar	double	kg/m³	`4500`	Name-value argument; upper bound on any component's density [kg/m^3] Validation: `mustBePositive`.
`max_iter`	scalar	double		`20`	Name-value argument; Gauss-Newton iterations for the geometry of components with `estimate_position` or `estimate_axis` Validation: `mustBeInteger, mustBeNonnegative`.
`step_km`	scalar	double	km	`0.010`	Name-value argument; trust-region radius of a step of a center coordinate [km] Validation: `mustBePositive`.
`position_sigma_km`	scalar	double	km	`Inf`	Name-value argument; a priori 1-sigma of each estimated center coordinate about its Validation: `mustBePositive`.
`step_deg`	scalar	double	deg	`1`	Name-value argument; trust-region radius of a step of an axis tilt or turn [deg] Validation: `mustBePositive`.
`axis_sigma_deg`	scalar	double	deg	`Inf`	Name-value argument; a priori 1-sigma of each estimated axis tilt and turn about the starting orientation [deg]; Inf = no constraint Validation: `mustBePositive`.
`max_shift_km`	scalar	double	km	`Inf`	Name-value argument; largest distance of an estimated center from its starting point [km]; Inf = no limit (a sphere is kept inside the body in any case) Validation: `mustBePositive`.
`max_tilt_deg`	scalar	double	deg	`Inf`	Name-value argument; largest angle of an estimated axis from its starting direction [deg]; Inf = no limit Validation: `mustBePositive`.
`geometry_tol`	scalar	double		`1e-3`	Name-value argument; the geometry has converged when an accepted step changes every geometry parameter by less than this fraction of its step limit Validation: `mustBeNonnegative`.
`max_turn_deg`	scalar	double	deg	`Inf`	Name-value argument; largest turn of an estimated ellipsoid about its axis from its starting orientation [deg]; Inf = no limit Validation: `mustBePositive`.

Outputs

Name	Description
`opts`	The settings. Fields: `opts`.

Description

`starting point [km]; Inf = no constraint. Needed when more`	
`geometry is estimated than the measurements determine.`	

Standard values: `moi_labels` {'Ixx','Iyy','Izz'}, regularization 0, background "estimate", `rho_max_kgm3` 4500, `max_iter` 20, `step_km` 0.010, `step_deg` 1, `position_sigma_km` Inf, `axis_sigma_deg` Inf, `max_shift_km` Inf, `max_tilt_deg` Inf, `max_turn_deg` Inf, `geometry_tol` 1e-3. The fields can also be changed afterwards (`opts.regularization` = 10).

Example

MATLAB, from the help text

```
opts = sbtk.invert.options(moi_labels={'libration'}, background="bulk");
opts = sbtk.invert.options(moi_labels={});               % gravity coefficients only
```

Related functions

Mentioned in the help text

`sbtk.invert.choose_regularization`

Called by

`sbtk.invert.solve`

Implementation: `toolbox/+sbtk/+invert/options.m`, line 1.

`sbtk.invert.replace_data`

Use another realization of the same measurements without repeating the setup.

Syntax

MATLAB

```
prob = sbtk.invert.replace_data(prob, obs)
```

Arguments

Name	Size	Class	Unit	Default	Description
`prob`	1 × 1	struct		required	Prepared inversion. Fields: `prob`.
`obs`	1 × 1	struct		required	Another realization of the same measurements (same labels, inertia weights, origin, degree and mass). Fields: `obs`.

Outputs

Name	Description
`prob`	The inversion with the new measurement values and uncertainties; geometry and design matrix reused. Fields: `prob`.

Description

obs must have the same coefficient labels, moment-of-inertia rows (weights), origin, degree and mass as the data prob was set up with; only the values and uncertainties may differ (e.g. another noise seed). The geometry and design matrix are reused and a run takes milliseconds.

Example

100 noise realizations

MATLAB fragment from the help text. Uses variables defined by earlier calls: `prob`, `exact`.

```
for k = 1:100
    fit = sbtk.invert.run(sbtk.invert.replace_data(prob, sbtk.data.add_noise(exact, k)));
end
```

Related functions

Mentioned in the help text

`sbtk.data.add_noise`, `sbtk.invert.run`

Calls

`sbtk.data.set_covariance`, `sbtk.data.set_moi_sigma`, `sbtk.data.set_uncertainties`, `sbtk.data.use_uncertainties`, `sbtk.invert.setup`

Implementation: `toolbox/+sbtk/+invert/replace_data.m`, line 1.

`sbtk.invert.run`

Estimate the densities for a prepared inversion (bounded, mass-constrained least squares).

Syntax

MATLAB

```
fit = sbtk.invert.run(prob)          % prob from sbtk.invert.setup
```

Arguments

Name	Size	Class	Unit	Default	Description
`prob`	1 × 1	struct		required	Prepared inversion from `sbtk.invert.setup` (or `sbtk.invert.replace_data`). Fields: `prob`.

Outputs

Name	Description
`fit`	Inversion result. Fields: `fit`.

Description

Unknowns: the background density, the relative density of each component, and the geometry asked for: the center of spheres, ellipsoids and tori with `estimate_position` (all coordinates, or those of a mask [x y z]) and the axis of ellipsoids and tori with `estimate_axis` = true. The total mass is an equality constraint and every component's density is bounded to [0, `opts.rho_max_kgm3`]. Measurements are whitened by the Cholesky factor of their covariance and the columns scaled to unit norm; the problem is solved with lsqlin (active set, or interior point if the active set does not converge). The geometry is refined by Gauss-Newton (at most `opts.max_iter` steps) with a trust region (`opts.step_km` for center coordinates, `opts.step_deg` for axis tilts): a step that does not lower the objective is undone and the trust region halved; it has converged when the objective changes by less than 1e-10 relative or a step changes every parameter by less than `opts.geometry_tol` of its step limit. Optional a priori constraints keep the geometry near its start (`opts.position_sigma_km`, `opts.axis_sigma_deg`), hard limits bound how far it may go (`opts.max_shift_km`, `opts.max_tilt_deg`), and spheres are kept inside the body. The parameters are listed by sbtk.internal.geometry_parameters. Optional Tikhonov regularization (`opts.regularization`) penalizes the relative densities. The covariance is the linearized one with the mass constraint eliminated (inequality bounds are not included; see `fit.at_bound`).

fit (a model: `rho_bg_kgm3`, components, `random_field`, `origin_km`, plus)

`rho_bg_kgm3, rho_bg_sigma_kgm3`	background density and its 1-sigma [kg/m^3]
`components`	the components with the estimated relative densities `drho_kgm3`
`drho_sigma_kgm3`	1-sigma of each relative density
`rho_kgm3, rho_sigma_kgm3`	density of each component (background + relative) and its 1-sigma
`center_sigma_km`	n x 3 1-sigma of the center coordinates (NaN unless estimated)
`axis_sigma_deg`	n x 3 1-sigma of the rotations about the component's first, second and third axes [deg]: the tilts of its axis and, for an ellipsoid, its turn about it (NaN unless estimated)
`at_bound`	density at 0 or `rho_max`
`resolved`	|relative density| > its 1-sigma
`determined`	false if the measurements (with any prior or regularization) do not determine all free parameters; cov is then a pseudo-inverse and the solution is not unique
`chi2, n_obs, dof`	chi-square, number of measurements, degrees of freedom (`n_obs` minus the effective number of parameters, trace of the data resolution matrix)
`chi2_reduced, p_value chi2/dof and P(chi2 >= observed) for a correct model`	
`consistent`	`p_value` >= 0.05 (model consistent with the measurements)
`labels, measured, predicted, sigma, normalized_residual`	
`every measurement, its predicted value, (predicted - measured)/sigma`	
`names, cov, corr`	parameters, their covariance and correlation
`iterations, regularization, moi_labels, background, body, data_name`	
`coef_about`	'origin' or '`center_of_mass`' (from the measurements)
`com_shift_km`	model's center of mass minus `origin_km`: the point the predicted coefficients of degree >= 2 and inertia are about ([0 0 0] for 'origin')
`frame_iterations`	Gauss-Newton steps on the model's center of mass (0 for 'origin')
`frame_offset_km`	change of that center in the last step (below 1e-6 km when converged)

When `obs.coef_about` is '`center_of_mass`' (packages read with sbtk.exchange.read_data, fields from sbtk.data.add_center_of_mass), the predicted coefficients of degree >= 2 and inertia are moved to the fitted model's own center of mass (exact translation), and the fit repeated until that center moves less than 1 mm. The model itself stays fixed: `origin_km` is the measured center of mass, from which layers and sectors are measured.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`.

```
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
fit  = sbtk.invert.run(prob);
sbtk.invert.summary(fit)
```

Related functions

Mentioned in the help text

sbtk.data.add_center_of_mass, sbtk.exchange.read_data, sbtk.internal.geometry_parameters, sbtk.invert.options, sbtk.invert.setup, sbtk.invert.summary

Calls

sbtk.gravity.component_integrals, sbtk.interior.no_random_field, sbtk.interior.set_axis, sbtk.invert.goodness_of_fit

Called by

sbtk.invert.choose_regularization, sbtk.invert.homogeneous, sbtk.invert.solve

Implementation: `toolbox/+sbtk/+invert/run.m`, line 1.

---

sbtk.invert.setup

Prepare an inversion: geometry and the design-matrix column of every component, computed once.

Syntax

MATLAB

```
prob = sbtk.invert.setup(body, components, obs, opts)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create (shape; the mass used is `obs.mass_kg`) Fields: `body`.
`components`	1 × any	struct		required	the model's components (sbtk.interior constructors). Their geometry is fixed except for the center of spheres, ellipsoids and tori with `estimate_position` (true, or a mask [x y z]) and the axis of ellipsoids and tori with `estimate_axis` = true (sbtk.invert.run). Their `drho_kgm3` is estimated (the value given is not used), except for components with `estimate_density` = false, whose value is kept. Fields: `component`.
`obs`	1 × 1	struct		required	the measurements (sbtk.data.simulate, sbtk.io.load_measurements, ...) Fields: `obs`.
`opts`	1 × 1	struct		required	from sbtk.invert.options Fields: `opts`.

Outputs

Name	Description
`prob`	everything sbtk.invert.run needs. Further runs with other options (`prob.opts.regularization` = ...), sbtk.invert.homogeneous and sbtk.invert.replace_data reuse it (Didymos, three components: setup 0.7 s, each run about 0.2 s; tools/benchmark.m). Fields: `prob`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`.

```
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
fit  = sbtk.invert.run(prob);                    % about 0.2 s
prob.opts.regularization = 5;                    % settings that do not select measurements
fit5 = sbtk.invert.run(prob);                    % can change between runs
```

Related functions

Mentioned in the help text

sbtk.body.create, sbtk.data.simulate, sbtk.invert.homogeneous, sbtk.invert.options, sbtk.invert.replace_data, sbtk.invert.run, sbtk.io.load_measurements

Calls

sbtk.data.add_center_of_mass, sbtk.data.set_covariance, sbtk.data.set_moi_sigma, sbtk.data.set_uncertainties, sbtk.data.use_uncertainties, sbtk.gravity.component_integrals, sbtk.gravity.geometry

Called by

sbtk.invert.replace_data, sbtk.invert.solve

Implementation: `toolbox/+sbtk/+invert/setup.m`, line 1.

---

sbtk.invert.solve

Setup and run in one call.

Syntax

MATLAB

```
fit = sbtk.invert.solve(body, components, obs)          % standard options
fit = sbtk.invert.solve(body, components, obs, opts)    % opts from sbtk.invert.options
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body (shape model; the mass used is `obs.mass_kg`). Fields: `body`.
`components`	1 × any	struct		required	The model's components; their geometry is fixed and their relative densities are estimated (the values given are not used) unless `estimate_density` is false. Fields: `component`.
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`opts`	1 × 1	struct		`sbtk.invert.options()`	Settings from `sbtk.invert.options`. Fields: `opts`.

Outputs

Name	Description
`fit`	Inversion result. Fields: `fit`.

Description

Same as sbtk.invert.run(sbtk.invert.setup(body, components, obs, opts)). Use setup + run when solving the same problem more than once (other options, sbtk.invert.homogeneous, other noise realizations).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`.

```
fit = sbtk.invert.solve(body, components, obs);
fit = sbtk.invert.solve(body, components, obs, sbtk.invert.options(moi_labels={'Izz'}));
sbtk.invert.summary(fit)
```

Related functions

Mentioned in the help text

`sbtk.invert.homogeneous`, `sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`, `sbtk.invert.summary`

Calls

`sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`

Implementation: `toolbox/+sbtk/+invert/solve.m`, line 1.

`sbtk.invert.summary`

Print an inversion result.

Syntax

MATLAB

```
sbtk.invert.summary(fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`fit`	1 × 1	struct		required	Inversion result to print. Fields: `fit`.

Example output

Didymos demo

Output

```
Didymos inversion of didymos_T1: 27 measurements
  chi2 = 26.74, dof = 24.0, reduced chi2 = 1.11, p = 0.32 (consistent with the measurements at the 5% level)
  component     density [kg/m^3]     relative density [kg/m^3]
  background      2708 +-    65
  core            3162 +-   205         +453 +-   268
```

Related functions

Called by

`sbtk.io.report_inversion`

Implementation: `toolbox/+sbtk/+invert/summary.m`, line 1.

`sbtk.compare`: Comparison with a true model

Function	Purpose
`evaluate`	Compare a fit with the true model, using a prepared reference (see sbtk.compare.reference).
`grid_summary`	Print a cell-by-cell comparison of two density grids.
`grids`	Compare two density grids cell by cell (e.g. a solution package with an interior-model package).
`reference`	The true model sampled once for comparisons with fits that use a given set of components.
`summary`	Print a comparison with the true model.
`with_truth`	Compare an estimated density with the true density (synthetic tests).

`sbtk.compare.evaluate`

Compare a fit with the true model, using a prepared reference (see sbtk.compare.reference).

Syntax

MATLAB

```
cmp = sbtk.compare.evaluate(ref, fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`ref`	1 × 1	struct		required	Prepared reference from `sbtk.compare.reference`. Fields: `ref`.
`fit`	1 × 1	struct		required	Inversion result with the same components and origin as the reference. Fields: `fit`.

Outputs

Name	Description
`cmp`	Comparison with the true model. Fields: `cmp`.

Description

fit must use the same components (same geometry) as ref. Returns the fields described in sbtk.compare.with_truth.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `components`, `prob`, `exact`.

```
% many fits of the same components against one true model
ref = sbtk.compare.reference(body, truth, components, prob.geo.origin_km);   % once, about 0.5 s
for k = 1:100
    fit = sbtk.invert.run(sbtk.invert.replace_data(prob, sbtk.data.add_noise(exact, k)));
    cmp = sbtk.compare.evaluate(ref, fit);                                   % milliseconds
    R2(k) = cmp.R2;
end
```

Related functions

Mentioned in the help text

`sbtk.compare.reference`, `sbtk.compare.with_truth`, `sbtk.data.add_noise`, `sbtk.invert.replace_data`, `sbtk.invert.run`

Calls

`sbtk.compare.with_truth`

Called by

`sbtk.compare.with_truth`

Implementation: `toolbox/+sbtk/+compare/evaluate.m`, line 1.

`sbtk.compare.grid_summary`

Print a cell-by-cell comparison of two density grids.

Syntax

MATLAB

```
lines = sbtk.compare.grid_summary(cmp)
```

Arguments

Name	Size	Class	Unit	Default	Description
`cmp`	1 × 1	struct		required	from sbtk.compare.grids Fields: `gridcmp`.

Outputs

Name	Description
`lines`	the printed lines (string array), e.g. for writelines into a results folder

Example

MATLAB, from the help text

```
cmp = sbtk.compare.grids(sbtk.exchange.read_grid("UCB-MOD1-Didymos/density_grid.txt"), ...
                         sbtk.exchange.read_grid("UCB-SOL_UCB1/density_grid.txt"));
sbtk.compare.grid_summary(cmp);
```

Related functions

Mentioned in the help text

`sbtk.compare.grids`, `sbtk.exchange.read_grid`

Implementation: `toolbox/+sbtk/+compare/grid_summary.m`, line 1.

`sbtk.compare.grids`

Compare two density grids cell by cell (e.g. a solution package with an interior-model package).

Syntax

MATLAB

```
cmp = sbtk.compare.grids(truth, estimate)
```

Arguments

Name	Size	Class	Unit	Default	Description
`truth`	1 × 1	struct		required	from sbtk.exchange.read_grid or sbtk.exchange.density_grid, with the same spacing; cells present in both are compared (every cell has the same volume, so all have equal weight) Fields: `grid`.
`estimate`	1 × 1	struct		required	from sbtk.exchange.read_grid or sbtk.exchange.density_grid, with the same spacing; cells present in both are compared (every cell has the same volume, so all have equal weight) Fields: `grid`.

Outputs

Name	Description
`cmp`	Comparison of the grids. Fields: `gridcmp`.

Description

`cmp.R2`	1 - sum (estimate - true)^2 / sum (true - mean true)^2: the fraction of the variance of the true density explained
`cmp.rms_kgm3`	RMS of estimate - true
`cmp.mean_error_kgm3`	mean of estimate - true
`cmp.std_true_kgm3`	standard deviation of the true density
`cmp.within_2sigma`	fraction of cells with |estimate - true| <= 2 sigma of the estimate (NaN if the estimate has no uncertainties)

`cmp.n_common`, `cmp.n_truth`, `cmp.n_estimate` numbers of cells

Example

MATLAB, from the help text

```
cmp = sbtk.compare.grids(sbtk.exchange.read_grid("UCB-MOD1-Didymos/density_grid.txt"), ...
                         sbtk.exchange.read_grid("XYZ-SOL_UCB1/density_grid.txt"));
```

Related functions

Mentioned in the help text

`sbtk.exchange.density_grid`, `sbtk.exchange.read_grid`

Implementation: `toolbox/+sbtk/+compare/grids.m`, line 1.

`sbtk.compare.reference`

The true model sampled once for comparisons with fits that use a given set of components.

Syntax

MATLAB

```
ref = sbtk.compare.reference(body, truth, components, origin_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.
`truth`	1 × 1	struct		required	True model from `sbtk.data.simulate`. Fields: `truth`.
`components`	1 × any	struct		required	the components of the fits to be compared (their geometry; densities are not used) Fields: `component`.
`origin_km`	1 × 3	double	km	required	the point their layers and sectors are measured from: the fits' `origin_km` (the true model's origin for data in its frame, the measured center of mass for data about the center of mass) Validation: `mustBeFinite`.

Outputs

Name	Description
`ref`	The prepared reference. Fields: `ref`.

Description

Evaluates the true density at points filling the body (24 radial shells per facet cone, weighted by volume, in the true model's frame), the membership of every point in every component, and `R2_max` for these components. sbtk.compare.evaluate(ref, fit) then compares any such fit in milliseconds.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `components`, `prob`.

```
ref = sbtk.compare.reference(body, truth, components, prob.geo.origin_km);
cmp = sbtk.compare.evaluate(ref, sbtk.invert.run(prob));
% (see sbtk.compare.evaluate for a loop over noise realizations)
```

Related functions

Mentioned in the help text

`sbtk.compare.evaluate`, `sbtk.invert.run`

Calls

`sbtk.gravity.geometry`

Called by

`sbtk.compare.with_truth`

Implementation: `toolbox/+sbtk/+compare/reference.m`, line 1.

`sbtk.compare.summary`

Print a comparison with the true model.

Syntax

MATLAB

```
sbtk.compare.summary(cmp)          % cmp from sbtk.compare.with_truth
```

Arguments

Name	Size	Class	Unit	Default	Description
`cmp`	1 × 1	struct		required	Comparison from `sbtk.compare.with_truth` or `sbtk.compare.evaluate`. Fields: `cmp`.

Example output

Didymos demo

Output

```
Didymos: estimate compared with the true model
  R2 = 0.89 (fraction of the variance of the true density explained); R2_max = 0.98 for these components
```

Related functions

Mentioned in the help text

`sbtk.compare.with_truth`

Called by

`sbtk.io.report_comparison`

Implementation: `toolbox/+sbtk/+compare/summary.m`, line 1.

`sbtk.compare.with_truth`

Compare an estimated density with the true density (synthetic tests).

Syntax

MATLAB

```
cmp = sbtk.compare.with_truth(body, truth, fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.
`truth`	1 × 1	struct		required	the true model, from sbtk.data.simulate Fields: `truth`.
`fit`	1 × 1	struct		required	the estimate, from sbtk.invert.run Fields: `fit`.

Outputs

Name	Description
`cmp`	Comparison with the true model. Fields: `cmp`.

Description

Both densities are evaluated at points filling the body (24 radial shells per facet cone, 1.18 million for Didymos, each weighted by its volume); the true model's layers and sectors are measured from `truth.origin_km`, the fit's from `fit.origin_km`. About 0.5 s for Didymos.

`cmp.R2`	coefficient of determination of the estimated density: R2 = 1 - sum w (`rho_est` - `rho_true`)^2 / sum w (`rho_true` - mean)^2, the fraction of the variance of the true density (about its volume-weighted mean) that the estimate explains. 1 = exact; 0 = no better than a homogeneous body; < 0 = worse than that.
`cmp.R2_max`	the largest R2 attainable with the same components (weighted least-squares fit of the true density by the background and the components): the limit set by the choice of components
`cmp.rms_kgm3`	volume-weighted RMS density error [kg/m^3]
`cmp.std_true_kgm3`	volume-weighted standard deviation of the true density [kg/m^3]
`cmp.components`	components with the same name in both: name, `estimate_kgm3`, `sigma_kgm3`, `true_kgm3` (densities), `deviation_sigma` (|estimate - true| / sigma)
`cmp.chi2_reduced, cmp.p_value`	copied from the fit

For many fits with the same components against one true model, call sbtk.compare.reference once and sbtk.compare.evaluate for each fit.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `fit`.

```
cmp = sbtk.compare.with_truth(body, truth, fit);
sbtk.compare.summary(cmp)
sbtk.plot.comparison(cmp);
```

Related functions

Mentioned in the help text

`sbtk.compare.evaluate`, `sbtk.compare.reference`, `sbtk.compare.summary`, `sbtk.data.simulate`, `sbtk.invert.run`, `sbtk.plot.comparison`

Calls

`sbtk.compare.evaluate`, `sbtk.compare.reference`

Called by

`sbtk.compare.evaluate`

Implementation: `toolbox/+sbtk/+compare/with_truth.m`, line 1.

Reference

`sbtk.exchange`: Exchange packages

Function	Purpose
`density_grid`	Density of a model at the centers of a cubic grid inside the body.
`locate`	The exchange package of a given kind in a run folder.
`read_data`	Measurements and body from a measurement package (sbtk.exchange.write_data format).
`read_grid`	Density grid from an interior-model or solution package (`density_grid.txt`).
`read_shadr`	Measurements and body from a gravity file in the PDS SHADR layout (e.g. other teams' .tab files).
`solution_name`	Name of a solution package: <team>-SOL_<data team><number>.
`write_data`	Write a measurement package to send to other teams (nothing about the interior).
`write_model`	Write the interior-model package behind a measurement package (released after the solutions).
`write_solution`	Write an inversion result as a solution package for the other teams.

`sbtk.exchange.density_grid`

Density of a model at the centers of a cubic grid inside the body.

Syntax

MATLAB

```
grid = sbtk.exchange.density_grid(body, model, spacing_km)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	from sbtk.body.create Fields: `body`.
`model`	1 × 1	struct		required	a true model (sbtk.data.simulate) or a fit (sbtk.invert.run)
`spacing_km`	scalar	double	km	required	cell edge [km] Validation: `mustBePositive`.

Outputs

Name	Description
`grid`	The density grid. Fields: `grid`.

Description

Cell centers lie at integer multiples of `spacing_km` along each axis of the shape-model frame, so grids with the same spacing coincide whoever makes them; cells whose center is inside the shape model are kept (tested along rays from the center of figure, so every model of a body has the same cells). The density is the model's density at the cell center (random field included).

`grid.spacing_km`, `grid.x_km` (n x 3), `grid.density_kgm3` (n x 1), `grid.density_sigma_kgm3` (n x 1, zero here; sbtk.exchange.write_solution fills it from the fit's covariance), `grid.member` (n x ncomp: the cell center is inside component c)

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`.

```
grid = sbtk.exchange.density_grid(body, truth, 0.020);              % 20 m cells
scatter3(grid.x_km(:,1), grid.x_km(:,2), grid.x_km(:,3), 4, grid.density_kgm3, 'filled');
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.data.simulate`, `sbtk.exchange.write_solution`, `sbtk.invert.run`

Calls

`sbtk.gravity.geometry`

Called by

`sbtk.exchange.write_model`, `sbtk.exchange.write_solution`

Implementation: `toolbox/+sbtk/+exchange/density_grid.m`, line 1.

`sbtk.exchange.locate`

The exchange package of a given kind in a run folder.

Syntax

MATLAB

```
folder = sbtk.exchange.locate(parent, kind)
```

Arguments

Name	Size	Class	Unit	Default	Description
`parent`	1 × 1	string		required	a run folder (or any folder holding a package) Validation: `mustBeFolder`.
`kind`	1 × 1	string		required	"DATA" (measurement package, <team>-DATA<n>...), "MOD" (interior-model package, <team>-MOD<n>...) or "SOL" (solution package, <team>-SOL_...) Validation: `mustBeMember(kind, ["DATA", "MOD", "SOL"])`.

Outputs

Name	Description
`folder`	Path of the package folder.

Description

Looks in parent and in its redacted/ and unredacted/ subfolders (the layout of simulation runs) and returns the one package of that kind. Errors if there is none or more than one.

Example

MATLAB, from the help text

```
run = sbtk.io.latest_run("hera_hlp6", "didymos_T1");
[obs, body] = sbtk.exchange.read_data(sbtk.exchange.locate(run, "DATA"));
model = sbtk.exchange.locate(run, "MOD");
```

Related functions

Mentioned in the help text

`sbtk.exchange.read_data`, `sbtk.io.latest_run`

Implementation: `toolbox/+sbtk/+exchange/locate.m`, line 1.

`sbtk.exchange.read_data`

Measurements and body from a measurement package (sbtk.exchange.write_data format).

Syntax

MATLAB

```
[obs, body] = sbtk.exchange.read_data(folder)
```

Arguments

Name	Size	Class	Unit	Default	Description
`folder`	1 × 1	string		required	the package folder (README.txt, `gravity_field.txt`, `gravity_covariance.txt`, `center_of_mass.txt`, `moments_of_inertia.txt`, conventions.txt, shape model) Validation: `mustBeFolder`.

Outputs

Name	Description
`obs`	the measurements, ready for sbtk.invert: coefficients of degree 2..L with their full covariance, expanded about the measured center of mass (`obs.origin_km`), plus degree-1 coefficients 0 +- sigma that carry the uncertainty of the center of mass, and the moment-of-inertia rows Fields: `obs`.
`body`	from the package's shape model (lengths in km), mass = GM / G and the package's reference radius Fields: `body`.

Description

A package of exact values (no 1-sigma columns, no `gravity_covariance.txt`) gives a set with NaN uncertainties: give it uncertainties (e.g. sbtk.data.use_uncertainties with the precision file of the test) before sbtk.invert.

The libration ratio (B - A)/C becomes the row (B - A) - ratio*Izz = 0 with sigma `ratio_sigma` * Izz, where Izz is the measured Izz if the package has one, otherwise the Izz of the homogeneous body.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `components`.

```
[obs, body] = sbtk.exchange.read_data("UCB-DATA1-Didymos");
fit = sbtk.invert.solve(body, components, obs, sbtk.invert.options());
```

Related functions

Mentioned in the help text

`sbtk.data.use_uncertainties`, `sbtk.invert.options`, `sbtk.invert.solve`

Calls

`sbtk.body.from_mass`, `sbtk.data.add_libration`, `sbtk.data.add_moi`, `sbtk.gravity.geometry`, `sbtk.shape.load`, `sbtk.units.G`

Implementation: `toolbox/+sbtk/+exchange/read_data.m`, line 1.

`sbtk.exchange.read_grid`

Density grid from an interior-model or solution package (`density_grid.txt`).

Syntax

MATLAB

```
grid = sbtk.exchange.read_grid(file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`file`	1 × 1	string		required	Path of a `density_grid.txt` file of an interior-model or solution package. Validation: `mustBeFile`.

Outputs

Name	Description
`grid`	The density grid (without `member`). Fields: `grid`.

Description

`grid.spacing_km`, `grid.x_km` (n x 3), `grid.density_kgm3`, `grid.density_sigma_kgm3` Compare two grids with sbtk.compare.grids.

Example

MATLAB, from the help text

```
grid = sbtk.exchange.read_grid("UCB-MOD1-Didymos/density_grid.txt");
```

Related functions

Mentioned in the help text

`sbtk.compare.grids`

Implementation: `toolbox/+sbtk/+exchange/read_grid.m`, line 1.

`sbtk.exchange.read_shadr`

Measurements and body from a gravity file in the PDS SHADR layout (e.g. other teams' .tab files).

Syntax

MATLAB

```
[obs, body] = sbtk.exchange.read_shadr(file, body_name, shape, origin_km, degree)
```

Arguments

Name	Size	Class	Unit	Default	Description
`file`	1 × 1	string		required	text file in the layout of the PDS Spherical Harmonic ASCII Data Record: a header row `R_ref` [km], GM [km^3/s^2], sigma GM, degree, order, normalization state (1 = fully normalized), reference longitude, latitude then one row per (l, m): l, m, C, S, `sigma_C`, `sigma_S` Values may be separated by commas or spaces; lines starting with '#' and lines that do not start with a number (column names) are skipped. Validation: `mustBeFile`.
`body_name`	1 × 1	string		required	name of the body, e.g. "Didymos"
`shape`	1 × 1	struct		required	from sbtk.shape.load: the shape model the field refers to Fields: `shape`.
`origin_km`	1 × 3	double	km	required	point the coefficients are expanded about, in shape-model coordinates [km]: [0 0 0] if the field is about the shape-model origin (it then has degree-1 terms), the center of mass if it is about the center of mass Validation: `mustBeFinite`.
`degree`	scalar	double		required	highest degree to use Validation: `mustBeInteger, mustBePositive`.

Outputs

Name	Description
`obs`	coefficients of degree 1..degree present in the file, covariance diag(sigma^2) (the format has no correlations), mass = GM / G Fields: `obs`.
`body`	sbtk.body.from_mass(`body_name`, shape, GM / G, `R_ref` of the file) Fields: `body`.

Description

A field about the center of mass has no degree-1 terms; add the measured center of mass with sbtk.data.add_center_of_mass.

Example

MATLAB, from the help text

```
shape = sbtk.shape.load("didymos_g_9309mm_spc_obj_0000n00000_v003.obj", "km");
[obs, body] = sbtk.exchange.read_shadr("Didymos_gravity.tab", "Didymos", shape, [0 0 0], 4);
```

Related functions

Mentioned in the help text

`sbtk.body.from_mass`, `sbtk.data.add_center_of_mass`, `sbtk.shape.load`

Calls

`sbtk.body.from_mass`, `sbtk.data.from_coefficients`, `sbtk.data.select_coefficients`, `sbtk.units.G`

Implementation: `toolbox/+sbtk/+exchange/read_shadr.m`, line 1.

`sbtk.exchange.solution_name`

Name of a solution package: <team>-SOL_<data team><number>.

Syntax

MATLAB

```
name = sbtk.exchange.solution_name(team, data_set)
```

Arguments

Name	Size	Class	Unit	Default	Description
`team`	1 × 1	string		required	the team that made the solution, e.g. "UCB"
`data_set`	1 × 1	string		required	the measurement package solved, named <data team>-DATA<number>[-comment]

Outputs

Name	Description
`name`	Package name `<team>-SOL_<data team><number>`.

Example

MATLAB, from the help text

```
sbtk.exchange.solution_name("UCB", "ROB-DATA1")          % "UCB-SOL_ROB1"
```

Implementation: `toolbox/+sbtk/+exchange/solution_name.m`, line 1.

`sbtk.exchange.write_data`

Write a measurement package to send to other teams (nothing about the interior).

Syntax

MATLAB

```
folder = sbtk.exchange.write_data(parent, data_set, body, obs, prepared_by)
```

Arguments

Name	Size	Class	Unit	Default	Description
`parent`	1 × 1	string		required	folder in which the package folder is created
`data_set`	1 × 1	string		required	package name, e.g. "UCB-DATA1-Didymos" (team - DATA<number> - comment)
`body`	1 × 1	struct		required	from sbtk.body.create; its shape-model file is copied into the package Fields: `body`.
`obs`	1 × 1	struct		required	the measurements with degree-1 coefficients (the center of mass): exact values (a maker's package; the receiving team applies its own uncertainties), or values with uncertainties and noise (sbtk.data.add_noise) Fields: `obs`.
`prepared_by`	1 × 1	string		required	who made the package, e.g. "University of Colorado Boulder, name (e-mail)"

Outputs

Name	Description
`folder`	Path of the package folder that was created.

Description

Creates <parent>/<data_set>/ with

`README.txt`	summary and list of files
`gravity_field.txt`	coefficients of degree 2..L about the center of mass
`gravity_covariance.txt`	their full covariance (values with uncertainties only)
`center_of_mass.txt`	center of mass and center of figure in the shape-model frame
`moments_of_inertia.txt`	inertia tensor elements and/or libration ratio
`conventions.txt`	units, frame, normalization (with code), degree 1
`<shape model>.obj the shape model`	

The files state only what applies: 1-sigma columns and the noise when the values have them (sbtk.internal.exchange_data_files). Nothing else of obs is written: not the expansion origin of the simulation (the true center of mass), the noise seed or the notes. Format: docs/exchange_format.md; read back with sbtk.exchange.read_data.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`, `body`, `exact`, `prepared_by`.

```
sbtk.exchange.write_data(fullfile(out.folder, "redacted"), "UCB-DATA1-Didymos", body, exact, prepared_by);
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.data.add_noise`, `sbtk.exchange.read_data`, `sbtk.internal.exchange_data_files`

Calls

`sbtk.data.set_covariance`, `sbtk.data.set_moi_sigma`, `sbtk.data.set_uncertainties`, `sbtk.data.use_uncertainties`, `sbtk.shape.properties`

Implementation: `toolbox/+sbtk/+exchange/write_data.m`, line 1.

`sbtk.exchange.write_model`

Write the interior-model package behind a measurement package (released after the solutions).

Syntax

MATLAB

```
folder = sbtk.exchange.write_model(parent, model_set, data_set, body, truth, obs_exact, spacing_km, prepared_by)
```

Arguments

Name	Size	Class	Unit	Default	Description
`parent`	1 × 1	string		required	folder in which the package folder is created
`model_set`	1 × 1	string		required	package name, e.g. "UCB-MOD1-Didymos"
`data_set`	1 × 1	string		required	name of the measurement package made from this model, e.g. "UCB-DATA1-Didymos"
`body`	1 × 1	struct		required	from sbtk.body.create Fields: `body`.
`truth`	1 × 1	struct		required	the true model (sbtk.data.simulate) Fields: `truth`.
`obs_exact`	1 × 1	struct		required	the measurements without noise: the exact values of sbtk.data.simulate, with or without uncertainties (without, their 1-sigma are written as NaN)
`spacing_km`	scalar	double	km	required	cell edge of the density grid [km] Validation: `mustBePositive`.
`prepared_by`	1 × 1	string		required	who made the package

Outputs

Name	Description
`folder`	Path of the package folder that was created.

Description

Creates <parent>/<model_set>/ with

`README.txt`	summary and list of files
`interior_model.txt`	background density and every component (type, densities, geometry), random density field, principal axes and moments of inertia
`density_grid.txt`	true density at the centers of a cubic grid (sbtk.exchange.density_grid)
`noise_free_data/`	the measurements of `data_set` without noise, in the same files
`figures/`	3-D view and slices of the true density

Compare a solution with it by sbtk.compare.grids on the two density grids.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`, `body`, `truth`, `exact`, `prepared_by`.

```
% exact: the measurements before sbtk.data.add_noise
sbtk.exchange.write_model(fullfile(out.folder, "unredacted"), "UCB-MOD1-Didymos", "UCB-DATA1-Didymos", ...
                          body, truth, exact, 0.020, prepared_by);
```

Related functions

Mentioned in the help text

`sbtk.body.create`, `sbtk.compare.grids`, `sbtk.data.add_noise`, `sbtk.data.simulate`, `sbtk.exchange.density_grid`

Calls

`sbtk.exchange.density_grid`, `sbtk.plot.interior3d`, `sbtk.plot.slices`, `sbtk.shape.properties`

Implementation: `toolbox/+sbtk/+exchange/write_model.m`, line 1.

`sbtk.exchange.write_solution`

Write an inversion result as a solution package for the other teams.

Syntax

MATLAB

```
folder = sbtk.exchange.write_solution(parent, solution_set, body, fit, homogeneous, spacing_km, prepared_by)
```

Arguments

Name	Size	Class	Unit	Default	Description
`parent`	1 × 1	string		required	folder in which the package folder is created
`solution_set`	1 × 1	string		required	package name, e.g. "UCB-`SOL_ROB1`" (team - SOL_<data team><number>)
`body`	1 × 1	struct		required	the body used in the inversion Fields: `body`.
`fit`	1 × 1	struct		required	from sbtk.invert.run (`fit.data_name` names the measurement package) Fields: `fit`.
`homogeneous`	1 × 1	struct		required	from sbtk.invert.homogeneous (the same data fitted by a homogeneous body) Fields: `fit`.

Name	Size	Class	Unit	Default	Description
`spacing_km`	scalar	double	km	required	cell edge of the density grid [km]; use the spacing of the interior-model package for a cell-by-cell comparison Validation: `mustBePositive`.
`prepared_by`	1 x 1	string		required	who made the solution

Outputs

Name	Description
`folder`	Path of the package folder that was created.

Description

Creates <parent>/<solution_set>/ with

`README.txt`	method, measurements used, chi-square test, list of files
`solution.txt`	background and component densities with 1-sigma, geometry
`solution_covariance.txt`	covariance of the estimated parameters
`fitted_measurements.txt`	every measurement used: measured, predicted, sigma, normalized residual
`density_grid.txt`	estimated density and its 1-sigma on the cubic grid
`figures/`	residuals, densities, 3-D view, slices

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`, `out`, `prepared_by`.

```
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
fit  = sbtk.invert.run(prob);
sbtk.exchange.write_solution(out.folder, sbtk.exchange.solution_name("UCB", obs.name), body, fit, ...
                             sbtk.invert.homogeneous(prob), 0.020, prepared_by);
```

Related functions

Mentioned in the help text

`sbtk.exchange.solution_name`, `sbtk.invert.homogeneous`, `sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`

Calls

`sbtk.exchange.density_grid`, `sbtk.plot.densities`, `sbtk.plot.interior3d`, `sbtk.plot.residuals`, `sbtk.plot.slices`

Implementation: `toolbox/+sbtk/+exchange/write_solution.m`, line 1.

Reference

`sbtk.io`: Results folders and text files

Function	Purpose
`finish_run`	Stop the run's log and say where the results are.
`latest_run`	Folder of the most recent run with a given name.
`load_fit`	Inversion result from a run folder (`fit.mat`) or a .mat file holding the variable fit.
`load_measurements`	Measurements from a run folder, a .mat file or a text file.
`load_truth`	True model from a run folder or a .mat file.
`new_run`	Create a results folder for one run and start its log.
`read_jpl_gravity`	Measurements from a JPL SSD gravity-field file (normalized coefficients + covariance).
`read_measurements`	Measurement set from the SBTK text format.
`report_comparison`	Save, print and plot a comparison with the true model.
`report_inversion`	Save, print and plot an inversion result.
`report_simulation`	Save, print and plot synthetic measurements and their true model.
`save_comparison`	comparison.mat and comparison.txt in the run folder.
`save_figure`	Save a figure as figures/<name>.png (200 dpi) in the run folder.
`save_fit`	`fit.mat` and `fit.txt` in the run folder.
`save_measurements`	measurements.mat and measurements.txt in the run folder.
`save_settings`	Write the settings of a run to `settings.json` (readable, reproducible).
`save_truth`	`truth.mat` and `truth.txt` in the run folder.
`subfolder`	A subfolder of a run folder that the save and report functions write to.
`write_comparison`	Plain-text copy of a comparison with the true model.
`write_fit`	Plain-text summary of an inversion result.
`write_measurements`	Plain-text copy of a measurement set (read back by sbtk.io.read_measurements).
`write_truth`	Plain-text description of a true model (keep private in blind tests).

`sbtk.io.finish_run`

Stop the run's log and say where the results are.

Syntax

MATLAB

```
sbtk.io.finish_run(out)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder whose log to close. Fields: `out`.

Example

MATLAB, from the help text

```
out = sbtk.io.new_run("my_project", "T1");
% ... everything printed until here goes to out.folder/log.txt
sbtk.io.finish_run(out);
```

Related functions

Mentioned in the help text

`sbtk.io.new_run`

Implementation: `toolbox/+sbtk/+io/finish_run.m`, line 1.

`sbtk.io.latest_run`

Folder of the most recent run with a given name.

Syntax

```
MATLAB
folder = sbtk.io.latest_run(project, name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`project`	1 × 1	string		required	Project name.
`name`	1 × 1	string		required	Run name; only folders whose name after the time stamp equals it match.

Outputs

Name	Description
`folder`	Path of the newest matching run folder.

Example

an inversion reads the newest simulation run

```
MATLAB, from the help text
obs = sbtk.io.load_measurements(sbtk.io.latest_run("hera_hlp6", "didymos_T1"));
```

Related functions

Mentioned in the help text

`sbtk.io.load_measurements`

Calls

`sbtk.root`

Implementation: `toolbox/+sbtk/+io/latest_run.m`, line 1.

`sbtk.io.load_fit`

Inversion result from a run folder (`fit.mat`) or a .mat file holding the variable fit.

Syntax

```
MATLAB
fit = sbtk.io.load_fit(source)
```

Arguments

Name	Size	Class	Unit	Default	Description
`source`	1 × 1	string		required	A run folder (reads `fit.mat`) or a .mat file holding the variable fit.

Outputs

Name	Description
`fit`	Inversion result. Fields: `fit`.

Example

```
MATLAB, from the help text
fit = sbtk.io.load_fit(sbtk.io.latest_run("hera_hlp6", "didymos_T1_inversion"));
```

Related functions

Mentioned in the help text

`sbtk.io.latest_run`

Implementation: `toolbox/+sbtk/+io/load_fit.m`, line 1.

`sbtk.io.load_measurements`

Measurements from a run folder, a .mat file or a text file.

Syntax

```
MATLAB
obs = sbtk.io.load_measurements(source)
```

Arguments

Name	Size	Class	Unit	Default	Description
`source`	1 × 1	string		required	a run folder (reads its measurements.mat, or unredacted/measurements.mat for a simulation run that splits its output), a .mat file holding the variable obs, or a text file in the SBTK format (sbtk.io.read_measurements). Measurement packages for other teams: sbtk.exchange.read_data.

Outputs

Name	Description
`obs`	The measurement set. Fields: `obs`.

Example

```
MATLAB, from the help text
obs = sbtk.io.load_measurements(sbtk.io.latest_run("hera_hlp6", "didymos_T1"));
obs = sbtk.io.load_measurements("their_data.txt");
```

Related functions

Mentioned in the help text

`sbtk.exchange.read_data`, `sbtk.io.latest_run`, `sbtk.io.read_measurements`

Calls

`sbtk.io.read_measurements`

Implementation: `toolbox/+sbtk/+io/load_measurements.m`, line 1.

`sbtk.io.load_truth`

True model from a run folder or a .mat file.

Syntax

```
MATLAB
truth = sbtk.io.load_truth(source)
```

Arguments

Name	Size	Class	Unit	Default	Description
`source`	1 × 1	string		required	a run folder (reads its `truth.mat`, or unredacted/truth.mat for a simulation run that splits its output) or a .mat file holding the variable truth

Outputs

Name	Description
`truth`	The true model. Fields: `truth`.

Example

```
MATLAB, from the help text
truth = sbtk.io.load_truth(sbtk.io.latest_run("hera_hlp6", "didymos_T1"));
```

Related functions

Mentioned in the help text

`sbtk.io.latest_run`

Implementation: `toolbox/+sbtk/+io/load_truth.m`, line 1.

`sbtk.io.new_run`

Create a results folder for one run and start its log.

Syntax

MATLAB

```
out = sbtk.io.new_run(project, name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`project`	1 × 1	string		required	Project name: the folder results/<project>/.
`name`	1 × 1	string		required	Run name; the folder is results/<project>/<yyyy-mm-`dd_HHMMSS`>_<name>/.

Outputs

Name	Description
`out`	The results folder. Fields: `out`.

Description

Creates <SBTK>/results/<project>/<yyyy-mm-`dd_HHMMSS`>_<name>/ (figures go to its figures/ subfolder) and records everything printed in the command window to log.txt in it until sbtk.io.finish_run(out). Standard file names inside:

```
  measurements.txt / .mat   truth.txt / .mat   fit.txt / .mat   homogeneous.txt
  comparison.txt / .mat
  settings.json              figures/<name>.png  log.txt
```

`out.project`, `out.name`, `out.stamp`, `out.folder`, `out.figures`

Example

MATLAB, from the help text

```
out = sbtk.io.new_run("hera_hlp6", "didymos_T1");
```

Related functions

Mentioned in the help text

`sbtk.io.finish_run`

Calls

`sbtk.root`

Implementation: `toolbox/+sbtk/+io/new_run.m`, line 1.

`sbtk.io.read_jpl_gravity`

Measurements from a JPL SSD gravity-field file (normalized coefficients + covariance).

Syntax

MATLAB

```
obs = sbtk.io.read_jpl_gravity(file, body, name, origin_km, degree)
```

Arguments

Name	Size	Class	Unit	Default	Description
`file`	1 × 1	string		required	JPL SSD gravity file in MATLAB-assignment form (`REF_RADIUS`, NORMALIZED, NAMES = {...}, VALS = [...], COV = [...]), e.g. the OSIRIS-REx Bennu field Validation: `mustBeFile`.
`body`	1 × 1	struct		required	from sbtk.body.create; `body.R_ref_km` must equal the file's `REF_RADIUS` Fields: `body`.
`name`	1 × 1	string		required	name of the data set
`origin_km`	1 × 3	double	km	required	point the field is expanded about, in shape-model coordinates [km] (the center of mass of the gravity solution) Validation: `mustBeFinite`.
`degree`	scalar	double		required	highest degree to keep (2 .. the file's DEGREE) Validation: `mustBeInteger, mustBeGreaterThanOrEqual(degree, 2)`.

Outputs

Name	Description
`obs`	The measurement set: coefficients of degree 2 … degree with their covariance, `coef_about` = 'origin'. Fields: `obs`.

Description

The file is parsed as text (not executed). Zonal terms `J_l` are converted to `C_l0` = -`J_l` (with the matching sign changes in the covariance); `C_lm`, `S_lm` are kept. The covariance of the kept coefficients is the full submatrix of the file's covariance (GM row and column removed). The mass is GM / G. Degree-1 terms are absent in such files (the expansion is about the center of mass); add them with sbtk.data.add_center_of_mass.

Example

MATLAB, from the help text

```
body = sbtk.body.create("Bennu", sbtk.shape.bundled("bennu"), 1190, 0.290);
file = fullfile(sbtk.root, "data", "gravity", "bennu_osirisrex_jpl_2019.txt");
obs  = sbtk.io.read_jpl_gravity(file, body, "bennu_jpl2019", [0 0 0], 4);
obs  = sbtk.data.add_center_of_mass(obs, [0 0 0], 0.001);
```

Related functions

Mentioned in the help text

sbtk.body.create, sbtk.data.add_center_of_mass, sbtk.root, sbtk.shape.bundled

Calls

sbtk.units.G

Implementation: `toolbox/+sbtk/+io/read_jpl_gravity.m`, line 1.

---

sbtk.io.read_measurements

Measurement set from the SBTK text format.

Syntax

MATLAB

```
obs = sbtk.io.read_measurements(file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`file`	1 × 1	string		required	Path of a text file in the SBTK measurement format. Validation: `mustBeFile`.

Outputs

Name	Description
`obs`	The measurement set (diagonal covariance). Fields: `obs`.

Description

Keywords (see sbtk.io.write_measurements and docs/data_formats.md):

```
body, name, mass_kg, GM_km3_s2, R_ref_km, degree
origin_km          expansion origin, shape-model coordinates [km]
com_minus_cof_km   center-of-mass offset from the center of figure [km]
coef l m C S sigma_C sigma_S        one line per degree and order
inertia label value sigma w(1..6)   one line per moment-of-inertia measurement
```

Other keywords are ignored. This is the entry point for data from other teams: convert their files to this format, or build the measurements from arrays with sbtk.data.from_coefficients.

Example

MATLAB, from the help text

```
obs = sbtk.io.read_measurements("measurements.txt");
```

Related functions

Mentioned in the help text

sbtk.data.from_coefficients, sbtk.io.write_measurements

Called by

sbtk.io.load_measurements

Implementation: `toolbox/+sbtk/+io/read_measurements.m`, line 1.

`sbtk.io.report_comparison`

Save, print and plot a comparison with the true model.

Syntax

MATLAB

```
sbtk.io.report_comparison(out, body, truth, fit, cmp)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`body`	1 × 1	struct		required	The body. Fields: `body`.
`truth`	1 × 1	struct		required	The true model. Fields: `truth`.
`fit`	1 × 1	struct		required	The inversion result. Fields: `fit`.
`cmp`	1 × 1	struct		required	Their comparison from `sbtk.compare.with_truth`. Fields: `cmp`.

Description

Writes to `out.folder`: comparison.txt/.mat, `settings.json`, figures/comparison.png and figures/truth_vs_estimate.png.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `fit`.

```
out = sbtk.io.new_run("my_project", "T1_comparison");
sbtk.io.report_comparison(out, body, truth, fit, sbtk.compare.with_truth(body, truth, fit));
sbtk.io.finish_run(out);
```

Related functions

Mentioned in the help text

`sbtk.compare.with_truth`, `sbtk.io.finish_run`, `sbtk.io.new_run`

Calls

`sbtk.compare.summary`, `sbtk.io.save_comparison`, `sbtk.io.save_figure`, `sbtk.io.save_settings`, `sbtk.plot.comparison`, `sbtk.plot.truth_vs_estimate`

Implementation: `toolbox/+sbtk/+io/report_comparison.m`, line 1.

`sbtk.io.report_inversion`

Save, print and plot an inversion result.

Syntax

MATLAB

```
sbtk.io.report_inversion(out, body, fit, homogeneous)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`body`	1 × 1	struct		required	The body. Fields: `body`.
`fit`	1 × 1	struct		required	from sbtk.invert.run Fields: `fit`.
`homogeneous`	1 × 1	struct		required	from sbtk.invert.homogeneous (the same data fitted by a homogeneous body) Fields: `fit`.

Description

Writes to `out.folder`:

`fit.txt/.mat`	densities with 1-sigma uncertainties and normalized residuals
`homogeneous.txt`	the homogeneous fit
`settings.json`	body, data, components, options
`figures/residuals.png, densities.png, estimate_3d.png, estimate_slices.png`	

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `components`, `obs`.

```
out  = sbtk.io.new_run("my_project", "T1_inversion");
prob = sbtk.invert.setup(body, components, obs, sbtk.invert.options());
sbtk.io.report_inversion(out, body, sbtk.invert.run(prob), sbtk.invert.homogeneous(prob));
sbtk.io.finish_run(out);
```

Related functions

Mentioned in the help text

`sbtk.invert.homogeneous`, `sbtk.invert.options`, `sbtk.invert.run`, `sbtk.invert.setup`, `sbtk.io.finish_run`, `sbtk.io.new_run`

Calls

`sbtk.invert.summary`, `sbtk.io.save_figure`, `sbtk.io.save_fit`, `sbtk.io.save_settings`, `sbtk.io.write_fit`, `sbtk.plot.densities`, `sbtk.plot.interior3d`, `sbtk.plot.residuals`, `sbtk.plot.slices`

Implementation: `toolbox/+sbtk/+io/report_inversion.m`, line 1.

`sbtk.io.report_simulation`

Save, print and plot synthetic measurements and their true model.

Syntax

MATLAB

```
sbtk.io.report_simulation(out, body, obs, truth)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder (or a subfolder from `sbtk.io.subfolder`). Fields: `out`.
`body`	1 × 1	struct		required	The body. Fields: `body`.
`obs`	1 × 1	struct		required	The simulated measurements. Fields: `obs`.
`truth`	1 × 1	struct		required	The true model. Fields: `truth`.

Description

Writes to `out.folder`:

`measurements.txt/.mat`	the measurements (the .txt is the exchange format)
`truth.txt/.mat`	the true model (keep private in blind tests)
`settings.json`	body, degree, uncertainties, components, random field, noise seed
`figures/truth_3d.png, truth_slices.png, measurements.png`	

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `obs`, `truth`.

```
out = sbtk.io.new_run("my_project", "T1");
sbtk.io.report_simulation(sbtk.io.subfolder(out, "unredacted"), body, obs, truth);   % or out itself
sbtk.io.finish_run(out);
```

Related functions

Mentioned in the help text

`sbtk.io.finish_run`, `sbtk.io.new_run`, `sbtk.io.subfolder`

Calls

`sbtk.data.summary`, `sbtk.interior.summary`, `sbtk.io.save_figure`, `sbtk.io.save_measurements`, `sbtk.io.save_settings`, `sbtk.io.save_truth`, `sbtk.plot.interior3d`, `sbtk.plot.measurements`, `sbtk.plot.slices`

Implementation: `toolbox/+sbtk/+io/report_simulation.m`, line 1.

`sbtk.io.save_comparison`

comparison.mat and comparison.txt in the run folder.

Syntax

```
MATLAB
sbtk.io.save_comparison(out, cmp)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`cmp`	1 × 1	struct		required	Comparison from `sbtk.compare.with_truth`. Fields: `cmp`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`, `cmp`.

```
sbtk.io.save_comparison(out, cmp);          % comparison.mat and comparison.txt in out.folder
```

Related functions

Calls

`sbtk.io.write_comparison`

Called by

`sbtk.io.report_comparison`

Implementation: `toolbox/+sbtk/+io/save_comparison.m`, line 1.

`sbtk.io.save_figure`

Save a figure as figures/<name>.png (200 dpi) in the run folder.

Syntax

```
MATLAB
file = sbtk.io.save_figure(out, fig, name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder; the file goes to `out.figures`. Fields: `out`.
`fig`	1 × 1	matlab.ui.Figure		required	Figure handle.
`name`	1 × 1	string		required	File name without extension.

Outputs

Name	Description
`file`	Path of the PNG file written.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`, `fit`.

```
sbtk.io.save_figure(out, sbtk.plot.residuals(fit), "residuals");   % out.folder/figures/residuals.png
```

Related functions

Mentioned in the help text

`sbtk.plot.residuals`

Called by

`sbtk.io.report_comparison`, `sbtk.io.report_inversion`, `sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+io/save_figure.m`, line 1.

`sbtk.io.save_fit`

`fit.mat` and `fit.txt` in the run folder.

Syntax

```
MATLAB
sbtk.io.save_fit(out, fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`fit`	1 × 1	struct		required	Inversion result. Fields: `fit`.

Example

```
MATLAB fragment from the help text. Uses variables defined by earlier calls: out, fit.
sbtk.io.save_fit(out, fit);              % fit.mat and fit.txt in out.folder
```

Related functions

Calls

`sbtk.io.write_fit`

Called by

`sbtk.io.report_inversion`

Implementation: `toolbox/+sbtk/+io/save_fit.m`, line 1.

`sbtk.io.save_measurements`

measurements.mat and measurements.txt in the run folder.

Syntax

```
MATLAB
sbtk.io.save_measurements(out, obs)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.

Example

```
MATLAB fragment from the help text. Uses variables defined by earlier calls: out, obs.
sbtk.io.save_measurements(out, obs);        % measurements.mat and measurements.txt in out.folder
```

Related functions

Calls

`sbtk.io.write_measurements`

Called by

`sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+io/save_measurements.m`, line 1.

`sbtk.io.save_settings`

Write the settings of a run to `settings.json` (readable, reproducible).

Syntax

```
MATLAB
sbtk.io.save_settings(out, settings)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`settings`	1 × 1	struct		required	any struct: body name, shape file, components, uncertainties, seeds, inversion options... Matrices and struct arrays are written as JSON.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`.

```
sbtk.io.save_settings(out, struct('noise_seed', 1, 'note', "first try"));
```

Related functions

Calls

`sbtk.version`

Called by

`sbtk.io.report_comparison`, `sbtk.io.report_inversion`, `sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+io/save_settings.m`, line 1.

`sbtk.io.save_truth`

`truth.mat` and `truth.txt` in the run folder.

Syntax

MATLAB

```
sbtk.io.save_truth(out, truth)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	Results folder. Fields: `out`.
`truth`	1 × 1	struct		required	The true model. Fields: `truth`.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `out`, `truth`.

```
sbtk.io.save_truth(out, truth);            % truth.mat and truth.txt in out.folder
```

Related functions

Calls

`sbtk.io.write_truth`

Called by

`sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+io/save_truth.m`, line 1.

`sbtk.io.subfolder`

A subfolder of a run folder that the save and report functions write to.

Syntax

MATLAB

```
sub = sbtk.io.subfolder(out, name)
```

Arguments

Name	Size	Class	Unit	Default	Description
`out`	1 × 1	struct		required	from sbtk.io.new_run Fields: `out`.
`name`	1 × 1	string		required	Name of the subfolder.

Outputs

Name	Description
`sub`	the same run with `sub.folder` = <run>/<name> and `sub.figures` = <run>/<name>/figures; the log stays in <run>/log.txt Fields: `out`.

Example

keep the true model apart from the data package to send

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `obs`, `truth`.

```
out = sbtk.io.new_run("hera_hlp6", "didymos_T1");
sbtk.io.report_simulation(sbtk.io.subfolder(out, "unredacted"), body, obs, truth);
```

Related functions

Mentioned in the help text

`sbtk.io.new_run`, `sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+io/subfolder.m`, line 1.

`sbtk.io.write_comparison`

Plain-text copy of a comparison with the true model.

Syntax

MATLAB

```
sbtk.io.write_comparison(cmp, file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`cmp`	1 × 1	struct		required	Comparison from `sbtk.compare.with_truth`. Fields: `cmp`.
`file`	1 × 1	string		required	Path of the text file to write.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `cmp`.

```
sbtk.io.write_comparison(cmp, "comparison.txt");
```

Related functions

Called by

`sbtk.io.save_comparison`

Implementation: `toolbox/+sbtk/+io/write_comparison.m`, line 1.

`sbtk.io.write_fit`

Plain-text summary of an inversion result.

Syntax

MATLAB

```
sbtk.io.write_fit(fit, file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`fit`	1 × 1	struct		required	Inversion result. Fields: `fit`.
`file`	1 × 1	string		required	Path of the text file to write.

Description

Lines starting with '#' describe the lines below them. Example:

```
  chi2 26.742   dof 24.0   chi2_reduced 1.114   p_value 0.316
  density core 453.423 268.160 3161.834 205.147
  residual C20 -3.822967e-02 -3.823088e-02 1.000000e-04 -0.012
```

Related functions

Called by

`sbtk.io.report_inversion`, `sbtk.io.save_fit`

Implementation: `toolbox/+sbtk/+io/write_fit.m`, line 1.

`sbtk.io.write_measurements`

Plain-text copy of a measurement set (read back by sbtk.io.read_measurements).

Syntax

MATLAB

```
sbtk.io.write_measurements(obs, file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.
`file`	1 × 1	string		required	Path of the text file to write.

Description

One keyword per line, values separated by spaces, '#' lines are comments, so every block can be pasted into a spreadsheet or another code. Only the diagonal of the coefficient covariance is written (the .mat keeps it all). Keywords: body, name, `mass_kg`, `GM_km3_s2`, `R_ref_km`, degree, `origin_km`, `com_minus_cof_km`, coef (l m C S `sigma_C` `sigma_S`), inertia (label value sigma `w_Ixx` `w_Iyy` `w_Izz` `w_Ixy` `w_Ixz` `w_Iyz`), plus informational lines.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `obs`.

```
sbtk.io.write_measurements(obs, "measurements.txt");
```

Related functions

Calls

`sbtk.units.norm_to_moi`

Called by

`sbtk.io.save_measurements`

Implementation: `toolbox/+sbtk/+io/write_measurements.m`, line 1.

`sbtk.io.write_truth`

Plain-text description of a true model (keep private in blind tests).

Syntax

MATLAB

```
sbtk.io.write_truth(truth, file)
```

Arguments

Name	Size	Class	Unit	Default	Description
`truth`	1 × 1	struct		required	The true model. Fields: `truth`.
`file`	1 × 1	string		required	Path of the text file to write.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `truth`.

```
sbtk.io.write_truth(truth, "truth.txt");
```

Related functions

Calls

`sbtk.version`

Called by

`sbtk.io.save_truth`

Implementation: `toolbox/+sbtk/+io/write_truth.m`, line 1.

Reference

`sbtk.plot`: Figures

Function	Purpose
`comparison`	Estimate compared with the true model: R2 and each component's density.
`densities`	Estimated density of the background and each component with its 1-sigma uncertainty.
`difference_from_homogeneous`	Gravity coefficients minus those of the homogeneous body, coefficient by coefficient.
`interior3d`	See-through 3D view of the body and the components of a model.
`measurements`	What the data contain: gravity signal and noise per degree, and the inertia rows.
`residuals`	Normalized residuals of a fit, (predicted - measured)/sigma, per measurement.
`shape`	The body's shape model with its center of figure and axes.
`slices`	Absolute density on the equatorial plane and a meridional plane.
`truth_vs_estimate`	Truth, estimate and their difference on two planes.

`sbtk.plot.comparison`

Estimate compared with the true model: R2 and each component's density.

Syntax

MATLAB

```
fig = sbtk.plot.comparison(cmp)          % cmp from sbtk.compare.with_truth
```

Arguments

Name	Size	Class	Unit	Default	Description
`cmp`	1 × 1	struct		required	Comparison from `sbtk.compare.with_truth`. Fields: `cmp`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Left: R2 of the estimated density (fraction of the variance of the true density explained) next to `R2_max`, the largest R2 the same components can reach. Right: each component's estimated density with +-1 sigma (bar) and the true density (cross).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `fit`.

```
fig = sbtk.plot.comparison(sbtk.compare.with_truth(body, truth, fit));
```

Related functions

Mentioned in the help text

`sbtk.compare.with_truth`

Called by

`sbtk.io.report_comparison`

Implementation: `toolbox/+sbtk/+plot/comparison.m`, line 1.

`sbtk.plot.densities`

Estimated density of the background and each component with its 1-sigma uncertainty.

Syntax

MATLAB

```
fig = sbtk.plot.densities(fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`fit`	1 × 1	struct		required	Inversion result. Fields: `fit`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

One row per component (background first): the density [kg/m^3] and a +-1 sigma bar. The dashed line is the background density. Gray: components whose relative density is not resolved (|relative density| < 1 sigma).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `fit`.

```
fig = sbtk.plot.densities(fit);
```

Related functions

Called by

`sbtk.exchange.write_solution`, `sbtk.io.report_inversion`

Implementation: `toolbox/+sbtk/+plot/densities.m`, line 1.

`sbtk.plot.difference_from_homogeneous`

Gravity coefficients minus those of the homogeneous body, coefficient by coefficient.

Syntax

MATLAB

```
[fig, differences] = sbtk.plot.difference_from_homogeneous(body, sets, names, scale, frame)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	the body: its shape model gives the homogeneous body (density M/V everywhere) Fields: `body`.
`sets`	1 × any	cell		required	cell array of coefficient sets, each one of a measurement set exact values (sbtk.data.simulate), measurements (sbtk.data.add_noise), a package (sbtk.exchange.read_data), ... a true model truth from sbtk.data.simulate an inversion result fit from sbtk.invert.run: the field of the fitted model
`names`				required	one legend entry per set, e.g. ["true model" "estimate"] Validation: `mustBeText`.
`scale`	1 × 1	string		required	"linear": the signed difference, with +-3 sigma bars for sets that have uncertainties (measurement sets with `coef_cov`); "log": the absolute difference, with 3 sigma drawn as a separate line Validation: `mustBeMember(scale, ["linear", "log"])`.
`frame`	1 × 1	string		required	the axes both fields are expressed in (sbtk.gravity.center_of_mass_frame): "shape": the axes of the shape-model frame; "spin": z along the axis of the largest moment of the set, x the shape-model x-axis projected onto the plane normal to it, as in body-fixed radio-science fields (the OSIRIS-REx field of Bennu). The set has C21 = S21 = 0, so the homogeneous body's C21, S21 show how far its axis of the largest moment is tilted from the set's; "principal": the principal axes of inertia of the set (x the axis of the smallest moment, z of the largest). The set has C21 = S21 = S22 = 0, so the homogeneous body's C21, S21, S22 show how its principal axes are tilted and turned. Validation: `mustBeMember(frame, ["shape", "spin", "principal"])`.

Outputs

Name	Description
`fig`	the figure
`differences`	struct array, one per set: name, labels (as plotted: C10, C11, S11, J2, ...), difference, sigma (1-sigma; NaN where the set has none), `axes_deg` ([tilt turn]: angle between the axes of the largest moment of the set and of the homogeneous body, and angle about that axis between their axes of the smallest moment, deg)

Description

Both fields are about their own center of mass, as radio-science fields are:

```
  degrees >= 2   the set about its center of mass minus the homogeneous body about its
                 center of mass (the center of figure)
  degree 1       the set's center of mass minus the center of figure, as C11, S11, C10 =
                 (x, y, z)/(sqrt(3) R_ref) in the axes of the plot: about its own center of
                 mass each field's degree 1 is zero, so this shows where the interior moves
                 the center of mass instead
```

sbtk.gravity.center_of_mass_frame moves each set to its center of mass and into the axes (it states how the center of mass of each kind of set is found); the homogeneous field is rotated into the same axes. Differences are fully normalized, in SBTK order; zonal coefficients are shown as `J_l` = -`C_l0`, so their difference has the opposite sign of the `C_l0` difference. The reference radius is the set's (`body.R_ref_km` for truth and fit). One geometry is built per set (about 1 s for 49,152 facets at degree 4, about a minute for 3 million facets).

The plot shows how far the gravity field of an interior departs from that of a uniform density, and, against 3 sigma, which departures measurements resolve. The three frames show the same information: the spin and principal axes move the difference in orientation of the principal axes into the homogeneous body's degree-2 terms.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `interior`, `unc`, `components`.

```
[exact, truth] = sbtk.data.simulate(body, interior, 4, "T1");
sbtk.plot.difference_from_homogeneous(body, {truth}, "true model", "linear", "principal");
obs = sbtk.data.add_noise(sbtk.data.set_uncertainties(exact, unc), 1);
fit = sbtk.invert.solve(body, components, obs);
sbtk.plot.difference_from_homogeneous(body, {obs, fit}, ["measured" "estimate"], "log", "shape");
```

Related functions

Mentioned in the help text

sbtk.data.add_noise, sbtk.data.set_uncertainties, sbtk.data.simulate, sbtk.exchange.read_data, sbtk.gravity.center_of_mass_frame, sbtk.invert.run, sbtk.invert.solve

Calls

sbtk.gravity.center_of_mass_frame, sbtk.gravity.forward, sbtk.gravity.geometry, sbtk.interior.create, sbtk.interior.no_components

Implementation: `toolbox/+sbtk/+plot/difference_from_homogeneous.m`, line 1.

---

sbtk.plot.interior3d

See-through 3D view of the body and the components of a model.

Syntax

MATLAB

```
fig = sbtk.plot.interior3d(body, model, title_str)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.
`model`	1 × 1	struct		required	a truth or a fit (anything with components and `origin_km`)
`title_str`	1 × 1	string		required	Title of the figure.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

The surface is drawn translucent; spheres, ellipsoids and tori are drawn where they lie inside the body, layers by their lower boundary. Color is the relative density (blue lighter, red denser than the background). Sectors are not drawn (see sbtk.plot.slices).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `fit`.

```
fig = sbtk.plot.interior3d(body, truth, "true model");
fig = sbtk.plot.interior3d(body, fit, "estimate");
```

Related functions

Mentioned in the help text

sbtk.plot.slices

Calls

sbtk.gravity.geometry

Called by

sbtk.exchange.write_model, sbtk.exchange.write_solution, sbtk.io.report_inversion, sbtk.io.report_simulation

Implementation: `toolbox/+sbtk/+plot/interior3d.m`, line 1.

`sbtk.plot.measurements`

What the data contain: gravity signal and noise per degree, and the inertia rows.

Syntax

```
MATLAB
fig = sbtk.plot.measurements(obs)
```

Arguments

Name	Size	Class	Unit	Default	Description
`obs`	1 × 1	struct		required	The measurement set. Fields: `obs`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Left: root-mean-square size of the coefficients of each degree (dots) and of their 1-sigma uncertainties (line). Degrees where the dots sit well above the line carry information about the interior; where they meet, the data are noise (degree 1 is near zero about the center of mass). Right: each moment-of-inertia measurement with its 1-sigma ('libration' is the row (B - A) - ratio*Izz, measured as 0).

Example

```
MATLAB fragment from the help text. Uses variables defined by earlier calls: obs.
fig = sbtk.plot.measurements(obs);
```

Related functions

Called by

`sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+plot/measurements.m`, line 1.

`sbtk.plot.residuals`

Normalized residuals of a fit, (predicted - measured)/sigma, per measurement.

Syntax

```
MATLAB
fig = sbtk.plot.residuals(fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`fit`	1 × 1	struct		required	Inversion result. Fields: `fit`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Gray band: +-1 sigma; dashed: +-2 sigma. Blue within 1 sigma, amber 1-2, red beyond 2. For a correct model about 68% of the residuals fall within +-1 sigma; systematic residuals in one group of coefficients (e.g. all of degree 3) indicate structure the model does not contain.

Example

```
MATLAB fragment from the help text. Uses variables defined by earlier calls: fit.
fig = sbtk.plot.residuals(fit);
```

Related functions

Called by

`sbtk.exchange.write_solution`, `sbtk.io.report_inversion`

Implementation: `toolbox/+sbtk/+plot/residuals.m`, line 1.

`sbtk.plot.shape`

The body's shape model with its center of figure and axes.

Syntax

MATLAB

```
fig = sbtk.plot.shape(body)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Example

MATLAB, from the help text

```
fig = sbtk.plot.shape(sbtk.body.from_mass("Didymos", sbtk.shape.bundled("didymos"), 5.273e11, 0.43));
```

Related functions

Mentioned in the help text

`sbtk.body.from_mass`, `sbtk.shape.bundled`

Implementation: `toolbox/+sbtk/+plot/shape.m`, line 1.

`sbtk.plot.slices`

Absolute density on the equatorial plane and a meridional plane.

Syntax

MATLAB

```
fig = sbtk.plot.slices(body, models, titles)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.
`models`	1 × any	cell		required	cell array of models (truth from sbtk.data.simulate, fit from sbtk.invert.run): one row of panels each, one shared color scale
`titles`	1 × any	cell		required	cell array, one label per model

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Left: the plane z = 0 (looking down the spin axis). Right: the plane y = 0. Both pass through the first model's `origin_km` (its center of mass).

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`.

```
fig = sbtk.plot.slices(body, {truth}, {'truth'});
```

Related functions

Mentioned in the help text

`sbtk.data.simulate`, `sbtk.invert.run`

Calls

`sbtk.gravity.geometry`

Called by

`sbtk.exchange.write_model`, `sbtk.exchange.write_solution`, `sbtk.io.report_inversion`, `sbtk.io.report_simulation`

Implementation: `toolbox/+sbtk/+plot/slices.m`, line 1.

`sbtk.plot.truth_vs_estimate`

Truth, estimate and their difference on two planes.

Syntax

MATLAB

```
fig = sbtk.plot.truth_vs_estimate(body, truth, fit)
```

Arguments

Name	Size	Class	Unit	Default	Description
`body`	1 × 1	struct		required	The body. Fields: `body`.
`truth`	1 × 1	struct		required	The true model. Fields: `truth`.
`fit`	1 × 1	struct		required	The inversion result. Fields: `fit`.

Outputs

Name	Description
`fig`	Handle of the new figure.

Description

Rows: truth, estimate (same density color scale), and estimate minus truth (blue = estimate too light, red = too dense). Left: plane z = 0; right: plane y = 0, both through the truth's center of mass.

Example

MATLAB fragment from the help text. Uses variables defined by earlier calls: `body`, `truth`, `fit`.

```
fig = sbtk.plot.truth_vs_estimate(body, truth, fit);
```

Related functions

Calls

`sbtk.gravity.geometry`

Called by

`sbtk.io.report_comparison`

Implementation: `toolbox/+sbtk/+plot/truth_vs_estimate.m`, line 1.

Data structures

SBTK passes MATLAB structures between its functions. Each structure has a fixed set of fields, listed below; the field names carry their units (see Units). Sizes refer to the Didymos examples (49,152 facets, degree 4) unless stated: *n* is the number of measurements, *nc* the number of components, *nf* the number of facets, *L* the highest degree.

A *model* is any structure with the fields `rho_bg_kgm3`, `components`, `random_field` and `origin_km`: a true model (`truth`) or an inversion result (`fit`). Functions that evaluate, plot or compare densities accept either.

Structure	Contents	Created by
`shape`	Shape model	`sbtk.shape.load`, `sbtk.shape.bundled`, `sbtk.shape.from_arrays`, `sbtk.shape.ellipsoid`, `sbtk.shape.reduce`, `sbtk.shape.reduce_spc`
`props`	Shape properties	`sbtk.shape.properties`
`body`	Body	`sbtk.body.create`, `sbtk.body.from_mass`
`component`	Interior component	`sbtk.interior.sphere`, `sbtk.interior.ellipsoid`, `sbtk.interior.torus`, `sbtk.interior.layer`, `sbtk.interior.sector`, `sbtk.interior.sector_grid`
`interior`	Interior model	`sbtk.interior.create`
`random_field`	Gaussian random density field	`sbtk.interior.random_field`, `sbtk.interior.no_random_field`
`unc`	Uncertainties for a simulation	`sbtk.data.uncertainties`
`obs`	Measurement set	`sbtk.data.simulate`, `sbtk.data.from_coefficients`, `sbtk.exchange.read_data`, `sbtk.exchange.read_shadr`, `sbtk.io.read_jpl_gravity`, `sbtk.io.read_measurements`, `sbtk.io.load_measurements`
`moi`	Moment-of-inertia measurement row	`sbtk.data.add_moi`, `sbtk.data.add_moi_row`, `sbtk.data.add_libration`, `sbtk.data.simulate`
`truth`	True model of a simulation	`sbtk.data.simulate`, `sbtk.io.load_truth`
`geo`	Facet-cone geometry	`sbtk.gravity.geometry`
`integrals`	Integrals of a component	`sbtk.gravity.component_integrals`, `sbtk.gravity.random_field_integrals`
`fm`	Forward-model result	`sbtk.gravity.forward`
`cf`	Field about its center of mass	`sbtk.gravity.center_of_mass_frame`
`opts`	Inversion settings	`sbtk.invert.options`
`prob`	Prepared inversion	`sbtk.invert.setup`, `sbtk.invert.replace_data`
`fit`	Inversion result	`sbtk.invert.run`, `sbtk.invert.solve`, `sbtk.invert.homogeneous`, `sbtk.io.load_fit`
`cmp`	Comparison with the true model	`sbtk.compare.with_truth`, `sbtk.compare.evaluate`
`ref`	Prepared true model for comparisons	`sbtk.compare.reference`
`grid`	Density grid	`sbtk.exchange.density_grid`, `sbtk.exchange.read_grid`
`gridcmp`	Comparison of two density grids	`sbtk.compare.grids`
`out`	Results folder	`sbtk.io.new_run`, `sbtk.io.subfolder`
`pk`	Coefficient order	`sbtk.internal.sh_packing`
`samples`	Points filling the body	`sbtk.internal.body_samples`
`slices`	Plotting planes	`sbtk.internal.slice_planes`

`shape`: Shape model

Created by `sbtk.shape.load`, `sbtk.shape.bundled`, `sbtk.shape.from_arrays`, `sbtk.shape.ellipsoid`, `sbtk.shape.reduce`, `sbtk.shape.reduce_spc`.

A closed triangular mesh with vertex positions in kilometers.

Fields of `shape` (6)

Field	Size	Unit	Description
`name`	text		File name without extension, or the name given.
`faces`	nf × 3		Vertex indices of each triangle (1-based), ordered so that the normal points outward.
`vertices_km`	nv × 3	km	Vertex positions in the shape-model frame.
`source`	text		Path of the file the shape was read from (`'arrays'` or `'ellipsoid'` for shapes not read from a file). After `sbtk.body.to_frame`, the name of the original file without its path. `sbtk.exchange.write_data` copies the file into a measurement package if the shape model is that file unchanged.
`frame_origin_km`	1 × 3	km	Origin of the shape model's frame in the coordinates of `source`: `[0 0 0]`, or as set by `sbtk.body.to_frame` (combined over successive changes of frame).

Field	Size	Unit	Description
`frame_axes`	3 × 3		Axes of the shape model's frame (columns) in the coordinates of `source`: `eye(3)`, or as set by `sbtk.body.to_frame`. The vertices are `(source vertices - frame_origin_km)*frame_axes`.

`props`: Shape properties

Created by `sbtk.shape.properties`.

Fields of `props` (8)

Field	Size	Unit	Description
`volume_km3`	scalar	km³	Enclosed volume.
`centroid_km`	1 × 3	km	Centroid of the enclosed volume (center of figure).
`n_facets`	scalar		Number of triangles.
`n_vertices`	scalar		Number of vertices.
`extent_km`	2 × 3	km	Minimum (row 1) and maximum (row 2) vertex coordinate along each axis.
`closed`	logical		True if every edge is shared by exactly two facets that traverse it in opposite directions.
`outward`	logical		True if the facets are ordered outward (positive enclosed volume).
`star_shaped_about_centroid`	logical		True if every facet faces away from the centroid, which the facet-cone integration about the centroid requires.

`body`: Body

Created by `sbtk.body.create`, `sbtk.body.from_mass`.

A shape model with the total mass and the reference radius of the gravity field.

Fields of `body` (8)

Field	Size	Unit	Description
`name`	text		Name of the body, used in printed output and file contents.
`shape`	struct		The `shape`.
`bulk_density_kgm3`	scalar	kg/m³	Mass divided by the shape volume.
`R_ref_km`	scalar	km	Reference radius of the gravity coefficients.
`volume_km3`	scalar	km³	Shape volume.
`mass_kg`	scalar	kg	Total mass.
`GM_km3_s2`	scalar	km³/s²	Gravitational parameter, GM with G from `sbtk.units.G`.
`centroid_km`	1 × 3	km	Center of figure in the shape-model frame.

`component`: Interior component

Created by `sbtk.interior.sphere`, `sbtk.interior.ellipsoid`, `sbtk.interior.torus`, `sbtk.interior.layer`, `sbtk.interior.sector`, `sbtk.interior.sector_grid`.

One region of uniform relative density. Every component has every field, so components of different types can be concatenated into one structure array (`[c1, c2]`); fields that a type does not use are zero. The geometry definitions are in Density distributions.

Fields of `component` (17)

Field	Size	Unit	Description
`name`	text		Name; unique within a model.
`type`	text		`'sphere'`, `'ellipsoid'`, `'torus'`, `'layer'` or `'sector'`.
`drho_kgm3`	scalar	kg/m³	Relative density: density minus the background density. In an inversion it is estimated unless `estimate_density` is false.
`center_km`	1 × 3	km	Center of a sphere or ellipsoid in the shape-model frame; for a torus, the point on its axis from which `height_km` is measured (`[0 0 0]` unless changed by `sbtk.interior.to_frame`).
`radius_km`	scalar	km	Sphere radius.
`semi_axes_km`	1 × 3	km	Ellipsoid semi-axes along its own axes.
`axes`	3 × 3		Columns: the component's own axes in the shape-model frame. Ellipsoid: $R_z(\mathrm{yaw})\,R_y(\mathrm{pitch})\,R_x(\mathrm{roll})$. Torus (about its third axis) and sector (latitude and longitude in these axes): `eye(3)` unless changed by `sbtk.interior.to_frame`.
`ring_radius_km`	scalar	km	Torus: radius of the circle at the center of the tube.
`tube_radius_km`	scalar	km	Torus: radius of the tube.
`height_km`	scalar	km	Torus: distance of the plane of the center circle from `center_km` along the torus axis (z for a torus made by `sbtk.interior.torus`).

Field	Size	Unit	Description
`depth_km`	1 × 2	km	Layer: top and bottom depth below the surface, measured along rays from the expansion origin.
`frac`	1 × 2		Sector: inner and outer fraction of the distance from the expansion origin to the surface (0 at the origin, 1 at the surface).
`lat_deg`	1 × 2	deg	Sector: latitude range, within −90 to 90.
`lon_deg`	1 × 2	deg	Sector: longitude range, within −360 to 360 and at most 360 wide; ranges beyond ±180 wrap.
`estimate_density`	logical		True (set by the constructors): the inversion estimates `drho_kgm3`. False: the value is held fixed (a prescribed component).
`estimate_position`	logical, 1 or 1 × 3		Spheres, ellipsoids and tori. False (set by the constructors). True: the inversion also estimates `center_km`; a mask `[x y z]` estimates only those coordinates, e.g. `[false false true]` for the height (Estimated geometry).
`estimate_axis`	logical		Ellipsoids and tori. False (set by the constructors). True: the inversion also estimates the orientation: for a torus the direction of its axis (the third of `axes`), for an ellipsoid also its turn about that axis.

`interior`: Interior model

Created by `sbtk.interior.create`.

Input of `sbtk.data.simulate` and `sbtk.gravity.forward`. The background density is not part of it: it follows from the mass of the body.

Fields of `interior` (2)

Field	Size	Unit	Description
`components`	1 × nc struct		The `component` array.
`random_field`	struct		The `random_field`; standard deviation 0 means no field.

`random_field`: Gaussian random density field

Created by `sbtk.interior.random_field`, `sbtk.interior.no_random_field`.

A sum of 300 Fourier modes with random wave vectors and phases, $\delta\rho(\mathbf{x}) = a\sum_{j=1}^{300}\cos(\mathbf{k}_j\cdot\mathbf{x}+\varphi_j)$ with $a = \sigma\sqrt{2\,/\,300}$ and $\mathbf{k}_j$ drawn from a normal distribution with standard deviation $1\,/\,L$ per component, which gives the correlation $\exp(-|\Delta\mathbf{x}|^2\,/\,(2L^2))$. The field is defined at every point and does not depend on the expansion origin.

Fields of `random_field` (6)

Field	Size	Unit	Description
`sigma_kgm3`	scalar	kg/m³	Standard deviation σ of the density perturbation; 0 means no field.
`corr_length_km`	scalar	km	Correlation length L.
`seed`	scalar		Seed of the random number stream; the same seed gives the same field.
`k_per_km`	300 × 3	1/km	Wave vectors $\mathbf{k}_j$.
`phase`	300 × 1	rad	Phases φ_j.
`amp_kgm3`	scalar	kg/m³	Amplitude a of each mode.

`unc`: Uncertainties for a simulation

Created by `sbtk.data.uncertainties`.

1σ uncertainties by degree and by inertia quantity, which `sbtk.data.set_uncertainties` gives to a measurement set (for example the exact values of `sbtk.data.simulate`, which have none) and `sbtk.data.add_noise` then draws noise from.

Fields of `unc` (3)

Field	Size	Unit	Description
`coef_sigma_by_degree`	1 × L		1σ of every fully normalized coefficient of degree 1, 2, ..., L. For degree 1, `sbtk.units.offset_to_degree1` converts a center-of-mass uncertainty in km.
`moi_sigma`	scalar		1σ of each inertia element divided by MR_{ref}^2.
`libration_sigma`	scalar		1σ of the libration ratio $(B-A)\,/\,C$.

`obs`: Measurement set

Created by `sbtk.data.simulate`, `sbtk.data.from_coefficients`, `sbtk.exchange.read_data`, `sbtk.exchange.read_shadr`, `sbtk.io.read_jpl_gravity`, `sbtk.io.read_measurements`, `sbtk.io.load_measurements`.

Gravity coefficients with their covariance, moment-of-inertia measurements, mass, reference radius and expansion origin. See Measurement sets.

Fields of `obs` (18)

Field	Size	Unit	Description
`body`	text		Name of the body.
`name`	text		Name of the data set (for packages, the package name).
`mass_kg`	scalar	kg	Total mass; the inversion keeps the model mass equal to it.
`GM_km3_s2`	scalar	km³/s²	GM.
`R_ref_km`	scalar	km	Reference radius of the coefficients.
`degree`	scalar		Highest degree L.
`origin_km`	1 × 3	km	Expansion origin in the shape-model frame.
`coef_about`	text		`'origin'`: all coefficients are about `origin_km`. `'center_of_mass'`: coefficients of degree ≥ 2 are about the body's center of mass and the degree-1 coefficients give its measured position relative to `origin_km` (Density estimation).
`com_minus_cof_km`	1 × 3	km	Center of mass minus center of figure.
`coef_labels`	n × 1 cell		Names of the coefficients, e.g. `'C10'`, `'S22'`, in the order of `coef`; C00 is not included.
`coef`	n × 1		Fully normalized coefficients.
`coef_cov`	n × n		Covariance of `coef` (diagonal or full); NaN for exact values without uncertainties.
`moi`	1 × k struct		Moment-of-inertia measurements, one `moi` row each.
`moi_tensor_norm`	3 × 3		Inertia tensor divided by MR_{ref}^2, assembled from the rows `Ixx` … `Iyz`; NaN if any of them is missing. Informational: the inversion uses the rows.
`libration_ratio`	scalar		$(B - A) / C$ used in the `libration` row; NaN if there is none.
`libration_sigma`	scalar		1σ of the libration ratio; NaN if there is none.
`noise_seed`	scalar		Seed of the simulated noise; NaN for exact values and real data.
`notes`	text		Origin of the data.

`moi`: Moment-of-inertia measurement row

Created by `sbtk.data.add_moi`, `sbtk.data.add_moi_row`, `sbtk.data.add_libration`, `sbtk.data.simulate`.

One measurement of a linear combination of the inertia elements: $\mathbf{w} \cdot [I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}] \,/\, (MR_{\mathrm{ref}}^2) = \mathrm{value} \pm \sigma$.

Fields of `moi` (4)

Field	Size	Unit	Description
`label`	text		`'Ixx'`, `'Iyy'`, `'Izz'`, `'Ixy'`, `'Ixz'`, `'Iyz'`, `'libration'`, or a name given to `sbtk.data.add_moi_row`.
`weights`	1 × 6		Weights $\mathbf{w}$ on $[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}]$. Single elements have one weight 1; the libration row is $[-\cos(2a), \cos(2a), -k, 2\sin(2a), 0, 0]$ (Moments of inertia).
`value`	scalar		Measured value (0 for the libration row).
`sigma`	scalar		1σ (for the libration row, the 1σ of the ratio times $I_{zz} \,/\, (MR_{\mathrm{ref}}^2)$); NaN for exact values without uncertainties.

`truth`: True model of a simulation

Created by `sbtk.data.simulate`, `sbtk.io.load_truth`.

A model with the exact values computed from it.

Fields of `truth` (11)

Field	Size	Unit	Description
`body`	text		Name of the body.
`name`	text		Name of the data set.
`rho_bg_kgm3`	scalar	kg/m³	Background density, set so that the model mass equals the body mass.
`components`	1 × nc struct		The `component` array with the true relative densities.
`random_field`	struct		The `random_field`.
`origin_km`	1 × 3	km	The expansion origin: the center of mass of the model. Layers and sectors are measured from it.
`mass_kg`	scalar	kg	Total mass.
`degree`	scalar		Highest degree.
`coef`	1 × (L+1)²		Exact fully normalized coefficients about `origin_km`, in the order of `pk`, C00 included.
`com_km`	1 × 3	km	Exact center of mass.
`inertia_norm`	1 × 6		Exact $[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}] \,/\, (MR_{\mathrm{ref}}^2)$ about `origin_km`.

`geo`: Facet-cone geometry

Created by `sbtk.gravity.geometry`.

The integrals of every facet cone about one expansion origin, per unit density; the workspace of the forward model and the inversion (Gravity and moment calculations). Not saved to files (about 30 MB for Didymos).

Fields of `geo` (26)

Field	Size	Unit	Description
`body`	text		Name of the body.
`origin_km`	1 × 3	km	Expansion origin (the apex of every cone).
`degree`	scalar		Highest degree L.
`R_ref_km`	scalar	km	Reference radius.
`pk`	struct		Coefficient order (pk).
`nf`	scalar		Number of facets.
`K`	scalar		Number of sub-rays per facet (9).
`faces`	nf × 3		Vertex indices of each facet.
`verts`	nv × 3	km	Vertices relative to `origin_km`.
`D`	nf × 3	km	First corner of each facet, relative to the origin.
`E`	nf × 3	km	Second corner.
`F`	nf × 3	km	Third corner.
`det6`	nf × 1	km³	Six times the cone volume, $\mathbf{D}\cdot(\mathbf{E}\times\mathbf{F})$.
`EF`	nf × 3	km²	$\mathbf{E}\times\mathbf{F}$, used in ray–facet tests.
`FD`	nf × 3	km²	$\mathbf{F}\times\mathbf{D}$.
`DE`	nf × 3	km²	$\mathbf{D}\times\mathbf{E}$.
`T`	nf × (L+1)²	km³	Per cone, $\int (r/R)^l \bar{Y}_{lm}\, dV$ (Gravity and moment calculations).
`V`	nf × 1	km³	Per-cone volume.
`m1`	nf × 3	km⁴	Per-cone first moment $\int \mathbf{x}\, dV$.
`Q`	nf × 6	km⁵	Per-cone second moments $\int [x^2, y^2, z^2, xy, xz, yz]\, dV$.
`U_body`	1 × (L+1)²	km³	Sum of `T` over the body.
`V_body`	scalar	km³	Body volume.
`m1_body`	1 × 3	km⁴	First moment of the body.
`Q_body`	1 × 6	km⁵	Second moments of the body.
`ray_t`	nf·K × 1	km	Distance from the origin to the surface along each sub-ray.
`ray_dir`	nf·K × 3		Unit direction of each sub-ray.

`integrals`: Integrals of a component

Created by `sbtk.gravity.component_integrals`, `sbtk.gravity.random_field_integrals`.

Integrals over the part of a component inside the body, about `geo.origin_km`, per unit density. From `sbtk.gravity.random_field_integrals` the same fields are weighted by the density perturbation (units multiplied by kg/m³).

Fields of `integrals` (4)

Field	Size	Unit	Description
`U`	1 × (L+1)²	km³	$\int (r/R)^l \bar{Y}_{lm}\, dV$ in the order of pk.
`V`	scalar	km³	Volume inside the body.
`m1`	1 × 3	km⁴	First moment.
`Q`	1 × 6	km⁵	Second moments $[xx, yy, zz, xy, xz, yz]$.

`fm`: Forward-model result

Created by `sbtk.gravity.forward`.

Fields of `fm` (5)

Field	Size	Unit	Description
`rho_bg_kgm3`	scalar	kg/m³	Background density that gives the stated mass.
`coef`	1 × (L+1)²		Fully normalized coefficients about `geo.origin_km`, order of pk.

Field	Size	Unit	Description
`com_km`	1 × 3	km	Center of mass in the shape-model frame.
`inertia_norm`	1 × 6		$[I_{xx}, I_{yy}, I_{zz}, I_{xy}, I_{xz}, I_{yz}] / (MR_{\mathrm{ref}}^2)$ about `geo.origin_km`.
`mass_kg`	scalar	kg	The mass given.

`cf`: Field about its center of mass

Created by `sbtk.gravity.center_of_mass_frame`.

A gravity field moved to its center of mass, in the shape-model axes or its principal axes (Frames of a gravity field). $n = (L+1)^2 - 1$.

Fields of `cf` (10)

Field	Size	Unit	Description
`labels`	1 × n		Names of the coefficients of degree 1 to L, order of `pk`.
`coef`	1 × n		Fully normalized coefficients about the center of mass in the axes `axes`: degree 1 is 0, and in principal axes $\bar{C}_{21} = \bar{S}_{21} = \bar{S}_{22} = 0$. NaN for a coefficient a measurement set does not have.
`sigma`	1 × n		1σ, NaN without uncertainties; degree 1: of the center of mass.
`cov`	n × n		Covariance propagated through the translation and the rotation.
`com_km`	1 × 3	km	Center of mass in the shape-model frame.
`axes`	3 × 3		Columns: the axes of `coef` as unit vectors in the shape-model axes (`eye(3)` for `"shape"`).
`principal`	3 × 3		Columns: the principal axes of the smallest, middle and largest moment in the shape-model axes.
`moments`	1 × 3		Principal moments minus their mean, $A \le B \le C$, as $I / (MR_{\mathrm{ref}}^2)$; the degree-2 field fixes them up to their trace.
`R_ref_km`	scalar	km	Reference radius.
`degree`	scalar		Highest degree L.

`opts`: Inversion settings

Created by `sbtk.invert.options`.

Fields of `opts` (13)

Field	Size	Unit	Description
`moi_labels`	1 × k cell		Labels of the `moi` rows to fit. Standard value `{'Ixx','Iyy','Izz'}`; `{}` fits gravity coefficients only.
`regularization`	scalar		Tikhonov parameter $\lambda \ge 0$. Standard value 0.
`background`	string		`"estimate"` (standard): the background density is a free parameter. `"bulk"`: it is held at the bulk density M / V.
`rho_max_kgm3`	scalar	kg/m³	Upper bound on the background density and on every component's density. Standard value 4500.
`max_iter`	scalar		Maximum Gauss–Newton iterations for estimated geometry. Standard value 20.
`step_km`	scalar	km	Trust-region radius of one step of a center coordinate. Standard value 0.010.
`step_deg`	scalar	deg	Trust-region radius of one step of an axis tilt or turn. Standard value 1.
`position_sigma_km`	scalar	km	A priori 1σ of each estimated center coordinate about its starting value; `Inf` (standard) means no a priori constraint.
`axis_sigma_deg`	scalar	deg	A priori 1σ of each estimated axis tilt and turn about the starting orientation; `Inf` (standard) means none.
`max_shift_km`	scalar	km	Largest distance of an estimated center from its starting point; `Inf` (standard) means no limit.
`max_tilt_deg`	scalar	deg	Largest angle of an estimated axis from its starting direction; `Inf` (standard) means no limit.
`max_turn_deg`	scalar	deg	Largest turn of an estimated ellipsoid about its axis from its starting orientation; `Inf` (standard) means no limit.
`geometry_tol`	scalar		The geometry has converged when an accepted step changes every parameter by less than this fraction of its step limit. Standard value 10^{-3}.

`prob`: Prepared inversion

Created by `sbtk.invert.setup`, `sbtk.invert.replace_data`.

Everything `sbtk.invert.run` needs, computed once. Sizes for 27 measurements and two components.

Fields of `prob` (27)

Field	Size	Unit	Description
`body`	struct		The `body`.
`obs_name`	text		Name of the measurement set.

Field	Size	Unit	Description
`obs`	struct		The `obs`.
`coef_about`	text		Copied from `obs.coef_about`.
`opts`	struct		The `opts`; fields that do not select measurements may be changed between runs.
`geo`	struct		Geometry about `obs.origin_km` (`geo`).
`mass_kg`	scalar	kg	Mass of the measurement set.
`rho_mean_kgm3`	scalar	kg/m³	Bulk density M / V.
`jc`	1 × n		Position of each measured coefficient in the packed vector of `pk`.
`W`	k × 6		Weights of the inertia rows used.
`y`	n × 1		Measurement vector (coefficients, then inertia rows), including the linearization offset of the center-of-mass iteration.
`Cov`	n × n		Measurement covariance: the coefficient covariance and the inertia variances.
`Lw`	n × n		Lower Cholesky factor of `Cov`, used to whiten the problem.
`labels`	n × 1 cell		Measurement names.
`shift_km`	1 × 3	km	Point the predicted coefficients of degree ≥ 2 and inertia are about, relative to the origin.
`T`	(L+1)² × (L+1)²		Solid-harmonic translation matrix for `shift_km`.
`D`	n × 3		Derivative of the predictions with respect to the expansion point, the coupling term of the center-of-mass iteration.
`y_measured`	n × 1		Measurement vector as given.
`Ibody`	struct		Integrals of the whole body (`integrals`).
`gbody`	n × 1		Predicted measurements of a body of unit density.
`components`	1 × nc struct		The `component` array.
`Ic`	1 × nc struct		`integrals` of each component.
`mov`	1 × m		Indices of the components whose geometry is estimated.
`gp`	1 × p struct		The estimated geometry parameters, one per scalar (`sbtk.internal.geometry_parameters`): `component`, `kind` (`'position'` or `'tilt'`: a rotation about the component's own axis `index`), `index`, `name`.
`start`	1 × nc struct		The components as given: priors and limits on the geometry refer to them.
`G`	n × nc		Design matrix: change of each measurement per unit relative density of each component (kg/m³).
`Vc`	1 × nc	km³	Volume of each component inside the body.

`fit`: Inversion result

Created by `sbtk.invert.run`, `sbtk.invert.solve`, `sbtk.invert.homogeneous`, `sbtk.io.load_fit`.

A model (`rho_bg_kgm3`, `components`, `random_field`, `origin_km`) with the estimates, their uncertainties and the fit statistics. See Density estimation and Goodness of fit.

Fields of `fit` (37)

Field	Size	Unit	Description
`body`	text		Name of the body.
`data_name`	text		Name of the measurement set.
`rho_bg_kgm3`	scalar	kg/m³	Estimated background density.
`components`	1 × nc struct		Components with the estimated `drho_kgm3` (and `center_km`, `axes` where estimated).
`random_field`	struct		No field (standard deviation 0).
`origin_km`	1 × 3	km	Origin of the model geometry: `obs.origin_km`.
`rho_bg_sigma_kgm3`	scalar	kg/m³	1σ of the background density.
`drho_sigma_kgm3`	1 × nc	kg/m³	1σ of each relative density.
`rho_kgm3`	1 × nc	kg/m³	Density of each component: background plus relative density.
`rho_sigma_kgm3`	1 × nc	kg/m³	1σ of each component density, including the covariance with the background.
`center_sigma_km`	nc × 3	km	1σ of the estimated center coordinates; NaN where not estimated.
`axis_sigma_deg`	nc × 3	deg	1σ of the estimated rotations about the component's first, second and third axes (the tilts of its axis and, for an ellipsoid, the turn about it); NaN where not estimated.
`at_bound`	1 × nc logical		Density at 0 or at `opts.rho_max_kgm3`. The stated 1σ does not account for the bound.

Field	Size	Unit	Description
`resolved`	1 × nc logical		Absolute relative density larger than its 1σ.
`determined`	logical		False if the measurements, with any regularization or a priori constraint, do not determine every free parameter; the covariance is then a pseudo-inverse.
`chi2`	scalar		χ^2 of the measurements.
`n_obs`	scalar		Number of measurements n.
`dof`	scalar		Degrees of freedom $\nu = n - p_{\text{eff}}$ (Goodness of fit).
`chi2_reduced`	scalar		χ^2 / ν.
`p_value`	scalar		$P(\chi^2_\nu \geq \chi^2)$.
`consistent`	logical		`p_value >= 0.05`.
`labels`	n × 1 cell		Measurement names.
`measured`	n × 1		Measured values.
`predicted`	n × 1		Values predicted by the fitted model.
`sigma`	n × 1		1σ of each measurement (square root of the covariance diagonal).
`normalized_residual`	n × 1		(predicted − measured)/σ.
`names`	1 × p cell		Names of the parameters: `'background'`, component names, and `<name>_x`, `_y`, `_z` for estimated centers.
`cov`	p × p		Covariance of the parameters (kg²/m⁶, kg/m³·km, km²).
`corr`	p × p		Correlation matrix of the parameters.
`iterations`	scalar		Gauss–Newton iterations for sphere positions (1 if none are estimated).
`regularization`	scalar		Tikhonov parameter used.
`moi_labels`	1 × k cell		Inertia rows used.
`background`	text		`'estimate'` or `'bulk'`.
`frame_iterations`	scalar		Iterations of the center-of-mass frame (0 when `coef_about` is `'origin'`).
`frame_offset_km`	scalar	km	Change of the model's center of mass in the last frame iteration.
`coef_about`	text		Copied from the measurements.
`com_shift_km`	1 × 3	km	Model center of mass minus `origin_km`: the point the predicted coefficients of degree ≥ 2 and inertia are about (`[0 0 0]` when `coef_about` is `'origin'`).

`cmp`: Comparison with the true model

Created by `sbtk.compare.with_truth`, `sbtk.compare.evaluate`.

See Comparison with a true model.

Fields of `cmp` (8)

Field	Size	Unit	Description
`body`	text		Name of the body.
`R2`	scalar		Fraction of the volume-weighted variance of the true density explained by the estimate.
`R2_max`	scalar		Largest R^2 the same components can reach.
`rms_kgm3`	scalar	kg/m³	Volume-weighted root-mean-square density error.
`std_true_kgm3`	scalar	kg/m³	Volume-weighted standard deviation of the true density.
`chi2_reduced`	scalar		Copied from the fit.
`p_value`	scalar		Copied from the fit.
`components`	struct array		Components present in both models, with fields `name`, `estimate_kgm3`, `sigma_kgm3`, `true_kgm3` (densities) and `deviation_sigma` (absolute difference divided by `sigma_kgm3`).

`ref`: Prepared true model for comparisons

Created by `sbtk.compare.reference`.

Sizes for Didymos (1,179,648 sample points).

Fields of `ref` (9)

Field	Size	Unit	Description
`body`	text		Name of the body.

Field	Size	Unit	Description
`truth`	struct		The `truth`.
`components`	1 × nc struct		Components of the fits to be compared.
`origin_km`	1 × 3	km	Origin of those fits.
`w`	1179648 × 1	km³	Volume of each sample point.
`rho_true`	1179648 × 1	kg/m³	True density at each sample point.
`member`	1179648 × nc sparse		1 where a sample point lies inside a component.
`var_true`	scalar	(kg/m³)²	Volume-weighted variance of the true density.
`R2_max`	scalar		Largest R^2 of these components.

`grid`: Density grid

Created by `sbtk.exchange.density_grid`, `sbtk.exchange.read_grid`.

Fields of `grid` (5)

Field	Size	Unit	Description
`spacing_km`	scalar	km	Cell edge.
`x_km`	n × 3	km	Cell centers, at integer multiples of the spacing in the shape-model frame.
`density_kgm3`	n × 1	kg/m³	Density at the cell centers.
`density_sigma_kgm3`	n × 1	kg/m³	1σ of the density: 0 for a true model; propagated from the parameter covariance in a solution package.
`member`	n × nc logical		Cell center inside each component. Only from `sbtk.exchange.density_grid`.

`gridcmp`: Comparison of two density grids

Created by `sbtk.compare.grids`.

Fields of `gridcmp` (8)

Field	Size	Unit	Description
`R2`	scalar		$1 - \sum(\rho_{\mathrm{est}} - \rho_{\mathrm{true}})^2 \,/\, \sum(\rho_{\mathrm{true}} - \bar{\rho}_{\mathrm{true}})^2$ over the common cells.
`rms_kgm3`	scalar	kg/m³	Root-mean-square of estimate minus truth.
`mean_error_kgm3`	scalar	kg/m³	Mean of estimate minus truth.
`std_true_kgm3`	scalar	kg/m³	Standard deviation of the true density.
`within_2sigma`	scalar		Fraction of cells with an absolute error at most twice the 1σ of the estimate; NaN if the estimate has no uncertainties.
`n_common`	scalar		Number of cells in both grids.
`n_truth`	scalar		Number of cells in the true grid.
`n_estimate`	scalar		Number of cells in the estimated grid.

`out`: Results folder

Created by `sbtk.io.new_run`, `sbtk.io.subfolder`.

Fields of `out` (5)

Field	Size	Unit	Description
`project`	text		Project name.
`name`	text		Run name.
`stamp`	text		Creation time, `yyyy-mm-dd_HHMMSS`.
`folder`	text		Folder the save and report functions write to: `results/<project>/<stamp>_<name>`, or a subfolder of it.
`figures`	text		Folder for figures: `<folder>/figures`.

`pk`: Coefficient order

Created by `sbtk.internal.sh_packing`.

Packed coefficient vectors are ordered C00, C10, C11, S11, C20, C21, S21, C22, S22, C30, ...: for each degree, C_{l0}, then C_{lm}, S_{lm} for $m = 1 \ldots l$. Sizes for $L = 4$.

Fields of `pk` (6)

Field	Size	Unit	Description
`deg`	1 × 25		Degree of each entry.

Field	Size	Unit	Description
`ord`	1 × 25		Order of each entry.
`isS`	1 × 25 logical		True for sine coefficients.
`idx`	1 × 25		Linear index into an (L+1) × (L+1) matrix with `C(l+1, m+1)` = C_{lm}.
`n`	scalar		Number of entries, $(L+1)^2$.
`labels`	1 × 25 cell		Names, `'C00'` … `'S44'`.

`samples`: Points filling the body

Created by `sbtk.internal.body_samples`.

Each facet cone is divided into 24 radial shells of equal steps in the radial fraction; each shell is represented by its centroid and volume. Used by the comparisons and the random-field integrals. Sizes for Didymos.

Fields of `samples` (5)

Field	Size	Unit	Description
`X`	1179648 × 3	km	Shell centroids in the shape-model frame.
`w`	1179648 × 1	km³	Shell volumes.
`ts`	1179648 × 1	km	Distance from the origin to the surface along each point's ray.
`ds`	24 × np		$s_k^p - s_{k-1}^p$ for the shell bounds s and powers $p = 3 \ldots \max(L+3, 5)$.
`nr`	scalar		Number of shells per cone (24).

`slices`: Plotting planes

Created by `sbtk.internal.slice_planes`.

A 1 × 2 structure array: plane 1 is $z = 0$ and plane 2 is $y = 0$, both through the geometry's origin.

Fields of `slices` (7)

Field	Size	Unit	Description
`g`	1 × 280	km	Grid coordinates along both plane axes.
`in`	280 × 280 logical		Grid point inside the body.
`X`	k × 3	km	Those points in the shape-model frame.
`ts`	k × 1	km	Surface distance along each point's ray.
`outline`	1441 × 2	km	Outline of the body in the plane.
`xlabel`	text		Axis label.
`ylabel`	text		Axis label.

Reference

Error identifiers

SBTK raises an error instead of substituting a value when an input is invalid or a computation cannot proceed. Every error has an identifier `sbtk:<package>:<condition>`, usable with `verifyError` in tests. The messages below are the fixed parts of the message text; values inserted at run time appear as format specifiers (`%s`, `%d`, `%g`).

Identifier	Raised by	Message
`sbtk:compare:differentComponents`	`sbtk.compare.evaluate`	The fit's components differ from the reference's or origin (e.g. a sphere position was estimated): use sbtk.compare.with_truth.
`sbtk:compare:gridSpacing`	`sbtk.compare.grids`	The grids have different spacings (%g and %g km).
`sbtk:data:axisymmetric`	`sbtk.internal.libration_weights`	Iyy - Ixx = Ixy = 0 (C22 = S22 = 0): the body has no preferred equatorial axis, so (B - A)/C is not defined.
`sbtk:data:coefLabel`	`sbtk.data.select_coefficients`	The data have no coefficient %s.
`sbtk:data:coefSize`	`sbtk.data.from_coefficients`	C, S, C_sigma and S_sigma must all be (L+1) x (L+1) with L >= 1.
`sbtk:data:covPositive`	`sbtk.data.set_covariance`	The covariance must be positive definite.
`sbtk:data:covSize`	`sbtk.data.set_covariance`	Covariance is %d x %d; the data have %d coefficients.
`sbtk:data:covSymmetric`	`sbtk.data.set_covariance`	The covariance must be symmetric.
`sbtk:data:degreeOne`	`sbtk.data.add_center_of_mass`	The data already have degree-1 coefficients.
`sbtk:data:duplicateMoi`	`sbtk.data.add_moi_row`	The data already have a moment-of-inertia row named %s.
`sbtk:data:incompleteDegree`	`sbtk.data.to_frame`	Rotating the coefficients needs every coefficient of each degree present; degree %d is incomplete.
`sbtk:data:moiLabel`	`sbtk.data.select_moi`	The data have no moment-of-inertia measurement %s.
`sbtk:data:moiLabel`	`sbtk.data.set_moi_sigma`	The data have no moment-of-inertia measurement %s.
`sbtk:data:moveOrigin`	`sbtk.data.move_origin`	Only measurements about the center of mass can move their origin; these are about obs.origin_km.
`sbtk:data:negativeDensity`	`sbtk.data.simulate`	The interior has negative density (minimum %.0f kg/m^3): make the light components less light.
`sbtk:data:noC22`	`sbtk.data.add_libration`	The libration ratio needs the measured C22 and S22 (they give the direction of the principal axes).
`sbtk:data:noCommonCoefficients`	`sbtk.data.use_uncertainties`	The two data sets have no gravity coefficient in common.
`sbtk:data:noIzz`	`sbtk.data.set_moi_sigma`	The 1-sigma of the libration row is the ratio's 1-sigma times Izz / (M R_ref^2) of the Izz row, which these data do not have: set it before sbtk.data.select_moi.
`sbtk:data:noIzz`	`sbtk.data.set_uncertainties`	The 1-sigma of the libration row is the ratio's 1-sigma times Izz / (M R_ref^2) of the Izz row, which these data do not have: set the uncertainties before sbtk.data.select_moi.
`sbtk:data:noUncertainties`	`sbtk.data.add_noise`	The measurements have no uncertainties to draw noise from: give them with sbtk.data.set_uncertainties, sbtk.data.use_uncertainties, sbtk.data.set_covariance or sbtk.data.set_moi_sigma.
`sbtk:data:noUncertainties`	`sbtk.exchange.write_data`	Some of the measurements have uncertainties and some do not. Give all of them (sbtk.data.set_uncertainties, sbtk.data.use_uncertainties, sbtk.data.set_covariance, sbtk.data.set_moi_sigma) or none (exact values).
`sbtk:data:noUncertainties`	`sbtk.invert.replace_data`	The measurements have no uncertainties, which the inversion needs as weights: give them with sbtk.data.set_uncertainties, sbtk.data.use_uncertainties, sbtk.data.set_covariance or sbtk.data.set_moi_sigma.
`sbtk:data:noUncertainties`	`sbtk.invert.setup`	The measurements have no uncertainties, which the inversion needs as weights: give them with sbtk.data.set_uncertainties, sbtk.data.use_uncertainties, sbtk.data.set_covariance or sbtk.data.set_moi_sigma.
`sbtk:data:transformMeasured`	`sbtk.data.to_frame`	Inertia rows can be expressed in new axes only from the full inertia tensor without uncertainties: transform the exact values of sbtk.data.simulate, then use sbtk.data.set_uncertainties, sbtk.data.select_moi and sbtk.data.add_noise.
`sbtk:data:uncertainties`	`sbtk.data.set_uncertainties`	unc.coef_sigma_by_degree has %d values; degree %d needs one per degree.
`sbtk:data:zeroSigma`	`sbtk.data.from_coefficients`	Uncertainty of %s is zero; every used coefficient needs sigma > 0.
`sbtk:exchange:correlatedCenterOfMass`	`sbtk.internal.exchange_data_files`	The center of mass is correlated with the other coefficients; the exchange format gives them separately.
`sbtk:exchange:covariance`	`sbtk.exchange.read_data`	gravity_covariance.txt has no row for %s.
`sbtk:exchange:dataSetName`	`sbtk.exchange.solution_name`	Data set %s is not named <team>-DATA<number>.
`sbtk:exchange:locate`	`sbtk.exchange.locate`	Found %d %s packages in %s (and its redacted/, unredacted/); expected one.

Identifier	Raised by	Message
`sbtk:exchange:missingKey`	`sbtk.internal.exchange_number`	The file has no "# %s:" line.
`sbtk:exchange:moiRow`	`sbtk.internal.exchange_data_files`	Moment-of-inertia row %s has no exchange-format equivalent.
`sbtk:exchange:noCenterOfMass`	`sbtk.internal.exchange_data_files`	The data need the degree-1 coefficients (the measured center of mass): add them with sbtk.data.add_center_of_mass.
`sbtk:exchange:normalization`	`sbtk.exchange.read_shadr`	The header of %s gives normalization state %g; SBTK needs 1 (fully normalized).
`sbtk:frame:notRotation`	`sbtk.internal.check_rotation`	The axes must be a rotation: orthonormal columns (the new axes as unit vectors in the current frame) with determinant +1.
`sbtk:gravity:incompleteSet`	`sbtk.gravity.center_of_mass_frame`	The field is about a point %.1f m from its center of mass and does not have every coefficient up to degree %d, which moving it needs.
`sbtk:gravity:incompleteSet`	`sbtk.gravity.center_of_mass_frame`	The principal axes need every coefficient of degree 2 to %d; the field lacks some.
`sbtk:gravity:notStarShaped`	`sbtk.gravity.geometry`	%d facets face the expansion origin: the shape is not star-shaped about [%g %g %g] km.
`sbtk:interior:duplicateName`	`sbtk.interior.create`	Component names must be unique (they label the results): %s
`sbtk:interior:noAxis`	`sbtk.interior.set_axis`	Component %s is a %s, which has no axis.
`sbtk:interior:noAxis`	`sbtk.interior.set_axis`	The direction for the axis of %s is zero.
`sbtk:interior:sectorCount`	`sbtk.interior.sector_grid`	Use 1 to 9 bands in each direction (got %d, %d, %d).
`sbtk:interior:sectorRange`	`sbtk.interior.sector`	Sector %s: each range must increase (frac %s, lat %s, lon %s) and span at most 360 deg of longitude.
`sbtk:interior:transformFit`	`sbtk.interior.to_frame`	An inversion result cannot be transformed. Transform the measurements (sbtk.data.to_frame) and the body (sbtk.body.to_frame) and invert in the new frame.
`sbtk:interior:unknownComponent`	`sbtk.interior.align_with_spin_axis`	The interior has no component named %s.
`sbtk:invert:coefLabel`	`sbtk.invert.setup`	Coefficient %s is above obs.degree = %d.
`sbtk:invert:differentData`	`sbtk.invert.replace_data`	The new data differ in labels, origin, degree, mass or inertia weights: run sbtk.invert.setup instead.
`sbtk:invert:duplicateName`	`sbtk.invert.setup`	Component names must be unique: %s
`sbtk:invert:geometryParameter`	`sbtk.internal.geometry_parameters`	estimate_position of %s must be true, false or a mask of 3 (x, y, z).
`sbtk:invert:geometryParameter`	`sbtk.internal.geometry_parameters`	Component %s is a %s, which has no center to estimate.
`sbtk:invert:geometryParameter`	`sbtk.internal.geometry_parameters`	Component %s is a %s, which has no axis to estimate.
`sbtk:invert:infeasible`	`sbtk.invert.run`	Prescribed densities violate the density bounds.
`sbtk:invert:moiLabel`	`sbtk.invert.setup`	opts.moi_labels asks for %s; the data have: %s.
`sbtk:invert:noCenterOfMass`	`sbtk.invert.setup`	Coefficients about the center of mass need its measured position as degree-1 coefficients (sbtk.data.add_center_of_mass).
`sbtk:invert:outsideBody`	`sbtk.invert.setup`	Component %s lies entirely outside the body.
`sbtk:invert:solver`	`sbtk.invert.run`	The bounded least-squares solution did not converge (lsqlin exit flag %d).
`sbtk:io:degree`	`sbtk.io.read_jpl_gravity`	The file has coefficients up to degree %d only.
`sbtk:io:jplFormat`	`sbtk.io.read_jpl_gravity`	NAMES, VALS and COV sizes do not agree in %s.
`sbtk:io:jplFormat`	`sbtk.io.read_jpl_gravity`	Coefficient %s is missing from %s.
`sbtk:io:jplFormat`	`sbtk.io.read_jpl_gravity`	Keyword %s not found.
`sbtk:io:jplFormat`	`sbtk.io.read_jpl_gravity`	Block %s not found.
`sbtk:io:noRun`	`sbtk.io.latest_run`	No run named %s in results/%s.
`sbtk:io:notNormalized`	`sbtk.io.read_jpl_gravity`	The coefficients in %s are not normalized.
`sbtk:io:referenceRadius`	`sbtk.io.read_jpl_gravity`	The file's reference radius is %g km; body.R_ref_km is %g km.

Identifier	Raised by	Message
`sbtk:iteration:notConverged`	`sbtk.internal.fixed_point`	The iteration for %s did not converge in %d steps (last change %.3g, tolerance %.3g).
`sbtk:plot:names`	`sbtk.plot.difference_from_homogeneous`	Give one name per set: %d sets, %d names.
`sbtk:shape:notDownloaded`	`sbtk.shape.bundled`	%s is not in data/shapes/: the high-resolution models are not in git; copy it from the Hera server (End-to-End Test/Shape/).
`sbtk:shape:notSpc`	`sbtk.shape.reduce_spc`	%d vertices and %d facets: not an SPC grid model (6 (q+1)^2 vertices, 12 q^2 facets).
`sbtk:shape:reduce`	`sbtk.shape.reduce`	The reduced shape (%d facets) is closed %d, outward %d, star-shaped %d: try another number of facets.
`sbtk:shape:spcReduce`	`sbtk.shape.reduce_spc`	The reduced grid (%d vertices) is closed %d, outward %d, star-shaped %d: the vertices are not in SPC grid order.
`sbtk:shape:spcStep`	`sbtk.shape.reduce_spc`	step %d does not divide the grid size q = %d.

Internal functions

Functions in `sbtk.internal` implement the supported functions. They are not part of the supported interface: their arguments and behavior may change without notice, and scripts should not call them. They are listed here because the methods pages refer to some of them. The toolbox contains no deprecated or legacy functions.

Function	Purpose	Used by (supported functions)
`sbtk.internal.body_samples`	Points that fill the body, with the volume each one represents.	`sbtk.compare.reference`, `sbtk.data.simulate`, `sbtk.gravity.random_field_integrals`
`sbtk.internal.body_settings`	The numbers that define a body, for `settings.json`.	`sbtk.io.report_comparison`, `sbtk.io.report_inversion`, `sbtk.io.report_simulation`
`sbtk.internal.check_rotation`	Stop unless axes is a proper rotation (orthonormal columns, right-handed).	`sbtk.body.to_frame`, `sbtk.data.to_frame`, `sbtk.interior.to_frame`
`sbtk.internal.component_columns`	Design-matrix columns and volumes of components idx at their current geometry.	`sbtk.invert.run`, `sbtk.invert.setup`
`sbtk.internal.component_member`	True for points X inside component c.	`sbtk.compare.reference`, `sbtk.exchange.density_grid`, `sbtk.gravity.component_integrals`
`sbtk.internal.component_settings`	Components as plain records for `settings.json` (only the fields each type uses).	`sbtk.io.report_inversion`, `sbtk.io.report_simulation`
`sbtk.internal.component_text`	One-line description of a component's geometry (printouts and text files).	`sbtk.exchange.write_model`, `sbtk.exchange.write_solution`, `sbtk.io.write_truth`
`sbtk.internal.ComputePolyhedralCSBar_mex`	Fully normalized spherical-harmonic coefficients of a uniform tetrahedron with one vertex at the expansion origin (compiled C).	`sbtk.gravity.geometry`
`sbtk.internal.density_at_points`	Density [kg/m^3] of a model at points inside the body.	`sbtk.compare.reference`, `sbtk.data.simulate`, `sbtk.exchange.density_grid`, `sbtk.interior.density_at`
`sbtk.internal.density_map`	Sequential dark-blue -> teal -> yellow color map (viridis-like) for densities.	`sbtk.plot.slices`, `sbtk.plot.truth_vs_estimate`
`sbtk.internal.difference_map`	Blue - white - red color map for signed differences (blue lighter, red denser).	`sbtk.plot.interior3d`, `sbtk.plot.truth_vs_estimate`
`sbtk.internal.empty_moi`	Empty list of moment-of-inertia measurement rows.	`sbtk.data.from_coefficients`, `sbtk.exchange.read_data`, `sbtk.io.read_jpl_gravity`, `sbtk.io.read_measurements`
`sbtk.internal.exchange_conventions`	Text of conventions.txt: units, frame, normalization (with code), degree 1.	other internal functions
`sbtk.internal.exchange_data_files`	Write the measurement files of an exchange package.	`sbtk.exchange.write_data`, `sbtk.exchange.write_model`
`sbtk.internal.exchange_grid_write`	Write `density_grid.txt` of an interior-model or solution package.	`sbtk.exchange.write_model`, `sbtk.exchange.write_solution`
`sbtk.internal.exchange_number`	Numeric value of a metadata key of an exchange file (error if missing).	`sbtk.exchange.read_data`, `sbtk.exchange.read_grid`
`sbtk.internal.exchange_read`	Read one exchange-format text file (see sbtk.internal.exchange_write).	`sbtk.exchange.read_data`, `sbtk.exchange.read_grid`
`sbtk.internal.exchange_write`	Write one exchange-format text file.	`sbtk.exchange.write_model`, `sbtk.exchange.write_solution`
`sbtk.internal.fixed_point`	Iterate x = update(x) until a step moves x by at most tol; at most `max_iter` steps.	`sbtk.data.simulate`, `sbtk.interior.align_with_spin_axis`
`sbtk.internal.geometry_deviation`	How far one geometry parameter of a component is from its start.	`sbtk.invert.run`
`sbtk.internal.geometry_parameters`	The geometry an inversion estimates: one entry per scalar parameter.	`sbtk.invert.setup`
`sbtk.internal.geometry_step`	A component with one geometry parameter changed by a step.	`sbtk.invert.run`
`sbtk.internal.icosphere`	Unit sphere mesh: an icosahedron with every triangle split into four LEVEL times.	`sbtk.shape.ellipsoid`
`sbtk.internal.inertia_from_Q`	Inertia tensor from second moments.	`sbtk.exchange.read_data`, `sbtk.gravity.forward`
`sbtk.internal.libration_weights`	Weights on [Ixx Iyy Izz Ixy Ixz Iyz] of the libration row (B - A) - ratio*C = 0.	`sbtk.data.add_libration`, `sbtk.data.simulate`, `sbtk.data.to_frame`

Function	Purpose	Used by (supported functions)
sbtk.internal.moi_tensor	3x3 normalized inertia tensor from measurement rows Ixx..Iyz (NaN if any is missing).	sbtk.data.add_moi_row, sbtk.data.add_noise, sbtk.data.select_moi, sbtk.data.simulate, sbtk.data.to_frame, sbtk.exchange.read_data, sbtk.io.read_measurements
sbtk.internal.new_component	Fields shared by every component (use the sbtk.interior constructors).	sbtk.interior.ellipsoid, sbtk.interior.layer, sbtk.interior.no_components, sbtk.interior.sector, sbtk.interior.sphere, sbtk.interior.torus
sbtk.internal.observe	Predicted measurements from per-unit-density integrals I (struct with U and Q).	sbtk.invert.run, sbtk.invert.setup
sbtk.internal.point_mass_sh	Normalized spherical harmonics of unit point masses.	sbtk.gravity.component_integrals
sbtk.internal.principal_axes	Principal axes of inertia from the degree-2 gravity coefficients.	sbtk.exchange.write_model, sbtk.gravity.center_of_mass_frame, sbtk.plot.difference_from_homogeneous
sbtk.internal.random_field_value	Random-field density perturbation [kg/m^3] at points X (n x 3, km).	sbtk.gravity.random_field_integrals
sbtk.internal.read_obj	Triangular faces and vertices of a Wavefront OBJ shape model.	sbtk.shape.load
sbtk.internal.rotate_inertia	Inertia elements [Ixx Iyy Izz Ixy Ixz Iyz] in rotated axes: axes' * I * axes.	sbtk.data.to_frame, sbtk.interior.to_frame
sbtk.internal.sh_packing	Ordering of packed spherical-harmonic coefficient vectors.	sbtk.data.from_coefficients, sbtk.data.select_degree, sbtk.data.set_uncertainties, sbtk.data.summary, sbtk.data.to_frame, sbtk.data.use_uncertainties, sbtk.exchange.read_shadr, sbtk.exchange.write_model, sbtk.gravity.center_of_mass_frame, sbtk.gravity.geometry, sbtk.gravity.labels, sbtk.interior.to_frame, sbtk.io.read_jpl_gravity, sbtk.io.report_simulation, sbtk.io.write_measurements, sbtk.plot.difference_from_homogeneous, sbtk.plot.measurements
sbtk.internal.sh_rotation	Matrix that expresses spherical-harmonic coefficients in rotated axes.	sbtk.data.to_frame, sbtk.gravity.center_of_mass_frame, sbtk.interior.to_frame, sbtk.plot.difference_from_homogeneous
sbtk.internal.sh_translation	Matrix that moves spherical-harmonic integrals to a shifted expansion origin.	sbtk.exchange.write_model, sbtk.gravity.center_of_mass_frame, sbtk.invert.run
sbtk.internal.shape_file_name	File name of a shape model in an exchange package.	other internal functions
sbtk.internal.slice_axes	Draw one density slice (image + body outline) on axes ax.	sbtk.plot.slices, sbtk.plot.truth_vs_estimate
sbtk.internal.slice_images	Absolute density of each model on the two slice planes.	sbtk.plot.slices, sbtk.plot.truth_vs_estimate
sbtk.internal.slice_planes	Grid points on the two plotting planes, for density slices.	other internal functions
sbtk.internal.surface_distance	Distance from the expansion origin to the surface along unit directions [km].	sbtk.exchange.density_grid, sbtk.gravity.component_integrals, sbtk.interior.density_at, sbtk.invert.run, sbtk.plot.interior3d
sbtk.internal.surface_distance_near	Distance from a nearby origin to the surface through given points [km].	sbtk.compare.reference

Validation and tests

SBTK 1.0.0 has 153 unit and integration tests in MATLAB's unit-test framework. They run with

```
MATLAB
run(fullfile(sbtk.root, "tests", "run_tests.m"))      % about 6 minutes; writes tests/reports/test_report.txt
```

This page states what the tests check and with what tolerance, as written in the test files. It makes no claims beyond them. Unit tests use a small ellipsoid body (`tests/unit/small_body.m`); validation tests use an ellipsoid with semi-axes 0.5 × 0.4 × 0.3 km (20,480 facets) and the 9.3 m Didymos model with a bulk density of 2600 kg/m³ and R_{ref} = 0.40 km.

Comparison with independent results

From `tests/integration/test_validation.m`:

Test	Check	Tolerance
`testKernelVolume`	The kernel's degree-0 integral of every facet cone equals the cone volume.	relative 10^{-12}
`testEllipsoidClosedForm`	$\bar{C}_{20}$, $\bar{C}_{22}$ and the principal moments of a homogeneous triaxial ellipsoid against the closed forms $C_{20} = (2c^2 - a^2 - b^2) / (10R^2)$, $C_{22} = (a^2 - b^2) / (20R^2)$, $I = M(b^2 + c^2) / 5$, …	relative 2 × 10^{-3} (20,480-facet mesh)
`testConeScaling`	Kernel output for a cone scaled by $s = 0.37$ against s^{l+3} times the unscaled integrals.	relative 10^{-13}
`testSpherePointMassVsRays`	A sphere integrated as a point mass against the same sphere integrated along rays (as an ellipsoid).	relative 10^{-3}
`testTorus`	Volume and I_{zz} of a buried torus against $2\pi^2 R a^2$ and $V(R^2 + 3a^2 / 4)$.	relative 10^{-4} (volume), 10^{-3} (I_{zz})
`testShellSector`	A full-sphere sector between radial fractions 0.5 and 0.75 has volume $(0.75^3 - 0.5^3)V$.	relative 10^{-12}
`testSectorsTileBody`	The 24 sectors of a sector grid fill the body exactly once.	relative 10^{-12}
`testRandomFieldMonteCarlo`	Mass and degree-2 integrals of a random field (σ = 120 kg/m³, L = 100 m) on Didymos against Monte Carlo integration with 4 × 10^6 points.	2% of the largest value
`testExactRecovery`	Exact Didymos data of a core (off-center), torus and regolith: densities and the core center are recovered, starting with the core at the origin.	10^{-6} kg/m³, 10^{-7} km, $\chi^2 < 10^{-10}$
`testExactRecoverySectors`	Exact data of a 24-sector interior with random densities are fitted by the same sectors.	$\chi^2 < 10^{-6}$, densities within 20 kg/m³
`testCalibration`	Over 40 noise realizations, the standard deviation of (estimate − truth)/σ.	within 0.25 of 1
`testFullCovariance`	The same with coefficients correlated by 0.6 and noise drawn from that covariance.	within 0.25 of 1
`testInertiaHelps`	Adding I_{xx}, I_{yy}, I_{zz} to the gravity coefficients reduces every density uncertainty.	strict
`testDimorphosExact`	Exact data of degree ≤ 3 and the libration ratio only (no other inertia data): an ellipsoidal core and a regolith layer are recovered.	10^{-6} kg/m³
`testTextRoundTrip`	Writing and reading the measurement text format preserves values, covariance, origin, seed and libration ratio, and the fits agree.	relative 10^{-14}; densities 10^{-6} kg/m³
`testFromCoefficients`	Data built from coefficient matrices equal the original measurement set and give the same fit.	exact; 10^{-6} kg/m³

Conventions and frames

From `tests/unit/test_exchange.m` and `tests/unit/test_data.m`:

Test	Check	Tolerance
`testDegreeTwoAndInertia`	The relations between $\bar{C}_{20}$, $\bar{C}_{21}$, $\bar{S}_{21}$, $\bar{C}_{22}$, $\bar{S}_{22}$ and the inertia elements (Moments of inertia).	10^{-12}
`testNormalizationCode`	The normalization function printed in `conventions.txt` gives $N_{20}, N_{21}, N_{22}, N_{30} = \sqrt{5}$, $\sqrt{5/3}$, $\sqrt{5/12}$, $\sqrt{7}$.	relative 10^{-15}
`testShTranslation`	Integrals moved by the solid-harmonic translation equal integrals computed about the shifted origin, including inertia.	10^{-12} of the largest value
`testPredictionsAboutTheModelCenterOfMass`	For data about the center of mass, the predicted coefficients and inertia of a fit equal a forward model computed from scratch about the fitted center of mass.	10^{-6}
`testCenterOfMassFrameMatchesTheTrueFrame`	Inverting a measurement package (expansion about the model's center of mass) gives the densities of an inversion about the true center of mass.	0.2σ; χ^2 within 10%
`testFrameIterationWithSectors`	The center-of-mass iteration converges with layers and many sectors.	offset < 10^{-6} km in fewer than 20 iterations
`testNothingAboutTheInterior`	A measurement package does not contain component names, the data-set name of the model, the seed, the expansion origin or the true center of mass.	

Test	Check	Tolerance
`testExactPackage`	A package of exact values has the columns `l,m,C,S`, no covariance file and no uncertainty or noise lines; it is read back with NaN uncertainties, the inversion asks for uncertainties, and with them it fits.	values 10^{-17}; $\chi^2 < 10^{-6}$
`testReportStatesWhatApplies`	Uncertainties, noise, the sign convention of the products of inertia and the libration ratio are stated only when the package has them; exact values with the center of mass as origin in principal axes, or with x, y turned 20° about z, are described as such in `conventions.txt`, and the package's shape model has its center of mass at the origin.	10^{-12} km; obj precision for the center of figure
`testGridDefinition`	Density-grid cell centers lie on multiples of the spacing and the cells fill the body; grids of different spacing are rejected.	volume within 5% at 50 m cells
`testLibrationRatioUsesPrincipalMoments`	The simulated libration ratio equals $(B - A) / C$ from the eigenvalues of the tensor, and the true tensor satisfies the libration row.	relative 10^{-12}, 10^{-14}
`testLibrationAmplitudeFormula`	$G = 2e / (3k - 1)$ against numerical integration of planar libration in a Keplerian orbit (e = 0.01; k = 0.07, 0.25, 0.5), and the inverse formula.	3%; relative 10^{-12}

Inversion behavior

From `tests/unit/test_invert.m`: exact recovery with $\nu = n - p$; the mass constraint (relative 10^{-10}); the homogeneous fit has the bulk density and $\nu = n$; regularization reduces the norm of the relative densities and increases the degrees of freedom; the p-value of $\chi^2 = 10$ with $\nu = 10$ equals 0.44049; density bounds hold and are flagged; settings by name and value; starting values of the relative densities are not used; unknown inertia labels and inconsistent replacement data are rejected. `testEstimateTorusAxisAndHeight`: the axis of a torus (true tilt 4.5°) and its height (12 m), started 3.5° and 15 m away, are recovered from exact values to 0.02° and 0.2 m with its density (5 kg/m³). `testGeometryLimits`: a hard limit on the tilt holds, an a priori constraint keeps the axis at its start, and geometry asked of components without a center or an axis is rejected. `testEstimateEllipsoidOrientation`: the full orientation of an ellipsoid (the direction of its third axis and its turn about it), started 8° away, is recovered from exact values to 0.05°, and a limit of 3° on the turn holds. The sphere-position results of `test_regression_baseline.m` (including the number of iterations) did not change when the estimated geometry was generalized.

Reproduction of published and earlier results

Test file	Check	Tolerance
`tests/integration/test_bennu_paper.m`	The Bennu inversion of Cavale et al. (Icarus) (Bennu): core relative density; boulders; torus mass contrast; core center; background equal to the bulk density 1190 kg/m³; coefficients of degrees 3–4 not fitted by the paper.	1 kg/m³; 2%; 1%; 0.5 m; 10^{-9} kg/m³; 0.15 of the JPL 1σ
`tests/integration/test_regression_baseline.m`	Geometry, integrals, sample points, simulation, noise, inversion, regularization choice, comparison and position estimation against values stored from the code before the toolkit (`tests/reference/baseline_v0.mat`). Results that include an ellipsoid or a torus differ because their ray crossings are now exact: the old code sampled 256 points per chord and missed rays that only graze them (the exact crossings equal 4096 samples to 10^{-15}).	relative 10^{-10} unless stated in the test; 2×10^{-6} with an ellipsoid or a torus

Other tests

Test file	Contents
`tests/unit/test_units.m`	G, length scales, inertia conversions, degree-1 conversion.
`tests/unit/test_shape.m`	OBJ reading with units, write–read round trip, ellipsoid volume (relative 3×10^{-3}) and centroid, detection of inward and open meshes, units required, bundled models, SPC reduction (closed, outward, star-shaped).
`tests/unit/test_body.m`	Mass from density, centroid of a shifted shape, bodies from mass, invalid densities rejected.
`tests/unit/test_interior.m`	Constructors, ellipsoid rotation, layer depths, latitude–longitude, duplicate names, sector grids and ranges, random field reproducible and with standard deviation within 15% of σ. `testSetAxis`: the axis points along the direction, by the smallest rotation. `testAlignWithSpinAxis`: a torus aligned with the axis of the largest moment of the model it belongs to (to 0.002°, in fewer than 10 simulations), the other components unchanged, unknown names and non-convergence reported.
`tests/unit/test_gravity.m`	Uniform body, mass kept by the background, center-of-mass shift, labels, non-star-shaped meshes rejected, components clipped to the body. `testCenterOfMassFrame`: about the center of mass degree 1 is 0; in principal axes $\bar{C}_{21} = \bar{S}_{21} = \bar{S}_{22} = 0$, each degree's power and each degree's total variance are unchanged, and $\bar{C}_{20}$, $\bar{C}_{22}$ equal the values from the principal moments; a measurement set, the same set without degree 1, and a set lacking a coefficient.
`tests/unit/test_frames.m`	`testSpinAxes`: in the spin axes $\bar{C}_{21} = \bar{S}_{21} = 0$ to 10^{-13}, z is the axis of the largest moment, x lies in the plane of the shape-model x-axis and z, $\frac{1}{2}\operatorname{atan2}(\bar{S}_{22}, \bar{C}_{22})$ is the longitude of the axis of the smallest moment, each degree's power is unchanged, and the comparison with the homogeneous body gives its own terms. `testChangeOfFrameEqualsSimulationInIt`: a body, an interior with every component type and a random field, carried into a rotated and shifted frame and simulated there, give the transformed true model, coefficients, inertia rows and libration ratio of the original simulation (to 10^{-7}), and the density grid in the new frame equals the original model at the same points (to 10^{-9} kg/m³). `testPackageInSpinAxes`: packages in the spin axes contain their own shape-model file, the data have $\bar{C}_{21}$, $\bar{S}_{21}$ zero within the noise, and the grid and the component descriptions are in the same frame. `testChangeOfFrameRules`: a shift applies to any set; new axes need exact inertia rows and complete degrees; fits are not transformed; non-rotations are rejected. `testNoLocalPathInPackage`: a package with a shape model changed in frame names the original file but not its folder. `testChangesOfFrameCombine`: after two changes of frame the shape model's vertices equal `(original vertices - frame_origin_km)*frame_axes` to 10^{-12} km. `testGeneratedShapeNamedAfterPackage`: a shape model changed in frame and written into two packages gets two file names, `<data set>_<shape name>.obj`, and the interior-model package names the same file as its data package.

Test file	Contents
`tests/unit/test_data.m`	Measurement fields, center-of-mass frame, simulation without uncertainties and `sbtk.data.set_uncertainties`, noise, negative density rejected, selection, covariance, inertia and libration rows, inertia uncertainties.
`tests/unit/test_compare.m`	$R^2 = 1$ for the true model; $R^2 = 0$ (within 0.01) for a homogeneous estimate.
`tests/unit/test_io.m`	Run folders, missing runs, report functions; a run that only makes a true model (simulation without uncertainties, the comparison with the homogeneous body in principal axes, the interior-model package with the principal axes and moments of the true model).
`tests/unit/test_plot.m`	Every figure function runs. `testDifferenceFromHomogeneous`: a homogeneous body differs from itself by 0; one interior as a true model, a package, a field about the shape-model origin and a fitted model gives the same differences. `testDifferenceInPrincipalAxes`: the rotated field has $C_{21} = S_{21} = S_{22} = 0$ to 10^{-13}, and the homogeneous field of the rotated shape model equals the rotated homogeneous coefficients to 10^{-12}.
`tests/integration/test_examples.m`	The eight tutorial scripts run, ex01 also section by section.
`tests/integration/test_template_project.m`	The project template runs: simulation, inversion, comparison.
`tests/integration/test_maker_scripts.m`	The six HLP6 maker scripts in `examples/` pass the code analyzer; `make_didymos_3` and `make_dimorphos_3` run, and their packages have exact values only, their own shape-model file with the center of mass at the origin, the frame stated in `conventions.txt`, $\tilde{C}_{21} = \tilde{S}_{21} = 0$ (and $\tilde{S}_{22} = 0$ for Dimorphos) to 10^{-12}, every boulder or void of the model, and no local paths.

Longer studies in `tests/studies/` (recovery over interiors and data sets, edge cases of the forward model against brute-force integration, uncertainty calibration) are run separately and are not part of the test suite.